

UNIVERSITAS NEGERI YOGYAKARTA
FAKULTAS TEKNIK
PROGRAM STUDI TEKNIK INDUSTRI

LAPORAN AKHIR
APLIKASI WEB DAN MOBILE
Semester Genap 2025/2026

LAPORAN AKHIR APLIKASI WEB AND MOBILE

Nama Mahasiswa : Saprina Melita Sari
NIM : 230514300012
Kelas : A1
Tanggal Praktikum : 9 Juni 2026
Dosen Pengampu : Dr. Eng. Ir. Aji Ery Burhandenny, S.T., M.AIT.

Dikumpulkan paling lambat 7 hari setelah pelaksanaan praktikum

DAFTAR ISI

MODUL 1	3
MODUL 2	21
MODUL 3	41
MODUL 4	61
MODUL 5	78
MODUL 6	98
MODUL 7	115
MODUL 8	140
MODUL 9	163
MODUL 10	178
MODUL 11	203
MODUL 12	222
MODUL 14	240
MODUL 15	260

MODUL 1

A. IDENTITAS & INFORMASI MODUL

Mata Kuliah	Aplikasi Web dan Mobile
Kode Modul	Modul 1 (Pertemuan 1 Pengenalan Pemrograman Web & HTML Dasar)
Topik Utama	Pengenalan Pemrograman Web & HTML Dasar
Sub-Topik	Pemrograman Web & HTML Dasar
Durasi Praktikum	340 Menit
Tanggal Praktikum	13 Februari 2026
Nama Mahasiswa	Saprina Melita Sari
NIM	23051430012
Kelas	A

B. TUJUAN PEMBELAJARAN

1	Memahami peran penting web application dalam sistem informasi industri.
2	Memahami peran penting web application dalam sistem informasi industri.
3	Mampu membuat halaman web statis dengan elemen teks, gambar, dan tabel.
4	Mampu menyusun formulir input data sederhana (misal: data produksi).

C. ALAT DAN BAHAN

No.	Nama Alat / Bahan / Aplikasi	Versi / Spesifikasi	Keterangan
1	Visual Studio Code (VS Code)	Versi 1.8x ke atas	Code Editor
2	Google Chrome / Browser Modern	Versi terbaru	Testing & Debugging

3	Node.js & npm	Node 18+ / npm 9+	Runtime JS (Modul tertentu)
4	Laptop	Asus	Device utama

D. DASAR TEORI

Perkembangan teknologi informasi telah menjadikan internet sebagai bagian penting dalam kehidupan sehari-hari, di mana *website* menjadi salah satu medium utama untuk menyampaikan informasi secara cepat dan luas kepada pengguna di seluruh dunia melalui browser tanpa dibatasi jarak ataupun waktu. Evolusi HTML (*HyperText Markup Language*) hingga versi terbarunya yaitu HTML5 menunjukkan bagaimana struktur web telah berkembang dari sekadar tampilan statis menjadi fondasi yang mampu mendukung berbagai aplikasi web modern, termasuk integrasi multimedia, API, dan interoperabilitas lintas perangkat dalam konteks aplikasi web yang kompleks (*HTML5 and the evolution of HTML; tracing the origins of digital platforms*).

Dalam pembuatan website, HTML merupakan komponen dasar yang berfungsi untuk menyusun struktur halaman, seperti menentukan judul, paragraf, tabel, gambar, serta tautan sehingga browser dapat memproses dan menampilkannya secara sistematis (*HTML*). Struktur ini memberikan organisasi konten yang jelas dan logis sehingga menjadi pondasi utama sebelum dilakukan pengembangan yang lebih lanjut dengan CSS (untuk tampilan visual) dan JavaScript (untuk fitur interaktif). Tanpa penggunaan HTML yang benar sebagai struktur dasar dokumen, halaman web tidak akan memiliki hierarki atau kerangka yang dapat diinterpretasikan oleh browser.

Lebih jauh lagi, beberapa penelitian deskriptif mengenai evolusi HTML menunjukkan bahwa perkembangan dari HTML1 hingga HTML5 membawa peningkatan kemampuan dalam perancangan dan pengembangan sistem web yang tidak hanya menampilkan data tetapi juga mampu mendukung elemen interaktif dan integrasi data yang lebih kompleks (*Studi Deskriptif Evolusi Website Dari Html1 Sampai Html5 Dan Pengaruhnya Terhadap Perancangan Dan Pengembangan Website*). Hal ini menjadi penting ketika website digunakan sebagai basis sistem informasi operasional dalam organisasi maupun industri karena struktur konten yang baik berkontribusi pada keterbacaan, pemeliharaan, dan pengembangan sistem informasi secara berkelanjutan.

Selain itu, pemahaman HTML menjadi langkah awal yang penting bagi pengembang web pemula karena dari sinilah semua struktur konten dikendalikan. Setelah pemahaman HTML, biasanya dikombinasikan dengan CSS untuk mempercantik tampilan visual dan JavaScript untuk memberikan fungsi interaktif pada halaman, sehingga menciptakan sebuah aplikasi web yang tidak hanya informatif tetapi juga responsif serta menarik bagi pengguna.

D.1 Konsep Utama

Konsep utama pada pertemuan pertama adalah memahami fondasi pengembangan aplikasi web melalui struktur dasar HTML sebagai kerangka (structure) halaman web. Pertemuan ini berfokus pada bagaimana sebuah halaman web dibangun dari sisi front-end menggunakan HTML sebelum masuk ke aspek desain (CSS) dan logika/interaktivitas (JavaScript).

D.2 Keterkaitan dengan Teknik Industri

Teknik Industri berfokus pada perancangan, perbaikan, dan integrasi sistem yang melibatkan manusia, material, informasi, peralatan, dan energi. Dalam era digital, pengelolaan informasi menjadi salah satu elemen kunci sistem tersebut, dan teknologi web merupakan media utama dalam mendukung integrasi informasi.

Sebagai contoh suatu perusahaan di klaten dapat merancang untuk menampilkan jumlah output harian, tingkat cacat produk, dan waktu henti mesin (downtime).

E. LANGKAH KERJA DAN HASIL PRAKTIKUM

E.1 Langkah 1 – Membuat File HTML

Membuat kerangka HTML dasar.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Data Mesin Produksi - Lini A</title>
</head>
<body>

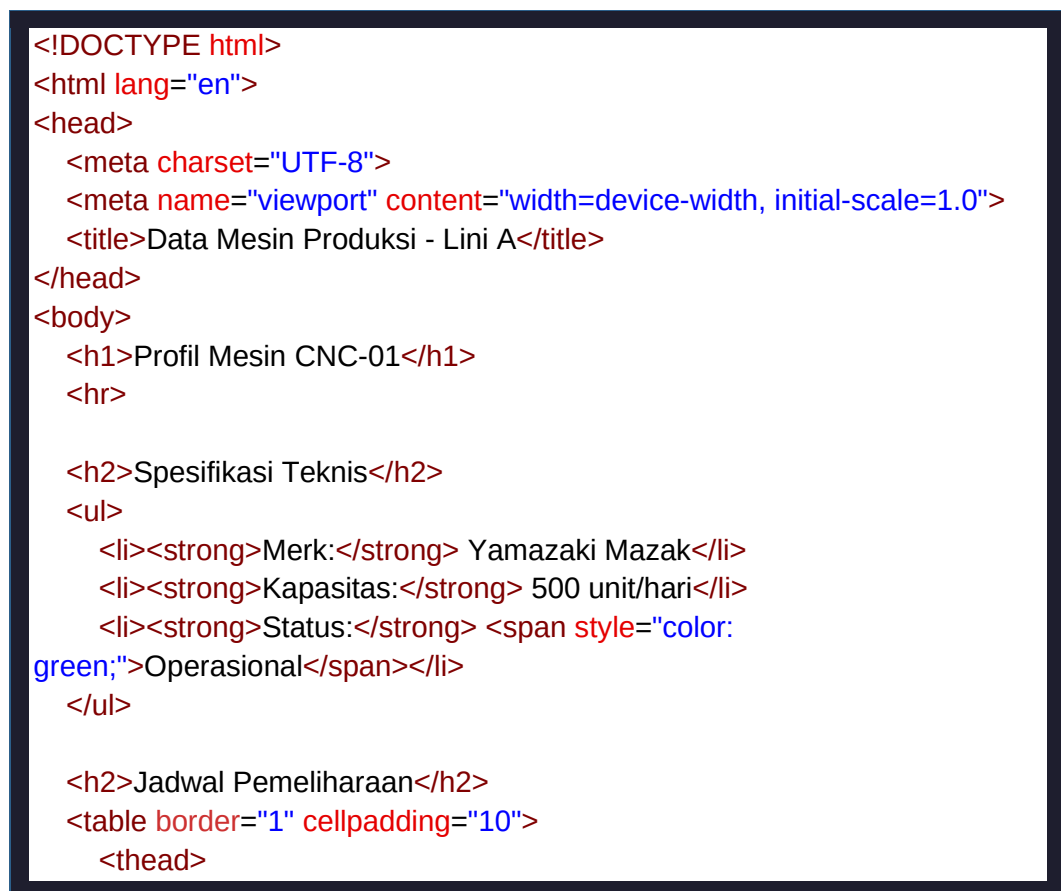
</body>
</html>
```

Screenshot Hasil:



Gambar E.1 – Hasil Langkah 1

E.2 Langkah 2 – Menyusun Profil Lini Produksi



```
<tr>
  <th>Tanggal</th>
  <th>Jenis Perawatan</th>
  <th>Penanggung Jawab</th>
</tr>
</thead>
<tbody>
  <tr>
    <td>12 Jan 2024</td>
    <td>Penggantian Oli</td>
    <td>Teknisi A</td>
  </tr>
  <tr>
    <td>15 Feb 2024</td>
    <td>Kalibrasi Presisi</td>
    <td>Teknisi B</td>
  </tr>
</tbody>
</table>
</body>
</html>
```

Screenshot Hasil:

The screenshot displays a web interface for a CNC machine profile. It includes a title 'Profil Mesin CNC-01', a technical specification section, and a maintenance schedule table.

Profil Mesin CNC-01

Spesifikasi Teknis

- Merk: Yamazaki Mazak
- Kapasitas: 500 unit/hari
- Status: Operasional

Jadwal Pemeliharaan

Tanggal	Jenis Perawatan	Penanggung Jawab
12 Jan 2024	Penggantian Oli	Teknisi A
15 Feb 2024	Kalibrasi Presisi	Teknisi B

Gambar E.2 – Hasil Langkah 2

E.3 Langkah 3 – Membuat Form Input Data Produksi

```

<hr>
<h2>Input Data Harian</h2>
<form action="">
  <label for="tanggal">Tanggal:</label><br>
  <input type="date" id="tanggal" name="tanggal" required><br><br>

  <label for="shift">Shift Kerja:</label><br>
  <select id="shift" name="shift">
    <option value="Pagi">Pagi (08:00 - 16:00)</option>
    <option value="Siang">Siang (16:00 - 24:00)</option>
    <option value="Malam">Malam (00:00 - 08:00)</option>
  </select><br><br>

  <label for="Jumlah">Jumlah Produksi (Unit):</label><br>
  <input type="number" id="jumlah" name="jumlah" min="0"
placeholder="Masukkan jumlah..."><br><br>

  <label for="catatan">Catatan Kerusakan:</label><br>
  <textarea id="Catatan" name="catatan" rows="4" cols="50"
placeholder="Deskripsikan jika ada kendala..."></textarea><br><br>

  <button type="submit">Simpan Data</button>
</form>

```

Screenshot Hasil:

Gambar E.3 – Hasil Langkah 3

E.4 Langkah 4 – Menjalankan File

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Data Mesin Produksi - Lini A</title>
</head>
<body>
  <h1>Profil Mesin CNC-01</h1>
  <hr>

  <h2>Spesifikasi Teknis</h2>
  <ul>
    <li><strong>Merk:</strong> Yamazaki Mazak</li>
    <li><strong>Kapasitas:</strong> 500 unit/hari</li>
    <li><strong>Status:</strong> <span style="color:
green;">Operasional</span></li>
  </ul>

  <h2>Jadwal Pemeliharaan</h2>
  <table border="1" cellpadding="10">
    <thead>
      <tr>
        <th>Tanggal</th>
        <th>Jenis Perawatan</th>
        <th>Penanggung Jawab</th>
      </tr>
    </thead>
    <tbody>
      <tr>
        <td>12 Jan 2024</td>
        <td>Penggantian Oli</td>
        <td>Teknisi A</td>
      </tr>
      <tr>
        <td>15 Feb 2024</td>
        <td>Kalibrasi Presisi</td>
        <td>Teknisi B</td>
      </tr>
    </tbody>
  </table>
  <hr>
  <h2>Input Data Harian</h2>
  <form action="">
    <label for="tanggal">Tanggal:</label><br>
```

```

<input type="date" id="tanggal" name="tanggal" required><br><br>

<label for="shift">Shift Kerja:</label><br>
<select id="shift" name="shift">
  <option value="Pagi">Pagi (08:00 - 16:00)</option>
  <option value="Siang">Siang (16:00 - 24:00)</option>
  <option value="Malam">Malam (00:00 - 08:00)</option>
</select><br><br>

<label for="Jumlah">Jumlah Produksi (Unit):</label><br>
<input type="number" id="jumlah" name="jumlah" min="0"
placeholder="Masukkan jumlah..."><br><br>

<label for="catatan">Catatan Kerusakan:</label><br>
<textarea id="Catatan" name="catatan" rows="4" cols="50"
placeholder="Deskripsikan jika ada kendala..."></textarea><br><br>

<button type="submit">Simpan Data</button>
</form>
</body>
</html>

```

Screenshot Hasil:

Profil Mesin CNC-01

Spesifikasi Teknis

- Merk: Yamazaki Mazak
- Kapasitas: 500 unit/hari
- Status: Operasional

Jadwal Pemeliharaan

Tanggal	Jenis Perawatan	Penanggung Jawab
12 Jan 2024	Penggantian Oli	Teknisi A
15 Feb 2024	Kalibrasi Presisi	Teknisi B

Gambar E.4 – Hasil Langkah 4

F. LATIHAN DAN TUGAS MANDIRI

F.1 Latihan 1 – Pengayaan Tabel

Pertanyaan / Instruksi Latihan:

Tambahkan satu kolom baru di tabel "Jadwal Pemeliharaan" bernama "Status" dan isi datanya secara manual (Selesai/Pending).

Jawaban / Kode Solusi:

```
<h2>Jadwal Pemeliharaan</h2>
<table border="1" cellpadding="10">
  <thead>
    <tr>
      <th>Tanggal</th>
      <th>Jenis Perawatan</th>
      <th>Penanggung Jawab</th>
      <th>Status</th>
    </tr>
  </thead>
  <tbody>
    <tr>
      <td>12 Jan 2024</td>
      <td>Penggantian Oli</td>
      <td>Teknisi A</td>
      <td>Pending</td>
    </tr>
    <tr>
      <td>15 Feb 2024</td>
      <td>Kalibrasi Presisi</td>
      <td>Teknisi B</td>
      <td>Selesai</td>
    </tr>
  </tbody>
</table>
<hr>
```

Jadwal Pemeliharaan

Tanggal	Jenis Perawatan	Penanggung Jawab	Status
12 Jan 2024	Penggantian Oli	Teknisi A	Pending
15 Feb 2024	Kalibrasi Presisi	Teknisi B	Selesai

Gambar F.1 – Hasil Latihan 1

F.2 Latihan 2 – Validasi Dasar

Pertanyaan / Instruksi Latihan:

Pada input "Jumlah Produksi", tambahkan atribut `max` dengan nilai 600 (maksimal kapasitas mesin).

Jawaban / Kode Solusi:

```
<label for="Jumlah">Jumlah Produksi (Unit):</label><br>  
  <input type="number" id="jumlah" name="jumlah" min="0" max="600"  
  placeholder="Masukkan jumlah..."><br><br>
```

Input Data Harian

Tanggal:
02/21/2026

Shift Kerja:
Pagi (08:00 - 16:00)

Jumlah Produksi (Unit):
702

Value must be less than or equal to 600.

Simpan Data

Gambar F.2 – Hasil Latihan 2

F.3 Latihan 3 – Semantic HTML

Pertanyaan / Instruksi Latihan:

Bungkus judul "Profil Mesin" dan spesifikasinya menggunakan tag `` dan `` agar kode lebih rapi dan semantik.

Jawaban / Kode Solusi:


```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Data Mesin Produksi - Lini A</title>
</head>
<body>
  <header>
    <h1>Profil Mesin CNC-01</h1><hr>
  </header>

  <section>
    <h2>Spesifikasi Teknis</h2>
    <ul>
      <li><strong>Merk:</strong> Yamazaki Mazak</li>
      <li><strong>Kapasitas:</strong> 500 unit/hari</li>
      <li><strong>Status:</strong> <span style="color:
green;">Operasional</span></li>
    </ul>
  </section>

  <section>
    <h2>Jadwal Pemeliharaan</h2>
    <table border="1" cellpadding="10">
      <thead>
        <tr>
          <th>Tanggal</th>
          <th>Jenis Perawatan</th>
          <th>Penanggung Jawab</th>
          <th>Status</th>
        </tr>
      </thead>
      <tbody>
        <tr>
          <td>12 Jan 2024</td>
          <td>Penggantian Oli</td>
          <td>Teknisi A</td>
          <td>Pending</td>
        </tr>
        <tr>
          <td>15 Feb 2024</td>
          <td>Kalibrasi Presisi</td>
          <td>Teknisi B</td>
          <td>Selesai</td>
        </tr>
      </tbody>
    </table>
  </section>
</body>
</html>
```

```
</tr>
</tbody>
</table>
<hr>
</section>

<section>
  <h2>Input Data Harian</h2>
  <form action="">
    <label for="tanggal">Tanggal:</label><br>
    <input type="date" id="tanggal" name="tanggal" required><br><br>

    <label for="shift">Shift Kerja:</label><br>
    <select id="shift" name="shift">
      <option value="Pagi">Pagi (08:00 - 16:00)</option>
      <option value="Siang">Siang (16:00 - 24:00)</option>
      <option value="Malam">Malam (00:00 - 08:00)</option>
    </select><br><br>

    <label for="Jumlah">Jumlah Produksi (Unit):</label><br>
    <input type="number" id="jumlah" name="jumlah" min="0" max="600"
placeholder="Masukkan jumlah..."><br><br>

    <label for="catatan">Catatan Kerusakan:</label><br>
    <textarea id="Catatan" name="catatan" rows="4" cols="50"
placeholder="Deskripsikan jika ada kendala..."></textarea><br><br>

    <button type="submit">Simpan Data</button>
  </form>
</section>

</body>
</html>
```

Profil Mesin CNC-01

Spesifikasi Teknis

- Merk: Yamazaki Mazak
- Kapasitas: 500 unit/hari
- Status: Operasional

Jadwal Pemeliharaan

Tanggal	Jenis Perawatan	Penanggung Jawab	Status
12 Jan 2024	Penggantian Oli	Teknisi A	Pending
15 Feb 2024	Kalibrasi Presisi	Teknisi B	Selesai

Input Data Harian

Tanggal:

Shift Kerja:

Jumlah Produksi (Unit):

Catatan Kerusakan:

Gambar F.3 – Hasil Latihan 3

F.3 Tugas Proyek Mini – Kartu Karyawan

Deskripsi proyek mini:

Buatlah halaman HTML baru bernama `laporan\_kualitas.html`. Halaman ini berisi formulir "Laporan Inspeksi Kualitas" dengan field:

- ♣ ID Produk (Text)
- ♣ Nama Inspektur (Text)
- ♣ Jenis Cacat (Radio Button: Geometri, Permukaan, Fungsional)

- ♣ Jumlah Cacat (Number)
- ♣ Foto Produk (Input type file - placeholder saja)
- ♣ Tombol Submit

Penjelasan pendekatan/solusi yang digunakan:

[Proyek mini ini diselesaikan dengan merancang form laporan inspeksi kualitas menggunakan HTML untuk menginput ID produk, nama inspektur, jenis cacat, jumlah cacat, dan foto produk. Struktur form dibuat menggunakan elemen <form> dan berbagai tipe input seperti text, radio, number, dan file agar data dapat diisi dengan lengkap. Selanjutnya, form dapat dikembangkan dengan menambahkan method POST dan enctype untuk mendukung upload file serta dihubungkan ke backend agar data dapat disimpan.

Kode Utama:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Laporan Inspeksi Kualitas</title>
</head>
<body>
  <header>
    <h1>Laporan Inspeksi Kualitas</h1><hr>
  </header>

  <main>
    <section>
      <form action="">
        <label for="id_produk">ID Produk:</label><br>
        <input type="text" id="id_produk" name="id_produk"
placeholder="Masukkan ID Produk..." required><br><br>

        <label for="nama_inspektur">Nama Inspektur:</label><br>
        <input type="text" id="nama_inspektur" name="nama_inspektur"
placeholder="Masukkan Nama Inspektur..." required><br><br>

        <label for="jenis_cacat">Jenis Cacat:</label><br>
        <input type="radio" id="geometri" name="jenis_cacat"
value="Geometri" required>
        <label for="geometri">Geometri</label><br>
```

```

        <input type="radio" id="permukaan" name="jenis_cacat"
value="Permukaan">
        <label for="permukaan">Permukaan</label><br>

        <input type="radio" id="fungsional" name="jenis_cacat"
value="Fungsional">
        <label for="fungsional">Fungsional</label><br><br>

        <label for="jumlah">Jumlah Cacat (Unit):</label><br>
        <input type="number" id="jumlah" name="jumlah" min="0"
placeholder="Masukkan Jumlah Cacat..."><br><br>

        <label for="foto_produk">Foto Produk:</label><br>
        <input type="file" id="foto_produk" name="foto_produk"
placeholder="Upload Foto Produk Dari Perangkat Anda"><br><br>

        <button type="submit">Submit</button>
    </form>
</section>
</main>

</body>
</html>

```

Gambar F.3 – Hasil Pembuatan Mini Proyek

G. PEMBAHASAN

G.1 Analisis Kesesuaian Hasil dengan Teori

Hasil praktikum yang diperoleh telah sesuai dengan teori dan konsep yang dipelajari pada Modul 1 mengenai struktur dasar HTML sebagai fondasi pembuatan halaman web. Selain itu, latihan dan hasil proyek mini dapat dipahami alur cara mengerjakan sesuai dengan cara pada praktikum 1.

G.2 Kendala yang Ditemukan dan Solusinya

Kendala / Error yang Ditemukan	Solusi yang Diterapkan
Pembuatan radio button	Mencari bagaimana cara membuat radio button di internet.

G.3 Kaitan dengan Penerapan di Industri

Kemudahan dalam menginput barang atau produk. Misalnya pada suatu pabrik terdapat produk *defect* maka dapat diinput jumlah tanpa menggunakan kertas atau ditulis secara manual. Hasil dari input an akan tersimpan secara otomatis dan akan membantu dalam pengarsipan dokumen.

H. KESIMPULAN

1	<hr> digunakan untuk garis.
2	Textarea digunakan untuk menulis dengan text panjang.
3	Penggunaan placeholder digunakan agar user mengetahui harus mengisi apa.
4	Penulisan label for dan input id harus sama agar kode berjalan.

I. DAFTAR PUSTAKA

No.	Referensi (Format APA 7th Edition)
[1]	Burhandenny, A. E. (2026). Manual Praktikum Aplikasi Web dan Mobile Semester Genap 2025/2026. Program Studi Teknik Industri, Universitas Negeri Yogyakarta.
[2]	Sama, H., & Hartanto, E. (2021, March). Studi Deskriptif Evolusi Website Dari Html1 Sampai Html5 Dan Pengaruhnya Terhadap Perancangan Dan Pengembangan Website. In <i>CoMBInES-Conference on Management, Business, Innovation, Education and Social Sciences</i> (Vol. 1, No. 1, pp. 589-596).
[3]	Tabarés, R. (2021). HTML5 and the evolution of HTML; tracing the origins of digital platforms. <i>Technology in Society</i> , 65, 101529.

J. LEMBAR PENILAIAN DAN PENGESAHAN

RUBRIK PENILAIAN LAPORAN					
Aspek Penilaian		Bobot			
No.	Aspek Penilaian	Bobot (%)	Nilai (0-100)	Skor Akhir	Catatan Penilai
1	Kelengkapan Isi (Semua bagian A–I terisi lengkap)	20%			
2	Kualitas Dasar Teori (Ditulis dengan bahasa sendiri, relevan, minimal 3 paragraf)	15%			
3	Dokumentasi Langkah Kerja (Kode & Screenshot sesuai, jelas, dan terbaca)	25%			
4	Tugas/Latihan Mandiri (Semua latihan & proyek mini dikerjakan dan berfungsi)	20%			
5	Pembahasan & Analisis (Kritis, mendalam, dan relevan dengan industri)	15%			
6	Kesimpulan & Tata Bahasa (Spesifik, rapi, mengikuti format yang ditentukan)	5%			
TOTAL NILAI AKHIR		100%			

Mahasiswa	Rekan Tim	Diperiksa oleh Asisten / Dosen
 _____ Saprina Melita Sari NIM:23051430012	 _____ Ardita Natalia NIM:23051430022	 _____ Dr. Eng. Ir. Aji Ery Burhandenny, S.T., M.AIT. Tanggal: _____

MODUL 2

A. IDENTITAS & INFORMASI MODUL

Mata Kuliah	Aplikasi Web dan Mobile
Kode Modul	Modul 2 (CSS Dasar & Styling)
Topik Utama	CSS Dasar & Styling
Sub-Topik	Pengenalan CSS, Selectors, Box Model, dan Styling Form
Durasi Praktikum	340 Menit
Tanggal Praktikum	20 Februari 2026
Nama Mahasiswa	Saprina Melita Sari
NIM	23051430012
Kelas	A

B. TUJUAN PEMBELAJARAN

1	Memahami cara kerja CSS untuk mempercantik antarmuka pengguna (UI).
2	Mampu memanipulasi warna, font, dan ukuran elemen.
3	Menguasai konsep Box Model (Margin, Border, Padding) untuk tata letak.
4	Menerapkan CSS pada formulir agar terlihat profesional dan mudah digunakan.

C. ALAT DAN BAHAN

No.	Nama Alat / Bahan / Aplikasi	Versi / Spesifikasi	Keterangan
1	Visual Studio Code (VS Code)	Versi 1.8x ke atas	Code Editor
2	Google Chrome / Browser Modern	Versi terbaru	Testing & Debugging

3	Node.js & npm	Node 18+ / npm 9+	Runtime JS (Modul tertentu)
4	Laptop	Asus	Device utama
5	HTML	-	Struktur
6	CSS	-	Styling

D. DASAR TEORI

Menurut Tanjung (2024), HTML merupakan bahasa standar web yang di kelola penggunaanya oleh W3S (*Word Wide Web Consortium*) berupa tag tag yang menyusun setiap element dari website. HTML berperan sebagai penyusun struktur halaman website yang menempatkan setiap *element website*. Sedangkan CSS adalah dokumen yang berfungsi mengatur *element* HTML dengan berbagai property yang tersedia sehingga dapat tampil dengan gaya yang diinginkan (Bangun et al., 2020).

Dalam konteks pengembangan web, HTML menyediakan kerangka dasar dan struktur konten, sedangkan CSS memberikan **atur gaya dan estetika** sehingga halaman web tidak hanya fungsional tetapi juga menarik secara visual bagi pengguna. Hal ini konsisten dengan kajian dalam literatur yang menyatakan bahwa pemahaman dasar HTML dan CSS merupakan langkah pertama dalam pengembangan web yang efektif, di mana HTML digunakan untuk membangun struktur halaman dan CSS digunakan untuk memperindah serta mengatur layout sesuai kebutuhan desain (Mardiansyah et al., 2023).

Penguasaan HTML dan CSS sangat penting dalam konteks pendidikan dan praktik pengembangan web, karena dengan memahami kedua teknologi ini pengguna mampu menciptakan halaman yang tidak hanya terstruktur dengan baik tetapi juga estetis dan responsif terhadap berbagai ukuran layar.

D.1 Konsep Utama

Menentukan elemen html yang akan diberikan style, misalnya warna, jarak, dan ukuran. CSS juga membuat tampilan sesuai dengan ukuran layar.

D.2 Keterkaitan dengan Teknik Industri

Penggunaan CSS digunakan untuk styling suatu coding sehingga terlihat rapi dan menarik. Selain itu, CSS digunakan untuk standarisasi style di seluruh website supaya tampilan lebih konsisten. User yang melihat atau mengakses akan lebih menyukai.

E. LANGKAH KERJA DAN HASIL PRAKTIKUM

E.1 Langkah 1 – Membuat File CSS Eksternal

Membuat kerangka HTML dasar.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Data Mesin Produksi - Lini A</title>
  <link rel="stylesheet" href="Praktikum 2 style.css">
</head>
<body>
```

Screenshot Hasil:



```
1  <!DOCTYPE html>
2  <html lang="en">
3  <head>
4      <meta charset="UTF-8">
5      <meta name="viewport" content="width=device-width, initial-scale=1.0">
6      <title>Data Mesin Produksi - Lini A</title>
7      <link rel="stylesheet" href="Praktikum 2 style.css">
8  </head>
```

Gambar E.1 – Hasil Langkah 1

E.2 Langkah 2 – Styling Body dan Tipografi

```
/* Import Font dari Google Fonts */
@import url('https://fonts.googleapis.com/css2?family=Roboto:wght
@400;700&display=swap');

body {
```

```

font-family: 'Roboto', sans-serif;
background-color: #f4f4f4; /* Warna abu-abu muda untuk background */
color: #333;
margin: 0;
padding: 20px;
}

h1 {
color: #2c3e50; /* Biru tua gelap */
text-align: center;
border-bottom: 2px solid #2c3e50;
padding-bottom: 10px;
}

h2 {
color: #34495e;
margin-top: 30px;
}

```

Screenshot Hasil:

Profil Mesin CNC-01

Spesifikasi Teknis

- Merk: Yamazaki Mazak
- Kapasitas: 500 unit/hari
- Status: Operasional

Jadwal Pemeliharaan

Tanggal	Jenis Perawatan	Penanggung Jawab
12 Jan 2024	Penggantian Oli	Teknisi A
15 Feb 2024	Kalibrasi Presisi	Teknisi B

Input Data Harian

Tanggal:

Shift Kerja:

Jumlah Produksi (Unit):

Catatan Kerusakan:

Gambar E.2 – Hasil Langkah 2

E.3 Langkah 3 – Styling Tabel Data

```

table {
width: 100%;
border-collapse: collapse; /* Menggabungkan border ganda menjadi satu */
margin-top: 20px;
box-shadow: 0 2px 5px rgba(0,0,0,0.1); /* Efek bayangan halus */
}

```

```

th, td {
    padding: 12px;
    text-align: left;
    border-bottom: 1px solid #ddd;
}

th {
    background-color: #2980b9; /* Biru profesional */
    color: white;
}

tr:hover {
    background-color: #f1f1f1; /* Highlight baris saat mouse diarahkan */
}

```

Screenshot Hasil:

Profil Mesin CNC-01

Spesifikasi Teknis

- Merk: Yamazaki Mazak
- Kapasitas: 500 unit/hari
- Status: Operasional

Jadwal Pemeliharaan

Tanggal	Jenis Perawatan	Pemanggung Jawab
12 Jan 2024	Penggantian Oli	Teknisi A
15 Feb 2024	Kalibrasi Presisi	Teknisi B

Input Data Harian

Tanggal:

Shift Kerja:

Jumlah Produksi (Unit):

Catatan Kerusakan:

Gambar E.3 – Hasil Langkah 3

E.4 Langkah 4 – Styling Formulir (Fokus pada UX)

```

form {
    background-color: white;
    padding: 20px;
    border-radius: 8px; /* Sudut melengkung */
    box-shadow: 0 2px 5px rgba(0,0,0,0.1);
    max-width: 600px;
    margin: 0 auto; /* Formulir berada di tengah */
}

label {
    display: block;
}

```

```
margin-bottom: 5px;
font-weight: bold;
}

input[type="text"],
input[type="number"],
input[type="date"],
select,
textarea {
  width: 100%; /* Lebar penuh */
  padding: 10px;
  margin-bottom: 15px;
  border: 1px solid #ccc;
  border-radius: 4px;
  box-sizing: border-box; /* Padding tidak menambah lebar total */
}

/* Styling khusus tombol */
button {
  background-color: #27ae60; /* Hijau */
  color: white;
  padding: 12px 20px;
  border: none;
  border-radius: 4px;
  cursor: pointer;
  width: 100%;
  font-size: 16px;
}

button:hover {
  background-color: #219150; /* Hijau lebih gelap saat hover */
}
```

Screenshot Hasil:

Tanggal:

mm/dd/yyyy

Shift Kerja:

Pagi (08:00 - 16:00)

Jumlah Produksi (Unit):

Masukkan jumlah...

Catatan Kerusakan:

Deskripsikan jika ada kendala...

Simpan Data

Gambar E.4 – Hasil Langkah 4

E.4 Langkah 5 – Melihat Hasil

```
/* Import Font dari Google Fonts */
@import url('https://fonts.googleapis.com/css2?family=Roboto:
wght@400;700&display=swap');

body {
  font-family: 'Roboto', sans-serif;
  background-color: #f4f4f4; /* Warna abu-abu muda untuk background */
  color: #333;
  margin: 0;
  padding: 20px;
}

h1 {
  color: #2c3e50; /* Biru tua gelap */
  text-align: center;
```

```
border-bottom: 2px solid #2c3e50;
padding-bottom: 10px;
}

h2 {
color: #34495e;
margin-top: 30px;
}

table {
width: 100%;
border-collapse: collapse; /* Menggabungkan border ganda menjadi satu */
margin-top: 20px;
box-shadow: 0 2px 5px rgba(0,0,0,0.1); /* Efek bayangan halus */
}

th, td {
padding: 12px;
text-align: left;
border-bottom: 1px solid #ddd;
}

th {
background-color: #2980b9; /* Biru profesional */
color: white;
}

tr:hover {
background-color: #f1f1f1; /* Highlight baris saat mouse diarahkan */
}

form {
background-color: white;
padding: 20px;
border-radius: 8px; /* Sudut melengkung */
box-shadow: 0 2px 5px rgba(0,0,0,0.1);
max-width: 600px;
margin: 0 auto; /* Formulir berada di tengah */
}

label {
display: block;
margin-bottom: 5px;
font-weight: bold;
}
```



```
input[type="text"],
input[type="number"],
input[type="date"],
select,
textarea {
  width: 100%; /* Lebar penuh */
  padding: 10px;
  margin-bottom: 15px;
  border: 1px solid #ccc;
  border-radius: 4px;
  box-sizing: border-box; /* Padding tidak menambah lebar total */
}

/* Styling khusus tombol */
button {
  background-color: #27ae60; /* Hijau */
  color: white;
  padding: 12px 20px;
  border: none;
  border-radius: 4px;
  cursor: pointer;
  width: 100%;
  font-size: 16px;
}

button:hover {
  background-color: #219150; /* Hijau lebih gelap saat hover */
}
```

Screenshot Hasil:

Profil Mesin CNC-01

Spesifikasi Teknis

- Merk: Yamazaki Mazak
- Kapasitas: 500 unit/hari
- Status: Operasional

Jadwal Pemeliharaan

Tanggal	Jenis Perawatan	Pemegang Jarak
12 Jan 2024	Penggantian Oli	Teknisi A
15 Feb 2024	Kalibrasi Presisi	Teknisi B

Input Data Harian

Tanggal:

mm/dd/yyyy

Shift Kerja:

Pagi (08.00 - 16.00)

Jumlah Produksi (Unit):

Masukkan jumlah...

Catatan Kerusakan:

Berikan detail jika ada masalah...

Simpan Data

Gambar E.4 – Hasil Langkah 5

F. LATIHAN DAN TUGAS MANDIRI

F.1 Latihan 1 – *Class Selector*

Pertanyaan / Instruksi Latihan:

Berikan kelas `status-ok` pada teks status operasional di HTML, dan berikan warna hijau tebal di CSS. Berikan kelas `status-down` dan berikan warna merah.

Jawaban / Kode Solusi:

```

<h2>Spesifikasi Teknis</h2>
<ul>
  <li><strong>Merk:</strong> Yamazaki Mazak</li>
  <li><strong>Kapasitas:</strong> 500 unit/hari</li>
  <li><strong>Status:</strong> <span class="status-ok">Operasional</span></li>
</ul>

<ul>
  <li><strong>Merk:</strong> Vianoruu Yuuji</li>
  <li><strong>Kapasitas:</strong> 1000 unit/hari</li>
  <li><strong>Status:</strong> <span class="status-down">Tidak
Operasional</span></li>
</ul>

```

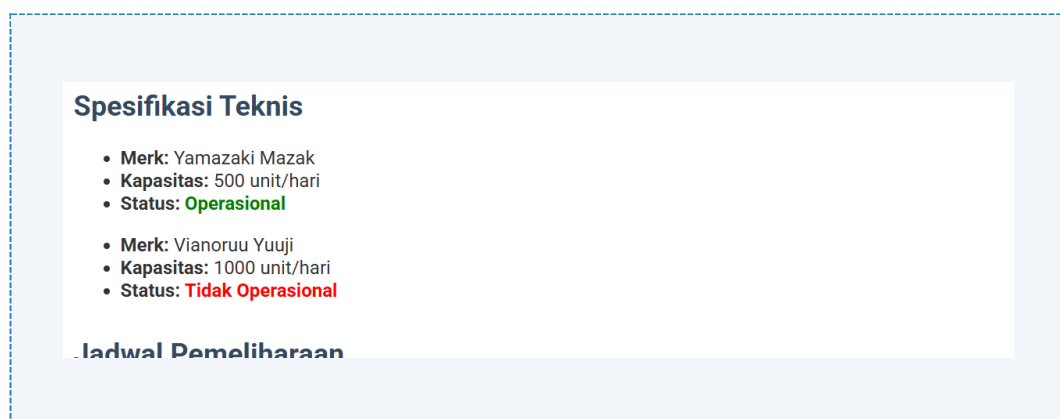
Dan untuk coding di CSS ditambahkan

```

.status-ok {
  color: green;
  font-weight: bold;
}

.status-down {
  color: red;
  font-weight: bold;
}

```



Gambar F.1 – Hasil Latihan 1

F.2 Latihan 2 – Layout Container

Pertanyaan / Instruksi Latihan:

Buat sebuah `

` dengan id `container` di HTML yang membungkus seluruh konten. Di CSS, berikan `max-width: 800px`, `margin: 0 auto`, dan `background: white` agar konten terlihat seperti kertas di tengah layar.

Jawaban / Kode Solusi:

```
<body>
  <div id="container">
    <h1>Profil Mesin CNC-01</h1>
    <hr>

    <h2>Spesifikasi Teknis</h2>
    <ul>
      <li><strong>Merk:</strong> Yamazaki Mazak</li>
      <li><strong>Kapasitas:</strong> 500 unit/hari</li>
      <li><strong>Status:</strong> <span class="status-
ok">Operasional</span></li>
    </ul>

    <ul>
      <li><strong>Merk:</strong> Vianoruu Yuuji</li>
      <li><strong>Kapasitas:</strong> 1000 unit/hari</li>
      <li><strong>Status:</strong> <span class="status-down">Tidak
Operasional</span></li>
    </ul>

    <h2>Jadwal Pemeliharaan</h2>
    <table border="1" cellpadding="10">
      <thead>
        <tr>
          <th>Tanggal</th>
          <th>Jenis Perawatan</th>
          <th>Penanggung Jawab</th>
        </tr>
      </thead>
      <tbody>
        <tr>
          <td>12 Jan 2024</td>
          <td>Penggantian Oli</td>
          <td>Teknisi A</td>
        </tr>
        <tr>
          <td>15 Feb 2024</td>
          <td>Kalibrasi Presisi</td>
          <td>Teknisi B</td>
        </tr>
      </tbody>
    </table>
  </div>
```

```
</tbody>
</table>
<hr>
<h2>Input Data Harian</h2>
<form action="">
  <label for="tanggal">Tanggal:</label><br>
  <input type="date" id="tanggal" name="tanggal" required><br><br>

  <label for="shift">Shift Kerja:</label><br>
  <select id="shift" name="shift">
    <option value="Pagi">Pagi (08:00 - 16:00)</option>
    <option value="Siang">Siang (16:00 - 24:00)</option>
    <option value="Malam">Malam (00:00 - 08:00)</option>
  </select><br><br>

  <label for="Jumlah">Jumlah Produksi (Unit):</label><br>
  <input type="number" id="jumlah" name="jumlah" min="0"
placeholder="Masukkan jumlah..."><br><br>

  <label for="catatan">Catatan Kerusakan:</label><br>
  <textarea id="Catatan" name="catatan" rows="4" cols="50"
placeholder="Deskripsikan jika ada kendala..."></textarea><br><br>

  <button type="submit">Simpan Data</button>
</form>
</div>
</body>

</html>
```

Dan coding CSS ditambahkan

```
#container {
  max-width: 800px;
  margin: 0 auto;
  background: white;
  padding: 20px;
}
```

Profil Mesin CNC-01

Spesifikasi Teknis

- Merk: Yamazaki Mazak
- Kapasitas: 500 unit/hari
- Status: **Operasional**
- Merk: Vianoruu Yuuji
- Kapasitas: 1000 unit/hari
- Status: **Tidak Operasional**

Jadwal Pemeliharaan

Tanggal	Jenis Perawatan	Penanggung Jawab
12 Jan 2024	Penggantian Oli	Teknisi A
15 Feb 2024	Kalibrasi Presisi	Teknisi B

Input Data Harian

Tanggal:

Shift Kerja:

Pagi (08:00 - 16:00)

Jumlah Produksi (Unit):

Catatan Kerusakan:

Deskripsikan jika ada kendala...

Simpan Data

Gambar F.2 – Hasil Latihan 2

F.3 Tugas Proyek Mini – Kartu Karyawan

Deskripsi proyek mini:

Terapkan styling CSS yang sama ke file `laporan\_kualitas.html` yang Anda buat di pertemuan 1.

Penjelasan pendekatan/solusi yang digunakan:

Pada Styling CSS untuk laporan kualitas hampir sama seperti pada latihan dan praktikum. Namun yang membedakan adalah hanya membutuhkan untuk h1 dan form. Selain itu, styling khusus tombol ditambahkan margin top sebesar 10 untuk memisahkan antara tombol reset dan submit. Penghapusan `<br>` pada html juga dilakukan agar spasi antara misal IDE produk dengan tabel dibawahnya tidak terlalu jauh. Penambahan # di container dilakukan karena coding di html memanggil dengan id.

Kode Utama:

```
/* Import Font dari Google Fonts */
@import url('https://fonts.googleapis.com/css2?family=Roboto:
wght@400;700&display=swap');

body {
  font-family: 'Roboto', sans-serif;
  background-color: #f4f4f4; /* Warna abu-abu muda untuk background */
  color: #333;
  margin: 0;
  padding: 20px;
}

h1 {
  color: #2c3e50; /* Biru tua gelap */
  text-align: center;
  border-bottom: 2px solid #2c3e50;
  padding-bottom: 10px;
}

form {
  background-color: white;
  padding: 20px;
  border-radius: 8px; /* Sudut melengkung */
  box-shadow: 0 2px 5px rgba(0,0,0,0.1);
  max-width: 600px;
  margin: 0 auto; /* Formulir berada di tengah */
}
```

```
label {
  display: block;
  margin-bottom: 5px;
  font-weight: bold;
}

.label-radio {
  display: inline-block;
  font-weight: lighter;
}

input[type="text"],
input[type="number"],
input[type="date"],
input[type="file"],
select,
textarea {
  width: 100%; /* Lebar penuh */
  padding: 10px;
  margin-bottom: 15px;
  border: 1px solid #ccc;
  border-radius: 4px;
  box-sizing: border-box; /* Padding tidak menambah lebar total */
}

/* Styling khusus tombol */
button {
  background-color: #f1f1f1;
  color: white;
  padding: 12px 20px;
  border: none;
  border-radius: 4px;
  cursor: pointer;
  width: 100%;
  font-size: 16px;
  margin-top: 10px; /* Supaya tombol tidak gabung */
}

button[type="submit"] {
  background-color: #28a745;
  color: white;
}

button[type="submit"]:hover {
  background-color: #218838;
}
```



```
button[type="reset"] {  
  background-color: #c0392b;  
  color: white;  
}  
  
button[type="reset"]:hover {  
  background-color: #c82333;  
}  
  
#container {  
  max-width: 800px;  
  margin: auto;  
  background: white;  
  padding: 20px;  
  border-radius: 15px;  
}
```

Laporan Inspeksi Kualitas

ID Produk:

Nama Inspektur:

Jenis Cacat:
☐ Geometri
☐ Permukaan
☐ Fungsional

Jumlah Cacat (Unit):

Foto Produk:
 No file chosen

Gambar F.3 – Hasil Pembuatan Mini Proyek

G. PEMBAHASAN

G.1 Analisis Kesesuaian Hasil dengan Teori

Hasil modul dengan output sesuai namun memerlukan beberapa percobaan untuk menyesuaikan antara coding html dan css. Html dengan css harus berkesinambungan dan href untuk css harus sesuai dengan nama file yang diberikan.

G.2 Kendala yang Ditemukan dan Solusinya

Kendala / Error yang Ditemukan	Solusi yang Diterapkan
Radio button yang mengarah ke bawah	Menggunakan display inline block untuk .label.
Tombol submit dan reset bergabung	Menambahkan margin top di styling khusus tombol button.
ID Produk dll terlalu jauh dengan tabel	Menghapus di html.

G.3 Kaitan dengan Penerapan di Industri

Penerapan modul ini berguna bagi industri yang ingin membuat suatu web atau mobile dimana harus memperhatikan style suatu coding. Semakin bagus styling maka semakin enak untuk dilihat karena rapi.

H. KESIMPULAN

1	 untuk spasi.
2	Id di html harus disertai # di css.
3	Penggunaan CSS membantu dalam style suatu coding

I. DAFTAR PUSTAKA

No.	Referensi (Format APA 7th Edition)
[1]	Burhandenny, A. E. (2026). Manual Praktikum Aplikasi Web dan Mobile Semester Genap 2025/2026. Program Studi Teknik Industri, Universitas Negeri Yogyakarta.
[2]	Bangun, R., Informasi, S., Talent, P., & Web, B. A. (2020). Jurnal Informatika Terpadu. 6(1), 29–37.
[3]	Mardiansyah, A., Kasah, B. N., Zamzami, H. R., Arabu, M. Y., Nasro, M. A., Kristanto, N., ... & Wulandari, Y. (2025). Pengenalan dasar HTML dan CSS: Langkah pertama dalam pengembangan web. Abdi Jurnal Publikasi, 3(3), 165-170.
[4]	Tanjung, A. H., Marlina, L., & Yazid, A. (2024). Pembuatan Aplikasi Pembelajaran Pengembangan Website HTML dan CSS dengan Animasi Interaktif Menggunakan Macromedia Flash 8. Jurnal Minfo Polgan, 13(2), 2557-2567.

J. LEMBAR PENILAIAN DAN PENGESAHAN

RUBRIK PENILAIAN LAPORAN					
Aspek Penilaian		Bobot			
No.	Aspek Penilaian	Bobot (%)	Nilai (0-100)	Skor Akhir	Catatan Penilai
1	Kelengkapan Isi (Semua bagian A–I terisi lengkap)	20%			
2	Kualitas Dasar Teori (Ditulis dengan bahasa sendiri, relevan, minimal 3 paragraf)	15%			
3	Dokumentasi Langkah Kerja (Kode & Screenshot sesuai, jelas, dan terbaca)	25%			
4	Tugas/Latihan Mandiri (Semua latihan & proyek mini dikerjakan dan berfungsi)	20%			
5	Pembahasan & Analisis (Kritis, mendalam, dan relevan dengan industri)	15%			
6	Kesimpulan & Tata Bahasa (Spesifik, rapi, mengikuti format yang ditentukan)	5%			
TOTAL NILAI AKHIR		100%			

Mahasiswa	Rekan Tim	Diperiksa oleh Asisten / Dosen
 _____ Saprina Melita Sari NIM:23051430012	 _____ Ardita Natalia NIM:23051430022	 _____ Dr. Eng. Ir. Aji Ery Burhandenny, S.T., M.AIT. Tanggal: _____

MODUL 3

A. IDENTITAS & INFORMASI MODUL

Mata Kuliah	Aplikasi Web dan Mobile
Kode Modul	MODUL 3 [LAYOUT CSS LANJUTAN (Flexbox & Grid) + Bootstrap]
Topik Utama	CSS Flexbox, CSS Grid, dan Framework CSS (Bootstrap 5) untuk prototyping cepat.
Sub-Topik	CSS Flexbox, CSS Grid, dan Framework CSS (Bootstrap 5) untuk prototyping cepat.
Durasi Praktikum	340 Menit
Tanggal Praktikum	27 Februari 2026
Nama Mahasiswa	Saprina Melita Sari
NIM	23051430012
Kelas	A

B. TUJUAN PEMBELAJARAN

1	Memahami konsep layout modern menggunakan Flexbox dan Grid.
2	Mengenal dan menggunakan Framework CSS (Bootstrap) untuk mempercepat pengembangan UI
3	Mampu membuat layout dashboard responsif (Sidebar + Main Content).
4	Menerapkan komponen Bootstrap (Card, Navbar, Button) pada konteks industri.

C. ALAT DAN BAHAN

No.	Nama Alat / Bahan / Aplikasi	Versi / Spesifikasi	Keterangan
-----	------------------------------	---------------------	------------

1	Visual Studio Code (VS Code)	Versi 1.8x ke atas	Code Editor
2	Google Chrome / Browser Modern	Versi terbaru	Testing & Debugging
3	Node.js & npm	Node 18+ / npm 9+	Runtime JS (Modul tertentu)
4	Laptop	Asus	Device utama
5	HTML	-	Struktur
6	CSS	-	Styling

D. DASAR TEORI

Layout modern pada pengembangan web melibatkan penggunaan teknik seperti **Flexbox dan CSS Grid** untuk mengatur elemen halaman agar responsif dan fleksibel terhadap berbagai ukuran layar. Flexbox memungkinkan elemen diatur dalam satu dimensi (misalnya baris atau kolom), sedangkan CSS Grid menyediakan kontrol dua dimensi atas baris dan kolom layout. Kedua teknik ini membantu membuat struktur antarmuka yang adaptif tanpa bergantung pada metode lama seperti *float* atau *positioning* manual, sehingga pengalaman pengguna menjadi lebih baik di berbagai perangkat (data statistik penelitian menunjukkan penggunaan Flexbox dan Grid tinggi dalam proyek responsif) (Santoso, 2025).

Menurut Tijer R (2023), Framework CSS seperti Bootstrap digunakan dalam pengembangan UI untuk mempercepat pembuatan antarmuka yang konsisten dan responsif tanpa harus menulis banyak CSS dari awal. Bootstrap menyediakan **sistem grid 12 kolom**, komponen siap pakai seperti tombol, form, dan navigasi, serta kelas utilitas untuk margin, padding, dan responsivitas, sehingga pengembang dapat fokus pada konten dan logika aplikasi daripada detail layout teknis.

Dalam konteks pembuatan dashboard responsif yang terdiri dari sidebar dan konten utama, Bootstrap mempermudah pengaturan struktur layout dengan grid dan breakpoint untuk berbagai ukuran layar, sementara komponen seperti navbar, card, dan button membantu menyusun interface yang mudah digunakan. Pendekatan ini relevan dengan praktik industri karena memungkinkan tim pengembang menghasilkan UI yang cepat, adaptif, dan konsisten, sesuai dengan kebutuhan aplikasi web modern yang harus bekerja baik di desktop maupun perangkat seluler (Habib et al., 2025).

D.1 Konsep Utama

Layout Modern (Flexbox & CSS Grid) adalah dasar pengaturan elemen halaman secara fleksibel dan terstruktur, baik satu dimensi (Flexbox) maupun dua dimensi (Grid), untuk memastikan tampilan tetap rapi dan adaptif di berbagai ukuran layar.

D.2 Keterkaitan dengan Teknik Industri

Keterkaitan penerapan layout modern dan Bootstrap dengan teknik industri terlihat pada pembuatan dashboard dan sistem monitoring yang efisien. Dengan menggunakan Flexbox, CSS Grid, dan komponen Bootstrap, informasi seperti stok bahan baku, jadwal produksi, dan performa mesin dapat ditampilkan secara terstruktur dan mudah diakses di berbagai perangkat. Hal ini mempermudah pengambilan keputusan cepat, mengurangi kesalahan manusia, dan meningkatkan efisiensi operasional serta kontrol produksi dalam konteks industri.

E. LANGKAH KERJA DAN HASIL PRAKTIKUM

E.1 Langkah 1 – Integrasi Bootstrap 5

=Tambahkan CDN Bootstrap di dalam <head>

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Sistem Monitoring Produksi</title>
  <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0
/dist/css/bootstrap.min.css"
rel="stylesheet">
  <link rel="stylesheet" href="Praktikum 2_style.css">
</head>
<body>
  <script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0
/dist/js/bootstrap.bundle.min.js">
  </script>
</body>
</html>
```

Screenshot Hasil:

```

1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4   <meta charset="UTF-8">
5   <meta name="viewport" content="width=device-width, initial-scale=1.0">
6   <title>Sistem Monitoring Produksi</title>
7   <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css"
8   rel="stylesheet">
9   <link rel="stylesheet" href="Praktikum 2_style.css">
10 </head>
11 <body>
12   <script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/js/bootstrap.bundle.min.js">
13   </script>
14 </body>
15 </html>

```

Gambar E.1 – Hasil Langkah 1

E.2 Langkah 2 – Membuat Navigasi Bar (Navbar)

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Sistem Monitoring Produksi</title>
  <link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css"
rel="stylesheet">
  <link rel="stylesheet" href="Praktikum 2_style.css">
</head>
<body>
  <nav class="navbar navbar-expand-lg navbar-dark bg-dark">
    <div class="container-fluid">
      <a class="navbar-brand" href="">TekInd Systems</a>
      <button class="navbar-toggler" type="button" data-bs-
toggle="collapse" data-bs-target="#navbarNav">
        <span class="navbar-toggler-icon"></span>
      </button>
      <div class="collapse navbar-collapse" id="navbarNav">
        <ul class="navbar-nav">
          <li class="nav-item">
            <a class="nav-link active" href="">Dashboard</a>
          </li>
          <li class="nav-item">
            <a class="nav-link" href="">Input Produksi</a>
          </li>
          <li class="nav-item">
            <a class="nav-link" href="">Inventori</a>

```

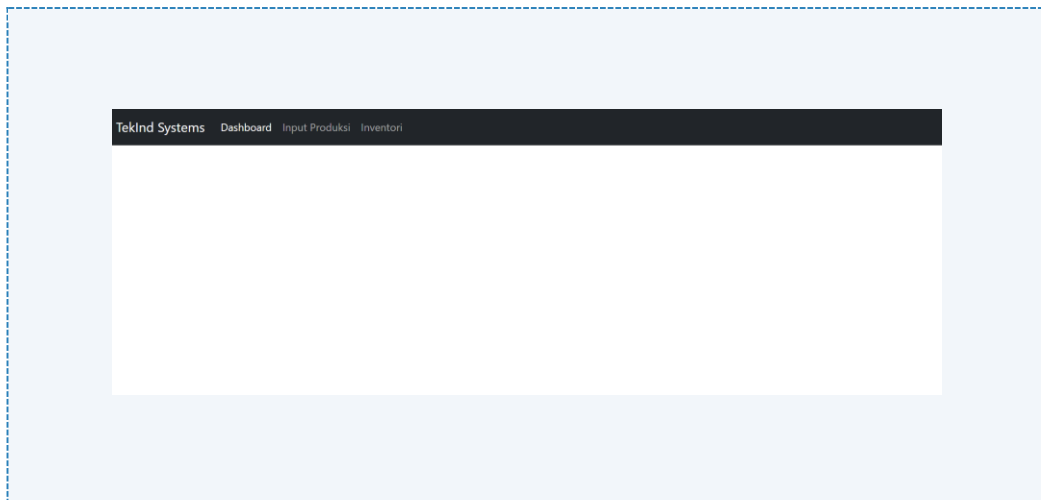


```

        </li>
      </ul>
    </div>
  </div>
</nav>
<script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0
/dist/js/bootstrap.bundle.min.js">
</script>
</body>
</html>

```

Screenshot Hasil:



Gambar E.2 – Hasil Langkah 2

E.3 Langkah 3 – Layout Grid Dashboard (Sistem Grid)

```

<div class="container mt-4">
  <!-- Row 1: KPI Cards (Menggunakan Grid System) -->
  <div class="row mb-4">
    <!-- Kolom lebar 4 (dari 12) -->
    <div class="col-md-4">
      <div class="card text-white bg-primary mb-3">
        <div class="card-header">Total Produksi</div>
        <div class="card-body">
          <h5 class="card-title">1,250 Unit</h5>
          <p class="card-text">Target Harian: 1,200 Unit</p>
        </div>
      </div>
    </div>
    <!-- Kolom lebar 4 -->

```

```

<div class="col-md-4">
  <div class="card text-white bg-success mb-3">
    <div class="card-header">Efisiensi Mesin</div>
    <div class="card-body">
      <h5 class="card-title">92%</h5>
      <p class="card-text">Status: Optimal</p>
    </div>
  </div>
</div>
<!-- Kolom lebar 4 -->
<div class="col-md-4">
  <div class="card text-white bg-warning mb-3">
    <div class="card-header">Reject Rate</div>
    <div class="card-body">
      <h5 class="card-title">1.5%</h5>
      <p class="card-text">Perlu perhatian shift malam</p>
    </div>
  </div>
</div>
</div>
<!-- Row 2: Tabel Data -->
<div class="row">
  <div class="col-12">
    <h3>Data Log Produksi Real-time</h3>
    <table class="table table-striped table-hover">
      <thead class="table-dark">
        <tr>
          <th>Waktu</th>
          <th>Mesin</th>
          <th>Produk</th>
          <th>Jumlah</th>
          <th>Status</th>
        </tr>
      </thead>
      <tbody>
        <tr>
          <td>08:00</td>
          <td>CNC-01</td>
          <td>Part A-12</td>
          <td>50</td>
          <td><span class="badge bg-success">OK</span></td>
        </tr>
        <tr>
          <td>08:15</td>
          <td>CNC-02</td>
          <td>Part B-05</td>
          <td></td>
          <td></td>
        </tr>
      </tbody>
    </table>
  </div>
</div>

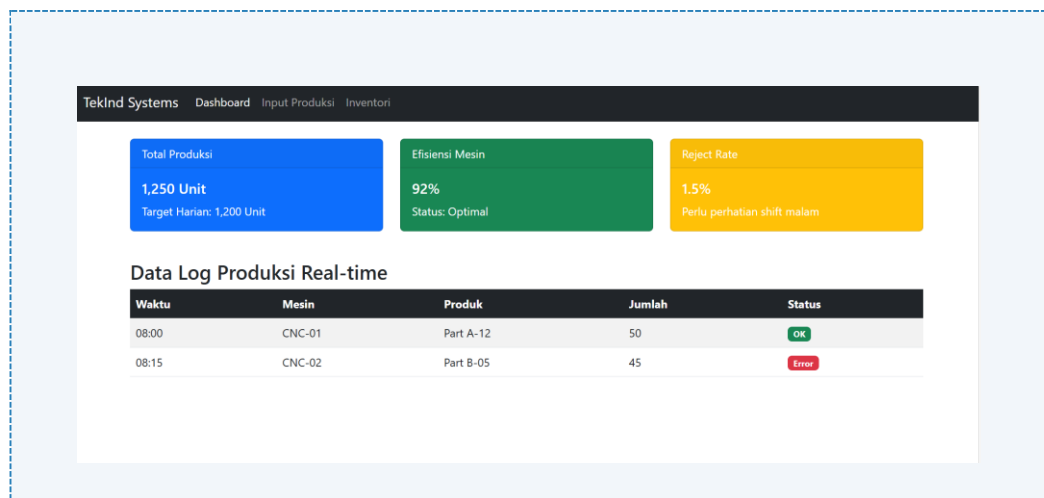
```

```

        <td>45</td>
        <td><span class="badge bg-danger">Error</span></td>
      </tr>
    </tbody>
  </table>
</div>
</div>
</div>

```

Screenshot Hasil:



Gambar E.3 – Hasil Langkah 3

E.4 Langkah 4 – Styling Form dengan Bootstrap Classes

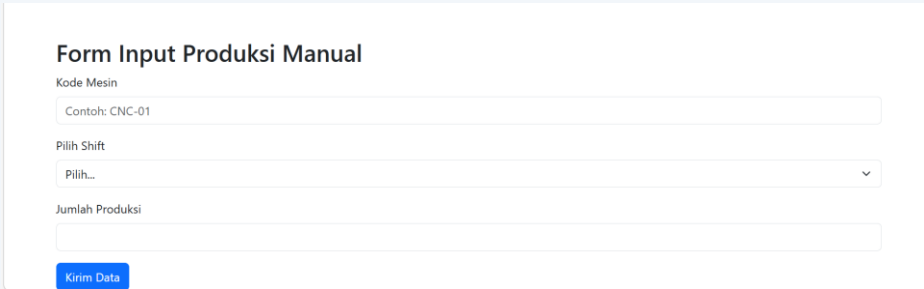
```

<div class="container mt-5">
  <h2>Form Input Produksi Manual</h2>
  <form>
    <div class="mb-3">
      <label for="kodeMesin" class="form-label">Kode Mesin</label>
      <input type="text" class="form-control" id="kodeMesin"
placeholder="Contoh: CNC-01">
    </div>
    <div class="mb-3">
      <label for="shiftSelect" class="form-label">Pilih Shift</label>
      <select class="form-select" id="shiftSelect">
        <option selected>Pilih...</option>
        <option value="1">Shift 1 (Pagi)</option>
        <option value="2">Shift 2 (Siang)</option>
        <option vaue="3">Shift 3(malam)</option>
      </select>
    </div>
  </form>
</div>

```

```
</div>
<div class="mb-3">
  <label for="jumlahProduksi" class="form-label">Jumlah
Produksi</label>
  <input type="number" class="form-control" id="jumlahProduksi">
</div>
<button type="submit" class="btn btn-primary">Kirim Data</button>
</form>
</div>
```

Screenshot Hasil:



Form Input Produksi Manual

Kode Mesin
Contoh: CNC-01

Pilih Shift
Pilih...

Jumlah Produksi

Kirim Data

Gambar E.4 – Hasil Langkah 4

E.5 Langkah 5 – Melihat Hasil

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Sistem Monitoring Produksi</title>
  <link href="https://cdn.jsdelivr.net/npm
/bootstrap@5.3.0/dist/css/bootstrap.min.css"
rel="stylesheet">
  <link rel="stylesheet" href="Praktikum 2_style.css">
</head>
<body>
  <nav class="navbar navbar-expand-lg navbar-dark bg-dark">
    <div class="container-fluid">
      <a class="navbar-brand" href="">TekInd Systems</a>
      <button class="navbar-toggler"
type="button" data-bs-toggle="collapse" data-bs-target="#navbarNav">
        <span class="navbar-toggler-icon"></span>
      </button>
      <div class="collapse navbar-collapse" id="navbarNav">
        <ul class="navbar-nav">
          <li class="nav-item">
            <a class="nav-link active" href="">Dashboard</a>
          </li>
          <li class="nav-item">
            <a class="nav-link" href="">Input Produksi</a>
          </li>
          <li class="nav-item">
            <a class="nav-link" href="">Inventori</a>
          </li>
        </ul>
      </div>
    </div>
  </nav>
  <div class="container mt-4">
    <!-- Row 1: KPI Cards (Menggunakan Grid System) -->
    <div class="row mb-4">
      <!-- Kolom lebar 4 (dari 12) -->
      <div class="col-md-4">
        <div class="card text-white bg-primary mb-3">
          <div class="card-header">Total Produksi</div>
          <div class="card-body">
            <h5 class="card-title">1,250 Unit</h5>
            <p class="card-text">Target Harian: 1,200 Unit</p>
          </div>
        </div>
      </div>
    </div>
  </div>
```

```

</div>
<!-- Kolom lebar 4 -->
<div class="col-md-4">
  <div class="card text-white bg-success mb-3">
    <div class="card-header">Efisiensi Mesin</div>
    <div class="card-body">
      <h5 class="card-title">92%</h5>
      <p class="card-text">Status: Optimal</p>
    </div>
  </div>
</div>
<!-- Kolom lebar 4 -->
<div class="col-md-4">
  <div class="card text-white bg-warning mb-3">
    <div class="card-header">Reject Rate</div>
    <div class="card-body">
      <h5 class="card-title">1.5%</h5>
      <p class="card-text">Perlu perhatian shift malam</p>
    </div>
  </div>
</div>
</div>
<!-- Row 2: Tabel Data -->
<div class="row">
  <div class="col-12">
    <h3>Data Log Produksi Real-time</h3>
    <table class="table table-striped table-hover">
      <thead class="table-dark">
        <tr>
          <th>Waktu</th>
          <th>Mesin</th>
          <th>Produk</th>
          <th>Jumlah</th>
          <th>Status</th>
        </tr>
      </thead>
      <tbody>
        <tr>
          <td>08:00</td>
          <td>CNC-01</td>
          <td>Part A-12</td>
          <td>50</td>
          <td><span class="badge bg-success">OK</span></td>
        </tr>
        <tr>
          <td>08:15</td>

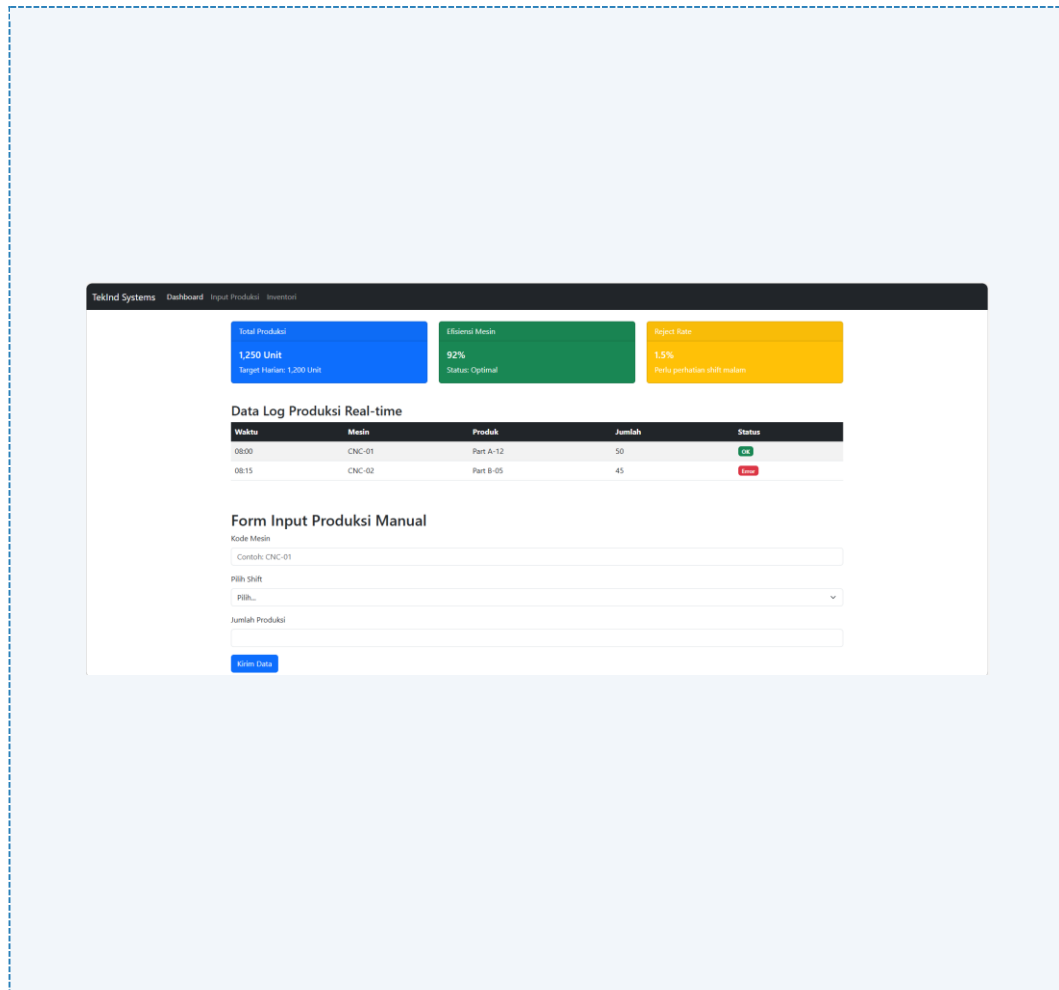
```

```

        <td>CNC-02</td>
        <td>Part B-05</td>
        <td>45</td>
        <td><span class="badge bg-danger">Error</span></td>
    </tr>
</tbody>
</table>
</div>
</div>
</div>
<div class="container mt-5">
    <h2>Form Input Produksi Manual</h2>
    <form>
        <div class="mb-3">
            <label for="kodeMesin" class="form-label">Kode Mesin</label>
            <input type="text" class="form-control" id="kodeMesin"
placeholder="Contoh: CNC-01">
        </div>
        <div class="mb-3">
            <label for="shiftSelect" class="form-label">Pilih Shift</label>
            <select class="form-select" id="shiftSelect">
                <option selected>Pilih...</option>
                <option value="1">Shift 1 (Pagi)</option>
                <option value="2">Shift 2 (Siang)</option>
                <option vaue="3">Shift 3(malam)</option>
            </select>
        </div>
        <div class="mb-3">
            <label for="jumlahProduksi" class="form-label">Jumlah
Produksi</label>
            <input type="number" class="form-control" id="jumlahProduksi">
        </div>
        <button type="submit" class="btn btn-primary">Kirim Data</button>
    </form>
</div>
<script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/js/
bootstrap.bundle.min.js">
</script>
</body>
</html>

```

Screenshot Hasil:



Gambar E.5 – Hasil Langkah 5

F. LATIHAN DAN TUGAS MANDIRI

F.1 Latihan 1 – Grid Responsif

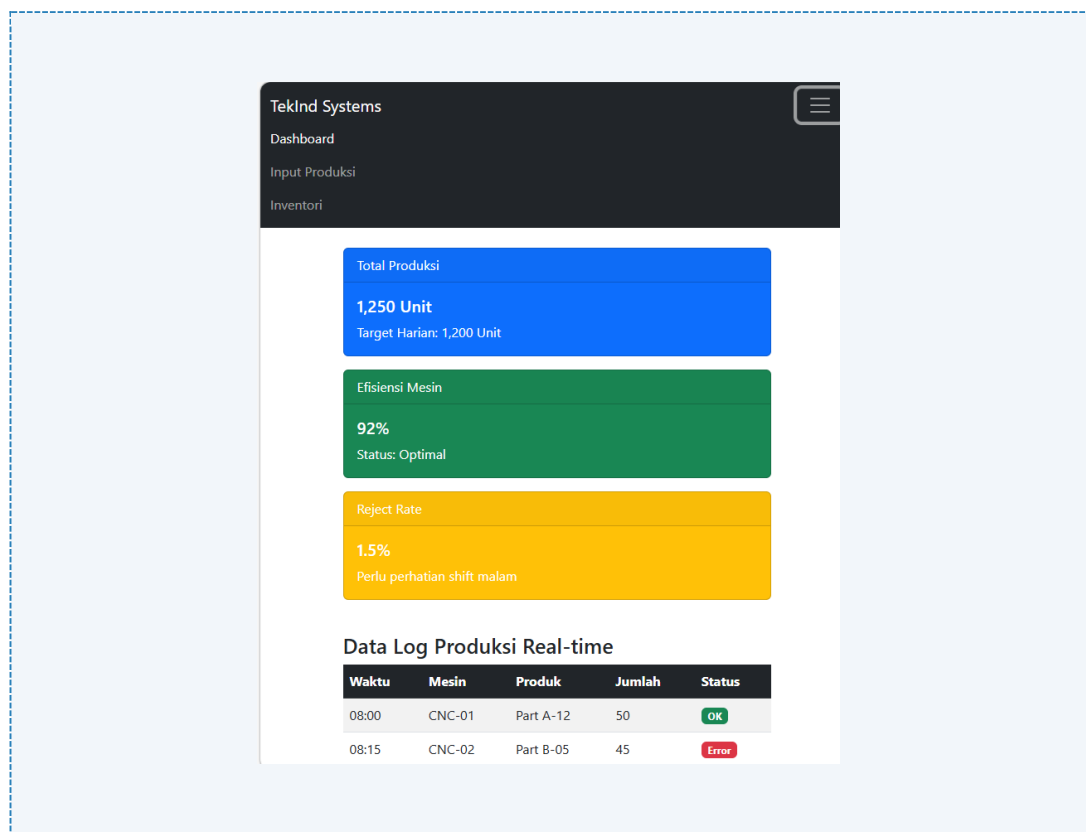
Coba ubah ukuran layar browser Anda. Perhatikan bagaimana kartu KPI yang tadinya sejajar (3 kolom) akan menumpuk ke bawah (1 kolom) di layar HP. Ini adalah kekuatan Grid System (`col-md-4` artinya 4 kolom di layar medium ke atas, otomatis 12 kolom/1 baris di layar kecil).

Jawaban / Kode Solusi:


```

</div>
<!-- Kolom lebar 4 -->
<div class="col-md-4">
  <div class="card text-white bg-success mb-3">
    <div class="card-header">Efisiensi Mesin</div>
    <div class="card-body">
      <h5 class="card-title">92%</h5>
      <p class="card-text">Status: Optimal</p>
    </div>
  </div>
</div>
<!-- Kolom lebar 4 -->
<div class="col-md-4">
  <div class="card text-white bg-warning mb-3">
    <div class="card-header">Reject Rate</div>
    <div class="card-body">
      <h5 class="card-title">1.5%</h5>
      <p class="card-text">Perlu perhatian shift malam</p>
    </div>
  </div>
</div>
</div>

```



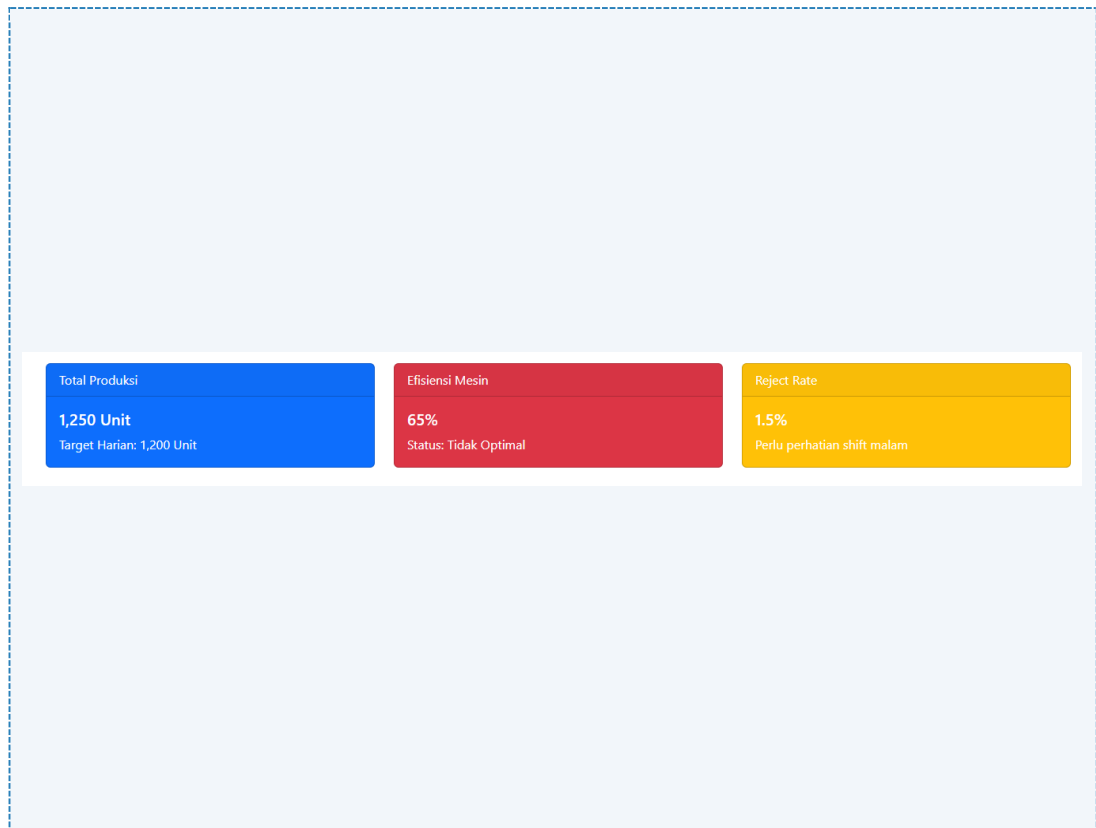
Gambar F.1 – Hasil Latihan 1

F.2 Latihan 2 – Modifikasi Card

Ubah kartu "Efisiensi Mesin" menjadi warna merah (`bg-danger`) jika angkanya diubah menjadi di bawah 80%.

Jawaban / Kode Solusi:

```
<!-- Kolom lebar 4 -->
<div class="col-md-4">
  <div class="card text-white bg-danger mb-3">
    <div class="card-header">Efisiensi Mesin</div>
    <div class="card-body">
      <h5 class="card-title">65%</h5>
      <p class="card-text">Status: Tidak Optimal</p>
    </div>
  </div>
</div>
```



Gambar F.2 – Hasil Latihan 2

F.3 Tugas Proyek Mini – Industri

Deskripsi proyek mini:

- Buatlah struktur dashboard baru bernama `dashboard\_inventori.html`.
- Gunakan Bootstrap Grid.
 - Bagian atas: Judul "Gudang Bahan Baku".
 - Bagian kiri (4 kolom): Form Stok Masuk (Input nama barang, qty, supplier).
 - Bagian kanan (8 kolom): Tabel Stok Saat Ini (Gunakan tabel Bootstrap).

Penjelasan pendekatan/solusi yang digunakan:

Pendekatan yang digunakan adalah memadukan form input dan tabel monitoring dalam satu tampilan web yang responsif menggunakan Bootstrap 5, sehingga pengguna dapat melihat data stok sekaligus menambahkan stok baru dengan mudah. Form stock masuk menyediakan field standar seperti nama barang, quantity, dan supplier, dengan dropdown untuk supplier agar data lebih terstandarisasi dan meminimalkan kesalahan input. Tabel stock saat ini menampilkan informasi lengkap mulai dari stok awal, barang masuk, barang keluar, sisa stok, hingga kondisi, yang dilengkapi dengan badge warna untuk menunjukkan status stok secara visual seperti hijau untuk tersedia, kuning untuk menipis, dan merah untuk kritis.

Kode Utama:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Gudang Bahan Baku</title>
  <link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css"
rel="stylesheet">
</head>
<body>
  <nav class="navbar navbar-expand-lg bg-primary navbar-dark">
    <div class="container">
      <h1 class="mx-auto mb-0 fw-bold text-center">Gudang Bahan
Baku</h1>
    </div>
  </nav>
  <div class="container mt-4">
    <div class="row g-4">
      <!-- FORM (4 KOLOM) -->
      <div class="col-md-4">
        <div class="card shadow-sm">
```

```

<div class="card-header bg-success text-white text-
center">Form Stock Masuk</div>

<div class="card-body">
  <form>
    <div class="mb-3">
      <label class="form-label">Nama Barang</label>
      <input type="text" class="form-control"
placeholder="Contoh: Tepung Terigu">
    </div>

    <div class="mb-3">
      <label class="form-label">Quantity</label>
      <input type="number" class="form-control"
placeholder="Masukkan jumlah">
    </div>

    <div class="mb-3">
      <label class="form-label">Supplier</label>
      <select class="form-select">
        <option selected disabled>Pilih Supplier</option>
        <option>PT Pangan Sejahtera</option>
        <option>CV Sumber Rejeki</option>
        <option>PT Indo Distribusi</option>
      </select>
    </div>

    <button type="submit" class="btn btn-primary w-100">
      Simpan Data
    </button>
  </form>
</div>
</div>
</div>

<!-- TABEL (8 KOLOM) -->
<div class="col-md-8">
  <div class="card shadow-sm">
    <div class="card-header bg-secondary text-white text-center">
      Stock Saat Ini
    </div>
    <div class="card-body">
      <table class="table table-hover table-striped text-center">
        <thead class="table-dark">
          <tr>
            <th>No</th>

```

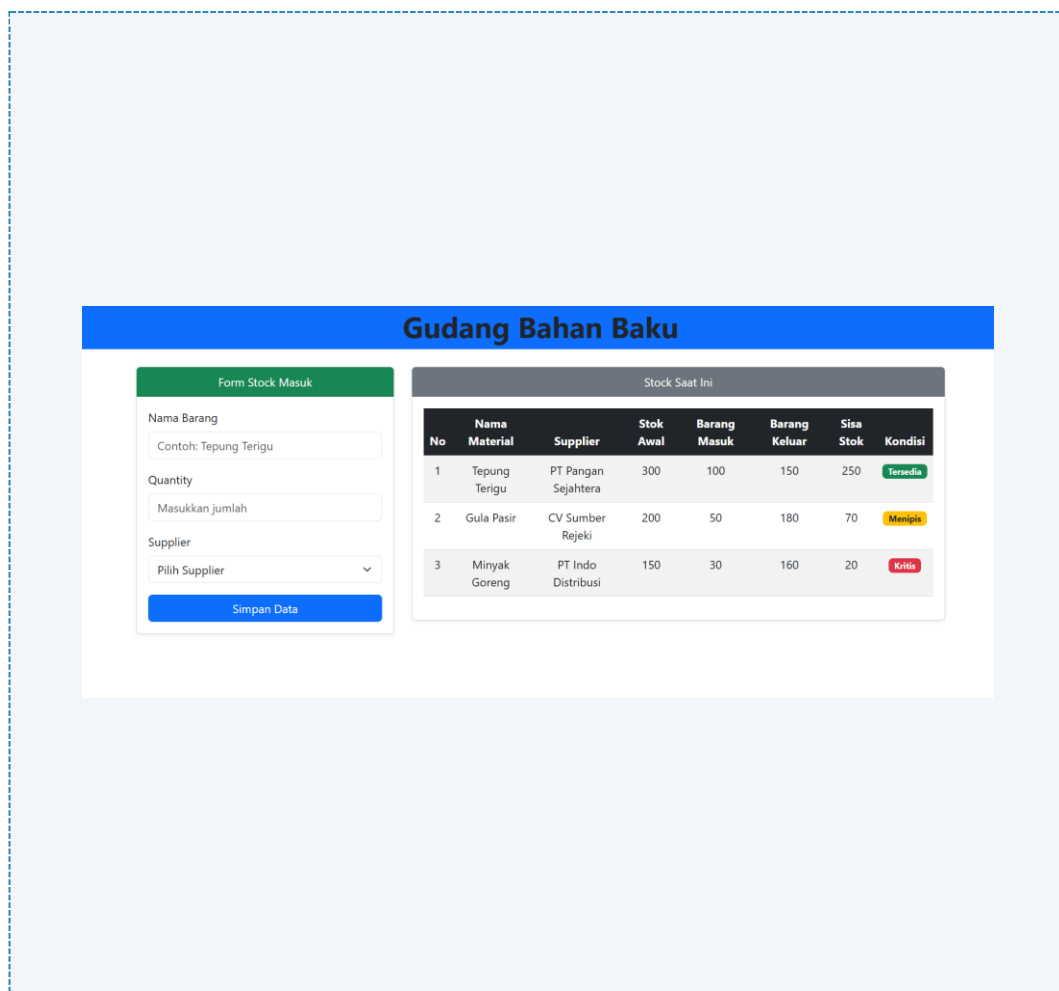
```

        <th>Nama Material</th>
        <th>Supplier</th>
        <th>Stok Awal</th>
        <th>Barang Masuk</th>
        <th>Barang Keluar</th>
        <th>Sisa Stok</th>
        <th>Kondisi</th>
    </tr>
</thead>
<tbody>
<tr>
    <td>1</td>
    <td>Tepung Terigu</td>
    <td>PT Pangan Sejahtera</td>
    <td>300</td>
    <td>100</td>
    <td>150</td>
    <td>250</td>
    <td><span class="badge bg-
success">Tersedia</span></td>
</tr>
<tr>
    <td>2</td>
    <td>Gula Pasir</td>
    <td>CV Sumber Rejeki</td>
    <td>200</td>
    <td>50</td>
    <td>180</td>
    <td>70</td>
    <td><span class="badge bg-warning text-
dark">Menipis</span></td>
</tr>
<tr>
    <td>3</td>
    <td>Minyak Goreng</td>
    <td>PT Indo Distribusi</td>
    <td>150</td>
    <td>30</td>
    <td>160</td>
    <td>20</td>
    <td><span class="badge bg-
danger">Kritis</span></td>
</tr>
</tbody>
</table>
</div>

```

```
</div>
</div>

</div>
</div>
<script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/js/
bootstrap.bundle.min.js"></script>
</body>
</html>
```



Gambar F.4 – Hasil Proyek Mini

G. PEMBAHASAN

G.1 Analisis Kesesuaian Hasil dengan Teori

Dalam sistem “Gudang Bahan Baku”, berbagai elemen coding memiliki fungsi masing-masing. Tag `<link>` digunakan untuk memanggil Bootstrap CSS sehingga tampilan website menjadi rapi dan responsif. Tag `<nav>` membuat **navbar** untuk judul atau menu navigasi, sedangkan `<div class="container">` dan `<div class="row">` digunakan untuk mengatur layout grid sehingga form dan tabel tampil berdampingan sesuai desain. Elemen `<form>` bersama `<input>` dan `<select>` berfungsi sebagai tempat pengguna memasukkan data stok, dengan atribut seperti `type="text"` atau `type="number"` untuk menyesuaikan jenis input. Tombol `<button>` digunakan untuk mengirim data, sementara `<table>` bersama `<thead>` dan `<tbody>` menampilkan data stok dalam bentuk tabel agar mudah dibaca. Tag `<span>` dengan class badge memberikan indikator visual kondisi stok melalui warna. Terakhir, tag `<script>` dipakai untuk memanggil JavaScript atau library Bootstrap yang membuat interaksi seperti dropdown dan perhitungan stok menjadi dinamis.

G.2 Kendala yang Ditemukan dan Solusinya

Kendala / Error yang Ditemukan	Solusi yang Diterapkan

G.3 Kaitan dengan Penerapan di Industri

Digunakan untuk manajemen inventory dan persediaan stok, misal pada suatu perusahaan akan mengirimkan produknya maka dapat melihat tingkat persediaan yang ada.

H. KESIMPULAN

1	Bootstrap 5 untuk mempermudah pembuatan tampilan web yang rapi, konsisten, dan responsif tanpa harus menulis CSS dari nol.
2	Col-md-4 digunakan untuk mengatur lebar kolom dalam grid pada layout halaman.
3	Badge digunakan untuk menampilkan status atau indikator penting secara visual.

I. DAFTAR PUSTAKA

No.	Referensi (Format APA 7th Edition)
1.	Habib, H. N., Efendi, A., & Prasetyo, D. E. (2025). Pemanfaatan Bootstrap Sebagai Framework Desain Responsif Untuk Meningkatkan Keterampilan Pemrograman Web.
2.	Santoso, M. F. (2025). Analisis efektivitas Bootstrap dan Tailwind CSS dalam pengembangan UI web responsif. <i>Jurnal Teknologi dan Sistem Informasi Bisnis</i> .
3.	Tijer, R. (2023). Implementasi Flexbox dan CSS Grid pada layout web responsif. <i>Journal of New Research in Information Design</i> , 8(1), 12-25.

J. LEMBAR PENILAIAN DAN PENGESAHAN

RUBRIK PENILAIAN LAPORAN					
Aspek Penilaian		Bobot			
No.	Aspek Penilaian	Bobot (%)	Nilai (0-100)	Skor Akhir	Catatan Penilai
1	Kelengkapan Isi (Semua bagian A–I terisi lengkap)	20%			
2	Kualitas Dasar Teori (Ditulis dengan bahasa sendiri, relevan, minimal 3 paragraf)	15%			
3	Dokumentasi Langkah Kerja (Kode & Screenshot sesuai, jelas, dan terbaca)	25%			
4	Tugas/Latihan Mandiri (Semua latihan & proyek mini dikerjakan dan berfungsi)	20%			
5	Pembahasan & Analisis (Kritis, mendalam, dan relevan dengan industri)	15%			
6	Kesimpulan & Tata Bahasa (Spesifik, rapi, mengikuti format yang ditentukan)	5%			
TOTAL NILAI AKHIR		100%			

Mahasiswa	Rekan Tim	Diperiksa oleh Asisten / Dosen
		
Saprina Melita Sari NIM:23051430012	Ardita Natalia NIM:23051430022	Dr. Eng. Ir. Aji Ery Burhandenny, S.T., M.AIT. Tanggal:

MODUL 4

A. IDENTITAS & INFORMASI MODUL

Mata Kuliah	Aplikasi Web dan Mobile
Kode Modul	MODUL 4 (Pertemuan 4)
Topik Utama	JavaScript Dasar (Variabel, Tipe Data, Operator, Control Flow)
Sub-Topik	Logika Pemrograman, Variabel, Tipe Data, Operator Aritmatika/logika, dan percabangan.
Durasi Praktikum	340 Menit
Tanggal Praktikum	06 Maret 2026
Nama Mahasiswa	Saprina Melita Sari
NIM	23051430012
Kelas	A

B. TUJUAN PEMBELAJARAN

1	Memahami peran JavaScript sebagai "otak" dari halaman web.
2	Mampu mendeklarasikan variabel dan memahami aturan penamaannya.
3	Menguasai operator aritmatika untuk perhitungan sederhana.
4	Mampu menggunakan Control Flow (`if/else`, `switch`) untuk pengambilan keputusan otomatis.

C. ALAT DAN BAHAN

No.	Nama Alat / Bahan / Aplikasi	Versi / Spesifikasi	Keterangan
1	Visual Studio Code (VS Code)	Versi 1.8x ke atas	Code Editor
2	Google Chrome / Browser Modern	Versi terbaru	Testing & Debugging
3	Node.js & npm	Node 18+ / npm 9+	Runtime JS (Modul tertentu)

4	Laptop	Asus RAM 4 GB	Device utama
5	HTML	-	Struktur
6	JavaScript	-	
7	Console		Melihat error di Javascript

D. DASAR TEORI

JavaScript merupakan salah satu bahasa pemrograman yang digunakan dalam pengembangan website untuk menambahkan interaktivitas dan logika pada halaman web. Dalam pengembangan web modern, JavaScript bekerja bersama dengan HTML dan CSS. HTML berfungsi sebagai struktur dasar halaman web, CSS digunakan untuk mengatur tampilan atau desain halaman, sedangkan JavaScript berperan dalam mengatur perilaku atau interaksi yang terjadi pada halaman tersebut. Oleh karena itu, JavaScript sering disebut sebagai “otak” dari halaman web karena mampu memproses input pengguna, menjalankan logika program, serta memberikan respon secara dinamis tanpa harus memuat ulang halaman secara keseluruhan.

Dalam pemrograman JavaScript, variabel digunakan sebagai tempat untuk menyimpan data yang akan diproses oleh program. Variabel dapat berisi berbagai tipe data seperti teks (string), angka (number), maupun nilai logika (boolean). Dalam JavaScript, variabel dapat dideklarasikan menggunakan kata kunci seperti `let`, `const`, dan `var`. Penamaan variabel juga harus mengikuti aturan tertentu, misalnya tidak boleh diawali dengan angka, tidak boleh mengandung spasi, serta sebaiknya menggunakan nama yang jelas agar mudah dipahami. Pemahaman mengenai variabel sangat penting karena hampir semua proses dalam pemrograman melibatkan penyimpanan dan pengolahan data menggunakan variabel.

Selain variabel, JavaScript juga menyediakan operator aritmatika yang digunakan untuk melakukan perhitungan matematis dalam program. Operator aritmatika yang umum digunakan antara lain penjumlahan (+), pengurangan (-), perkalian (*), pembagian (/), serta modulus (%). Operator tersebut memungkinkan program untuk melakukan berbagai perhitungan sederhana seperti menghitung total produksi, nilai rata-rata, atau perhitungan gaji karyawan. Penggunaan operator aritmatika membantu sistem dalam mengolah data numerik secara otomatis sehingga menghasilkan informasi yang lebih cepat dan akurat.

Dalam proses pengambilan keputusan pada program, JavaScript menggunakan konsep control flow atau alur kendali. Control flow berfungsi untuk menentukan perintah mana yang akan dijalankan berdasarkan kondisi tertentu. Salah satu struktur yang sering digunakan adalah `if/else`. Struktur ini memungkinkan program untuk mengevaluasi suatu kondisi, kemudian menjalankan perintah yang berbeda sesuai dengan hasil evaluasi tersebut.

Dengan adanya control flow, program dapat memberikan respon yang berbeda terhadap berbagai situasi yang mungkin terjadi selama program dijalankan (Hidayatullah & Paniran, 2023).

Selain if/else, JavaScript juga menyediakan struktur switch yang digunakan untuk menangani beberapa pilihan kondisi berdasarkan nilai tertentu. Struktur switch biasanya digunakan ketika program harus memilih satu dari beberapa kemungkinan yang tersedia, seperti mengkonversi kode shift kerja menjadi nama shift tertentu. Dengan menggunakan switch, kode program dapat menjadi lebih terstruktur dan mudah dipahami dibandingkan dengan penggunaan banyak kondisi if/else. Oleh karena itu, penguasaan konsep control flow seperti if/else dan switch sangat penting dalam membangun logika program yang mampu melakukan pengambilan keputusan secara otomatis.

D.1 Konsep Utama

- 1) JavaScript berfungsi sebagai bahasa pemrograman yang **memberikan** logika dan interaktivitas pada halaman web. Jika HTML membangun struktur dan CSS mengatur tampilan, maka JavaScript mengatur perilaku dan proses yang terjadi di halaman web.
- 2) variabel digunakan untuk menyimpan data yang akan diproses dalam program. Dalam JavaScript variabel dapat dideklarasikan menggunakan var, let, atau const, serta harus mengikuti aturan penamaan tertentu.
- 3) Operator aritmatika digunakan untuk melakukan perhitungan matematis **sederhana** seperti penjumlahan (+), pengurangan (-), perkalian (*), pembagian (/), dan modulus (%).
- 4) Control flow digunakan untuk mengatur alur program berdasarkan kondisi **tertentu**. Dalam materi ini digunakan:
 - if / else untuk keputusan berdasarkan kondisi
 - switch untuk memilih aksi berdasarkan beberapa kemungkinan nilai.

D.2 Keterkaitan dengan Teknik Industri

Dalam Teknik Industri, banyak proses produksi atau monitoring yang perlu otomatis dan real-time. JavaScript bisa digunakan untuk membuat dashboard produksi yang menampilkan data operasional mesin, jumlah produksi, efisiensi, atau reject rate secara langsung di browser. Misalnya, data dari mesin dikirim ke web, dan JavaScript memproses serta menampilkan statusnya otomatis.

E. LANGKAH KERJA DAN HASIL PRAKTIKUM

E.1 Langkah 1 – Hello World & Variabel di Console

Buat file baru 'script.js' dan hubungkan ke 'indeks.html' (taruh di atas '</body>'):

```
<script src="script.js"></script>
```

```
// Komentar single line

// 1. Variabel & Tipe Data
let namaMesin = "CNC-Mazak-01"; // String
let targetHarian = 500; // Number
let isOperational = true; // Boolean

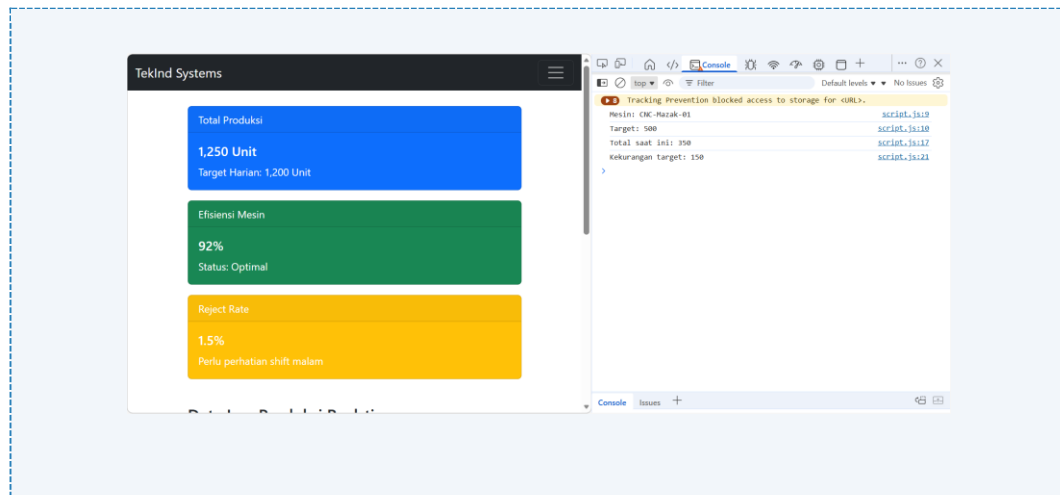
// Menampilkan ke console browser (Tekan F12 -> Console)
console.log("Mesin: " + namaMesin);
console.log("Target: " + targetHarian);

// 2. Operator Aritmatika
let produksiPagi = 200;
let produksiSiang = 150;
let totalProduksi = produksiPagi + produksiSiang;

console.log("Total saat ini: " + totalProduksi);

// Hitung sisa kekurangan target
let kekurangan = targetHarian - totalProduksi;
console.log("Kekurangan target: " + kekurangan);
```

Screenshot Hasil:



Gambar E.1 – Hasil Langkah 1

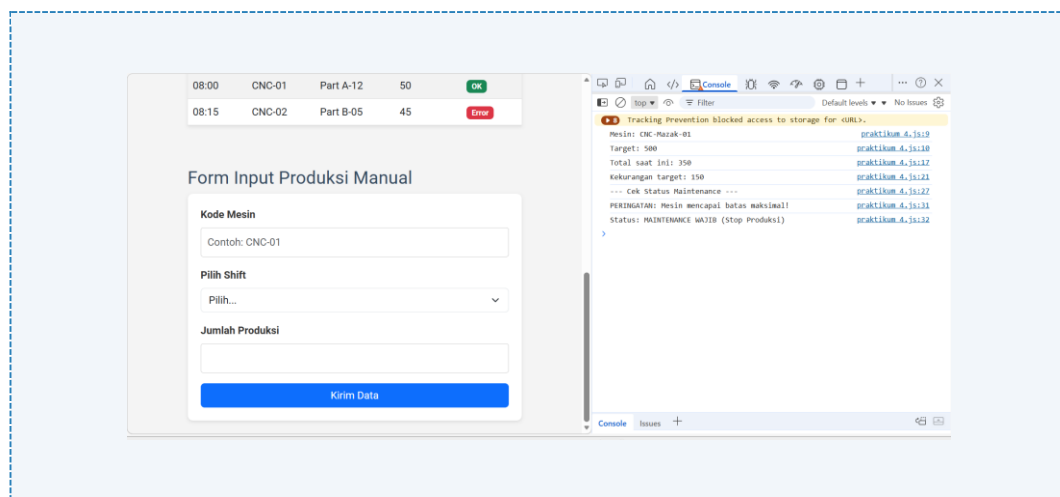
E.2 Langkah 2 – Control Flow (Pengambilan Keputusan)

```
// Simulasi data pembacaan sensor (jam operasional)
let jamOperasional = 1250; // dalam jam
let batasMaksimal = 1200; // batas sebelum maintenance wajib

console.log("--- Cek Status Maintenance ---");

// Logika If/Else
if (jamOperasional >= batasMaksimal) {
  console.log("PERINGATAN: Mesin mencapai batas maksimal!");
  console.log("Status: MAINTENANCE WAJIB (Stop Produksi)");
} else if (jamOperasional > 1000) {
  console.log("Status: SIAP HATI-HATI (Segera jadwalkan maintenance)");
} else {
  console.log("Status: BERJALAN NORMAL");
}
}
```

Screenshot Hasil:



Gambar E.2 – Hasil Langkah 2

E.3 Langkah 3 – Studi Kasus Industri (Perhitungan OEE Sederhana)

```
// Data Input
let jamKerjaPlanned = 8; // Jam
let jamKerjaAktual = 6.5; // Jam (Ada 1.5 jam breakdown)

// Perhitungan
let availability = (jamKerjaAktual / jamKerjaPlanned) * 100;

// Pembulatan 2 angka di belakang koma
availability = availability.toFixed(2);
```

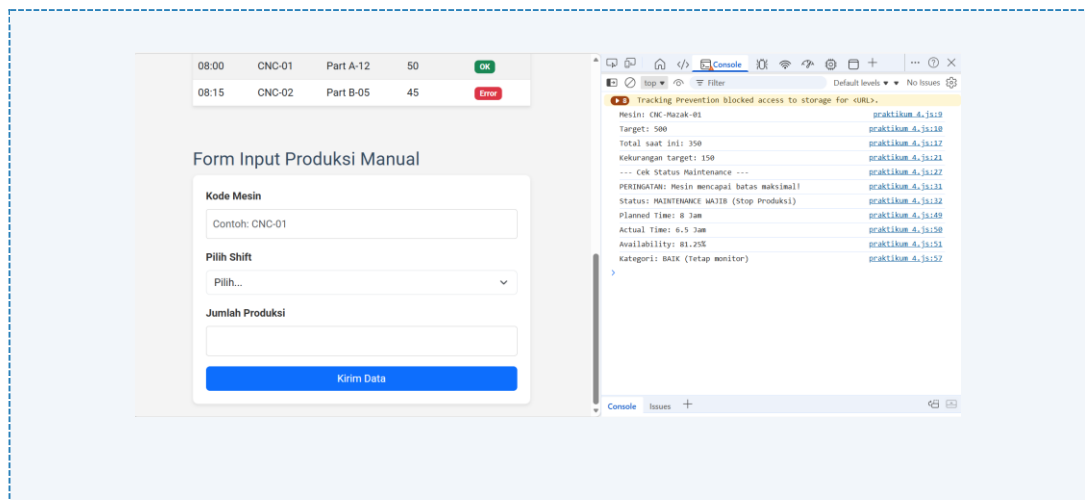
```

console.log("Planned Time: " + jamKerjaPlanned + " Jam");
console.log("Actual Time: " + jamKerjaAktual + " Jam");
console.log("Availability: " + availability + "%");

// Logika Penilaian Kualitas Availability
if (availability >= 90) {
  console.log("Kategori: WORLD CLASS");
} else if (availability >= 80) {
  console.log("Kategori: BAIK (Tetap monitor)");
} else {
  console.log("Kategori: BURUK (Perlu investigasi penyebab breakdown)");
}

```

Screenshot Hasil:



Gambar E.3 – Hasil Langkah 3

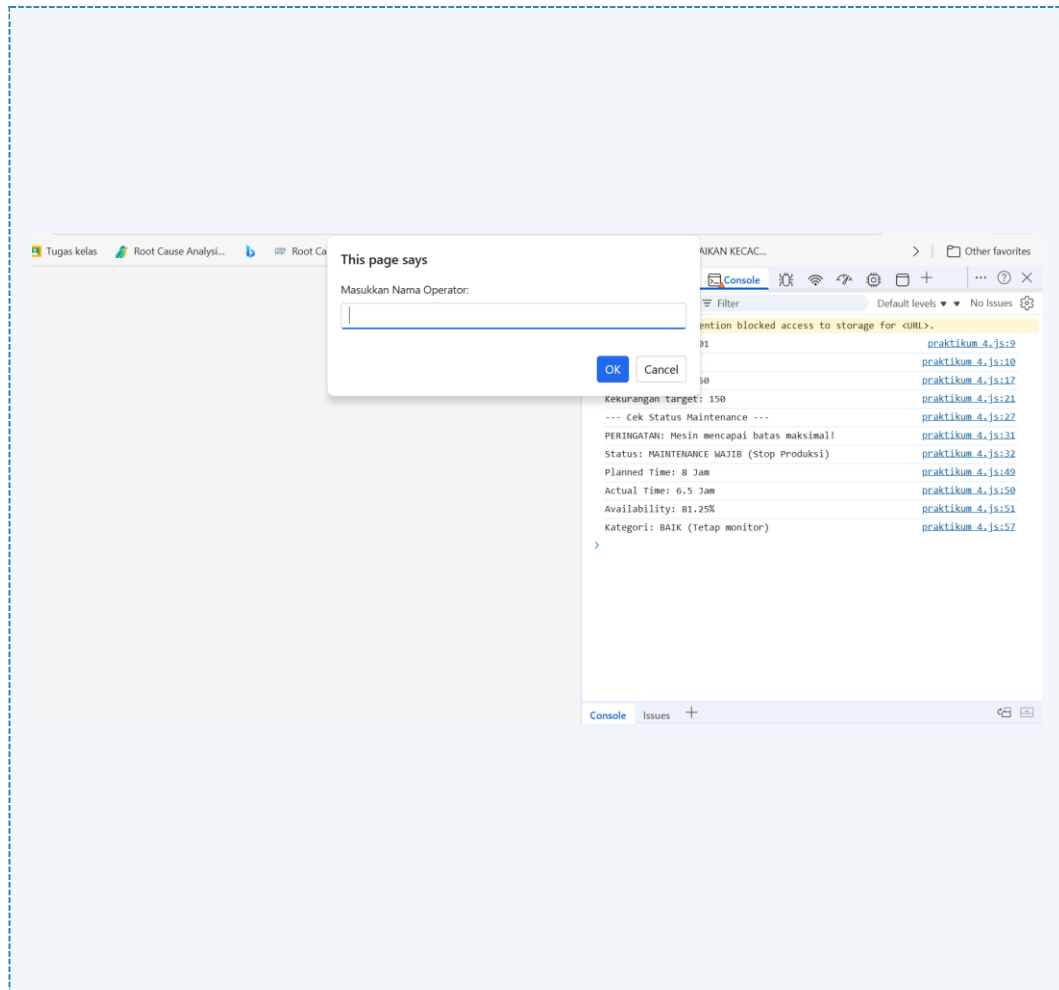
E.4 Langkah 4 – Interaksi DOM Sederhana (Alert & Prompt)

```

let namaOperator = prompt("Masukkan Nama Operator:");
let shiftKerja = prompt("Masukkan Shift (Pagi/Siang/Malam):");
if (shiftKerja === "Malam") {
  alert("Halo " + namaOperator + ", Shift malam memiliki tambahan uang makan sebesar Rp 20.000.");
} else {
  alert("Halo " + namaOperator + ", Selamat bekerja. Tetap semangat!");
}

```

Screenshot Hasil:



Gambar E.4 – Hasil Langkah 4

F. LATIHAN DAN TUGAS MANDIRI

F.1 Latihan 1 – Matematika

Buat variabel `gajiPokok` dan `jamLembur`. Hitung `totalGaji` dengan asumsi upah lembur per jam adalah 1.5x dari gaji pokok dibagi 173 (jam kerja bulanan). Tampilkan hasilnya di console.

Jawaban / Kode Solusi:

HTML

```
<!-- Row 2: Menghitung Total Gaji (Menggunakan Grid System) -->
```

```
<div class="row mb-4">
  <!-- Kolom lebar 4 (dari 12) -->
  <div class="col-md-4">
    <div class="card text-white bg-primary mb-3">
      <div class="card-header">Gaji Pokok</div>
      <div class="card-body">
        <h5 class="card-title">3.000.000</h5>
      </div>
    </div>
  </div>
  <!-- Kolom lebar 4 -->
  <div class="col-md-4">
    <div class="card text-white bg-success mb-3">
      <div class="card-header">Jam Lembur</div>
      <div class="card-body">
        <h5 class="card-title">3 Jam</h5>
      </div>
    </div>
  </div>
  <!-- Kolom lebar 4 -->
  <div class="col-md-4">
    <div class="card text-white bg-warning mb-3">
      <div class="card-header">Total Gaji</div>
      <div class="card-body">
        <h5 class="card-title">3078034.68</h5>
      </div>
    </div>
  </div>
</div>
```

JavaScript

```
// Variabel
```

```
let gajiPokok = 3000000; // contoh gaji pokok per bulan
```

```
let jamLembur = 3; // jumlah jam lembur
```

```
// Upah lembur per jam = 1.5 x (gaji pokok / 173)
```

```
let upahLemburPerJam = 1.5 * (gajiPokok / 173);
```

```
// Total gaji
```

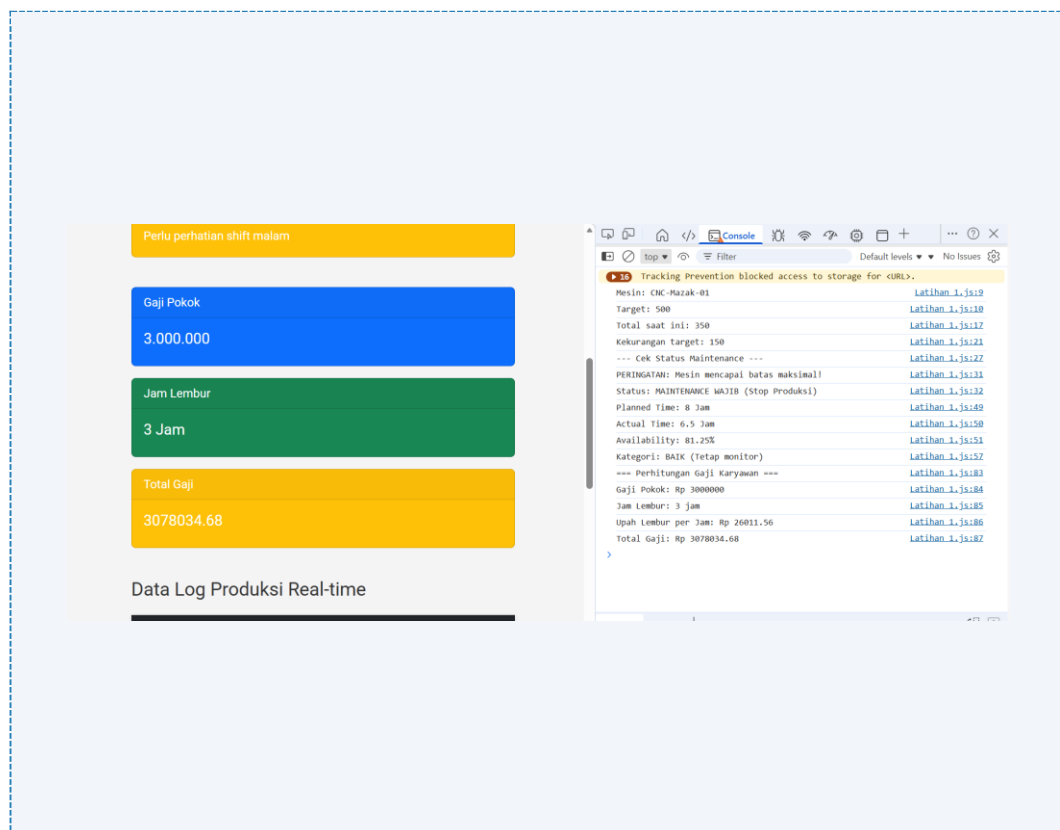
```
let totalGaji = gajiPokok + (jamLembur * upahLemburPerJam);
```

```
// Tampilkan di console
```

```
console.log("=== Perhitungan Gaji Karyawan ===");
```

```
console.log("Gaji Pokok: Rp " + gajiPokok);
```

```
console.log("Jam Lembur: " + jamLembur + " jam");  
console.log("Upah Lembur per Jam: Rp " + upahLemburPerJam.toFixed(2));  
console.log("Total Gaji: Rp " + totalGaji.toFixed(2));
```



Gambar F.1 – Hasil Latihan 1

F.2 Latihan 2 – Logika Switch

Gunakan `switch` untuk mengkonversi kode shift angka (1, 2, 3) menjadi nama shift (Pagi, Siang, Malam). Jika input selain 1-3, tampilkan "Shift Tidak Valid".

Jawaban / Kode Solusi:

```
// Latihan 2 (Logika Switch)

// Input kode shift
let namaOperator = prompt("Masukkan Nama Operator:");
let kodeShift = prompt("Masukkan Kode Shift (1=Pagi, 2=Siang, 3=Malam):");
let namaShift;

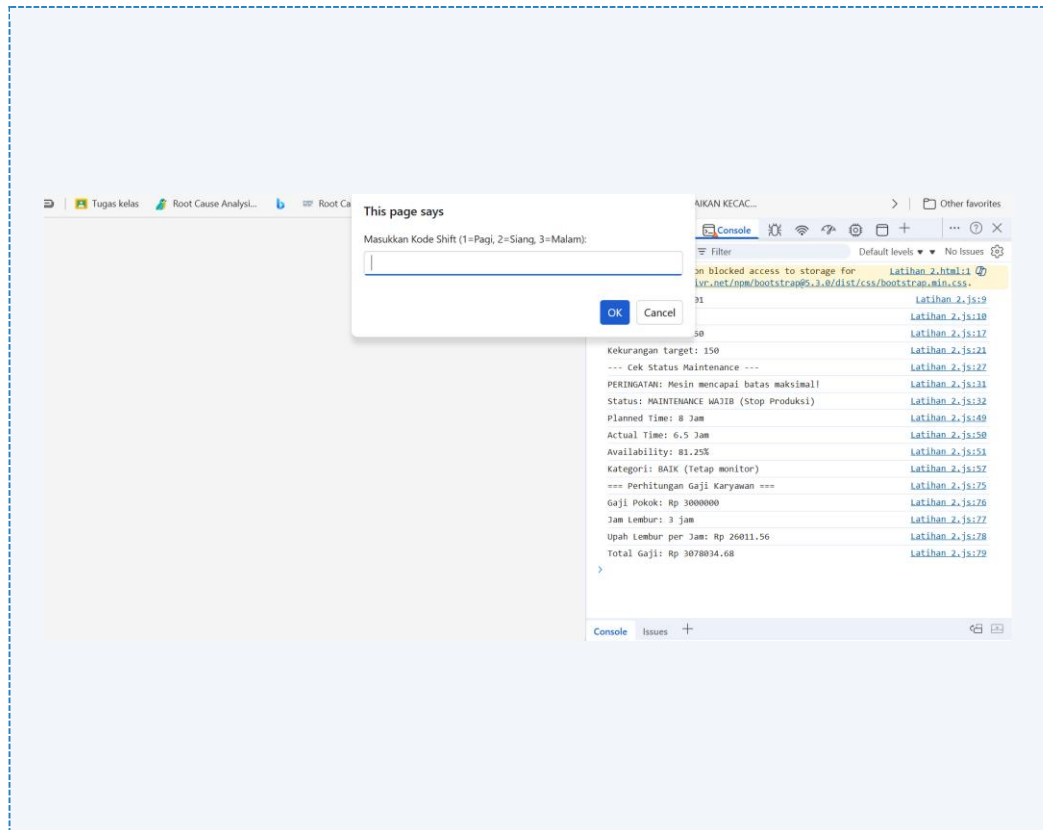
// Logika switch
switch (kodeShift) {
  case "1":
    namaShift = "Pagi";
    alert("Halo " + namaOperator + ", Selamat bekerja. Tetap semangat!");
    break;

  case "2":
    namaShift = "Siang";
    alert("Halo " + namaOperator + ", Selamat bekerja. Tetap semangat!");
    break;

  case "3":
    namaShift = "Malam";
    alert("Halo " + namaOperator + ", Shift malam memiliki tambahan uang makan sebesar Rp 20.000.");
    break;

  default:
    namaShift = "Shift Tidak Valid";
}

// Tampilkan hasil
alert("Shift Anda: " + namaShift);
console.log("Kode Shift: " + kodeShift);
console.log("Nama Shift: " + namaShift);
```



Gambar F.2 – Hasil Latihan 2

F.3 Tugas Proyek Mini – Kalkulator Biaya Produksi

Deskripsi proyek mini:

Buat program di `script.js` yang menghitung biaya produksi total per unit.

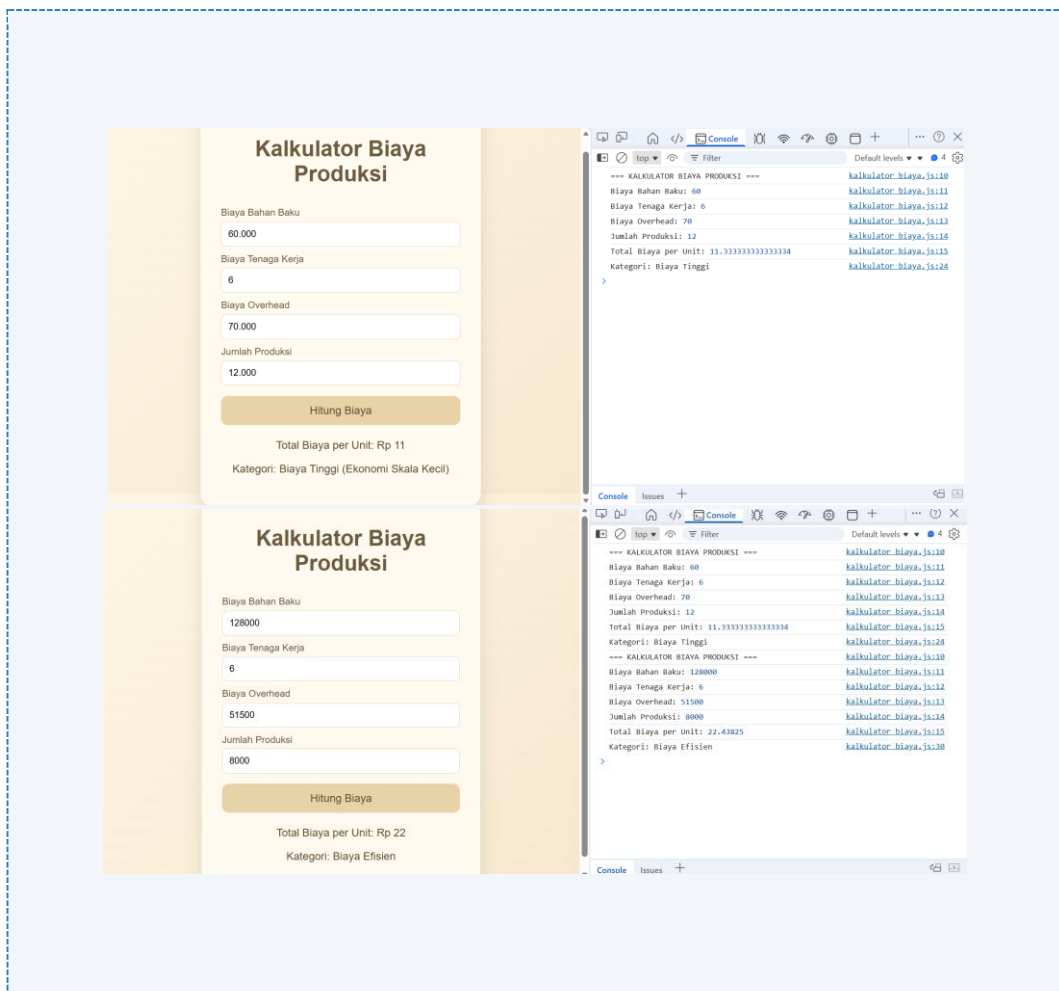
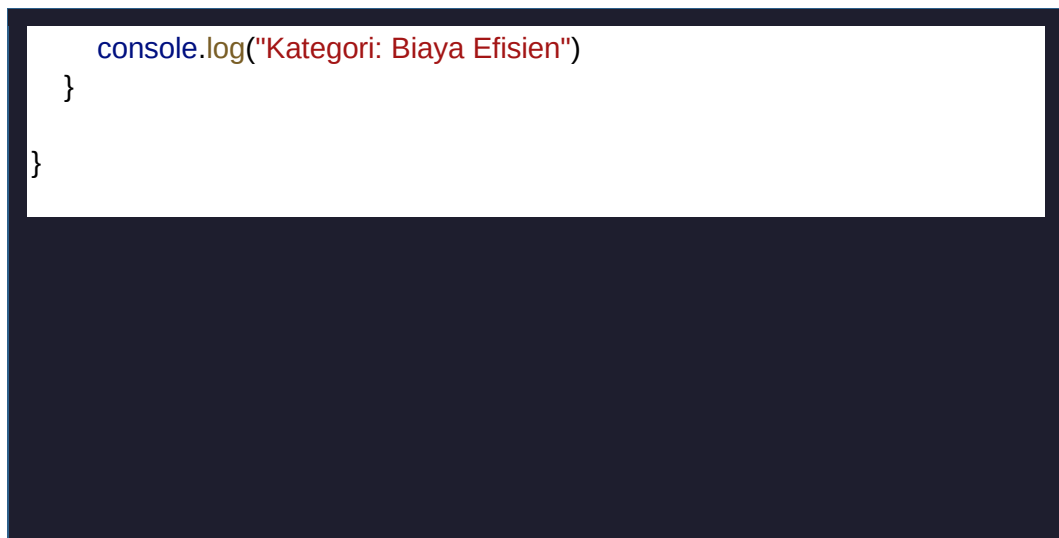
- Variabel: `biayaBahanBaku`, `biayaTenagaKerja`, `biayaOverhead`, `jumlahProduksi`.
- Rumus: $\text{totalPerUnit} = (\text{biayaBahanBaku} + \text{biayaTenagaKerja} + \text{biayaOverhead}) / \text{jumlahProduksi}$.
- Gunakan `if/else` untuk mengecek: jika `jumlahProduksi` < 100, tampilkan "Biaya Tinggi (Ekonomi Skala Kecil)". Jika ≥ 100 , tampilkan "Biaya Efisien".

Penjelasan pendekatan/solusi yang digunakan:

- Deklarasi variabel: untuk menyimpan biaya dan jumlah produksi.
- Operator aritmatika: + dan / untuk menghitung biaya total per unit.
- If/else: untuk memberikan penilaian otomatis berdasarkan jumlah produksi.
- **console.log** / **alert**: untuk menampilkan hasil perhitungan dan evaluasi biaya.

Kode Utama:

```
function hitungBiaya() {  
  
    let biayaBahanBaku =  
    parseInt(document.getElementById("bahanBaku").value)  
    let biayaTenagaKerja =  
    parseInt(document.getElementById("tenagaKerja").value)  
    let biayaOverhead =  
    parseInt(document.getElementById("overhead").value)  
    let jumlahProduksi =  
    parseInt(document.getElementById("jumlahProduksi").value)  
  
    let totalPerUnit = (biayaBahanBaku + biayaTenagaKerja + biayaOverhead)  
    / jumlahProduksi  
  
    console.log("=== KALKULATOR BIAYA PRODUKSI ===")  
    console.log("Biaya Bahan Baku:", biayaBahanBaku)  
    console.log("Biaya Tenaga Kerja:", biayaTenagaKerja)  
    console.log("Biaya Overhead:", biayaOverhead)  
    console.log("Jumlah Produksi:", jumlahProduksi)  
    console.log("Total Biaya per Unit:", totalPerUnit)  
  
    document.getElementById("hasilBiaya").innerText =  
        "Total Biaya per Unit: Rp " + Math.round(totalPerUnit).toLocaleString()  
  
    if (jumlahProduksi < 100) {  
        document.getElementById("kategoriBiaya").innerText =  
            "Kategori: Biaya Tinggi (Ekonomi Skala Kecil)"  
  
        console.log("Kategori: Biaya Tinggi")  
    }  
    else {  
        document.getElementById("kategoriBiaya").innerText =  
            "Kategori: Biaya Efisien"    }  
}
```



Gambar F.4 – Hasil Proyek Mini

G. PEMBAHASAN

G.1 Analisis Kesesuaian Hasil dengan Teori

Program yang dibuat menggunakan JavaScript ini sudah sesuai dengan teori dasar pemrograman. Variabel `biayaBahanBaku`, `biayaTenagaKerja`, `biayaOverhead`, dan `jumlahProduksi` digunakan untuk menyimpan data numerik yang akan dihitung, sesuai dengan konsep bahwa variabel merupakan tempat penyimpanan data dalam program. Perhitungan `totalPerUnit` menggunakan operator aritmatika (+ dan /) sehingga teori tentang operator aritmatika untuk perhitungan matematis sederhana juga terpenuhi. Struktur kontrol `if/else` digunakan untuk mengevaluasi jumlah produksi dan menentukan kategori biaya, sehingga program dapat mengambil keputusan secara otomatis sesuai kondisi, sesuai dengan teori control flow.

G.2 Kendala yang Ditemukan dan Solusinya

Kendala / Error yang Ditemukan	Solusi yang Diterapkan

G.3 Kaitan dengan Penerapan di Industri

Teknik Industri sering menangani proses produksi, monitoring mesin, dan manajemen operasional. Dengan JavaScript, bisa membuat **program yang otomatis menghitung biaya, produksi, atau efisiensi**, sehingga mendukung prinsip otomatisasi dan digitalisasi sistem industri. Misalnya, perhitungan biaya produksi per unit, jam lembur karyawan, atau total output bisa dilakukan secara otomatis di web.

H. KESIMPULAN

1	JavaScript berfungsi sebagai “otak” halaman web, yaitu untuk membuat halaman web interaktif .
2	If/Else digunakan untuk mengambil keputusan berdasarkan kondisi numerik.
3	Switch digunakan salah satunya untuk mengonversi kode shift menjadi nama shift.

I. DAFTAR PUSTAKA

No.	Referensi (Format APA 7th Edition)
1.	Fatikasari, P., & Fargazzi, M. 2025. Dominasi JavaScript dalam Pengembangan Web. Kohesi: Jurnal Sains dan Teknologi.
2.	Hidayatullah, M. A., & Paniran. 2023. Penerapan JavaScript pada Pembuatan Landing Page Website UMKM. Jurnal Nasional Komputasi dan Teknologi Informasi.
3.	Lathifaturrohman & Sujatmiko. 2025. Rancang Bangun LMS Berbasis Web Mengimplementasikan Project Based Learning untuk Meningkatkan Kompetensi Pemrograman JavaScript. IT-Edu: Jurnal Information Technology and Education.

J. LEMBAR PENILAIAN DAN PENGESAHAN

RUBRIK PENILAIAN LAPORAN					
Aspek Penilaian		Bobot			
No.	Aspek Penilaian	Bobot (%)	Nilai (0-100)	Skor Akhir	Catatan Penilai
1	Kelengkapan Isi (Semua bagian A–I terisi lengkap)	20%			
2	Kualitas Dasar Teori (Ditulis dengan bahasa sendiri, relevan, minimal 3 paragraf)	15%			
3	Dokumentasi Langkah Kerja (Kode & Screenshot sesuai, jelas, dan terbaca)	25%			
4	Tugas/Latihan Mandiri (Semua latihan & proyek mini dikerjakan dan berfungsi)	20%			
5	Pembahasan & Analisis (Kritis, mendalam, dan relevan dengan industri)	15%			
6	Kesimpulan & Tata Bahasa (Spesifik, rapi, mengikuti format yang ditentukan)	5%			
TOTAL NILAI AKHIR		100%			

Mahasiswa	Rekan Tim	Diperiksa oleh Asisten / Dosen
 _____ Saprina Melita Sari NIM:23051430012	 _____ Ardita Natalia NIM:23051430022	 _____ Dr. Eng. Ir. Aji Ery Burhandenny, S.T., M.AIT. Tanggal: _____

MODUL 5

A. IDENTITAS & INFORMASI MODUL

Mata Kuliah	Aplikasi Web dan Mobile
Kode Modul	MODUL 5 (Pertemuan 5)
Topik Utama	Function, Array, Object di JavaScript
Sub-Topik	Modularitas kode (Function), pengelolaan data list (Array), dan struktur data kompleks (Object).
Durasi Praktikum	340 Menit
Tanggal Praktikum	13 Maret 2026
Nama Mahasiswa	Saprina Melita Sari
NIM	23051430012
Kelas	A

B. TUJUAN PEMBELAJARAN

1	Memahami konsep Function untuk menghindari pengulangan kode (DRY - Don't Repeat Yourself).
2	Menguasai penggunaan Array untuk menyimpan sekumpulan data (misal: daftar cacat produksi).
3	Memahami Object untuk merepresentasikan entitas nyata (misal: Spesifikasi Mesin) dalam bentuk data.
4	Mampu melakukan iterasi (perulangan) terhadap Array dan Object.

C. ALAT DAN BAHAN

No.	Nama Alat / Bahan / Aplikasi	Versi / Spesifikasi	Keterangan
1	Visual Studio Code (VS Code)	Versi 1.8x ke atas	Code Editor
2	Google Chrome / Browser Modern	Versi terbaru	Testing & Debugging

3	Live Server	Versi 5.7.9	Extentions Vs Code untuk menjalankan HTML
4	Prettier	Verso 12.3.0	Extentions Vs Code untuk merapikan code
5	Bootsrap	Versi 5.3.8	Framework front-end open source untuk mempercepat pengembangan website.

D. DASAR TEORI

D.1 Konsep Utama

Modul ini membahas tiga konsep utama dalam JavaScript yang berperan penting dalam penyusunan program yang lebih rapi dan terstruktur, yaitu function, array, dan object. Function merupakan kumpulan instruksi yang disusun dalam satu blok kode sehingga dapat dipanggil kembali saat diperlukan. Dengan adanya function, penulisan kode menjadi lebih efisien karena programmer tidak perlu mengulang perintah yang sama berkali-kali. Penerapan function juga mendukung prinsip DRY (Don't Repeat Yourself), yaitu konsep yang bertujuan meminimalkan pengulangan kode dalam proses pengembangan perangkat lunak.

Selain itu, modul ini menjelaskan mengenai array sebagai struktur data yang digunakan untuk menyimpan banyak nilai dalam satu variabel. Array membantu pengelolaan data yang memiliki jenis serupa, seperti daftar barang, data produksi, maupun hasil pengukuran. Setiap data dalam array dapat diakses melalui indeks tertentu dan dapat diolah menggunakan perulangan sehingga proses pencarian, perhitungan, maupun pengolahan data lainnya dapat dilakukan dengan lebih teratur dan sistematis.

Konsep berikutnya adalah object, yaitu struktur data yang dipakai untuk menggambarkan suatu entitas beserta atribut atau karakteristik yang dimilikinya. Object umumnya digunakan untuk menyimpan data dengan berbagai properti berbeda, misalnya spesifikasi mesin yang mencakup nama mesin, kapasitas, daya listrik, serta kondisi operasionalnya. Dengan mengombinasikan function,

array, dan object, program JavaScript mampu menangani data yang lebih kompleks sehingga aplikasi yang dibuat menjadi lebih terorganisir dan sesuai dengan kebutuhan sistem nyata di dunia industri.

D.2 Keterkaitan dengan Teknik Industri

Konsep function, array, dan object pada JavaScript sangat relevan dengan penerapan teknologi informasi di bidang Teknik Industri, khususnya dalam mendukung pengolahan data serta otomatisasi aktivitas operasional. Function dapat diibaratkan sebagai prosedur kerja yang dirancang agar suatu proses dapat dijalankan secara berulang dengan langkah yang konsisten. Dalam dunia industri, function sering dimanfaatkan untuk melakukan proses otomatis seperti menghitung biaya operasional, menentukan kapasitas produksi, maupun menganalisis waktu kerja mesin tanpa harus menuliskan perintah yang sama berkali-kali.

Array berperan penting dalam pengelolaan data yang jumlahnya banyak dan memiliki jenis serupa. Pada kegiatan industri, data seperti hasil produksi harian, stok barang, jumlah produk cacat, hingga daftar mesin produksi perlu disusun secara rapi agar mudah dianalisis. Dengan menggunakan array, data-data tersebut dapat disimpan dalam satu struktur sehingga proses pengolahan seperti pencarian data, perhitungan total produksi, maupun evaluasi performa dapat dilakukan dengan lebih cepat dan sistematis.

Sementara itu, object digunakan untuk menggambarkan suatu komponen atau entitas dalam sistem industri beserta karakteristik yang dimilikinya. Sebagai contoh, sebuah mesin produksi dapat direpresentasikan dalam bentuk object yang memuat informasi seperti nama mesin, kapasitas kerja, konsumsi daya listrik, dan kondisi operasional. Pendekatan ini membantu pengembangan sistem informasi industri menjadi lebih terorganisir karena setiap elemen dapat dimodelkan sesuai kondisi nyata di lapangan. Dengan memadukan function, array, dan object, pembuatan aplikasi industri seperti sistem inventori, pengendalian produksi, hingga ERP dapat dilakukan dengan lebih efektif dan terstruktur.

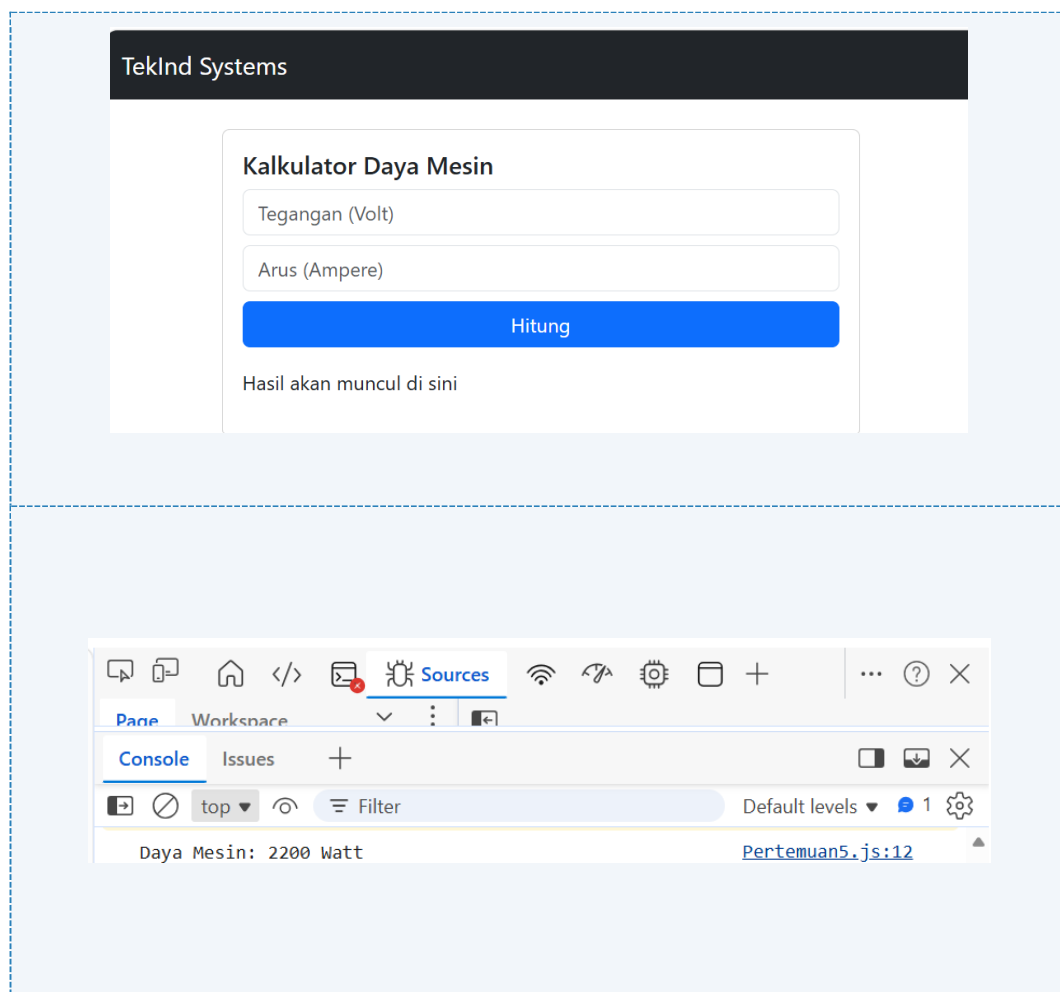
E. LANGKAH KERJA DAN HASIL PRAKTIKUM

E.1 Langkah 1 – Membuat Function Perhitungan

```
// 1. FUNCTION Declaration
// Fungsi untuk menghitung konsumsi daya (P = V x I)
function hitungDaya(tegangan, arus) {
  let daya = tegangan * arus;
  return daya; // Mengembalikan hasil
}

// Pemanggilan Function
let teganganMesin = 220; // Volt
let arusMesin = 10; // Ampere
let totalDaya = hitungDaya(teganganMesin, arusMesin);
console.log("Daya Mesin: " + totalDaya + " Watt");
```

Screenshot Hasil:



Gambar E.1 – Hasil Langkah 1

E.2 Langkah 2 – Array dan Looping (Daftar Produk Cacat)

```
// 2. ARRAY
let daftarCacat = ["C-001", "C-005", "C-012", "C-001", "C-020"]; // C-001 terjadi 2 kali

console.log("Jumlah Cacat: " + daftarCacat.length);

// Looping (Perulangan) untuk menampilkan setiap cacat
console.log("--- Laporan Detail Cacat ---");
for (let i = 0; i < daftarCacat.length; i++) {
  console.log("Item ke-" + (i + 1) + " : " + daftarCacat[i]);
}

// Menambahkan data baru ke array
daftarCacat.push("C-099");
console.log("Setelah penambahan: " + daftarCacat);
```

Screenshot Hasil:

Laporan Cacat Produksi

Jumlah Cacat: 6

Item ke-1 : C-001

Item ke-2 : C-005

Item ke-3 : C-012

Item ke-4 : C-001

Tambah Cacat

Jumlah Cacat: 5	Pertemuan5.js:17
--- Laporan Detail Cacat ---	Pertemuan5.js:20
Item ke-1 : C-001	Pertemuan5.js:22
Item ke-2 : C-005	Pertemuan5.js:22
Item ke-3 : C-012	Pertemuan5.js:22
Item ke-4 : C-001	Pertemuan5.js:22
Item ke-5 : C-020	Pertemuan5.js:22
Setelah penambahan: C-001,C-005,C-012,C-001,C-020,C-099	Pertemuan5.js:27

Gambar E.2 – Hasil Langkah 2

E.3 Langkah 3 – Object (Rekaman Data Mesin)

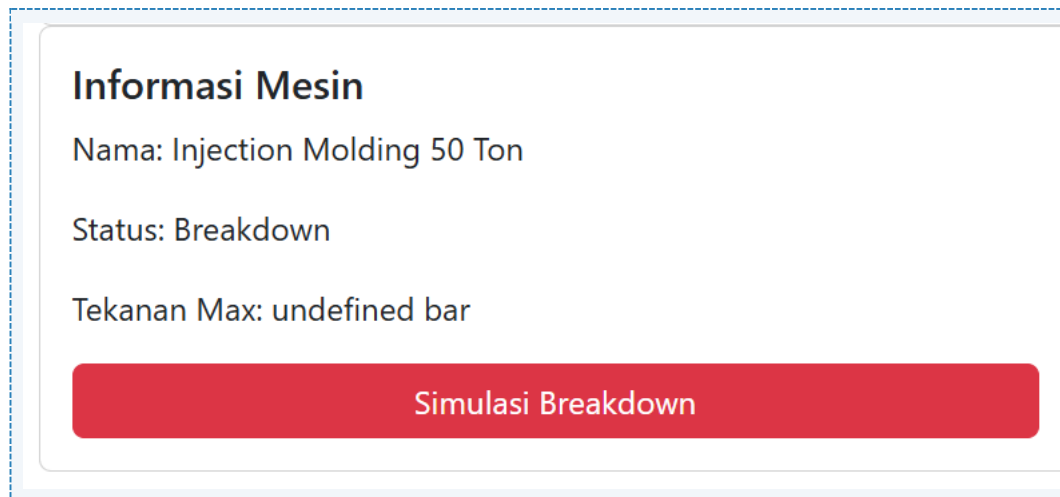
```
// 3. OBJECT
let mesinA = {
  id: "M-01",
  nama: "Injection Molding 50 Ton",
  merk: "Toshiba",
  tahunBeli: 2018,
  status: "Running",
  spesifikasi: {
    kecepatanMax: 200, // mm/s
    // tekananMax: 150 // bar
  }
};

// Mengakses data object
console.log("Nama Mesin: " + mesinA.nama);
console.log("Status: " + mesinA.status);
console.log("Tekanan Max: " + mesinA.spesifikasi.tekananMax + " bar");

// Mengubah data object (Simulasi mesin rusak)
mesinA.status = "Breakdown";
console.log("Status Baru: " + mesinA.status);
```

Screenshot Hasil:

Nama Mesin: Injection Molding 50 Ton	Pertemuan5.js:43
Status: Running	Pertemuan5.js:44
Tekanan Max: undefined bar	Pertemuan5.js:45
Status Baru: Breakdown	Pertemuan5.js:49



Gambar E.3 – Hasil Langkah 3

E.4 Langkah 4 – Gabungan (Array of Objects)

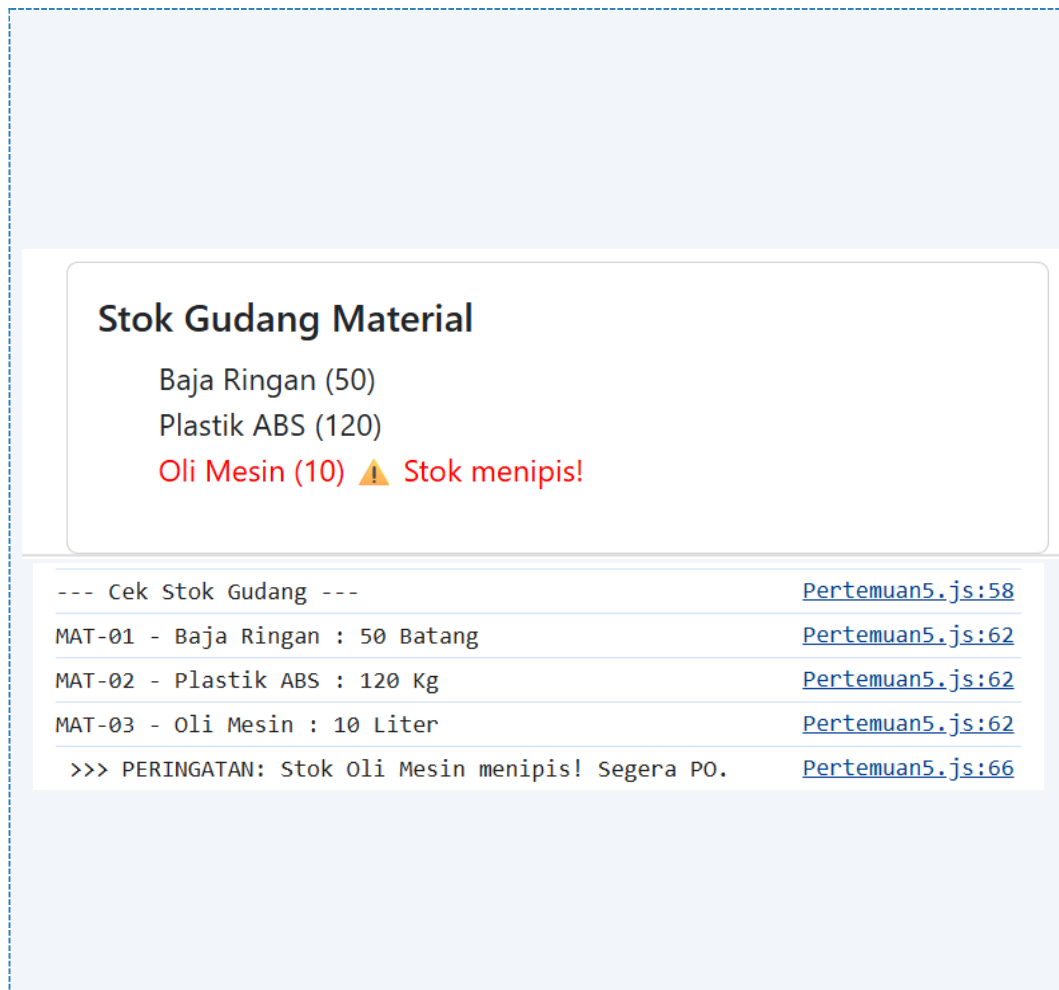
```
// Array berisi beberapa object
let gudangMaterial = [
  { kode: "MAT-01", nama: "Baja Ringan", stok: 50, satuan: "Batang" },
  { kode: "MAT-02", nama: "Plastik ABS", stok: 120, satuan: "Kg" },
  { kode: "MAT-03", nama: "Oli Mesin", stok: 10, satuan: "Liter" }
];

console.log("--- Cek Stok Gudang ---");

// Menggunakan forEach untuk iterasi array object
gudangMaterial.forEach(function(item) {
  console.log(item.kode + " - " + item.nama + " : " + item.stok + " " +
    item.satuan);

  // Logika Re-order
  if (item.stok < 20) {
    console.log(">>> PERINGATAN: Stok " + item.nama + " menipis!
    Segera PO.");
  }
});
```

Screenshot Hasil:



Gambar E.4 – Hasil Langkah 4

F. LATIHAN DAN TUGAS MANDIRI

F.1 Latihan 1 – Fuction

Buat function `hitungLingkaran(jariJari)` yang mengembalikan nilai Luas dan Keliling. Tampilkan kedua nilai tersebut.

Jawaban / Kode Solusi:

HTML

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Perhitungan Lingkaran</title>

  <!-- Bootstrap -->
  <link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css"
rel="stylesheet">

  <!-- CSS tambahan -->
  <link rel="stylesheet" href="latihan1.css">
</head>

<body>

<!-- NAVBAR -->
<nav class="navbar navbar-expand-lg navbar-dark bg-dark">
  <div class="container-fluid">
    <a class="navbar-brand" href="">TekInd Systems</a>
  </div>
</nav>

<div class="container mt-4">

  <!-- LATIHAN 1 -->
<div class="row mt-4">
  <div class="col-12">
    <div class="card p-3">
      <h5>Perhitungan Lingkaran</h5>
      <input type="number" id="jariJari" class="form-control mb-2"
placeholder="Masukkan Jari-Jari">
      <button class="btn btn-primary"
onclick="tampilkanLingkaran()">Hitung</button>
      <p class="mt-2" id="hasilLuas"></p>
      <p id="hasilKeliling"></p>
    </div>
  </div>
</div>

<script src="latihan1.js"></script>

</body>
```

```
</html>
```

```
JAVASRIPT
```

```
// LATIHAN 1 - Function hitungLingkaran
```

```
function hitungLingkaran(jariJari) {
```

```
    let luas = Math.PI * jariJari * jariJari;
```

```
    let keliling = 2 * Math.PI * jariJari;
```

```
    return {
```

```
        luas: luas.toFixed(2),
```

```
        keliling: keliling.toFixed(2)
```

```
    };
```

```
}
```

```
let hasilLingkaran = hitungLingkaran(7);
```

```
console.log("Luas Lingkaran: " + hasilLingkaran.luas);
```

```
console.log("Keliling Lingkaran: " + hasilLingkaran.keliling);
```

```
function tampilkanLingkaran() {
```

```
    let jariJari = document.getElementById("jariJari").value;
```

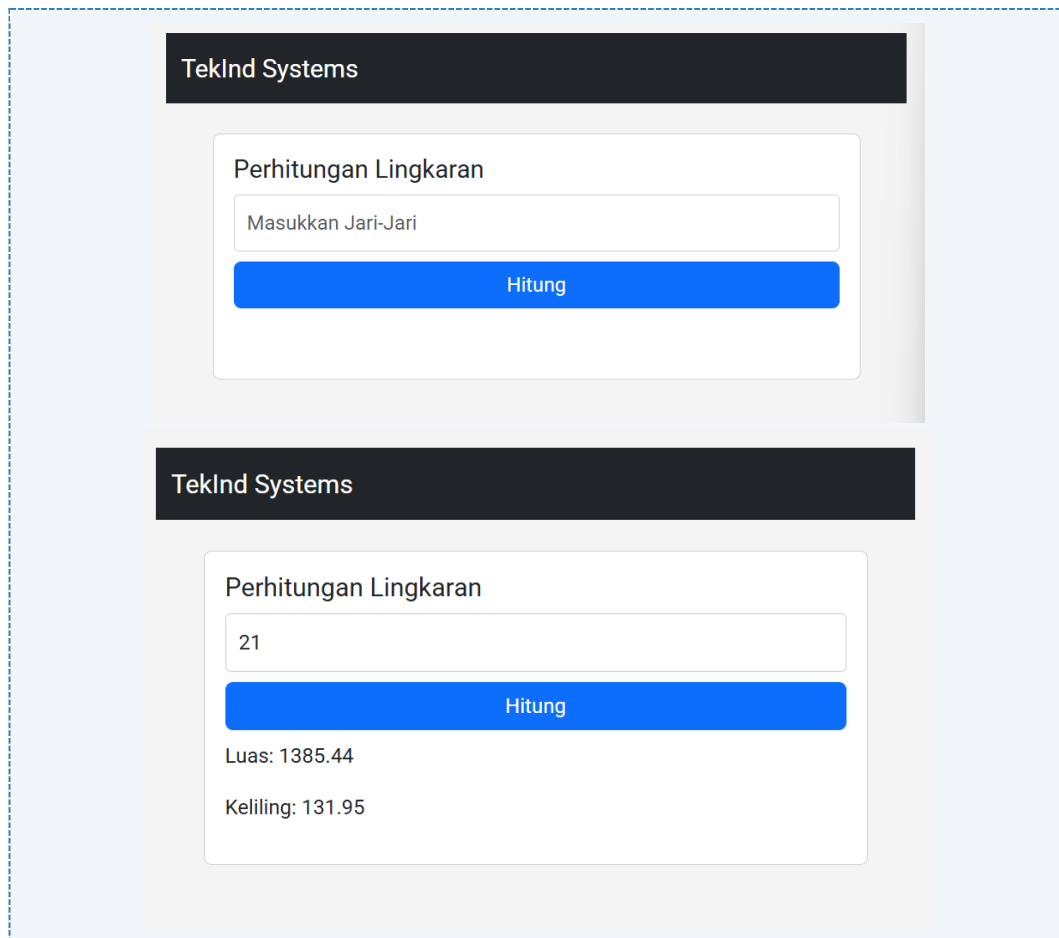
```
    let hasil = hitungLingkaran(jariJari);
```

```
    document.getElementById("hasilLuas").innerText = "Luas: " + hasil.luas;
```

```
    document.getElementById("hasilKeliling").innerText = "Keliling: " +
```

```
    hasil.keliling;
```

```
}
```



Gambar F.1 – Hasil Latihan 1

F.2 Latihan 2 – Array Manipulation

Dari array `daftarCacat` di atas, carilah berapa kali cacat "C-001" muncul (Gunakan loop dan if/counter).

Jawaban / Kode Solusi:

```
HTML
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Perhitungan Lingkaran</title>

  <!-- Bootstrap -->
  <link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css"
rel="stylesheet">
```

```
<!-- CSS tambahan -->
<link rel="stylesheet" href="latihan1.css">
</head>

<body>

<!-- NAVBAR -->
<nav class="navbar navbar-expand-lg navbar-dark bg-dark">
  <div class="container-fluid">
    <a class="navbar-brand" href="">TekInd Systems</a>
  </div>
</nav>

<div class="container mt-4">

<!-- LATIHAN 2 -->
<div class="row mt-4">
  <div class="col-12">
    <div class="card p-3">
      <h5>Latihan 2 - Cari Frekuensi Cacat</h5>
      <input type="text" id="inputCacat" class="form-control mb-2"
placeholder="Masukkan kode cacat (misal: C-001)">
      <button class="btn btn-primary" onclick="cariCacat()">Cari</button>
      <p class="mt-2" id="hasilCacat"></p>
    </div>
  </div>
</div>

<script src="latihan2.js"></script>

</body>
</html>

JAVASCRIPT
// LATIHAN 2 - Array Manipulation
// Taruh di PALING ATAS, sebelum semua function
let daftarCacat = ["C-001", "C-005", "C-012", "C-001", "C-020"];
let counter = 0;

for (let i = 0; i < daftarCacat.length; i++) {
  if (daftarCacat[i] === "C-001") {
    counter++;
  }
}

console.log("Cacat C-001 muncul sebanyak: " + counter + " kali");
```

```
function cariCacat() {  
  let target = document.getElementById("inputCacat").value;  
  let counter = 0;  
  
  for (let i = 0; i < daftarCacat.length; i++) {  
    if (daftarCacat[i] === target) {  
      counter++;  
    }  
  }  
  
  if (counter === 0) {  
    document.getElementById("hasilCacat").innerText = "Cacat " + target + "  
tidak ditemukan.";  
  } else {  
    document.getElementById("hasilCacat").innerText = "Cacat " + target + "  
muncul sebanyak: " + counter + " kali";  
  }  
}
```

Oth

TekInd Systems

Latihan 2 - Cari Frekuensi Cacat

C-001

Cari

Cacat C-001 muncul sebanyak: 2 kali

Gambar F.2 – Hasil Latihan 2

F.3 Tugas Proyek Mini – Sistem Antrian Job Shop

Deskripsi proyek mini:

- a. Buat Array of Objects bernama ``antrianMesin``. Isi dengan 3 object job: `{idJob: "J01", namaProses: "Drilling", durasi: 30}``.
- b. Buat function ``prosesAntrian(antrian)`` yang menerima parameter array tersebut.
- c. Di dalam function, tampilkan pesan: "Memproses Job [ID] - [Nama] selama [durasi] menit" untuk setiap job dalam antrian.
- d. Tambahkan 1 job baru ke array dan panggil function lagi.

Penjelasan pendekatan/solusi yang digunakan:

Program tersebut menggunakan pendekatan pemrograman terstruktur dengan memanfaatkan konsep array, object, function, looping, serta manipulasi DOM pada JavaScript untuk membuat simulasi sistem antrian job shop. Data antrian disimpan dalam bentuk array of objects, di mana setiap object merepresentasikan satu job produksi yang memiliki atribut seperti ID job, nama proses, dan durasi pengerjaan. Pendekatan ini memudahkan pengelolaan data karena setiap pekerjaan dapat disimpan secara terstruktur dalam satu variabel. Selain itu, program menggunakan function modular seperti `prosesAntrian()`, `tampilAntrian()`, dan `tambahJob()` agar setiap bagian program memiliki tugas tertentu sehingga kode menjadi lebih rapi, efisien, dan mudah dikembangkan.

Proses pembacaan data dilakukan menggunakan perulangan `for` untuk menampilkan seluruh job dalam antrian secara otomatis. Pendekatan ini memungkinkan sistem menangani jumlah data yang dapat berubah sewaktu-waktu tanpa perlu mengubah logika program utama. Penambahan job baru dilakukan menggunakan method `push()` sehingga sistem bersifat dinamis dan dapat menyesuaikan dengan adanya tambahan pekerjaan produksi baru. Selain itu, program juga menerapkan manipulasi DOM untuk menampilkan data antrian langsung pada halaman web melalui elemen HTML. Dengan cara ini, informasi antrian dapat diperbarui secara interaktif dan lebih mudah dipantau oleh pengguna.

Kode Utama:

HTML

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Sistem Monitoring Produksi</title>

  <!-- Bootstrap -->
  <link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css"
rel="stylesheet">

  <!-- CSS tambahan -->
  <link rel="stylesheet" href="Sistem Antrian.css">
</head>

<body>

<!-- NAVBAR -->
<nav class="navbar navbar-expand-lg navbar-dark bg-dark">
  <div class="container-fluid">
    <a class="navbar-brand" href="">TekInd Systems</a>
  </div>
</nav>

<!-- TUGAS -->
<div class="row mt-4">
  <div class="col-12">
    <div class="card p-3">
      <h5>Proyek Mini - Simulasi Antrian Mesin</h5>
      <button class="btn btn-primary me-2"
onclick="tampilAntrian()">Proses Antrian</button>
      <button class="btn btn-success" onclick="tambahJob()">Tambah Job
Baru</button>
      <ul id="listAntrian" class="list-group mt-3" style="max-height: 200px;
overflow-y: auto;"></ul>
    </div>
  </div>
</div>

<script src="Sistem Antrian.js"></script>

</body>
</html>
```


JAVASRIPT

```
// Proyek Mini - Sistem Antrian Job Shop
```

```
// a. Array of Objects
```

```
let antrianMesin = [  
  { idJob: "J01", namaProses: "Drilling", durasi: 30 },  
  { idJob: "J02", namaProses: "Milling", durasi: 45 },  
  { idJob: "J03", namaProses: "Turning", durasi: 20 }  
];
```

```
// b. & c. Function prosesAntrian
```

```
function prosesAntrian(antrian) {  
  console.log("--- Memulai Proses Antrian ---");  
  for (let i = 0; i < antrian.length; i++) {  
    console.log("Memproses Job " + antrian[i].idJob +  
      " - " + antrian[i].namaProses +  
      " selama " + antrian[i].durasi + " menit");  
  }  
  console.log("--- Antrian Selesai ---");  
}
```

```
// Panggil pertama kali
```

```
prosesAntrian(antrianMesin);
```

```
// d. Tambah 1 job baru lalu panggil lagi
```

```
antrianMesin.push({ idJob: "J04", namaProses: "Grinding", durasi: 15 });  
console.log("Job baru ditambahkan!");  
prosesAntrian(antrianMesin);
```

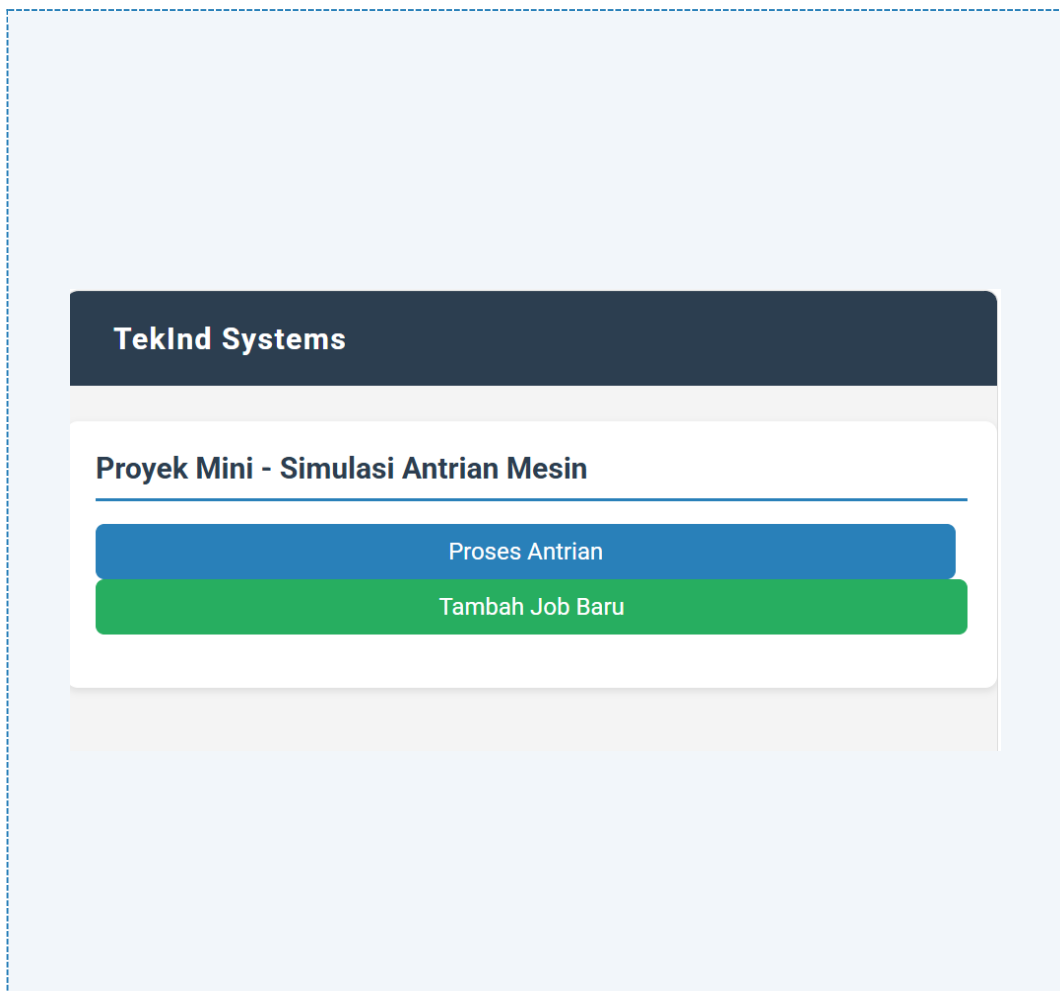
```
function tampilAntrian() {
```

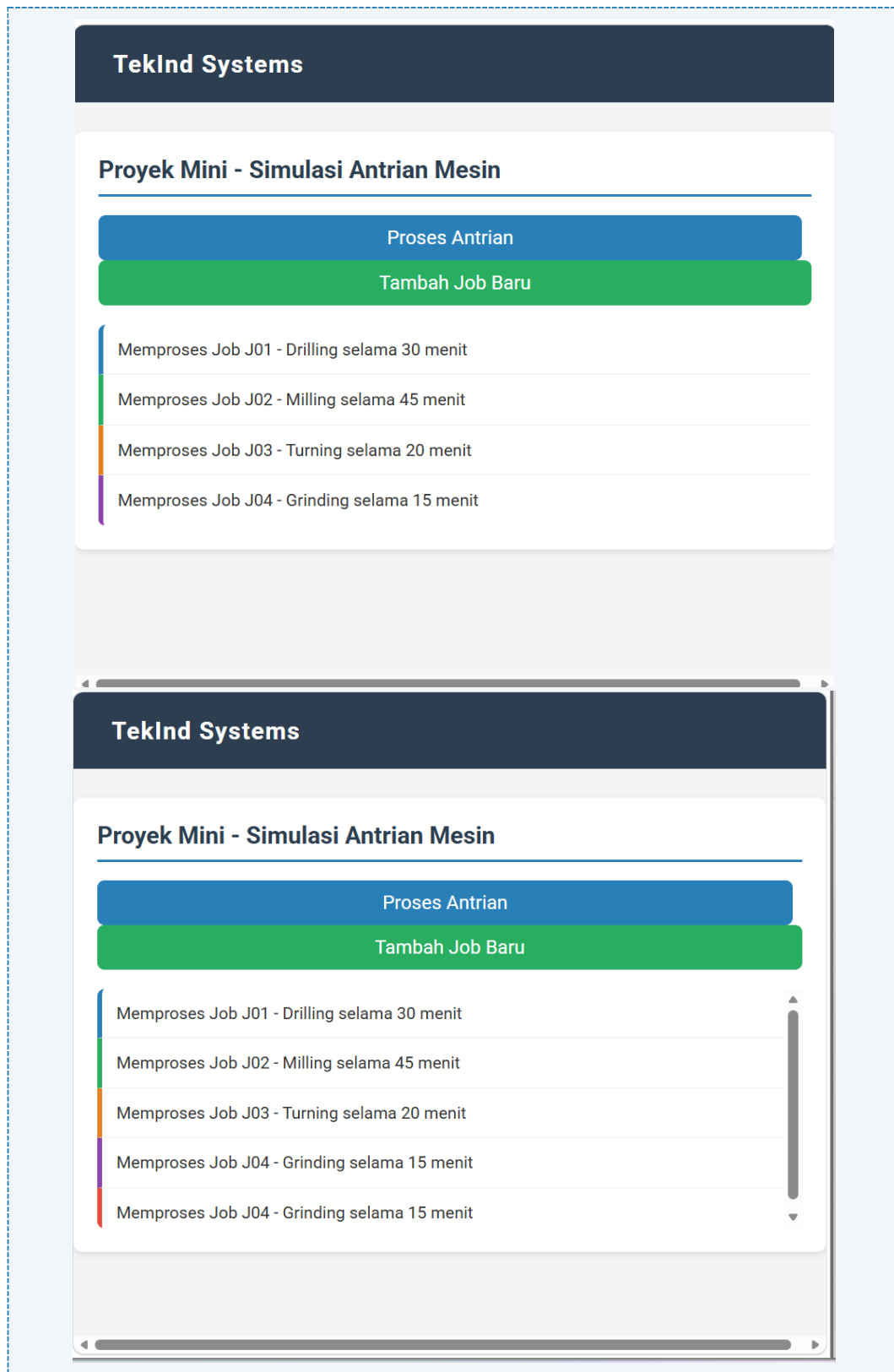
```
  let list = document.getElementById("listAntrian");  
  list.innerHTML = "";
```

```
  for (let i = 0; i < antrianMesin.length; i++) {  
    let li = document.createElement("li");  
    li.className = "list-group-item";  
    li.innerText = "Memproses Job " + antrianMesin[i].idJob +  
      " - " + antrianMesin[i].namaProses +  
      " selama " + antrianMesin[i].durasi + " menit";  
    list.appendChild(li);  
  }  
}
```

```
function tambahJob() {
```

```
  antrianMesin.push({ idJob: "J04", namaProses: "Grinding", durasi: 15 });  
  tampilAntrian();  
}
```





Gambar F.4 – Hasil Proyek Mini

G. PEMBAHASAN

G.1 Analisis Kesesuaian Hasil dengan Teori

Hasil yang diperoleh dari praktikum pada modul ini secara umum telah sesuai dengan teori yang dipelajari. Penerapan function pada program menunjukkan bahwa sekumpulan instruksi dapat disusun dalam satu blok kode dan dipanggil kembali kapan saja diperlukan. Penggunaan function membantu membuat program menjadi lebih teratur karena programmer tidak perlu menuliskan kode yang sama secara berulang. Selain itu, pendekatan ini juga membuat proses pengembangan program menjadi lebih efektif dan mudah dipahami.

Penggunaan array dalam praktikum juga telah menggambarkan fungsi array sebagai media penyimpanan banyak data dalam satu variabel. Data dapat disusun dalam bentuk daftar sehingga proses pengolahan, pencarian, maupun penampilan data menjadi lebih sederhana dan sistematis. Metode ini tentunya lebih efisien dibandingkan menyimpan setiap data pada variabel yang berbeda-beda.

Di sisi lain, penerapan object sudah menunjukkan konsep pengelompokan data berdasarkan atribut tertentu. Melalui object, beberapa informasi yang saling berkaitan dapat disimpan dalam satu struktur sehingga data lebih rapi dan mudah digunakan kembali saat diperlukan. Secara keseluruhan, hasil praktikum membuktikan bahwa kombinasi function, array, dan object mampu mendukung pembuatan program yang lebih terstruktur, efisien, serta mudah dikelola dalam proses pengembangan aplikasi.

G.2 Kendala yang Ditemukan dan Solusinya

Kendala / Error yang Ditemukan	Solusi yang Diterapkan
Kesalahan penulisan nama variabel atau function sehingga program tidak dapat dijalankan.	Memeriksa kembali penulisan syntax dan memastikan nama variabel serta function konsisten.
Hasil program awalnya hanya muncul pada console sehingga data tidak dapat dilihat langsung di halaman web.	Menambahkan function <code>tampilkanAntrian()</code> untuk menampilkan data array ke halaman HTML menggunakan <code>document.getElementById()</code> serta membuat elemen <code></code> secara dinamis.
Tombol tambah job tidak berfungsi saat diklik.	Memastikan event button sudah terhubung dengan function <code>tambahJob()</code> .

G.3 Kaitan dengan Penerapan di Industri

Konsep function, array, dan object pada JavaScript berhubungan erat dengan penerapan teknologi informasi di dunia industri, terutama dalam mendukung pengelolaan data serta otomatisasi proses pada bidang Teknik Industri. Dalam kegiatan industri, berbagai aktivitas memerlukan sistem yang mampu mengolah informasi secara cepat, akurat, dan terstruktur, seperti pencatatan hasil produksi, monitoring mesin, hingga evaluasi performa proses kerja. Pemrograman berperan penting dalam membantu membangun sistem yang dapat menyederhanakan pekerjaan tersebut sehingga proses operasional menjadi lebih efisien dan meminimalkan terjadinya human error.

Penerapan function dapat diibaratkan sebagai prosedur kerja yang dijalankan berulang kali dalam aktivitas operasional perusahaan. Contohnya, pada sistem produksi dapat dibuat program untuk menghitung biaya operasional, penggunaan energi mesin, maupun total hasil produksi harian. Dengan memanfaatkan function, proses perhitungan dapat dilakukan secara berulang tanpa harus menuliskan kembali kode yang sama, sehingga pengolahan data menjadi lebih praktis, konsisten, dan mudah dikelola.

H. KESIMPULAN

1	Object pada JavaScript digunakan untuk menyimpan data yang memiliki beberapa karakteristik atau atribut dalam satu kesatuan. Dengan pendekatan ini, data dapat disusun dengan lebih rapi dan mudah direpresentasikan sesuai kebutuhan program.
2	JavaScript tidak hanya dapat menampilkan output pada console, tetapi juga mampu menampilkan hasil langsung pada halaman web melalui manipulasi DOM, misalnya menggunakan <code>document.getElementById()</code> serta pembuatan elemen HTML secara dinamis.
3	Array sebagai struktur data memungkinkan banyak data disimpan dalam satu variabel sehingga proses pengelolaan data menjadi lebih mudah. Data yang tersimpan di dalam array dapat diproses atau ditampilkan satu per satu menggunakan perulangan secara lebih sistematis.

I. DAFTAR PUSTAKA

No.	Referensi (Format APA 7th Edition)
1.	Fatikasari, P., & Fargazzi, M. 2025. Dominasi JavaScript dalam Pengembangan Web. Kohesi: Jurnal Sains dan Teknologi.
2.	Hidayatullah, M. A., & Paniran. 2023. Penerapan JavaScript pada Pembuatan Landing Page Website UMKM. Jurnal Nasional Komputasi dan Teknologi Informasi.
3.	Lathifaturrohman & Sujatmiko. 2025. Rancang Bangun LMS Berbasis Web Mengimplementasikan Project Based Learning untuk Meningkatkan Kompetensi Pemrograman JavaScript. IT-Edu: Jurnal Information Technology and Education.

MODUL 6

A. IDENTITAS & INFORMASI MODUL

Mata Kuliah	Aplikasi Web dan Mobile
Kode Modul	MODUL 6 (Pertemuan 6)
Topik Utama	DOM Manipulation & Event Handling
Sub-Topik	Document Object Model (DOM), Seleksi Elemen, dan Event Listeners.
Durasi Praktikum	340 Menit
Tanggal Praktikum	27 Maret 2026
Nama Mahasiswa	Saprina Melita Sari
NIM	23051430012
Kelas	A

B. TUJUAN PEMBELAJARAN

1	Memahami struktur DOM dan bagaimana JavaScript melihat halaman web.
2	Mampu memanipulasi elemen HTML (mengubah teks, warna, kelas) secara dinamis menggunakan JavaScript.
3	Mampu menangani interaksi pengguna (klik tombol, ketik keyboard) menggunakan Event Listeners.
4	Menerapkan konsep ini untuk membuat simulasi Control Panel mesin industri.

C. ALAT DAN BAHAN

No.	Nama Alat / Bahan / Aplikasi	Versi / Spesifikasi	Keterangan
1	Visual Studio Code (VS Code)	Versi 1.8x ke atas	Code Editor

2	Google Chrome / Browser Modern	Versi terbaru	Testing & Debugging
3	Node.js & npm	Node 18+ / npm 9+	Runtime JS (Modul tertentu)
5	HTML	-	Struktur
6	CSS		Styling

D. DASAR TEORI

DOM Manipulation dan Event Handling dalam pengembangan web didasari oleh Document Object Model (DOM), yang merupakan antarmuka pemrograman untuk merepresentasikan dan memanipulasi dokumen HTML sebagai pohon node (Galahartlambang et al., 2021). DOM memungkinkan JavaScript berinteraksi secara dinamis dengan elemen halaman, mengubah struktur, konten, dan gaya tanpa memuat ulang halaman, sehingga mendukung aplikasi web interaktif modern. Di Indonesia, penerapan DOM sering dibahas dalam konteks performa aplikasi web untuk mengatasi keterlambatan rendering pada elemen besar.

Seleksi elemen DOM menggunakan metode seperti `getElementById()`, `querySelector()`, dan `querySelectorAll()` yang memanfaatkan selector CSS untuk akses presisi (Rosidi, 2025). Teknik ini esensial untuk manipulasi targeted, di mana hasil seleksi berupa objek node siap untuk operasi seperti insert atau delete, meningkatkan efisiensi kode JavaScript. Studi lokal menekankan seleksi ini dalam pengembangan sistem berbasis web untuk mengoptimalkan navigasi pohon DOM.

Event Handling direalisasikan melalui `addEventListener()` yang meregistrasikan fungsi callback pada event seperti 'click' atau 'input' (Galahartlambang et al., 2021). Fitur ini mendukung event delegation dan pencegahan propagasi, memungkinkan respons real-time terhadap interaksi user tanpa konflik multiple handler. Di konteks Indonesia, event handling dikaitkan dengan peningkatan user experience pada aplikasi single-page.

Teori keseluruhan menyoroti evolusi DOM dari abstraksi statis ke Virtual DOM untuk performa lebih baik, dengan aplikasi di framework seperti React dan Vue (Galahartlambang et al., 2021). Standarisasi W3C diadaptasi secara luas di penelitian lokal untuk mengukur metrik seperti durasi eksekusi dan penggunaan memori, mendukung pengembangan web skalabel.

D.1 Konsep Utama

Document Object Model (DOM) merupakan representasi struktural dari dokumen HTML dalam bentuk pohon node yang dapat dimanipulasi secara dinamis oleh JavaScript. Browser mem-parsing kode HTML menjadi objek DOM, sehingga memungkinkan pengaksesan elemen melalui metode seleksi seperti `querySelector()`, modifikasi konten dengan `innerHTML`, pengubahan atribut menggunakan `setAttribute()`, serta pengelolaan gaya CSS melalui properti `style`. Operasi penambahan dan penghapusan node dilakukan dengan `appendChild()` dan `removeChild()`, di mana setiap perubahan langsung tercermin pada tampilan halaman tanpa memerlukan reload penuh, sehingga meningkatkan efisiensi interaksi pengguna.

Event Handling berfungsi untuk mendeteksi dan merespons interaksi pengguna seperti klik, input teks, atau scroll melalui metode `addEventListener()` yang diterapkan pada objek `EventTarget`. Proses propagasi event terjadi melalui mekanisme bubbling (dari elemen target menuju root) atau capturing (sebaliknya), dengan objek event menyediakan informasi detail seperti `target`, `clientX`, dan `clientY`. Konsep event delegation memungkinkan pemantauan event pada satu elemen parent untuk menangani multiple child elements, yang sangat efektif dalam mengoptimalkan performa pada daftar elemen dinamis.

D.2 Keterkaitan dengan Teknik Industri

DOM Manipulation & Event Handling digunakan untuk:

1. Dashboard monitoring produksi real-time - Tampilkan OEE, cycle time, defect rate dari sensor IoT tanpa refresh halaman
2. Simulasi facility layout - Klik mesin → hitung rectilinear distance/centroid → update visual layout instan
3. Quality control web app - Event delegation pantau multiple Six Sigma charts dari satu listener parent

E. LANGKAH KERJA DAN HASIL PRAKTIKUM

E.1 Langkah 1 – Membuat UI Panel Kontrol

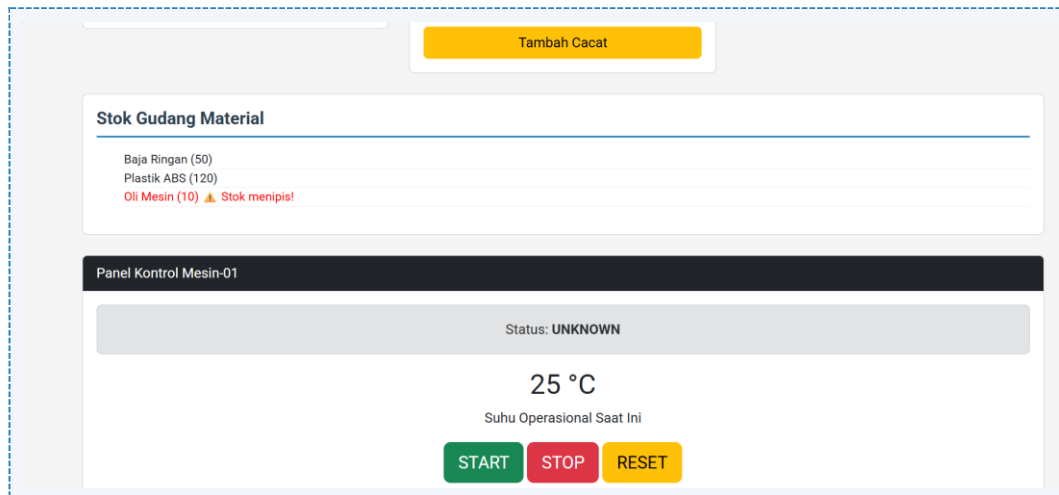

```

<!-- Simulasi Panel Kontrol Mesin (Meeting 6) -->
<div class="card mt-4">
  <div class="card-header bg-dark text-white">Panel Kontrol Mesin-
01</div>
  <div class="card-body text-center">
    <!-- Indikator Status -->
    <div id="statusIndicator" class="alert alert-secondary" role="alert">
      Status: <strong>UNKNOWN</strong>
    </div>

    <h2 id="suhuMesin">25 °C</h2>
    <p>Suhu Operasional Saat Ini</p>
    <div class="d-grid gap-2 d-md-block">
      <button id="btnStart" class="btn btn-success btn-
lg">START</button>
      <button id="btnStop" class="btn btn-danger btn-lg">STOP</button>
      <button id="btnReset" class="btn btn-warning btn-
lg">RESET</button>
    </div>
  </div>
</div>

```

Screenshot Hasil:



Gambar E.1 – Hasil Langkah 1

E.2 Langkah 2 – Seleksi Elemen di JavaScript

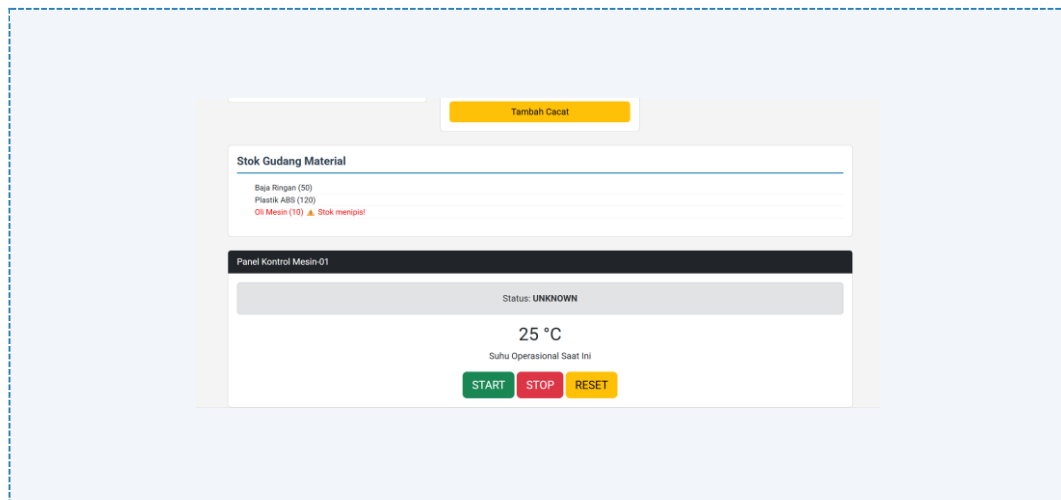
```

// 1. Seleksi Elemen DOM
const btnStart = document.getElementById('btnStart');
const btnStop = document.getElementById('btnStop');
const btnReset = document.getElementById('btnReset');
const statusIndicator = document.getElementById('statusIndicator');

```

```
const suhuMesin = document.getElementById('suhuMesin');
const teksStatus = statusIndicator.querySelector('strong'); // Mengambil
elemen <strong>
di dalam alert
// Variabel State
let suhu = 25;
let intervalId = null; // Untuk menyimpan ID timer
```

Screenshot Hasil:



Gambar E.2 – Hasil Langkah 2

E.3 Langkah 3 – Menambahkan Event Listener (Click)

```
// Event Listener Tombol START
btnStart.addEventListener('click', function() {
  // Ubah UI Status
  statusIndicator.className = 'alert alert-success'; // Ganti class Bootstrap
  (Hijau)
  teksStatus.innerText = 'RUNNING';

  // Logika simulasi kenaikan suhu
  intervalId = setInterval(function() {
    suhu += 1;
    suhuMesin.innerText = suhu + " °C";

    // Peringatan jika suhu overheating
    if (suhu > 80) {
      statusIndicator.className = 'alert alert-danger';
      teksStatus.innerText = 'OVERHEAT WARNING';
      suhuMesin.style.color = 'red';
    }
  }, 1000);
});
```

```

    }, 1000); // Jalankan setiap 1 detik

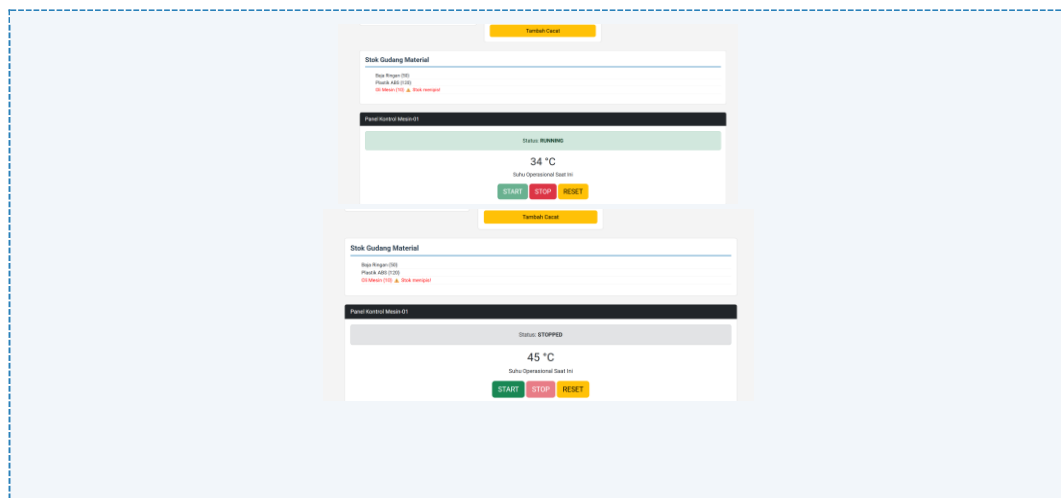
    // Matikan tombol Start agar tidak double click
    btnStart.disabled = true;
    btnStop.disabled = false;
  });
  // Event Listener Tombol STOP
  btnStop.addEventListener('click', function() {
    clearInterval(intervalId); // Hentikan timer kenaikan suhu
    statusIndicator.className = 'alert alert-secondary';
    teksStatus.innerHTML = 'STOPPED';

    // Reset tombol
    btnStart.disabled = false;
    btnStop.disabled = true;
  });
  // Event Listener Tombol RESET
  btnReset.addEventListener('click', function() {
    clearInterval(intervalId);
    suhu = 25;
    suhuMesin.innerHTML = suhu + " °C";
    suhuMesin.style.color = 'black';
    statusIndicator.className = 'alert alert-secondary';
    teksStatus.innerHTML = 'UNKNOWN';

    btnStart.disabled = false;
    btnStop.disabled = true;
  });

```

Screenshot Hasil:



Gambar E.3 – Hasil Langkah 3

E.4 Langkah 4 – Event Input (Validasi Real-Time)

HTML

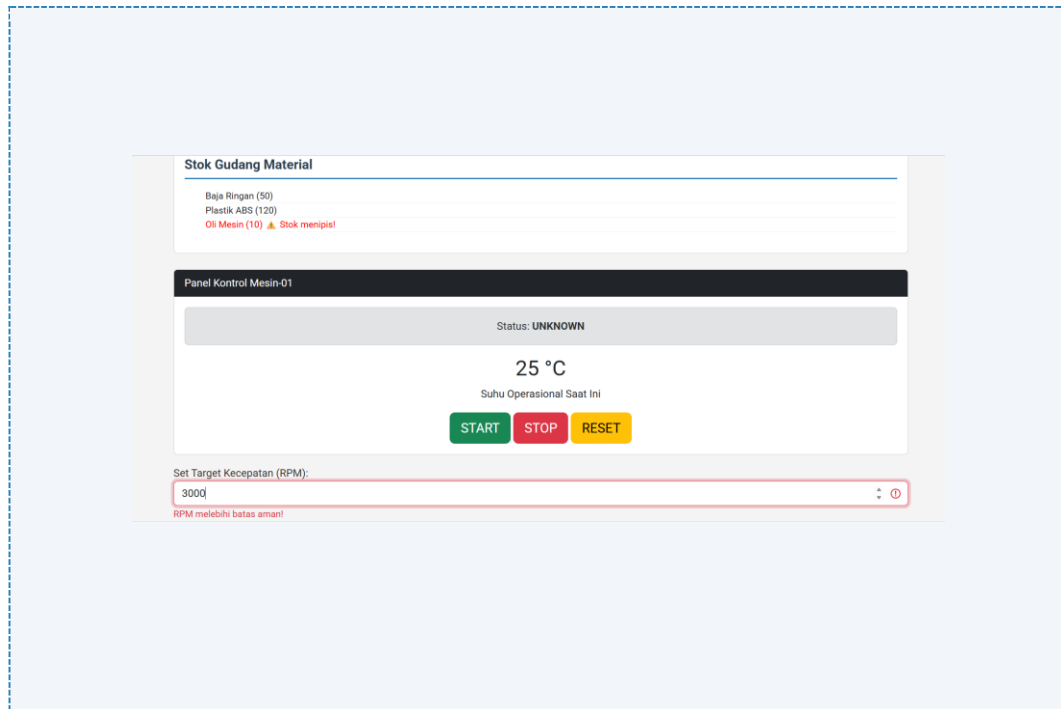
```
<div class="mt-3">
  <label>Set Target Kecepatan (RPM):</label>
  <input type="number" id="inputRPM" class="form-control"
placeholder="Max 2000 RPM">
  <small id="pesanError" class="text-danger d-none">RPM melebihi batas
aman!</small>
</div>
```

JAVASCRIPT

```
const inputRPM = document.getElementById('inputRPM');
const pesanError = document.getElementById('pesanError');
inputRPM.addEventListener('input', function() {
  let val = parseInt(this.value);

  if (val > 2000) {
    pesanError.classList.remove('d-none'); // Tampilkan pesan error
    this.classList.add('is-invalid'); // Berikan border merah Bootstrap
  } else {
    pesanError.classList.add('d-none'); // Sembunyikan pesan
    this.classList.remove('is-invalid');
  }
});
```

Screenshot Hasil:



Gambar E.4 – Hasil Langkah 4

F. LATIHAN DAN TUGAS MANDIRI

F.1 Latihan 1 – DOM Styling

Buatlah tombol tambahan bernama "Maintenance Mode". Saat diklik, background dari card menjadi abu-abu (`bg-light`), dan teks status berubah menjadi "MAINTENANCE".

Jawaban / Kode Solusi:

HTML

```
<button class="btn btn-secondary btn-lg"
onclick="maintenanceMode()">Maintenance Mode</button>
```

JAVASCRIPT

// LATIHAN 1

```
function maintenanceMode() {
  clearInterval(intervalId);
```

// Ubah teks status

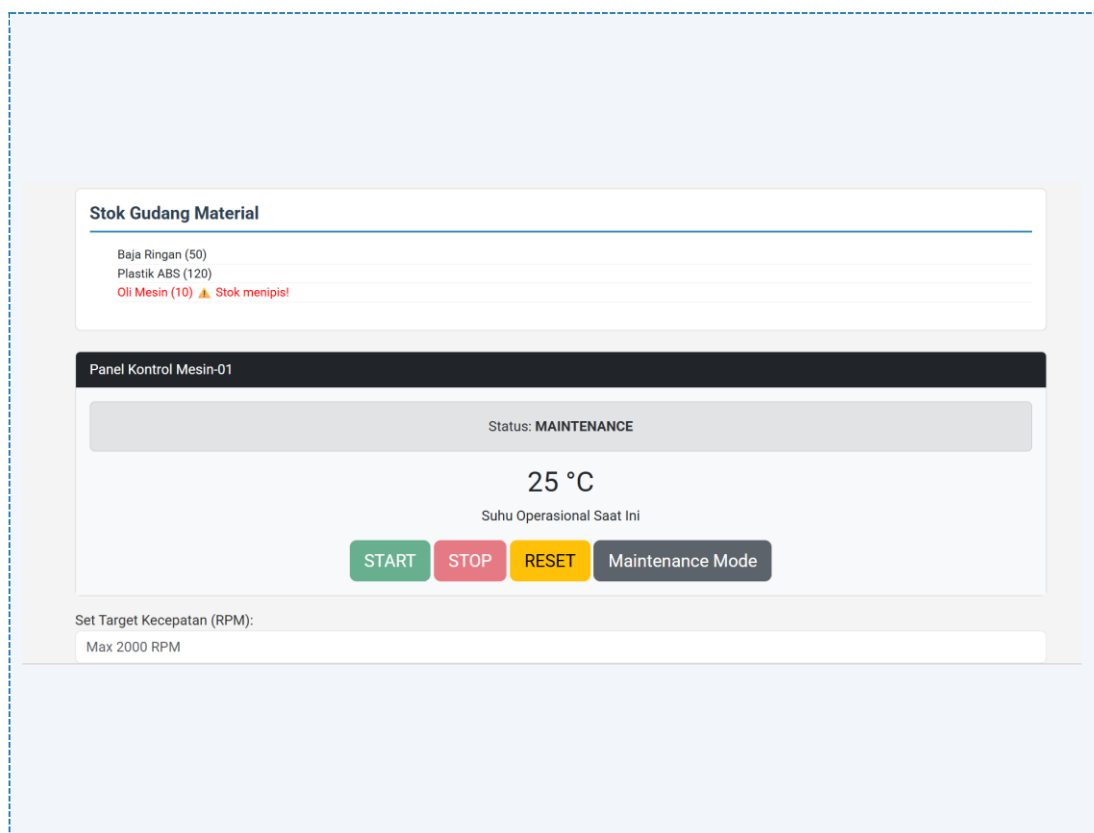
```
teksStatus.innerText = "MAINTENANCE";
```

// Ubah warna alert indicator

```
statusIndicator.className = "alert alert-secondary";
```

```
// Ambil card Panel Kontrol secara spesifik lewat statusIndicator
let cardBody = statusIndicator.closest('.card-body');
cardBody.classList.add('bg-light');

// Reset tombol
btnStart.disabled = true;
btnStop.disabled = true;
}
```



Gambar F.1 – Hasil Latihan 1

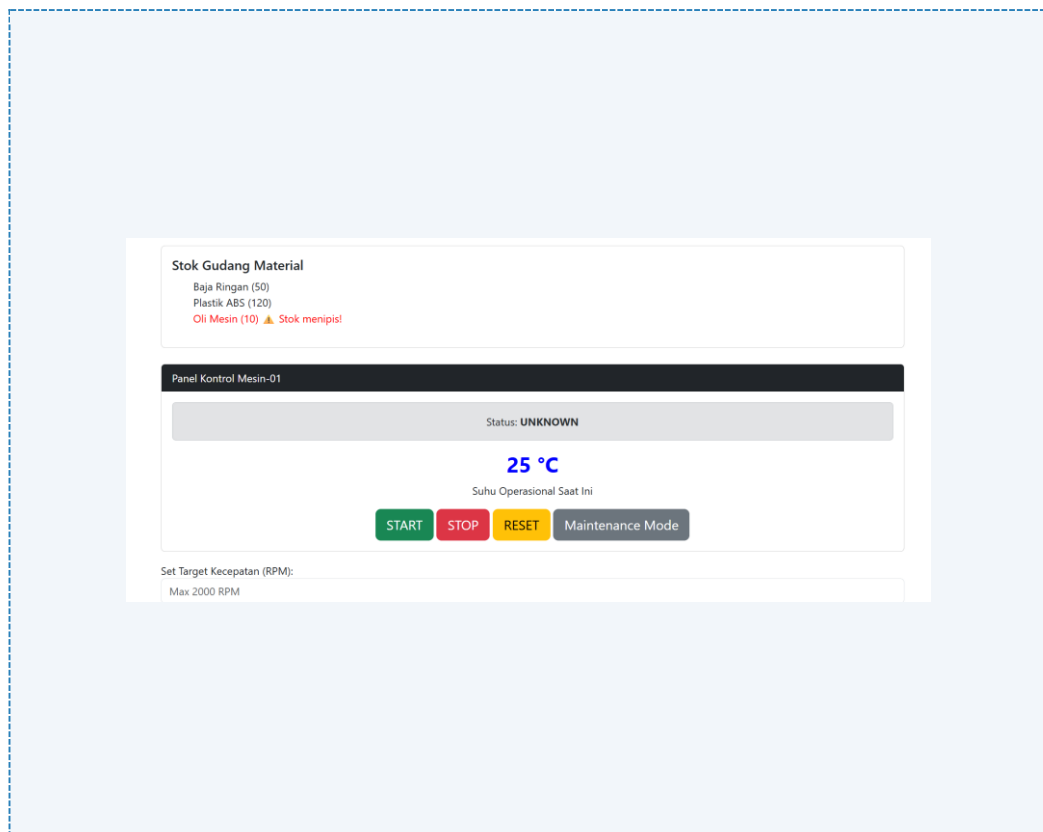
F.2 Latihan 2 – Mouse Events

Tambahkan efek saat mouse diarahkan (`mouseover`) ke teks suhu mesin, suhu tersebut menjadi tebal dan berwarna biru. Saat mouse keluar (`mouseout`), kembali normal.

Jawaban / Kode Solusi:

```
// LATIHAN 2 - Mouseover Suhu Mesin
suhuMesin.addEventListener('mouseover', function() {
    suhuMesin.style.fontWeight = 'bold';
    suhuMesin.style.color = 'blue';
});

suhuMesin.addEventListener('mouseout', function() {
    suhuMesin.style.fontWeight = 'normal';
    suhuMesin.style.color = 'black';
});
```



Gambar F.2 – Hasil Latihan 2

F.3 Tugas Proyek Mini – Kalkulator Tagihan Listrik

Deskripsi proyek mini:

- Buat UI dengan 2 input: "Daya (Watt)" dan "Jam Pemakaian".
- Saat user mengetik di salah satu input, secara real-time (tanpa tombol submit) tampilkan hasil "Total kWh" dan "Estimasi Biaya (Rp 1.500/kWh)" di elemen `di bawahnya.

Penjelasan pendekatan/solusi yang digunakan:

Pada proyek mini Kalkulator Tagihan Listrik, pendekatan yang dilakukan dimulai dari seleksi elemen DOM menggunakan `document.getElementById()` untuk mengambil elemen input Daya, input Jam, dan elemen hasil agar bisa dimanipulasi melalui JavaScript. Setelah elemen berhasil diambil, ditambahkan event listener dengan event input pada kedua input tersebut sehingga setiap kali user mengetik, function `hitungListrik()` langsung dipanggil secara real-time tanpa perlu menekan tombol apapun.

Di dalam function `hitungListrik()`, ditambahkan validasi input menggunakan `isNaN()` dan pengecekan nilai negatif untuk memastikan perhitungan hanya dilakukan jika input valid. Jika input tidak valid, elemen hasil disembunyikan kembali menggunakan `classList.add('d-none')`. Jika valid, dilakukan perhitungan konversi Watt ke kWh dengan rumus $(\text{daya} * \text{jam}) / 1000$ kemudian dikalikan tarif Rp 1.500 untuk mendapatkan estimasi biaya.

Kode Utama:

```
HTML
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Perhitungan Lingkaran</title>

  <!-- Bootstrap -->
  <link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css"
rel="stylesheet">

  <!-- CSS tambahan -->
  <link rel="stylesheet" href="latihan1.css">
</head>

<body>

<!-- NAVBAR -->
<nav class="navbar navbar-expand-lg navbar-dark bg-dark">
  <div class="container-fluid">
    <a class="navbar-brand" href="">TekInd Systems</a>
  </div>
</nav>
<div class="container mt-4">
<!-- PROYEK MINI - Kalkulator Tagihan Listrik -->
<div class="card mt-4">
  <div class="card-header bg-primary text-white">
    Kalkulator Tagihan Listrik
```



```

</div>
<div class="card-body">
  <div class="row">
    <div class="col-md-6">
      <label>Daya (Watt)</label>
      <input type="number" id="inputDaya" class="form-control mb-2"
placeholder="Masukkan daya (Watt)">
    </div>
    <div class="col-md-6">
      <label>Jam Pemakaian</label>
      <input type="number" id="inputJam" class="form-control mb-2"
placeholder="Masukkan jam pemakaian">
    </div>
  </div>
  <div id="hasilListrik" class="alert alert-info mt-3 d-none">
    <p class="mb-1">Total kWh: <strong id="totalKwh"></strong></p>
    <p class="mb-0">Estimasi Biaya: <strong
id="totalBiaya"></strong></p>
  </div>
</div>
</div>
</div>

<script src="Kalkulator Tagihan Listrik.js"></script>

</body>
</html>

```

JAVASCRIPT

```

// PROYEK MINI - Kalkulator Tagihan Listrik
const inputDaya = document.getElementById('inputDaya');
const inputJam = document.getElementById('inputJam');
const hasilListrik = document.getElementById('hasilListrik');
const totalKwh = document.getElementById('totalKwh');
const totalBiaya = document.getElementById('totalBiaya');

function hitungListrik() {
  let daya = parseFloat(inputDaya.value);
  let jam = parseFloat(inputJam.value);

  if (isNaN(daya) || isNaN(jam) || daya <= 0 || jam <= 0) {
    hasilListrik.classList.add('d-none');
    return;
  }
}

```

```
let kwh = (daya * jam) / 1000;  
let biaya = kwh * 1500;  
  
totalKwh.innerText = kwh.toFixed(2) + " kWh";  
totalBiaya.innerText = "Rp " + biaya.toLocaleString('id-ID');  
  
hasilListrik.classList.remove('d-none');  
}  
  
inputDaya.addEventListener('input', hitungListrik);  
inputJam.addEventListener('input', hitungListrik);
```

TekInd Systems

Kalkulator Tagihan Listrik

Daya (Watt) 900

Jam Pemakaian 7

Total kWh: 6.30 kWh
Estimasi Biaya: Rp 9.450

Gambar F.4 – Hasil Proyek Mini

G. PEMBAHASAN

G.1 Analisis Kesesuaian Hasil dengan Teori

Pertama, dari sisi teori DOM Manipulation, kode yang dibuat sudah sesuai karena menggunakan `document.getElementById()` untuk mengambil elemen HTML dan memanipulasinya melalui JavaScript. Hal ini sesuai dengan konsep DOM (Document Object Model) dimana JavaScript berinteraksi dengan elemen HTML melalui seleksi elemen terlebih dahulu sebelum melakukan perubahan.

Kedua, dari sisi teori Event Handling, penggunaan `addEventListener('input', hitungListrik)` sudah sesuai dengan konsep event-driven programming dimana program merespons aksi yang dilakukan user. Event input dipilih karena bersifat real-time sehingga function langsung dipanggil setiap kali user mengetik tanpa menunggu tombol ditekan, berbeda dengan event click yang hanya merespons saat tombol diklik.

Ketiga, dari sisi teori Function, pembuatan `hitungListrik()` sudah sesuai dengan konsep function declaration dimana logika perhitungan dikelompokkan dalam satu function yang dapat dipanggil dari berbagai tempat, baik dari event listener maupun dari tombol melalui onclick.

G.2 Kendala yang Ditemukan dan Solusinya

Kendala / Error yang Ditemukan	Solusi yang Diterapkan
Kendala yang sering ditemukan adalah lupa menambahkan return berupa object yang berisi dua nilai sekaligus yaitu luas dan keliling, karena biasanya function hanya mengembalikan satu nilai saja.	Mengembalikan dua nilai sekaligus dengan membungkusnya dalam sebuah object menggunakan kurung kurawal
Tambah Job bisa diklik berkali-kali sehingga job J04 masuk ke array lebih dari satu kali.	Menonaktifkan tombol setelah job baru berhasil ditambahkan menggunakan disabled, sehingga tombol hanya bisa diklik satu kali saja.
Tombol Tambah Cacat tidak bisa diklik karena function <code>tambahData()</code> belum didefinisikan di JS.	Mendefinisikan function <code>tambahData()</code> dan <code>renderListCacat()</code> di JS. <code>renderListCacat()</code> bertugas mengosongkan list lama lalu merender ulang seluruh isi array dari awal, sehingga setiap kali ada penambahan data tampilan selalu sinkron dengan isi array.

G.3 Kaitan dengan Penerapan di Industri

Ketiga solusi yang dipakai dalam praktikum ini sebetulnya mencerminkan hal-hal yang memang terjadi di dunia industri nyata. Pertama, soal return object di function `hitungLingkaran()`. dalam pekerjaan engineer di industri, jarang sekali satu perhitungan hanya menghasilkan satu angka saja. Misalnya waktu menghitung performa mesin, hasilnya bisa sekaligus berupa efisiensi energi dan efisiensi waktu, jadi masuk akal kalau satu function langsung mengembalikan dua nilai sekaligus daripada hitung dua kali. Kedua, soal tombol Tambah Job yang dibatasi supaya tidak bisa diklik berkali-kali — ini sebenarnya mirip dengan aturan di rantai produksi dimana satu pekerjaan tidak boleh dijadwalkan dua kali ke mesin yang sama, karena kalau sampai terjadi maka jadwal produksi bisa kacau dan mesin terpakai sia-sia. Ketiga, soal function pencatatan cacat yang langsung memperbarui tampilan setiap ada data baru — di pabrik sungguhan, data cacat tidak boleh menunggu sampai akhir shift baru dilaporkan karena supervisor butuh tahu kondisi kualitas produksi saat itu juga supaya bisa langsung ambil keputusan. Jadi ketiga solusi koding ini bukan sekadar menyelesaikan masalah teknis, tapi juga mencerminkan cara berpikir yang memang dibutuhkan dalam pengelolaan produksi dan kualitas di industri.

H. KESIMPULAN

1	Kode praktikum berhasil menerapkan konsep DOM Manipulation (getElementById()), Event Handling (addEventListener('input')), dan Function Declaration (hitungListrik()) sesuai prinsip event-driven programming untuk interaksi real-time.
2	Masalah return multiple value diatasi dengan object return, duplikasi data dicegah dengan tombol disabled, dan fungsi undefined diperbaiki dengan definisi tambahData() serta renderListCacat() untuk sinkronisasi tampilan-array.
3	

I. DAFTAR PUSTAKA

No.	Referensi (Format APA 7th Edition)
1.	Galahartlambang, Y., Khotiah, T., & Jumain. (2021). Analisa Performa Aplikasi Web Berbasis Manipulasi DOM dan Virtual DOM. Seminar Nasional Inovasi Teknologi (INOTEK), 36-40.
2.	Rosidi, H. (2025). Sistem Antrian Cucian Mobil Berbasis Website Menggunakan Konsep CRM dan Penerapan UML [Skripsi]. Universitas Bina Sarana Informatika.
3.	

J. LEMBAR PENILAIAN DAN PENGESAHAN

RUBRIK PENILAIAN LAPORAN					
Aspek Penilaian		Bobot			
No.	Aspek Penilaian	Bobot (%)	Nilai (0-100)	Skor Akhir	Catatan Penilai
1	Kelengkapan Isi (Semua bagian A–I terisi lengkap)	20%			
2	Kualitas Dasar Teori (Ditulis dengan bahasa sendiri, relevan, minimal 3 paragraf)	15%			
3	Dokumentasi Langkah Kerja (Kode & Screenshot sesuai, jelas, dan terbaca)	25%			
4	Tugas/Latihan Mandiri (Semua latihan & proyek mini dikerjakan dan berfungsi)	20%			
5	Pembahasan & Analisis (Kritis, mendalam, dan relevan dengan industri)	15%			
6	Kesimpulan & Tata Bahasa (Spesifik, rapi, mengikuti format yang ditentukan)	5%			
TOTAL NILAI AKHIR		100%			

Mahasiswa	Rekan Tim	Diperiksa oleh Asisten / Dosen
		
Saprina Melita Sari NIM:23051430012	Ardita Natalia NIM:23051430022	Dr. Eng. Ir. Aji Ery Burhandenny, S.T., M.AIT. Tanggal:

MODUL 7

A. IDENTITAS & INFORMASI MODUL

Mata Kuliah	Aplikasi Web dan Mobile
Kode Modul	MODUL 7 (Pertemuan 7)
Topik Utama	Validation & Local Storage
Sub-Topik	Validasi form tingkat lanjut dan penyimpanan data persisten di browser (Client-side Storage).
Durasi Praktikum	340 Menit
Tanggal Praktikum	3 April 2026
Nama Mahasiswa	Saprina Melita Sari
NIM	23051430012
Kelas	A

B. TUJUAN PEMBELAJARAN

1	Menerapkan validasi form sebelum data dikirim untuk mencegah data kotor.
2	Memahami cara kerja Web Storage API (LocalStorage).
3	Mampu menyimpan, mengambil, mengupdate, dan menghapus (CRUD sederhana) data di LocalStorage.
4	Membuat aplikasi "Buku Harian Produksi" sederhana yang datanya tidak hilang saat browser direfresh.

C. ALAT DAN BAHAN

No.	Nama Alat / Bahan / Aplikasi	Versi / Spesifikasi	Keterangan
1	Visual Studio Code (VS Code)	Versi 1.8x ke atas	Code Editor
2	Google Chrome / Browser Modern	Versi terbaru	Testing & Debugging

3	Node.js & npm	Node 18+ / npm 9+	Runtime JS (Modul tertentu)
5	HTML	-	Struktur
6	CSS		Styling

D. DASAR TEORI

Pengembangan aplikasi web modern mensyaratkan penggunaan teknik validasi form dan penyimpanan data yang efisien di sisi klien agar data yang diolah tetap bersih, akurat, dan persisten. *Validasi form* merupakan proses pemeriksaan input yang dilakukan sebelum data dikirimkan atau diproses lebih lanjut. Tujuan validasi form adalah untuk memastikan bahwa data yang diberikan oleh pengguna telah lengkap, sesuai format, dan bebas dari kesalahan sehingga mencegah terjadinya *data kotor* atau kesalahan dalam pengolahan. Validasi ini dapat dilakukan secara *client-side* menggunakan JavaScript dan HTML5 untuk memberikan respons langsung tanpa harus menunggu proses server, sehingga meningkatkan pengalaman pengguna dan performa aplikasi karena meminimalkan permintaan yang tidak perlu ke server.

Selain validasi form, aplikasi web juga sering membutuhkan mekanisme penyimpanan data agar bisa bertahan antar sesi penggunaan. *Web Storage API*, yang merupakan bagian dari teknologi HTML5, menyediakan dua jenis penyimpanan di sisi klien yaitu *localStorage* dan *sessionStorage*. *LocalStorage* merupakan fasilitas penyimpanan *key-value* yang memungkinkan data *persisten* tersimpan di browser tanpa batas waktu, bahkan setelah jendela atau tab browser ditutup, sehingga cocok untuk aplikasi yang membutuhkan data tidak hilang saat halaman direfresh atau browser ditutup. Sebaliknya, *sessionStorage* hanya menyimpan data selama sesi tab aktif berlangsung. *LocalStorage* memudahkan developer menyimpan, mengambil, memperbarui, dan menghapus data sederhana tanpa perlu backend atau database, sehingga sering digunakan pada aplikasi berskala kecil hingga menengah yang dijalankan sepenuhnya di sisi klien.

Dalam *aplikasi Buku Harian Produksi*, *localStorage* digunakan untuk menyimpan data produksi harian yang berupa array objek JSON, yang kemudian diolah dan ditampilkan kembali pada antarmuka tabel agar data tidak hilang setelah browser direfresh. Metode ini mengajarkan cara melakukan operasi CRUD sederhana pada data yang tersimpan di *localStorage* sehingga aplikasi tetap responsif dan data tetap tersedia untuk pengguna *tanpa memerlukan server*. Implementasi ini juga memperkenalkan konsep serialisasi dan deserialisasi objek JavaScript ke format JSON agar data kompleks dapat tersimpan dalam *localStorage* dengan benar.

D.1 Konsep Utama

Log Produksi Harian mengusung konsep utama pembangunan aplikasi pencatatan produksi yang interaktif, valid, dan persisten di browser. Setiap data produksi dimasukkan melalui form HTML yang telah dilengkapi dengan validasi di sisi klien untuk memastikan data lengkap, sesuai format, dan bebas dari kesalahan sebelum disimpan. Data produksi kemudian disimpan secara persisten menggunakan LocalStorage, memungkinkan operasi CRUD sederhana sehingga data tetap ada meskipun halaman direfresh. Semua data disimpan sebagai array objek, yang dapat dimanipulasi menggunakan fungsi filter untuk menampilkan operator tertentu dan sort untuk mengurutkan jumlah produksi dari terbesar ke terkecil, kemudian ditampilkan secara dinamis di tabel menggunakan manipulasi DOM. Interaktivitas aplikasi juga ditingkatkan melalui fitur reset form setelah submit, tombol hapus data untuk membersihkan LocalStorage dan tabel, serta input pencarian dan tombol sortir yang memudahkan pengguna dalam menganalisis dan meninjau riwayat produksi.

D.2 Keterkaitan dengan Teknik Industri

Pencatatan jumlah produksi harian, pemantauan kinerja operator, dan pengelolaan shift kerja merupakan bagian penting dari operasional pabrik untuk meningkatkan efisiensi dan efektivitas produksi. Dengan aplikasi ini, data produksi dapat dianalisis secara cepat, operator yang kurang produktif dapat diidentifikasi, dan keputusan perbaikan proses dapat diambil berdasarkan informasi real-time. Fitur filter dan sortir memudahkan manajer produksi untuk meninjau performa individu atau shift tertentu, sejalan dengan praktik *quality control*, *performance monitoring*, dan *production planning* dalam teknik industri.

E. LANGKAH KERJA DAN HASIL PRAKTIKUM

E.1 Langkah 1 – Membuat Form Input Produksi

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Log Produksi Harian</title>
  <link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css"
rel="stylesheet">
</head>
<body class="container mt-5">

  <div class="row">
    <!-- Kolom Kiri: Form Input -->
    <div class="col-md-4">
      <div class="card">
        <div class="card-header">Input Data</div>
        <div class="card-body">
          <form id="formProduksi">
            <div class="mb-3">
              <label>Tanggal</label>
              <input type="date" id="tanggal" class="form-control"
required>
            </div>
            <div class="mb-3">
              <label>Nama Operator</label>
              <input type="text" id="operator" class="form-control"
placeholder="Nama Lengkap" required>
            </div>
            <div class="mb-3">
              <label>Shift</label>
              <select id="shift" class="form-select" required>
                <option value="Pagi">Pagi</option>
                <option value="Siang">Siang</option>
                <option value="Malam">Malam</option>
              </select>
            </div>
            <div class="mb-3">
              <label>Jumlah Produksi (Unit)</label>
              <input type="number" id="jumlah" class="form-control"
min="1" required>
            </div>
            <button type="submit" class="btn btn-primary w-
100">Simpan Log</button>
            <button type="button" id="btnHapusSemua" class="btn btn-
danger w-100 mt-2">Hapus Semua Data</button>

```

```

        </form>
      </div>
    </div>
  </div>

  <!-- Kolom Kanan: Tabel Data -->
  <div class="col-md-8">
    <h3>Riwayat Produksi</h3>
    <table class="table table-bordered table-striped">
      <thead class="table-dark">
        <tr>
          <th>Tanggal</th>
          <th>Operator</th>
          <th>Shift</th>
          <th>Jumlah</th>
          <th>Aksi</th>
        </tr>
      </thead>
      <tbody id="tabelBody">
        <!-- Data akan masuk di sini lewat JS -->
      </tbody>
    </table>
  </div>
</div>

<script src="log_produksi.js"></script>
</body>
</html>

```

Screenshot Hasil:

The screenshot displays a web application interface. On the left, there is a form titled 'Input Data' with the following fields: 'Tanggal' (Date) with a placeholder 'mm/dd/yyyy' and a calendar icon, 'Nama Operator' (Full Name), 'Shift' (a dropdown menu currently showing 'Pagi'), and 'Jumlah Produksi (Unit)'. Below these fields are two buttons: a blue 'Simpan Log' (Save Log) button and a red 'Hapus Semua Data' (Delete All Data) button. On the right, there is a section titled 'Riwayat Produksi' (Production History) which contains a table with the following headers: 'Tanggal', 'Operator', 'Shift', 'Jumlah', and 'Aksi'.

Gambar E.1 – Hasil Langkah 1

E.2 Langkah 2 – Logika JavaScript (Simpan & Baca)

```
// 1. Seleksi Elemen
const formProduksi = document.getElementById('formProduksi');
const tabelBody = document.getElementById('tabelBody');
const btnHapusSemua = document.getElementById('btnHapusSemua');

// Kunci untuk LocalStorage
const STORAGE_KEY = 'DATA_PRODUKSI_INDUSTRI';

// Fungsi Load Data saat halaman dibuka
document.addEventListener('DOMContentLoaded', function() {
  loadDataFromStorage();
});

// 2. Event Listener: Submit Form
formProduksi.addEventListener('submit', function(event) {
  event.preventDefault(); // Mencegah refresh halaman

  // Ambil Value dari Form
  const tanggal = document.getElementById('tanggal').value;
  const operator = document.getElementById('operator').value;
  const shift = document.getElementById('shift').value;
  const jumlah = document.getElementById('jumlah').value;

  // Validasi Sederhana (JavaScript)
  if (jumlah <= 0) {
    alert("Jumlah produksi harus lebih dari 0!");
    return;
  }

  // Buat Object Data
  const dataBaru = {
    id: Date.now(), // ID unik berdasarkan waktu
    tanggal: tanggal,
    operator: operator,
    shift: shift,
    jumlah: parseInt(jumlah)
  };

  // Simpan ke LocalStorage
  saveData(dataBaru);

  // Reset Form
  formProduksi.reset();
```

```
// Refresh Tampilan Tabel
loadDataFromStorage();
});

// 3. Fungsi Simpan ke LocalStorage
function saveData(data) {
  // Ambil data lama (jika ada), jika tidak ada array kosong
  let dataLama = JSON.parse(localStorage.getItem(STORAGE_KEY)) || [];

  // Tambah data baru ke array
  dataLama.push(data);

  // Simpan kembali ke LocalStorage (Convert ke JSON String)
  localStorage.setItem(STORAGE_KEY, JSON.stringify(dataLama));
}

// 4. Fungsi Baca & Render Tabel
function loadDataFromStorage() {
  // Ambil data
  let data = JSON.parse(localStorage.getItem(STORAGE_KEY)) || [];

  // Kosongkan tabel dulu
  tabelBody.innerHTML = "";

  // Loop data dan buat baris tabel
  data.forEach(function(item) {
    const row = document.createElement('tr');
    row.innerHTML = `
      <td>${item.tanggal}</td>
      <td>${item.operator}</td>
      <td>${item.shift}</td>
      <td>${item.jumlah}</td>
      <td>
        <button class="btn btn-sm btn-danger"
        onclick="hapusData(${item.id})">Hapus</button>
      </td>
    `;
    tabelBody.appendChild(row);
  });
}

// 5. Fungsi Hapus Data Spesifik
// Kita pasang di window agar bisa dipanggil dari inline HTML onclick
window.hapusData = function(id) {
  if(confirm('Yakin ingin menghapus log ini?')) {
    let data = JSON.parse(localStorage.getItem(STORAGE_KEY)) || [];
```

```
// Filter data: hapus item yang id-nya cocok
let dataBaru = data.filter(item => item.id !== id);

// Simpan ulang
localStorage.setItem(STORAGE_KEY, JSON.stringify(dataBaru));

// Refresh tampilan
loadDataFromStorage();
}
}

// 6. Event Hapus Semua
btnHapusSemua.addEventListener('click', function() {
  if(confirm('PERINGATAN: Semua data akan dihapus permanen!')) {
    localStorage.removeItem(STORAGE_KEY);
    loadDataFromStorage();
  }
});
```

Screenshot Hasil:

The screenshot displays a web application interface. On the left, there is a form titled 'Input Data' with the following fields: 'Tanggal' (Date) with a date picker showing 'mm/dd/yyyy', 'Nama Operator' (Full Name) with a text input, 'Shift' with a dropdown menu showing 'Pagi', and 'Jumlah Produksi (Unit)' (Production Quantity) with a text input. Below these fields are two buttons: 'Simpan Log' (Save Log) in blue and 'Hapus Semua Data' (Delete All Data) in red. On the right, there is a table titled 'Riwayat Produksi' (Production History). The table has a dark header with columns: 'Tanggal', 'Operator', 'Shift', 'Jumlah', and 'Aksi'. The table body is currently empty.

Gambar E.2 – Hasil Langkah 2

F. LATIHAN DAN TUGAS MANDIRI

F.1 Latihan 1 – Filter Data

Tambahkan fitur filter atau pencarian. Buat input "Cari Nama Operator". Saat user mengetik, tabel hanya menampilkan data operator yang namanya cocok (Gunakan `filter` array).

Jawaban / Kode Solusi:

HTML

```
<input type="text" id="inputCari" class="form-control" placeholder="Cari Nama Operator...">
```

JAVASCRIPT

// 1. Seleksi Elemen

```
const formProduksi = document.getElementById('formProduksi');
const tabelBody = document.getElementById('tabelBody');
const btnHapusSemua = document.getElementById('btnHapusSemua');
// Seleksi elemen inputCari (latihan 1)
const inputCari = document.getElementById('inputCari');
// Seleksi elemen btnSortJumlah (latihan 2)
const btnSortJumlah = document.getElementById('btnSortJumlah');
// Kunci untuk LocalStorage
const STORAGE_KEY = 'DATA_PRODUKSI_INDUSTRI';
// Fungsi Load Data saat halaman dibuka
document.addEventListener('DOMContentLoaded', function () {
  loadDataFromStorage();
});
```

// 2. Event Listener: Submit Form

```
formProduksi.addEventListener('submit', function (event) {
  event.preventDefault(); // Mencegah refresh halaman
  // Ambil Value dari Form
  const tanggal = document.getElementById('tanggal').value;
  const operator = document.getElementById('operator').value;
  const shift = document.getElementById('shift').value;
  const jumlah = document.getElementById('jumlah').value;
  // Validasi Sederhana (JavaScript)
  if (jumlah <= 0) {
    alert("Jumlah produksi harus lebih dari 0!");
    return;
  }
  // Buat Object Data
  const dataBaru = {
    id: Date.now(), // ID unik berdasarkan waktu
    tanggal: tanggal,
    operator: operator,
    shift: shift,
    jumlah: parseInt(jumlah)
  };
  // Simpan ke LocalStorage
  saveData(dataBaru);
  // Reset Form
```

```
formProduksi.reset();

// Refresh Tampilan Tabel
loadDataFromStorage();
});
// 3. Fungsi Simpan ke LocalStorage
function saveData(data) {
    // Ambil data lama (jika ada), jika tidak ada array kosong
    let dataLama = JSON.parse(localStorage.getItem(STORAGE_KEY)) || [];

    // Tambah data baru ke array
    dataLama.push(data);

    // Simpan kembali ke LocalStorage (Convert ke JSON String)
    localStorage.setItem(STORAGE_KEY, JSON.stringify(dataLama));
}
// 4. Fungsi Baca & Render Tabel
function loadDataFromStorage() {
    // Ambil data
    let data = JSON.parse(localStorage.getItem(STORAGE_KEY)) || [];

    // Kosongkan tabel dulu
    tabelBody.innerHTML = "";
    // Loop data dan buat baris tabel
    data.forEach(function (item) {
        const row = document.createElement('tr');
        row.innerHTML = `
<td>${item.tanggal}</td>
<td>${item.operator}</td>
<td>${item.shift}</td>
<td>${item.jumlah}</td>
<td>
<button class="btn btn-sm btn-danger"
onclick="hapusData(${item.id})">Hapus</button>
</td>
`;
        tabelBody.appendChild(row);
    });
}
// 5. Fungsi Hapus Data Spesifik
// Kita pasang di window agar bisa dipanggil dari inline HTML onclick
window.hapusData = function (id) {
    if (confirm('Yakin ingin menghapus log ini?')) {
        let data = JSON.parse(localStorage.getItem(STORAGE_KEY)) || [];

        // Filter data: hapus item yang id-nya cocok
```



```
let dataBaru = data.filter(item => item.id !== id);

// Simpan ulang
localStorage.setItem(STORAGE_KEY, JSON.stringify(dataBaru));

// Refresh tampilan
loadDataFromStorage();
}
}

// 6. Event Hapus Semua
btnHapusSemua.addEventListener('click', function () {
  if (confirm('PERINGATAN: Semua data akan dihapus permanen!')) {
    localStorage.removeItem(STORAGE_KEY);
    loadDataFromStorage();
  }
});

// membuat function renderTable untuk menampilkan hasil filter
inputCari(latihan 1)
function renderTable(data) {
  tabelBody.innerHTML = "";

  data.forEach(function (item) {
    const row = document.createElement('tr');
    row.innerHTML = `
      <td>${item.tanggal}</td>
      <td>${item.operator}</td>
      <td>${item.shift}</td>
      <td>${item.jumlah}</td>
      <td>
        <button class="btn btn-sm btn-danger"
          onclick="hapusData(${item.id})">Hapus</button>
      </td>
    `;
    tabelBody.appendChild(row);
  });
}

// Membuat evenListener inputCari untuk filter nama operator (latihan 1)
inputCari.addEventListener('input', function () {
  const keyword = this.value.toLowerCase();
  const data = JSON.parse(localStorage.getItem(STORAGE_KEY)) || [];

  const hasilFilter = data.filter(item =>
    item.operator.toLowerCase().includes(keyword)
  );
});
```

```
renderTable(hasilFilter);
});

// Membuat evenListener btnSortJumlah untuk mengurutkan data (latihan 2)
btnSortJumlah.addEventListener('click', function () {
  const keyword = inputCari.value.toLowerCase();
  const data = JSON.parse(localStorage.getItem(STORAGE_KEY)) || [];
  // Filter dulu sesuai kondisi input saat ini
  const hasilFilter = data.filter(item =>
    item.operator.toLowerCase().includes(keyword)
  );
  // Urutkan hasil filter
  const dataUrut = hasilFilter.sort((a, b) => b.jumlah - a.jumlah);

  renderTable(dataUrut);
});
```

The screenshot displays a web application interface for managing production history. On the left, there is an 'Input Data' form with the following fields: 'Tanggal' (Date) with a date picker showing 'mm/dd/yyyy', 'Nama Operator' (Operator Name) with a text input labeled 'Nama Lengkap', 'Shift' (Shift) with a dropdown menu showing 'Pagi', and 'Jumlah Produksi (Unit)' (Production Quantity (Unit)) with a text input. Below these fields are two buttons: 'Simpan Log' (Save Log) in blue and 'Hapus Semua Data' (Delete All Data) in red. On the right, there is a 'Riwayat Produksi' (Production History) section. It includes a search bar labeled 'Cari Nama Operator...' and a table with the following columns: 'Tanggal', 'Operator', 'Shift', 'Jumlah', and 'Aksi'.

Gambar F.1 – Hasil Latihan 1

F.2 Latihan 2 –

Tambahkan tombol "Sortir berdasarkan Jumlah (Terbesar)" yang mengurutkan tabel dari jumlah produksi terbanyak ke terkecil.

Jawaban / Kode Solusi:

```
// 1. Seleksi Elemen
const formProduksi = document.getElementById('formProduksi');
const tabelBody = document.getElementById('tabelBody');
const btnHapusSemua = document.getElementById('btnHapusSemua');
// Seleksi elemen inputCari (latihan 1)
const inputCari = document.getElementById('inputCari');
// Seleksi elemen btnSortJumlah (latihan 2)
const btnSortJumlah = document.getElementById('btnSortJumlah');
// Kunci untuk LocalStorage
const STORAGE_KEY = 'DATA_PRODUKSI_INDUSTRI';
// Fungsi Load Data saat halaman dibuka
document.addEventListener('DOMContentLoaded', function () {
  loadDataFromStorage();
});
// 2. Event Listener: Submit Form
formProduksi.addEventListener('submit', function (event) {
  event.preventDefault(); // Mencegah refresh halaman
  // Ambil Value dari Form
  const tanggal = document.getElementById('tanggal').value;
  const operator = document.getElementById('operator').value;
  const shift = document.getElementById('shift').value;
  const jumlah = document.getElementById('jumlah').value;
  // Validasi Sederhana (JavaScript)
  if (jumlah <= 0) {
    alert("Jumlah produksi harus lebih dari 0!");
    return;
  }
  // Buat Object Data
  const dataBaru = {
    id: Date.now(), // ID unik berdasarkan waktu
    tanggal: tanggal,
    operator: operator,
    shift: shift,
    jumlah: parseInt(jumlah)
  };
  // Simpan ke LocalStorage
  saveData(dataBaru);
  // Reset Form
  formProduksi.reset();
```

```
// Refresh Tampilan Tabel
loadDataFromStorage();
});
// 3. Fungsi Simpan ke LocalStorage
function saveData(data) {
    // Ambil data lama (jika ada), jika tidak ada array kosong
    let dataLama = JSON.parse(localStorage.getItem(STORAGE_KEY)) || [];

    // Tambah data baru ke array
    dataLama.push(data);

    // Simpan kembali ke LocalStorage (Convert ke JSON String)
    localStorage.setItem(STORAGE_KEY, JSON.stringify(dataLama));
}
// 4. Fungsi Baca & Render Tabel
function loadDataFromStorage() {
    // Ambil data
    let data = JSON.parse(localStorage.getItem(STORAGE_KEY)) || [];

    // Kosongkan tabel dulu
    tabelBody.innerHTML = "";
    // Loop data dan buat baris tabel
    data.forEach(function (item) {
        const row = document.createElement('tr');
        row.innerHTML = `
<td>${item.tanggal}</td>
<td>${item.operator}</td>
<td>${item.shift}</td>
<td>${item.jumlah}</td>
<td>
<button class="btn btn-sm btn-danger"
onclick="hapusData(${item.id})">Hapus</button>
</td>
`;
        tabelBody.appendChild(row);
    });
}
// 5. Fungsi Hapus Data Spesifik
// Kita pasang di window agar bisa dipanggil dari inline HTML onclick
window.hapusData = function (id) {
    if (confirm('Yakin ingin menghapus log ini?')) {
        let data = JSON.parse(localStorage.getItem(STORAGE_KEY)) || [];

        // Filter data: hapus item yang id-nya cocok
        let dataBaru = data.filter(item => item.id !== id);
```

```
// Simpan ulang
localStorage.setItem(STORAGE_KEY, JSON.stringify(dataBaru));

// Refresh tampilan
loadDataFromStorage();
}
}

// 6. Event Hapus Semua
btnHapusSemua.addEventListener('click', function () {
  if (confirm('PERINGATAN: Semua data akan dihapus permanen!')) {
    localStorage.removeItem(STORAGE_KEY);
    loadDataFromStorage();
  }
});

// membuat function renderTable untuk menampilkan hasil filter
inputCari(latihan 1)
function renderTable(data) {
  tabelBody.innerHTML = "";

  data.forEach(function (item) {
    const row = document.createElement('tr');
    row.innerHTML = `
      <td>${item.tanggal}</td>
      <td>${item.operator}</td>
      <td>${item.shift}</td>
      <td>${item.jumlah}</td>
      <td>
        <button class="btn btn-sm btn-danger"
          onclick="hapusData(${item.id})">Hapus</button>
      </td>
    `;
    tabelBody.appendChild(row);
  });
}

// Membuat evenListener inputCari untuk filter nama operator (latihan 1)
inputCari.addEventListener('input', function () {
  const keyword = this.value.toLowerCase();
  const data = JSON.parse(localStorage.getItem(STORAGE_KEY)) || [];

  const hasilFilter = data.filter(item =>
    item.operator.toLowerCase().includes(keyword)
  );

  renderTable(hasilFilter);
});
```

```
// Membuat evenListener btnSortJumlah untuk mengurutkan data (latihan 2)
btnSortJumlah.addEventListener('click', function () {
  const keyword = inputCari.value.toLowerCase();
  const data = JSON.parse(localStorage.getItem(STORAGE_KEY)) || [];
  // Filter dulu sesuai kondisi input saat ini
  const hasilFilter = data.filter(item =>
    item.operator.toLowerCase().includes(keyword)
  );
  // Urutkan hasil filter
  const dataUrut = hasilFilter.sort((a, b) => b.jumlah - a.jumlah);

  renderTable(dataUrut);
});
```

Input Data

Tanggal
mm/dd/yyyy

Nama Operator
Nama Lengkap

Shift
Pagi

Jumlah Produksi (Unit)

Simpan Log

Hapus Semua Data

Riwayat Produksi

Cari Nama Operator... Sortir berdasarkan Jumlah (Terbesar)

Tanggal	Operator	Shift	Jumlah	Aksi
---------	----------	-------	--------	------

Gambar F.2 – Hasil Latihan 2

F.3 Tugas Proyek Mini – Aplikasi Ckecklist 5S

Deskripsi proyek mini:

Buat aplikasi checklist sederhana audit 5S (Seiri, Seiton, Seiso, Seiketsu, Shitsuke).

- a. Form berisi 5 checkbox (boolean) dan input nama auditor.
- b. Saat disimpan, hitung skor (jumlah checklist dicentang / 5 * 100).
- c. Simpan riwayat audit ke LocalStorage dan tampilkan di tabel (Tanggal, Auditor, Skor).

Penjelasan pendekatan/solusi yang digunakan:

1. Penyimpanan Data
Solusi: Riwayat audit disimpan di browser agar tetap ada meskipun halaman direfresh.
Kode/Teknis: Menggunakan LocalStorage (localStorage.setItem / localStorage.getItem) untuk menyimpan array objek JSON.
2. Form Input Dinamis
Solusi: Form digunakan untuk memasukkan nama auditor dan menandai checklist 5S.
Kode/Teknis: Menggunakan HTML form dan input type="text" untuk nama auditor, input type="checkbox" untuk checklist.
3. Pengambilan Status Checklist
Solusi: Status checkbox dibaca sebagai benar/salah untuk menghitung skor.
Kode/Teknis: Menggunakan properti .checked di checkbox, menghasilkan tipe Boolean (true/false).
4. Perhitungan Skor Otomatis
Solusi: Skor dihitung dari jumlah checklist yang dicentang dibagi 5, dikonversi ke persen.
Kode/Teknis: Array checklist di-filter untuk menghitung yang true, hasilnya dikalikan 100 → tipe data Number.
5. Penambahan Data Baru
Solusi: Data audit baru ditambahkan ke riwayat dan disimpan kembali di LocalStorage.
Kode/Teknis: Menggunakan array push() untuk menambah objek {tanggal, auditor, skor} ke array dataAudit, lalu disimpan dengan LocalStorage.
6. Tabel Riwayat Dinamis
Solusi: Tabel audit diperbarui otomatis saat ada data baru atau dihapus.
Kode/Teknis: Membuat fungsi renderTabel(), menggunakan DOM manipulation (document.createElement, .innerHTML) untuk menampilkan array data.
7. Interaktivitas Form
Solusi: Form otomatis direset setelah submit agar siap input baru.
Kode/Teknis: Memanggil form.reset() setelah menambah data → membersihkan input teks dan checkbox.
8. Penghapusan Data
Solusi: Ada tombol untuk menghapus semua data audit sekaligus, membersihkan LocalStorage dan tabel.
Kode/Teknis: Mengosongkan array dataAudit = [], localStorage.removeItem(), lalu memanggil renderTabel() untuk update tampilan.

Kode Utama:

```
HTML
<!DOCTYPE html>
<html lang="id">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Audit 5S</title>
  <link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css"
rel="stylesheet">
</head>
<body class="container mt-5">

  <h2>Checklist Audit 5S</h2>
  <div class="row">
    <div class="col-md-4">
      <div class="card">
        <div class="card-header">Form Audit</div>
        <div class="card-body">
          <form id="formAudit">
            <div class="mb-3">
              <label>Nama Auditor</label>
              <input type="text" id="auditor" class="form-control"
placeholder="Masukkan Nama" required>
            </div>
            <div class="mb-3">
              <div class="form-check">
                <input type="checkbox" class="form-check-input"
id="seiri">
                <label class="form-check-label" for="seiri">Seiri
(Sort)</label>
              </div>
              <div class="form-check">
                <input type="checkbox" class="form-check-input"
id="seiton">
                <label class="form-check-label" for="seiton">Seiton (Set
in Order)</label>
              </div>
              <div class="form-check">
                <input type="checkbox" class="form-check-input"
id="seiso">
                <label class="form-check-label" for="seiso">Seiso
(Shine)</label>
              </div>
            </div>
          </form>
        </div>
      </div>
    </div>
  </div>
```



```

        <div class="form-check">
            <input type="checkbox" class="form-check-input"
id="seiketsu">
            <label class="form-check-label" for="seiketsu">Seiketsu
(Standardize)</label>
        </div>
        <div class="form-check">
            <input type="checkbox" class="form-check-input"
id="shitsuke">
            <label class="form-check-label" for="shitsuke">Shitsuke
(Sustain)</label>
        </div>
    </div>
    <button type="submit" class="btn btn-primary w-100">Simpan
Audit</button>
    <button type="button" id="btnHapusSemua" class="btn btn-
danger w-100 mt-2">Hapus Semua Data</button>
</form>
</div>
</div>
</div>
</div>

<div class="col-md-8">
    <h3>Riwayat Audit</h3>
    <table class="table table-bordered table-striped">
        <thead class="table-dark">
            <tr>
                <th>Tanggal</th>
                <th>Auditor</th>
                <th>Skor (%)</th>
            </tr>
        </thead>
        <tbody id="tabelAudit">
            <!-- Data akan muncul di sini -->
        </tbody>
    </table>
</div>
</div>

<script src="aplikasi checklist.js"></script>
</body>
</html>

JAVASCRIPT
// =====
// AUDIT 5S

```

```
// =====

// 1. Ambil elemen
const formAudit = document.getElementById('formAudit');
const tabelAudit = document.getElementById('tabelAudit');
const btnHapusSemua = document.getElementById('btnHapusSemua');

// 2. Ambil data dari LocalStorage atau inisialisasi kosong
let dataAudit = JSON.parse(localStorage.getItem('dataAudit')) || [];

// 3. Fungsi render tabel
function renderTabel() {
  tabelAudit.innerHTML = '';
  dataAudit.forEach((item, index) => {
    const row = document.createElement('tr');
    row.innerHTML = `
      <td>${item.tanggal}</td>
      <td>${item.auditor}</td>
      <td>${item.skor}</td>
    `;
    tabelAudit.appendChild(row);
  });
}

// 4. Render tabel pertama kali
renderTabel();

// 5. Event submit form
formAudit.addEventListener('submit', function(e) {
  e.preventDefault();

  const auditor = document.getElementById('auditor').value;
  const seiri = document.getElementById('seiri').checked;
  const seiton = document.getElementById('seiton').checked;
  const seiso = document.getElementById('seiso').checked;
  const seiketsu = document.getElementById('seiketsu').checked;
  const shitsuke = document.getElementById('shitsuke').checked;

  // Hitung skor
  const total = [seiri, seiton, seiso, seiketsu, shitsuke].filter(v => v).length;
  const skor = Math.round((total / 5) * 100);

  // Tambahkan ke array
  const tanggal = new Date().toLocaleDateString('id-ID');
  dataAudit.push({ tanggal, auditor, skor });
});
```

```
// Simpan ke LocalStorage
localStorage.setItem('dataAudit', JSON.stringify(dataAudit));

// Render tabel
renderTabel();

// Reset form
formAudit.reset();
});

// 6. Hapus semua data
btnHapusSemua.addEventListener('click', () => {
  if (confirm('Apakah yakin ingin menghapus semua data audit?')) {
    dataAudit = [];
    localStorage.removeItem('dataAudit');
    renderTabel();
  }
});
```

Checklist Audit 5S

Form Audit

Nama Auditor

Masukkan Nama

☐ Seiri (Sort)

☐ Seiton (Set in Order)

☐ Seiso (Shine)

☐ Seiketsu (Standardize)

☐ Shitsuke (Sustain)

Simpan Audit

Hapus Semua Data

Riwayat Audit

Tanggal	Auditor	Skor (%)
4/4/2026	MEMO	40

Gambar F.3 – Hasil Proyek Mini

G. PEMBAHASAN

G.1 Analisis Kesesuaian Hasil dengan Teori

Form input dibuat menggunakan HTML dengan tipe data yang sesuai, yaitu tanggal, teks, select, dan number, sehingga validasi input bisa dilakukan dengan benar. Data produksi disimpan dalam bentuk array objek, yang memungkinkan manipulasi lebih lanjut seperti menambah data baru, menghapus, maupun menyimpan riwayat secara persisten menggunakan `LocalStorage`. Untuk latihan 1, fitur filter data menggunakan `Array.filter()`, sehingga ketika pengguna mengetik nama operator, tabel hanya menampilkan data yang sesuai, sesuai teori filter array. Sedangkan pada latihan 2, tombol sortir mengurutkan data dari jumlah produksi terbesar ke terkecil menggunakan `Array.sort()`, yang juga sesuai teori pengurutan data numerik. Tabel riwayat produksi diperbarui secara dinamis menggunakan manipulasi DOM, sehingga setiap perubahan data—baik penambahan, filter, maupun sortir—langsung tercermin di tampilan tabel.

G.2 Kendala yang Ditemukan dan Solusinya

Kendala / Error yang Ditemukan	Solusi yang Diterapkan
Data tidak tersimpan setelah halaman direfresh.	Menggunakan <code>LocalStorage</code> untuk menyimpan array data audit sehingga tetap ada meskipun halaman dimuat ulang.
Filter data operator tidak berjalan saat mengetik karena tabel tidak update.	Menggunakan <code>Array.filter()</code> untuk memilih data yang sesuai input dan render ulang tabel setiap kali input berubah.
Data tidak bisa diurutkan dari jumlah terbesar ke terkecil.	Menggunakan <code>Array.sort()</code> pada properti jumlah produksi dan render ulang tabel setelah sortir.

G.3 Kaitan dengan Penerapan di Industri

Log Produksi Harian memiliki kaitan yang erat dengan industri manufaktur **dan** produksi, di mana pencatatan jumlah produksi harian, pemantauan kinerja operator, dan pengelolaan shift kerja merupakan bagian penting dari operasional pabrik. Dengan aplikasi ini, manajemen dapat memantau produktivitas secara real-time, mengidentifikasi operator atau shift dengan performa tinggi atau rendah, serta memudahkan pengambilan keputusan berbasis data. Selain itu, fitur filter dan sortir memudahkan analisis data produksi, misalnya menemukan operator tertentu atau melihat unit produksi terbesar, yang sangat relevan dengan praktik industrial engineering, quality control, dan performance monitoring.

H. KESIMPULAN

1	Form input divalidasi di sisi klien untuk memastikan data lengkap dan bersih, sementara LocalStorage digunakan agar data produksi tetap tersimpan secara persisten.
2	Data disimpan sebagai array objek yang dapat difilter dan diurutkan; tabel riwayat produksi diperbarui secara dinamis, memudahkan analisis operator dan jumlah produksi.
3	Aplikasi meniru praktik pencatatan produksi di dunia nyata, mendukung pemantauan kinerja, quality control, dan pengambilan keputusan berbasis data di lingkungan manufaktur dan teknik industri.

I. DAFTAR PUSTAKA

No.	Referensi (Format APA 7th Edition)
1.	Al-Rousan, Thamer, et al. "A New Security Model for Web Browser Local Storage." <i>International Journal of Advanced Computer Science and Applications (IJACSA)</i> , Vol. 10, No. 8, 2019.
2.	Sinha, Akash Rakesh. "Navigating Client-Side Storage in Modern Web Applications: Mechanisms, Best Practices and Future Directions." <i>International Journal for Multidisciplinary Research (IJFMR)</i> , Vol. 2, Issue 5, September–October 2020.

J. LEMBAR PENILAIAN DAN PENGESAHAN

RUBRIK PENILAIAN LAPORAN					
Aspek Penilaian		Bobot			
No.	Aspek Penilaian	Bobot (%)	Nilai (0-100)	Skor Akhir	Catatan Penilai
1	Kelengkapan Isi (Semua bagian A–I terisi lengkap)	20%			
2	Kualitas Dasar Teori (Ditulis dengan bahasa sendiri, relevan, minimal 3 paragraf)	15%			
3	Dokumentasi Langkah Kerja (Kode & Screenshot sesuai, jelas, dan terbaca)	25%			
4	Tugas/Latihan Mandiri (Semua latihan & proyek mini dikerjakan dan berfungsi)	20%			
5	Pembahasan & Analisis (Kritis, mendalam, dan relevan dengan industri)	15%			
6	Kesimpulan & Tata Bahasa (Spesifik, rapi, mengikuti format yang ditentukan)	5%			
TOTAL NILAI AKHIR		100%			

Mahasiswa	Rekan Tim	Diperiksa oleh Asisten / Dosen
		
Saprina Melita Sari NIM:23051430012	Ardita Natalia NIM:23051430022	Dr. Eng. Ir. Aji Ery Burhandenny, S.T., M.AIT. Tanggal:

MODUL 8

A. IDENTITAS & INFORMASI MODUL

Mata Kuliah	Aplikasi Web dan Mobile
Kode Modul	Modul 08
Topik Utama	Form Validation & Local Storage
Sub-Topik	Konsep Asynchronous, Fetch API, Promise, Async/Await, dan integrasi REST API eksternal.
Durasi Praktikum	340 Menit
Tanggal Praktikum	17 April 2026
Nama Mahasiswa	Saprina Melita Sari
NIM	23051430012
Kelas	A1

B. TUJUAN PEMBELAJARAN

1	Memahami perbedaan kode sinkronus dan asinkronus
2	Mampu menggunakan `fetch()` untuk mengambil data dari server eksternal (API).
3	Memahami format data JSON (JavaScript Object Notation).
4	Menerapkan penanganan error saat request API gagal.

C. ALAT DAN BAHAN

No.	Nama Alat / Bahan / Aplikasi	Versi / Spesifikasi	Keterangan
1	Visual Studio Code (VS Code)	Versi 1.8x ke atas	Code Editor
2	Google Chrome	Versi terbaru	Testing & Debugging
3	Live Server	Versi 5.7.9	Extensions VS Code untuk menjalankan html

4	Prettier	Versi 12.3.0	Extenions VS Code untuk merapikan indentansi code
5	Bootstrap	Versi 5.3.8	Framework front-end open-source untuk mempercepat pengembangan website

D. DASAR TEORI

D.1 Konsep Utama

Modul ini berfokus pada penerapan Asynchronous JavaScript dan penggunaan Fetch API sebagai dasar komunikasi data antara sisi klien (client-side) dengan server eksternal (server-side). Secara konsep, pemrograman asinkron memungkinkan aplikasi web melakukan permintaan dan pemrosesan data di latar belakang tanpa mengganggu aktivitas pengguna pada antarmuka utama. Pada praktikum ini, konsep tersebut diterapkan melalui pengambilan data laporan dari REST API publik. Proses tersebut juga dilengkapi dengan indikator pemuatan (loading) sebagai bentuk umpan balik kepada pengguna selama pertukaran data melalui jaringan sedang berlangsung.

Selain itu, modul ini menekankan pentingnya pemanfaatan JavaScript Object Notation (JSON) sebagai standar pertukaran data modern beserta teknik pengolahannya. Data mentah yang diterima dari server harus melalui proses parsing JSON agar dapat diubah menjadi objek yang mudah dibaca dan diolah oleh sistem. Setelah berhasil diekstraksi, data diproses lebih lanjut melalui teknik mapping dan lokalisasi, seperti membatasi jumlah data yang ditampilkan serta mengganti sebagian isi dengan simulasi teks berbahasa Indonesia. Penerapan tersebut menunjukkan bahwa data dari sumber eksternal dapat disesuaikan dengan kebutuhan dan konteks operasional aplikasi yang dikembangkan.

Pada tahap akhir, modul mengombinasikan manipulasi Document Object Model (DOM) secara dinamis dengan penerapan mekanisme penanganan kesalahan (error handling) untuk meningkatkan ketahanan sistem. Data yang telah distandardisasi kemudian ditampilkan secara otomatis dalam bentuk kartu informasi, di mana setiap kartu memiliki identitas unik (ID) guna mendukung interaksi yang lebih akurat. Sebagai upaya menjaga stabilitas aplikasi, dilakukan pemeriksaan terhadap status respons serta penggunaan blok catch untuk menangani kemungkinan terjadinya kesalahan. Dengan demikian, ketika koneksi mengalami gangguan atau permintaan gagal diproses, aplikasi tetap dapat berjalan dengan baik dan memberikan pesan peringatan yang informatif kepada pengguna tanpa mengalami crash.

D.2 Keterkaitan dengan Teknik Industri

Praktikum pada modul ini memiliki keterkaitan yang erat dengan bidang Teknik Industri, terutama dalam pengembangan Sistem Informasi Manajemen (SIM) dan pemantauan proses produksi secara real-time. Dalam lingkungan manufaktur, informasi terkait kondisi operasional di rantai produksi perlu diperoleh dengan cepat tanpa menghambat aktivitas lain yang sedang berlangsung. Penerapan Asynchronous JavaScript dan Fetch API pada program ini menggambarkan cara kerja dasbor pemantauan industri, seperti Andon system, yang mampu mengambil data berupa status mesin, peringatan kerusakan, atau anomali sensor dari server pusat maupun sistem Enterprise Resource Planning (ERP) secara otomatis di latar belakang. Kemampuan memperbarui tampilan antarmuka tanpa menyebabkan lag atau pembekuan layar sangat membantu operator dan manajer dalam melakukan pengawasan produksi secara responsif dan berkesinambungan.

Hubungan dengan keilmuan Teknik Industri juga tampak pada proses pengolahan dan standardisasi data operasional. Pada era Industrial Internet of Things (IIoT), berbagai mesin, sensor, dan perangkat lunak saling bertukar data dalam jumlah besar. Tahapan pengambilan data melalui REST API, penguraian format JSON, hingga penyesuaian isi data menjadi laporan insiden yang terstandar mencerminkan prinsip interoperabilitas dalam sistem terintegrasi. Proses tersebut menunjukkan bagaimana data dari sistem eksternal atau vendor dapat diterjemahkan ke dalam format yang sesuai dengan kebutuhan operasional perusahaan. Kemampuan melakukan standardisasi data seperti ini menjadi kompetensi penting bagi insinyur industri dalam menyediakan informasi yang valid sebagai dasar evaluasi kinerja mesin dan pengambilan keputusan.

Selain itu, manipulasi elemen DOM yang diterapkan dalam program merepresentasikan alur kerja pada Computerized Maintenance Management System (CMMS) yang efektif dan minim kesalahan. Ketika tombol "Tindak Lanjut" pada kartu insiden dipilih, sistem secara otomatis memberikan umpan balik visual sekaligus menonaktifkan aksi lanjutan pada elemen tersebut. Mekanisme ini sejalan dengan konsep Poka-Yoke, yaitu pendekatan pencegahan kesalahan untuk menghindari pengiriman tiket perbaikan yang berulang atau terjadinya tumpang tindih tugas pada tim pemeliharaan. Didukung dengan penerapan error handling untuk mengantisipasi kegagalan jaringan, rancangan aplikasi ini mendukung terciptanya sistem kerja yang andal, terkoordinasi, mengurangi pemborosan waktu tunggu (waste of waiting), serta selaras dengan prinsip-prinsip Teknik Industri dalam meningkatkan efisiensi operasional.

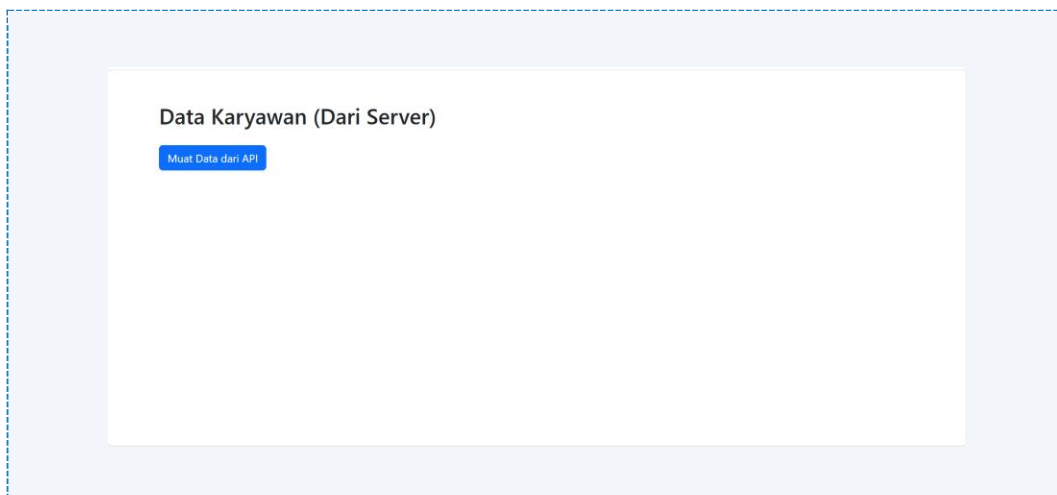
E. LANGKAH KERJA DAN HASIL PRAKTIKUM

E.1 Langkah 1 – Membuat HTML untuk Menampilkan Data

```
<!DOCTYPE html>
<html lang="id">
<head>
  <meta charset="UTF-8">
  <title>Integrasi API Data Karyawan</title>
  <link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css"
rel="stylesheet">
</head>
<body class="container mt-5">
  <h2 class="mb-4">Data Karyawan (Dari Server)</h2>

  <div class="mb-3">
    <button id="btnLoad" class="btn btn-primary">Muat Data dari
API</button>
    <div id="loading" class="spinner-border text-primary d-none"
role="status"></div>
  </div>
  <div class="row" id="containerKaryawan">
    <!-- Card karyawan akan muncul di sini -->
  </div>
  <script src="api_script.js"></script>
</body>
</html>
```

Screenshot Hasil:



Gambar E.1 – Hasil Langkah 1

E.2 Langkah 2 – Fetch Data dengan .then() (Promise)

```
const btnLoad = document.getElementById('btnLoad');
const container = document.getElementById('containerKaryawan');
const loading = document.getElementById('loading');

// Endpoint API (Simulasi Database)
const API_URL = 'https://jsonplaceholder.typicode.com/users';

btnLoad.addEventListener('click', function () {
  // Tampilkan loading
  loading.classList.remove('d-none');
  container.innerHTML = ""; // Bersihkan konten lama

  // Fetch Data
  fetch(API_URL)
    .then(function (response) {
      // Cek jika response sukses (kode 200-299)
      if (!response.ok) {
        throw new Error('Gagal mengambil data');
      }
      // Parsing data JSON
      return response.json();
    })
    .then(function (dataKaryawan) {
      // Data berhasil didapat
      console.log(dataKaryawan); // Cek di console browser
      renderData(dataKaryawan);
    })
    .catch(function (error) {
      // Jika ada error (misal putus internet)
      container.innerHTML = `

Saprina Melita Sari– 23051430012 | Laporan Akhir



Halaman 144 dari 285

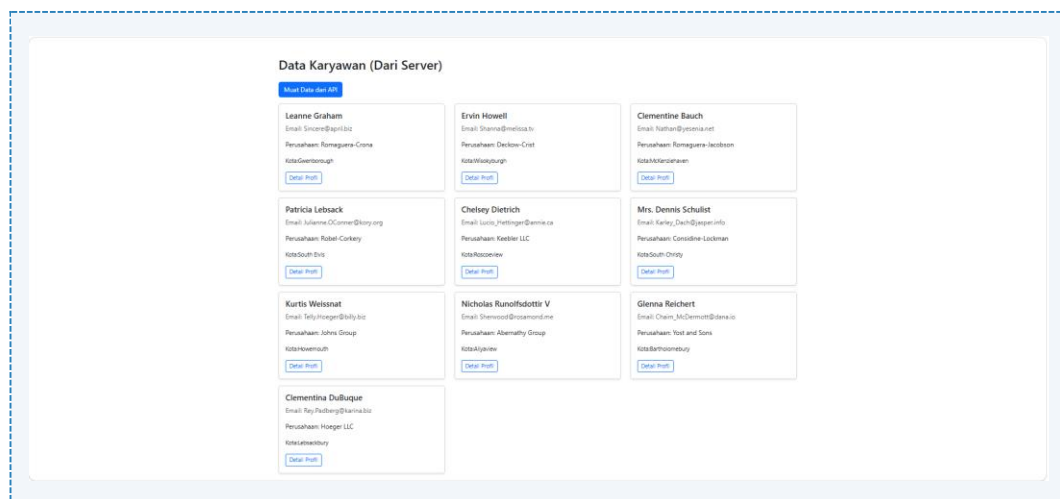

```

```

<h5 class="card-title">${karyawan.name}</h5>
<p class="card-text text-muted">Email: ${karyawan.email}</p>
<p class="card-text">Perusahaan: ${karyawan.company.name}</p>
<p class="card-
text"><small>Kota: ${karyawan.address.city}</small></p>
<a href="" class="btn btn-sm btn-outline-primary">Detail Profil</a>
</div>
</div>
`;
container.appendChild(col);
});
}

```

Screenshot Hasil:



Gambar E.2 – hasil fetch then()(Promise)

E.3 Langkah 3 – Fetch dengan Async/Await (Modern Style)

```

// Fungsi Async untuk mencari data spesifik
async function cariKaryawan(id) {
  try {
    console.log(`Mencari data ID: ${id}...`);

    const response = await
    fetch(`https://jsonplaceholder.typicode.com/users/${id}`);

    if (!response.ok) {
      throw new Error('Data tidak ditemukan');
    }

    const data = await response.json();
  }
}

```

```

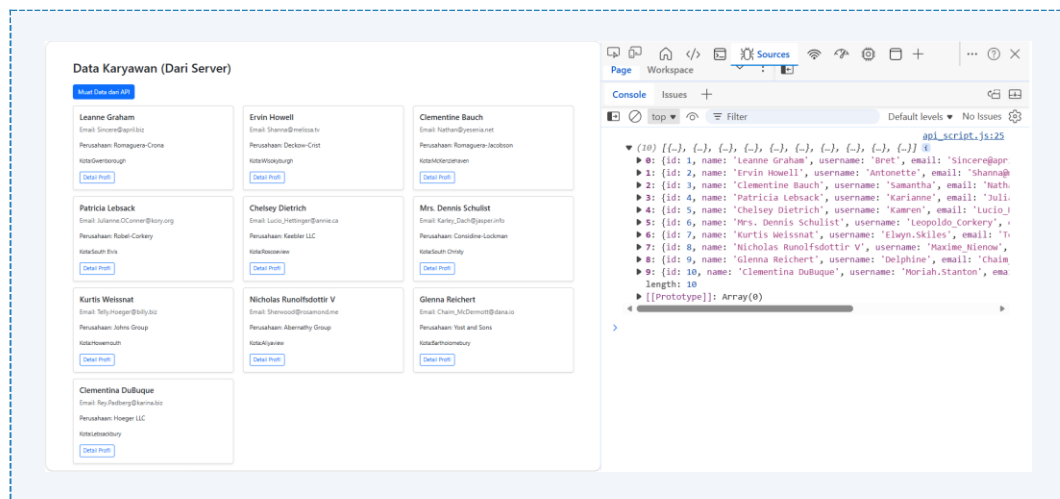
console.log("Ditemukan:", data);
alert(`Ditemukan: ${data.name} - bekerja di ${data.company.name}`);

} catch (error) {
  console.error(error);
  alert(error.message);
}
}

// Contoh pemanggilan: cariKaryawan(2);
// Anda bisa memanggil fungsi ini lewat console browser untuk tes.

```

Screenshot Hasil:



Gambar E.3 – Fetch dengan Async/Await

F. LATIHAN DAN TUGAS MANDIRI

F.1 Latihan 1 – Posting Data

Pertanyaan / Instruksi Latihan:

Buat tombol "Tambah Karyawan Baru". Gunakan method `fetch` dengan opsi `{ method: 'POST', body: ... }` untuk mengirim data dummy ke API.

Jawaban / Kode Solusi:

HTML

```
<button id="btnTambah" class="btn btn-success">Tambah Karyawan Baru</button>
```

JAVASCRIPT

```
document.addEventListener('DOMContentLoaded', function () {

    const API_URL = 'https://jsonplaceholder.typicode.com/users';
    const btnTambah = document.getElementById('btnTambah');

    if (!btnTambah) {
        console.error("Tombol tidak ditemukan!");
        return;
    }

    btnTambah.addEventListener('click', function () {

        console.log("Tombol diklik");

        // DATA LEBIH DARI SATU
        const dataKaryawan = [
            {
                name: "Budi Santoso",
                username: "budi123",
                email: "budi@email.com",
                address: { city: "Surabaya" }
            },
            {
                name: "Siti Rahma",
                username: "siti123",
                email: "siti@email.com",
                address: { city: "Semarang" }
            },
            {
                name: "Andi Wijaya",
                username: "andi123",
                email: "andi@email.com",
                address: { city: "Solo" }
            }
        ];

        // LOOP POST SATU-SATU
        dataKaryawan.forEach((karyawan) => {
            fetch(API_URL, {
                method: 'POST',
                headers: {
```

```
        'Content-Type': 'application/json'
      },
      body: JSON.stringify(karyawan)
    })
    .then(response => response.json())
    .then(data => {
      console.log("BERHASIL POST:", data);
    })
    .catch(error => {
      console.error("ERROR:", error);
    });
  });
});

CSS
/* Background halaman */
body {
  background: linear-gradient(to right, #eef2f3, #dfe9f3);
  font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
}

/* Judul */
h2 {
  font-weight: bold;
  color: #333;
}

/* Tombol */
#btnTambah {
  transition: all 0.3s ease;
}

#btnTambah:hover {
  background-color: #198754;
  transform: scale(1.05);
}

/* Card karyawan */
.card {
  border: none;
  border-radius: 12px;
  transition: all 0.3s ease;
}
```

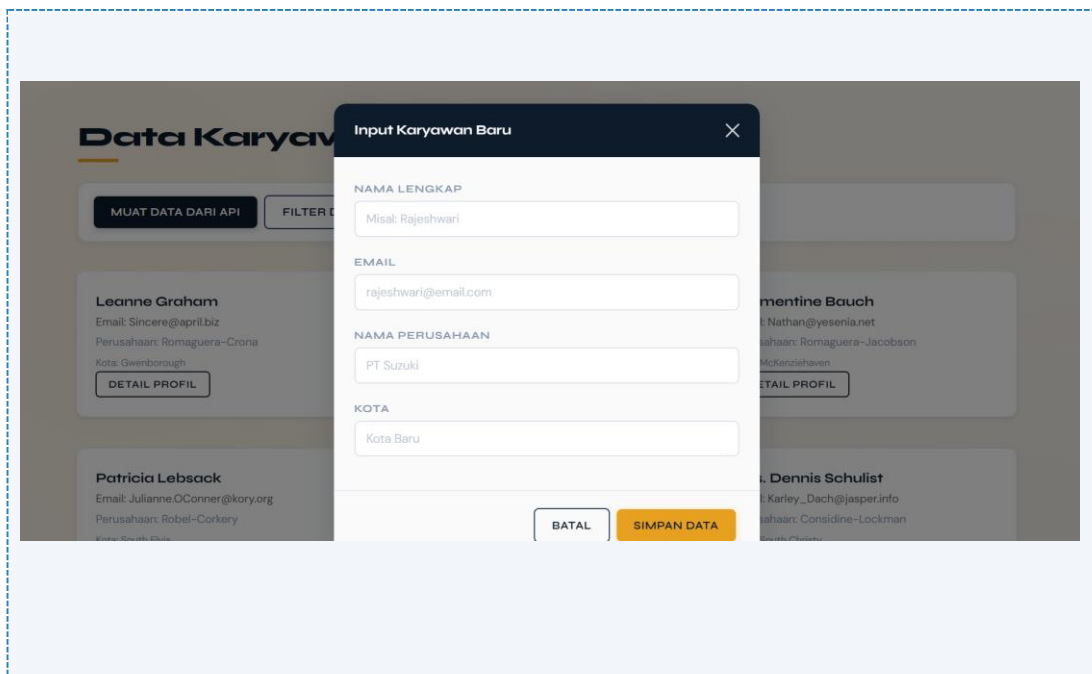


```
/* Hover card */
.card: hover {
  transform: translateY(-5px);
  box-shadow: 0 8px 20px rgba(0, 0, 0, 0.2);
}

/* Isi card */
.card-title {
  font-weight: bold;
  color: #0d6efd;
}

.card-text {
  font-size: 14px;
}

/* Spinner loading */
#loading {
  margin-left: 10px;
}
```



Gambar F.1 – Hasil pembuatan form dan button tambah karyawan

F.2 Latihan 2 – Filter Array

Pertanyaan / Instruksi Latihan:

Sebelum data dirender ke layar dengan `forEach`, lakukan filter terlebih dahulu. Tampilkan HANYA karyawan yang tinggal di kota yang mengandung huruf "s" (misal: "Southside", "London"). Petunjuk: Gunakan `data.filter(...)` sebelum `data.forEach(...)`.

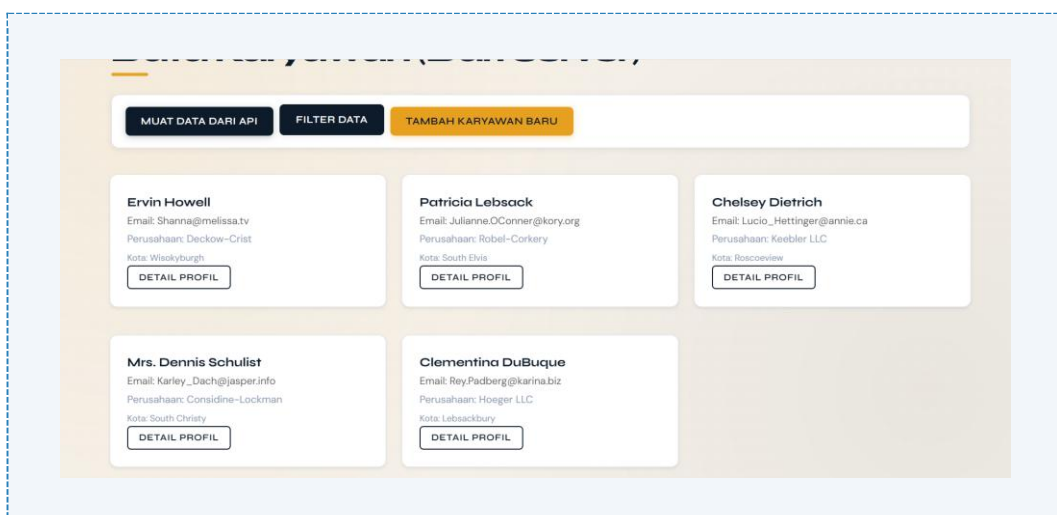
Jawaban / Kode Solusi:

```
const btnSortJumlah = document.getElementById('btnSortJumlah');

<button id="btnFilter" class="btn btn-secondary">Filter data</button>
btnFilter.addEventListener('click', function () {
  // Validasi jika data belum dimuat
  if (daftarKaryawan.length === 0) {
    alert("Harap klik 'Muat Semua Data' terlebih dahulu!");
    return;
  }

  // Filter dari variabel daftarKaryawan
  const hasilFilter = daftarKaryawan.filter(function (item) {
    return item.address.city.toLowerCase().includes('s');
  });

  // Tampilkan hasil filter ke layar
  container.innerHTML = "";
  renderData(hasilFilter);
});
```



Gambar F.2 – Hasil pembuatan button filter data

F.3 Tugas Proyek Mini – Integrasi Data Produksi

Deskripsi proyek mini:

Anggap API `https://jsonplaceholder.typicode.com/posts` Adalah database "Laporan insiden".

- Tampilkan daftar 10 laporan insiden terbaru (Title = Judul Insiden, Body = Deskripsi).
- Di setiap card, tambahkan tombol "Tindak Lanjut". Saat diklik, `alert()` menampilkan "Tiket ID sedang diproses oleh Tim Maintenance"

Penjelasan pendekatan/solusi yang digunakan:

Proses pengerjaan diawali dengan menyiapkan sepuluh data simulasi berbahasa Indonesia yang disesuaikan dengan kebutuhan pelaporan insiden di lingkungan kerja. Setelah itu, sistem diprogram untuk mengambil data melalui API ketika tombol dijalankan oleh pengguna. Selama proses pengambilan data berlangsung, ditampilkan indikator loading sebagai informasi bahwa sistem sedang memproses permintaan. Selain itu, mekanisme penanganan kesalahan juga diterapkan untuk mengantisipasi kemungkinan terjadinya gangguan koneksi.

Setelah data berhasil diperoleh, program secara otomatis membangun tampilan antarmuka dengan menampilkan kartu-kartu laporan pada halaman web. Setiap kartu beserta tombol yang terdapat di dalamnya diberikan identitas unik (ID) untuk memudahkan proses identifikasi dan pengelolaan elemen secara tepat. Pada tahap akhir, tombol "Tindak Lanjut" diprogram agar dapat memberikan respons interaktif. Ketika tombol diklik, sistem akan menampilkan pesan alert yang berisi informasi mengenai status tiket. Setelah pesan tersebut ditutup, kartu dan tombol akan mengalami perubahan tampilan sebagai indikator visual bahwa insiden telah diproses, sekaligus menonaktifkan tombol tersebut guna mencegah terjadinya pemrosesan berulang pada laporan yang sama.

Kode Utama:

```
const API_POSTS = "https://jsonplaceholder.typicode.com/posts";
const API_USERS = "https://jsonplaceholder.typicode.com/users";

let semuaKaryawan = [];

// Ambil elemen
const btnLoad = document.getElementById("btnLoad");
const btnFilter = document.getElementById("btnFilter");
const loading = document.getElementById("loading");
const cInsiden = document.getElementById("containerInsiden");
const cKaryawan = document.getElementById("containerKaryawan");
const statusBadge = document.getElementById("statusBadge");
```

```
const modalDetailBS = new
bootstrap.Modal(document.getElementById("modalDetail"));

// ===== UTILITY =====
const setLoading = (v) => {
  loading.classList.toggle("d-none", !v);
  btnLoad.disabled = v;
};

const inisial = (nama) =>
  nama.split(" ").slice(0, 2).map(w => w[0].toUpperCase()).join("");

const getStatus = (id) => {
  const list = [
    { label: "Kritis", cls: "badge-kritis" },
    { label: "Sedang", cls: "badge-sedang" },
    { label: "Rendah", cls: "badge-rendah" },
  ];
  return list[id % list.length];
};

// ===== INSIDEN (BAHASA INDONESIA)
=====
function renderInsiden(posts) {
  cInsiden.innerHTML = "";

  const data = posts.slice(0, 10);
  document.getElementById("jumlahInsiden").textContent =
    `${data.length} insiden`;

  const daftarMasalah = [
    "Kerusakan mesin produksi",
    "Keterlambatan proses produksi",
    "Kesalahan input bahan baku",
    "Gangguan sistem operasional",
    "Penurunan kualitas produk"
  ];

  data.forEach((post, i) => {
    const status = getStatus(post.id);

    const judul = daftarMasalah[post.id % daftarMasalah.length];
    const deskripsi = `Insiden berupa ${judul.toLowerCase()} sedang dalam
    proses penanganan oleh tim produksi. (ID: ${post.id})`;

    cInsiden.innerHTML += `
```

```

<div class="col-md-6 col-lg-4">
  <div class="card-insiden">
    <div class="insiden-meta">
      <span>TKT-${String(post.id).padStart(4,"0")}</span>
      <span class="${status.cls}">${status.label}</span>
    </div>

    <p class="insiden-title"><b>${judul}</b></p>
    <p class="insiden-desc">${deskripsi}</p>

    <div class="insiden-footer">
      <span>Pelapor: User #${post.userId}</span>
      <button class="btn-tindak" data-id="${post.id}">
        Tindak Lanjut
      </button>
    </div>
  </div>
</div>
`
  `;
});
}

// Klik tindak lanjut
cInsiden.addEventListener("click", (e) => {
  if (e.target.classList.contains("btn-tindak")) {
    alert(`Insiden ${e.target.dataset.id} sedang diproses oleh tim
maintenance`);
  }
});

// ===== KARYAWAN =====
function renderKaryawan(data) {
  cKaryawan.innerHTML = "";

  document.getElementById("jumlahKaryawan").textContent =
    `${data.length} karyawan`;

  if (data.length === 0) {
    cKaryawan.innerHTML = `
      <div class="col-12 text-center">
        <p>Tidak ada data karyawan</p>
      </div>`;
    return;
  }

  data.forEach((user) => {

```

```

cKaryawan.innerHTML += `
  <div class="col-md-6 col-lg-4">
    <div class="card-karyawan">
      <div class="avatar">${inisial(user.name)}</div>
      <p><b>${user.name}</b></p>
      <p>${user.email}</p>
      <p>${user.phone}</p>
      <p>${user.address.city}</p>

      <button class="btn-detail"
        data-user="${encodeURIComponent(JSON.stringify(user))}">
        Lihat Detail
      </button>
    </div>
  </div>
`;
});
}

// Detail modal
cKaryawan.addEventListener("click", (e) => {
  if (!e.target.classList.contains("btn-detail")) return;

  const user = JSON.parse(
    decodeURIComponent(e.target.dataset.user)
  );

  document.getElementById("detailNama").textContent = user.name;
  document.getElementById("detailUsername").textContent =
user.username;
  document.getElementById("detailEmail").textContent = user.email;
  document.getElementById("detailTelepon").textContent = user.phone;
  document.getElementById("detailWebsite").textContent = user.website;
  document.getElementById("detailPerusahaan").textContent =
user.company?.name ?? "-";
  document.getElementById("detailAlamat").textContent =
`${user.address.street}, ${user.address.suite}, ${user.address.city}
${user.address.zipcode}`;

  modalDetailBS.show();
});

// ===== FETCH DATA =====
async function muatData() {
  setLoading(true);

```

```
try {
  const [resPosts, resUsers] = await Promise.all([
    fetch(API_POSTS),
    fetch(API_USERS)
  ]);

  if (!resPosts.ok) throw new Error("Gagal mengambil data insiden");
  if (!resUsers.ok) throw new Error("Gagal mengambil data karyawan");

  const posts = await resPosts.json();
  const users = await resUsers.json();

  semuaKaryawan = users;

  renderInsiden(posts);
  renderKaryawan(users);

  statusBadge.className = "status-badge status-online";
  statusBadge.innerHTML = `<span class="status-dot"></span> Online`;

} catch (err) {
  alert(err.message);
}

setLoading(false);
}

// ===== FILTER =====
btnFilter.addEventListener("click", () => {
  if (semuaKaryawan.length === 0) {
    alert("Muat data terlebih dahulu");
    return;
  }

  const keyword = prompt("Masukkan nama kota:").toLowerCase();
  if (!keyword) return;

  const hasil = semuaKaryawan.filter(u =>
    u.address.city.toLowerCase().includes(keyword)
  );

  renderKaryawan(hasil);
});

// ===== TAMBAH KARYAWAN =====
```

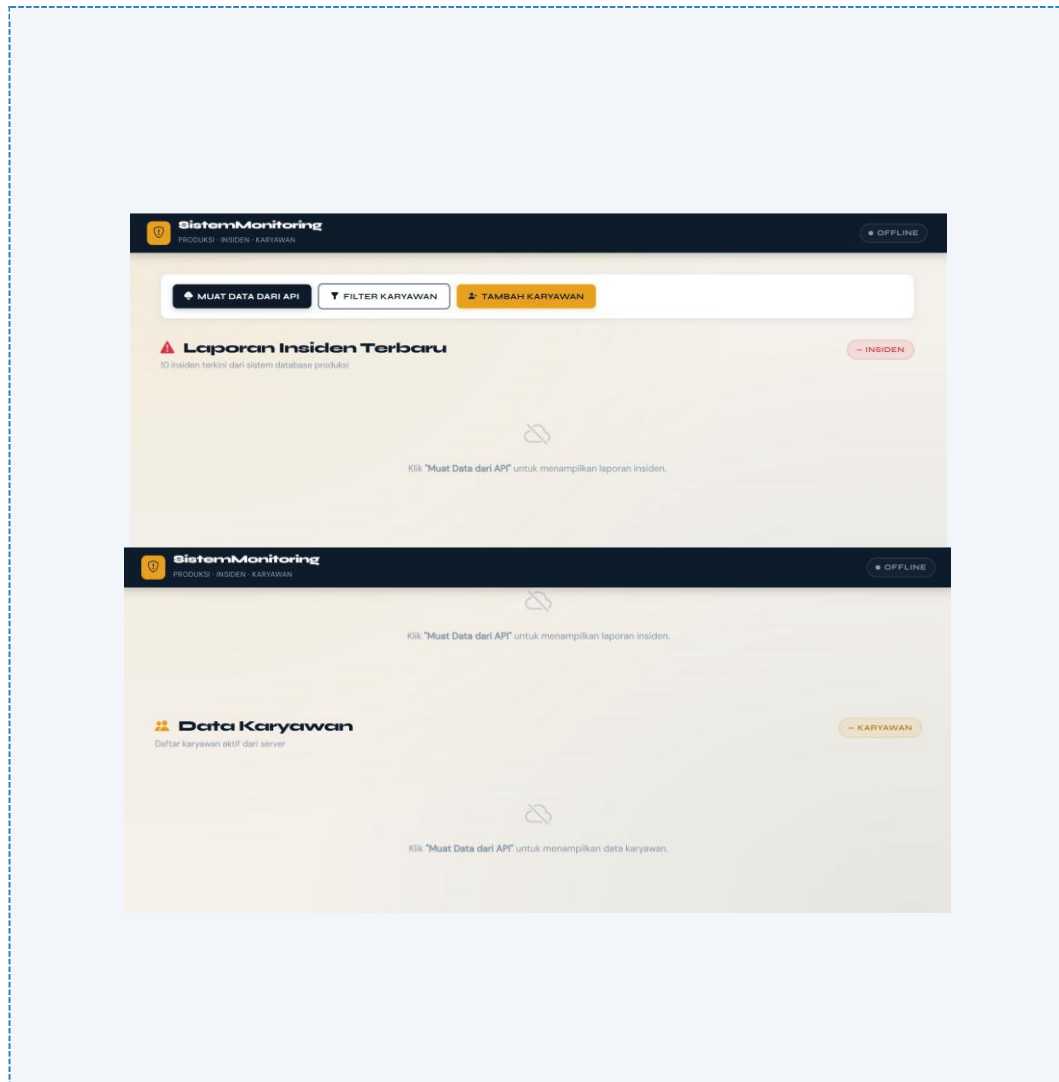
```
document.getElementById("formTambahKaryawan").addEventListener("submit", (e) => {
  e.preventDefault();

  const karyawanBaru = {
    id: semuaKaryawan.length + 1,
    name: document.getElementById("inputNama").value,
    email: document.getElementById("inputEmail").value,
    phone: "-",
    username:
document.getElementById("inputNama").value.toLowerCase().replace(/\s+/g,
"."),
    website: "-",
    company: { name: document.getElementById("inputPerusahaan").value },
    address: {
      street: "-",
      suite: "-",
      city: document.getElementById("inputKota").value,
      zipcode: "-"
    }
  };

  semuaKaryawan.push(karyawanBaru);
  renderKaryawan(semuaKaryawan);

  bootstrap.Modal.getInstance
(document.getElementById("modalTambah"))?.hide();
  e.target.reset();
});

// ===== LOAD =====
btnLoad.addEventListener("click", muatData);
```

Gambar F.3 – Hasil pembuatan mini proyek integrasi data produksi

G. PEMBAHASAN

G.1 Analisis Kesesuaian Hasil dengan Teori

Hasil program yang diperoleh menunjukkan kesesuaian yang baik dengan teori dasar Asynchronous JavaScript dalam proses komunikasi data dari sumber eksternal. Secara teoritis, pemrograman asinkron memungkinkan suatu proses dijalankan di latar belakang sehingga antarmuka aplikasi tetap responsif dan tidak mengalami pembekuan saat menunggu respons dari server. Pada program Laporan Insiden, konsep tersebut diterapkan melalui penggunaan Fetch API untuk mengambil data dari REST API publik. Implementasi ini terbukti mampu mendukung pertukaran data secara efisien, yang ditunjukkan oleh tampilan aplikasi yang tetap dapat digunakan dengan baik serta adanya indikator loading sebagai umpan balik visual selama proses pengambilan data berlangsung.

Kesesuaian dengan teori juga tampak pada penerapan Promise, pengolahan data berformat JavaScript Object Notation (JSON), serta

mekanisme penanganan kesalahan. Dalam pengembangan web modern, data yang diterima dari server perlu diuraikan terlebih dahulu agar dapat diproses oleh aplikasi di sisi klien. Sejalan dengan konsep tersebut, program ini melakukan parsing terhadap respons API menjadi objek JSON sebelum data dimanipulasi dan ditampilkan pada antarmuka. Selain itu, prinsip ketahanan sistem diwujudkan melalui penerapan error handling. Ketika terjadi kegagalan saat melakukan request ke API, aplikasi tidak berhenti secara tiba-tiba, tetapi mampu menangani kondisi tersebut dengan menampilkan pesan peringatan yang informatif kepada pengguna.

Penggabungan antara konsep asinkronus dan manipulasi DOM dalam program ini juga berhasil merepresentasikan teori integrasi aplikasi front-end dengan sistem data yang lebih luas. Secara konseptual, pendekatan tersebut memungkinkan aplikasi web berfungsi sebagai dasbor interaktif yang menghubungkan pengguna dengan sumber data terpusat. Dalam implementasinya, data yang diperoleh kemudian disesuaikan menjadi simulasi laporan insiden industri sehingga lebih mendekati kondisi nyata di lapangan. Hal ini sesuai dengan kebutuhan sistem operasional industri modern, di mana antarmuka pengguna harus mampu terhubung secara dinamis dengan database ERP maupun jaringan sensor IoT untuk mendukung proses maintenance yang cepat, efisien, dan mudah dipantau.

G.2 Kendala yang Ditemukan dan Solusinya

Kendala / Error yang Ditemukan	Solusi yang Diterapkan
Proses pengambilan data dari REST API membutuhkan waktu sehingga pengguna harus menunggu respons dari server.	Menambahkan indikator loading agar pengguna mengetahui bahwa sistem sedang memproses permintaan data.
Koneksi internet yang tidak stabil dapat menyebabkan request API gagal dijalankan.	Menerapkan error handling menggunakan blok catch untuk menampilkan pesan peringatan tanpa membuat aplikasi mengalami crash.
Data yang diterima dari API masih berupa format mentah sehingga belum dapat langsung digunakan pada aplikasi.	Melakukan parsing JSON untuk mengubah respons API menjadi objek JavaScript yang mudah diolah dan ditampilkan.

G.3 Kaitan dengan Penerapan di Industri

Praktikum pada modul ini memiliki hubungan yang erat dengan penerapan sistem informasi di dunia industri, terutama dalam aspek komunikasi data terpusat dan pemantauan operasional secara real-time.

Penggunaan Asynchronous JavaScript melalui Fetch API menggambarkan mekanisme pemrosesan paralel yang banyak diterapkan pada antarmuka digital industri. Dalam kondisi operasional yang dinamis, kemampuan sistem untuk mengambil pembaruan data tanpa menghambat tampilan antarmuka sangat penting. Konsep asinkronus memungkinkan proses komunikasi dengan server berlangsung di latar belakang sehingga operator tetap dapat berinteraksi dengan dasbor tanpa terganggu oleh waktu tunggu selama proses pemuatan data.

Selain itu, integrasi REST API dan pengolahan format data pada program ini mencerminkan prinsip interoperabilitas yang terdapat dalam sistem Industrial Internet of Things (IIoT) maupun Enterprise Resource Planning (ERP). Data mentah yang diperoleh dari server eksternal diproses, dikonversi, dan disesuaikan menjadi laporan berbahasa Indonesia sehingga menyerupai proses standarisasi informasi dari berbagai sumber ke dalam format yang digunakan perusahaan. Penerapan error handling juga menunjukkan pentingnya keandalan sistem dalam menjaga kontinuitas operasional. Ketika terjadi gangguan jaringan atau kegagalan pengambilan data, sistem mampu memberikan peringatan yang jelas kepada pengguna sehingga risiko kesalahan pengambilan keputusan akibat terputusnya aliran informasi dapat diminimalkan.

H. KESIMPULAN

1	Penggunaan fungsi <code>fetch()</code> memungkinkan proses pengambilan data dari REST API dilakukan secara asinkron sehingga antarmuka pengguna tetap responsif dan tidak mengalami pembekuan selama menunggu respons dari server.
2	Metode <code>.json()</code> digunakan untuk mengubah respons mentah dari server menjadi objek JSON sehingga data dapat dibaca, diolah, dimanipulasi, dan ditampilkan sesuai kebutuhan aplikasi.
3	Manipulasi DOM secara dinamis dengan pemberian ID unik pada setiap elemen memungkinkan perubahan status visual dan pengaturan atribut dilakukan secara tepat pada elemen tertentu tanpa memengaruhi elemen lainnya.
4	Penerapan error handling melalui blok <code>.catch()</code> serta pengecekan <code>!response.ok</code> berfungsi menjaga kestabilan sistem, mencegah aplikasi mengalami crash, dan memastikan pengguna memperoleh informasi yang jelas ketika terjadi kegagalan koneksi.

I. DAFTAR PUSTAKA

No.	Referensi (Format APA 7th Edition)
[1]	Burhandenny, A. E. (2026). Manual Praktikum Aplikasi Web dan Mobile Semester Genap 2025/2026. Program Studi Teknik Industri, Universitas Negeri Yogyakarta.
[2]	https://www.javascripttutorial.net/
[3]	https://www.javascripttutorial.net/javascript-dom/
[4]	https://developer.mozilla.org/en-US/docs/Web/JavaScript
[5]	https://www.w3schools.com/js/default.asp

J. LEMBAR PENILAIAN DAN PENGESAHAN

RUBRIK PENILAIAN LAPORAN					
Aspek Penilaian		Bobot			
No.	Aspek Penilaian	Bobot (%)	Nilai (0-100)	Skor Akhir	Catatan Penilai
1	Kelengkapan Isi (Semua bagian A–I terisi lengkap)	20%			
2	Kualitas Dasar Teori (Ditulis dengan bahasa sendiri, relevan, minimal 3 paragraf)	15%			
3	Dokumentasi Langkah Kerja (Kode & Screenshot sesuai, jelas, dan terbaca)	25%			
4	Tugas/Latihan Mandiri (Semua latihan & proyek mini dikerjakan dan berfungsi)	20%			
5	Pembahasan & Analisis (Kritis, mendalam, dan relevan dengan industri)	15%			
6	Kesimpulan & Tata Bahasa (Spesifik, rapi, mengikuti format yang ditentukan)	5%			
TOTAL NILAI AKHIR		100%			

Mahasiswa	Rekan Tim	Diperiksa oleh Asisten / Dosen
 _____ Saprina Melita Sari NIM:23051430012	 _____ Ardita Natalia NIM:23051430022	 _____ Dr. Eng. Ir. Aji Ery Burhandenny, S.T., M.AIT. Tanggal: _____

MODUL 9

A. IDENTITAS & INFORMASI MODUL

Mata Kuliah	Aplikasi Web dan Mobile
Kode Modul	Modul 09
Topik Utama	Pengenalan React.js (Component, JSX, Props)
Sub-Topik	Setup Lingkungan React, Konsep Component, JSX Syntax, dan Props (Data Passing).
Durasi Praktikum	340 Menit
Tanggal Praktikum	24 April 2026
Nama Mahasiswa	Saprina Melita Sari
NIM	23051430012
Kelas	A1

B. TUJUAN PEMBELAJARAN

1	Memahami konsep Library vs Framework dan mengapa React dipilih untuk aplikasi skala besar.
2	Mampu menyiapkan environment pengembangan React menggunakan Vite
3	Memahami sintaks JSX (JavaScript XML).
4	Mampu membuat Functional Component yang menerima dan menampilkan data melalui Props.

C. ALAT DAN BAHAN

No.	Nama Alat / Bahan / Aplikasi	Versi / Spesifikasi	Keterangan
1	Visual Studio Code (VS Code)	Versi 1.8x ke atas	Code Editor
2	Google Chrome / Browser Modern	Versi terbaru	Testing & Debugging

3	Node.js & npm	Node 18+ / npm 9+	Runtime JS (Modul tertentu)
4	Prettier	Versi 12.3.0	Extensions VS Code untuk merapikan indentansi code

D. DASAR TEORI

D.1 Konsep Utama

Konsep utama yang dipelajari pada materi ini adalah React.js sebagai teknologi yang digunakan untuk membangun antarmuka pengguna (user interface) dengan pendekatan berbasis komponen. Dalam React, tampilan aplikasi dibagi menjadi beberapa bagian kecil yang disebut component, di mana setiap komponen memiliki tugas dan fungsi tertentu. Komponen-komponen tersebut dapat digunakan kembali pada bagian lain dari aplikasi sehingga proses pengembangan menjadi lebih efisien, terorganisir, dan mudah dikelola, terutama ketika aplikasi berkembang menjadi lebih kompleks.

Selain itu, materi ini juga memperkenalkan JSX (JavaScript XML), yaitu sintaks khusus yang memungkinkan penulisan struktur tampilan menyerupai HTML langsung di dalam file JavaScript. Penggunaan JSX membantu pengembang menggabungkan logika program dan tampilan dalam satu tempat sehingga proses pembuatan antarmuka menjadi lebih praktis. Dengan pendekatan ini, hubungan antara data dan tampilan dapat dipahami dengan lebih mudah, sehingga proses pengembangan maupun debugging dapat dilakukan dengan lebih cepat.

Konsep berikutnya adalah Props (Properties), yaitu mekanisme yang digunakan untuk mengirimkan data dari komponen induk (parent) ke komponen anak (child). Props bekerja seperti parameter pada sebuah fungsi, sehingga memungkinkan setiap komponen menerima data yang berbeda sesuai kebutuhan. Melalui penggunaan props, pengelolaan data menjadi lebih fleksibel dan struktur aplikasi menjadi lebih modular. Oleh karena itu, kombinasi antara component, JSX, dan props menjadi fondasi penting dalam pengembangan aplikasi modern menggunakan React.

D.2 Keterkaitan dengan Teknik Industri

Dalam sudut pandang Teknik Industri, konsep pembagian sistem ke dalam komponen-komponen kecil pada React memiliki kesamaan dengan prinsip modularisasi dalam sistem produksi. Pada lingkungan manufaktur, suatu proses kerja umumnya dibagi menjadi beberapa stasiun kerja agar lebih mudah dikendalikan, diawasi, dan ditingkatkan efisiensinya. Penerapan konsep component pada React juga bertujuan serupa, yaitu memecah

antarmuka menjadi bagian-bagian yang lebih kecil sehingga proses pengembangan, pengujian, serta pemeliharaan aplikasi dapat dilakukan dengan lebih efektif. Pendekatan ini sejalan dengan prinsip Teknik Industri yang menekankan efisiensi dan optimalisasi proses kerja.

Selain itu, penggunaan props dalam React dapat dianalogikan sebagai mekanisme aliran informasi pada sistem operasi industri. Dalam kegiatan produksi, informasi seperti jadwal kerja, ketersediaan bahan baku, maupun kondisi mesin perlu disampaikan dari satu bagian ke bagian lainnya secara tepat dan akurat agar operasional berjalan lancar. Demikian pula pada React, props berfungsi sebagai sarana pertukaran data antar komponen sehingga setiap bagian aplikasi dapat bekerja secara terkoordinasi. Hal tersebut menunjukkan bahwa konsep komunikasi data dalam React memiliki relevansi dengan pengelolaan arus informasi dalam sistem industri.

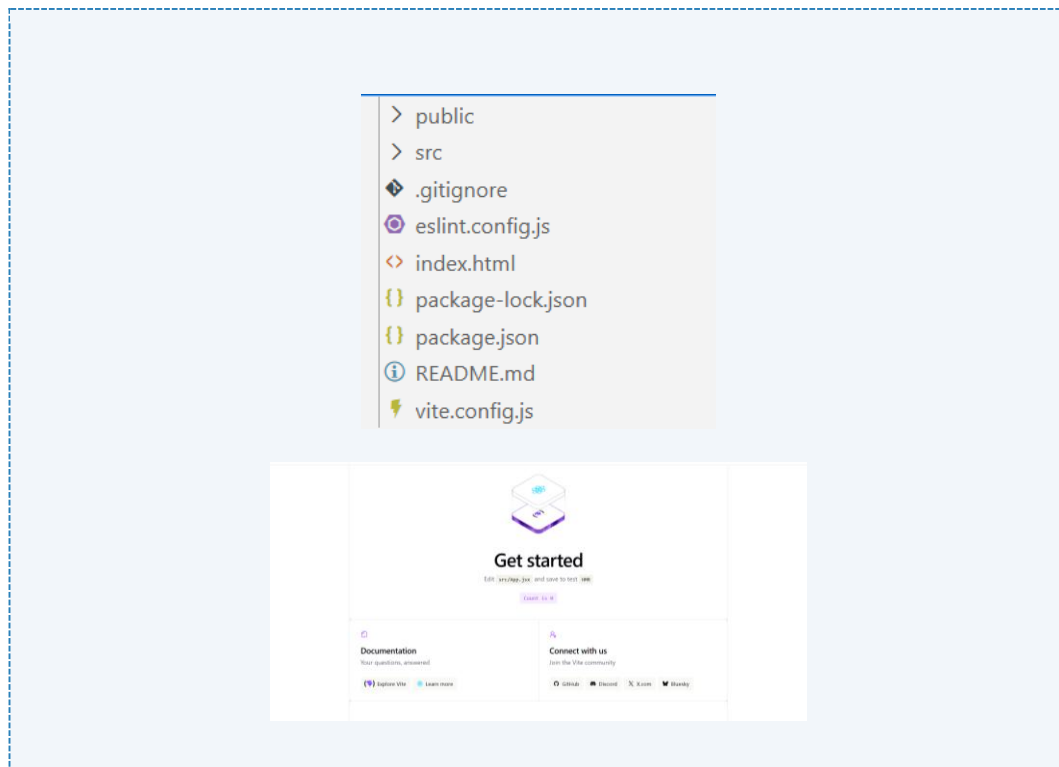
Dari aspek ergonomi dan perancangan sistem, React juga mendukung pembuatan antarmuka yang responsif, interaktif, dan mudah digunakan. Dalam Teknik Industri, desain antarmuka yang baik merupakan bagian dari human-machine interaction yang bertujuan meningkatkan kenyamanan, efektivitas, dan produktivitas pengguna. Melalui React, pengembang dapat menciptakan tampilan yang dinamis dan mudah beradaptasi dengan kebutuhan pengguna, sehingga mendukung efisiensi kerja dalam pengoperasian sistem informasi industri.

E. LANGKAH KERJA DAN HASIL PRAKTIKUM

E.1 Langkah 1 – Setup Project dengan Vite

```
/* ===== KODE / OUTPUT LANGKAH 1 ===== */  
npm create vite@latest sistem-industri -- --template react  
cd sistem-industri  
npm install  
npm run dev
```

Screenshot Hasil:

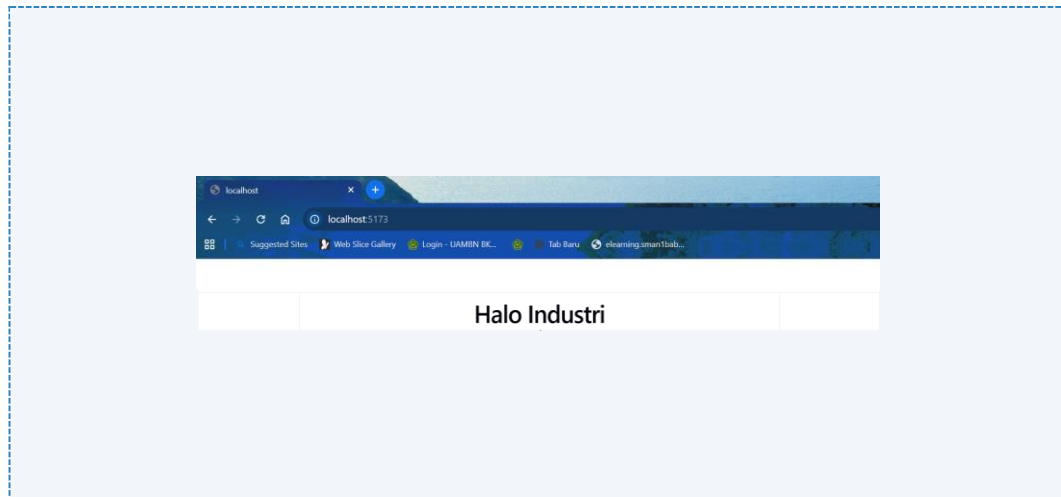


Gambar E.1 – Hasil instalasi

E.2 Langkah 2 – Membersihkan Project

```
/* ===== KODE / OUTPUT LANGKAH 2 ===== */  
import { useState } from 'react'  
import reactLogo from './assets/react.svg'  
import viteLogo from './assets/vite.svg'  
import heroImg from './assets/hero.png'  
import './App.css'  
  
function App() {  
  const [count, setCount] = useState(0)  
  
  return (  
    <>  
      <h1>Halo Industri</h1>  
    </>  
  )  
}  
  
export default App
```

Screenshot Hasil:

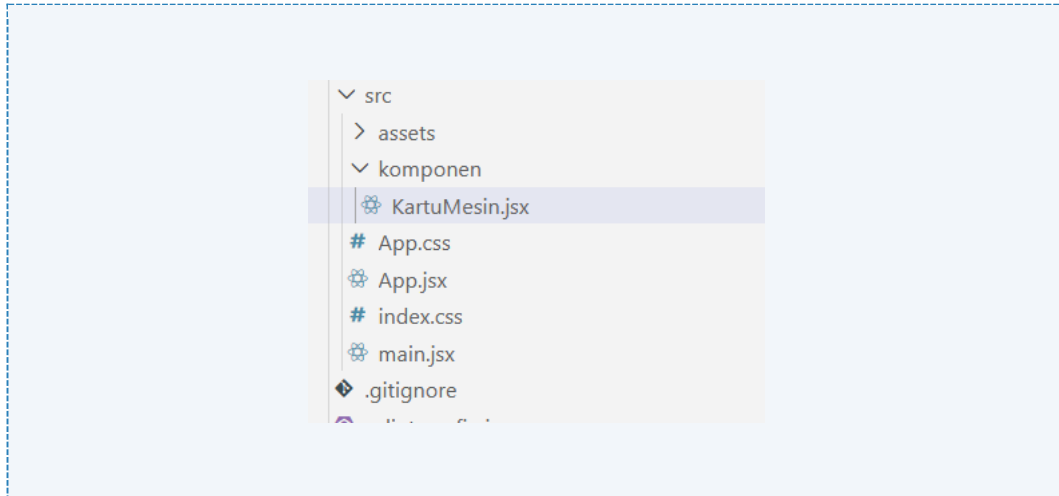


Gambar E.2 – Hasil Pembersihan

E.3 Langkah 3 – Membuat Komponen Pertama: KartuMesin

```
// 1. Import React
import React from 'react';
// 2. Membuat Komponen Fungsi yang menerima 'props'
function KartuMesin(props) {

  // Menerima data dari props
  const namaMesin = props.nama;
  const status = props.status;
  const produksi = props.produksi;
  // Logika penentuan warna badge berdasarkan status
  let badgeColor = 'bg-secondary';
  if (status === 'Running') badgeColor = 'bg-success';
  if (status === 'Stop') badgeColor = 'bg-danger';
  if (status === 'Maintenance') badgeColor = 'bg-warning';
  return (
    // Menggunakan class -> className di JSX
    <div className="card shadow-sm p-3 mb-3">
      <div className="card-body">
        <h5 className="card-title">{namaMesin}</h5>
        <span className={`badge
${badgeColor}`}>{status}</span>
        <hr />
        <p>Produksi Saat Ini: <strong>{produksi}</strong>
Unit</p>
      </div>
    </div>
  );
}
// 3. Export agar bisa dipakai di file lain
export default KartuMesin;
```

Screenshot Hasil:

Gambar E.3 – Hasil Pembuatan Komponen KartuMesin

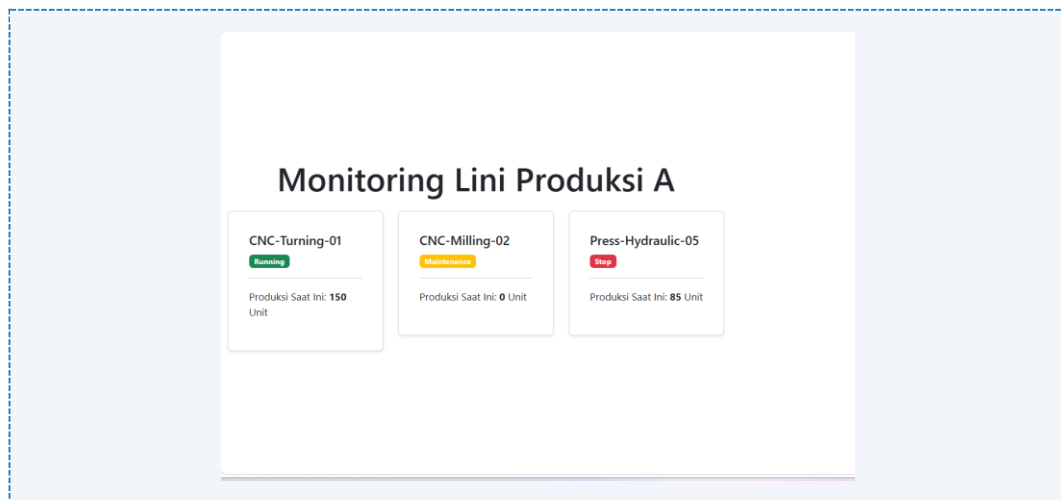
E.4 Langkah 4 – Menggunakan Komponen di App.jsx

```
import React from 'react';
// Import komponen yang sudah dibuat
import KartuMesin from './Komponen/KartuMesin';
function App() {
  return (
    <div className="container mt-4">
      <h1 className="text-center mb-4">Monitoring Lini Produksi
A</h1>

      <div className="row">
        {/* Kolom 1: Menggunakan komponen dengan data berbeda */}
        <div className="col-md-4">
          <KartuMesin
            nama="CNC-Turning-01"
            status="Running"
            produksi={150}
          />
        </div>
        {/* Kolom 2 */}
        <div className="col-md-4">
          <KartuMesin
            nama="CNC-Milling-02"
            status="Maintenance"
            produksi={0}
          />
        </div>
        {/* Kolom 3 */}
```

```
        <div className="col-md-4">
          <KartuMesin
            nama="Press-Hydraulic-05"
            status="Stop"
            produksi={85}
          />
        </div>
      </div>
    </div>
  );
}
export default App;
```

Screenshot Hasil:



Gambar E.4 – Hasil Penggunaan Komponen di App.jsx

F. LATIHAN DAN TUGAS MANDIRI

F.1 Latihan 1 – Destructuring Props

Pertanyaan / Instruksi Latihan:

Ubah cara deklarasi props di `KartuMesin.jsx` menggunakan destructuring: `function KartuMesin({ nama, status, produksi }) { ... }`. Sesuaikan kode di dalamnya.

Jawaban / Kode Solusi:

```
import React from 'react';

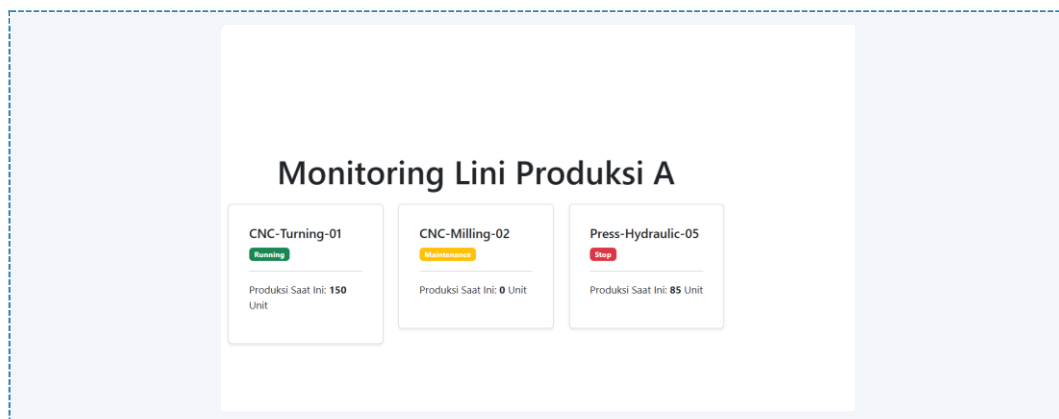
function Latihan1({ nama, status, produksi }) {

  let badgeColor = 'bg-secondary';

  if (status === 'Running') badgeColor = 'bg-success';
  if (status === 'Stop') badgeColor = 'bg-danger';
  if (status === 'Maintenance') badgeColor = 'bg-warning';

  return (
    <div className="card shadow-sm p-3 mb-3">
      <div className="card-body">
        <h5 className="card-title">{nama}</h5>
        <span className={`badge
${badgeColor}`}>{status}</span>
        <hr />
        <p>
          Produksi Saat Ini:
          <strong>{produksi}</strong> Unit
        </p>
      </div>
    </div>
  );
}

export default Latihan1;
```



Gambar F.1 – Hasil Pembuatan Destructuring Props

F.2 Latihan 2 – Default Props

Pertanyaan / Instruksi Latihan:

Berikan nilai default pada prop `produksi` menjadi 0, jika user lupa mengirim prop tersebut

Jawaban / Kode Solusi:

```
import React from 'react';

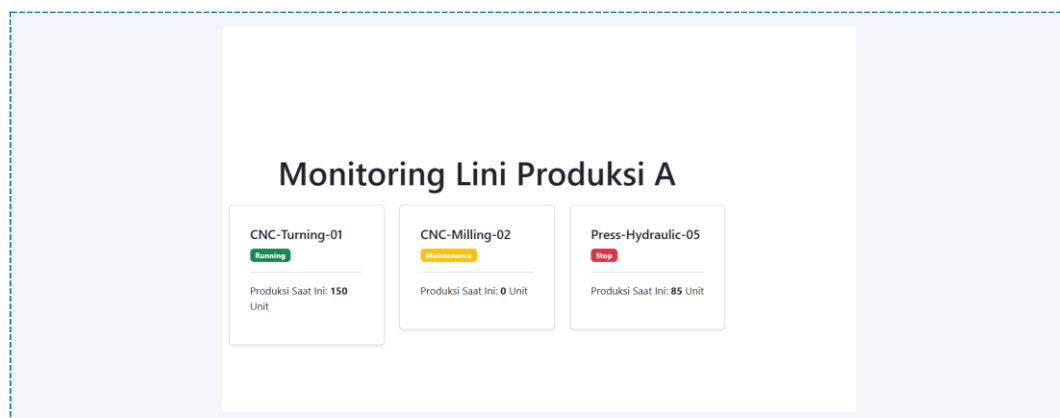
function Latihan2({ nama, status, produksi = 0 }) {

  let badgeColor = 'bg-secondary';

  if (status === 'Running') badgeColor = 'bg-success';
  if (status === 'Stop') badgeColor = 'bg-danger';
  if (status === 'Maintenance') badgeColor = 'bg-warning';

  return (
    <div className="card shadow-sm p-3 mb-3">
      <div className="card-body">
        <h5 className="card-title">{nama}</h5>
        <span className={`badge
${badgeColor}`}>{status}</span>
        <hr />
        <p>
          Produksi Saat Ini:
          <strong>{produksi}</strong> Unit
        </p>
      </div>
    </div>
  );
}

export default Latihan2;
```



Gambar F.2 – Hasil pemberian nilai default produksi = 0

F.3 Tugas Proyek Mini – Kartu Karyawan

Deskripsi proyek mini:

- Buat komponen baru bernama `KartuKaryawan.jsx`. Komponen ini menerima props: `nama`, `jabatan`, `bagian`.
- Tampilkan komponen ini 3 kali di `App.jsx` dengan data karyawan berbeda (Manager, Operator, QC).

Penjelasan pendekatan/solusi yang digunakan:

Proyek mini ini dikerjakan dengan mengembangkan komponen React sederhana yang berfungsi untuk menampilkan data karyawan dalam bentuk kartu informasi. Pendekatan yang diterapkan menggunakan konsep pemecahan antarmuka ke dalam komponen-komponen yang dapat digunakan kembali (*reusable component*). Pada implementasinya, komponen KartuKaryawan berperan sebagai template untuk menyajikan informasi setiap karyawan, sedangkan komponen App bertindak sebagai komponen utama yang mengatur susunan halaman serta menampilkan beberapa data karyawan secara bersamaan.

Komponen KartuKaryawan dirancang sebagai komponen fungsi yang menerima data melalui props, yaitu berupa nama, jabatan, dan bagian. Informasi tersebut kemudian ditampilkan menggunakan elemen JSX, seperti `div`, `h5`, dan `p`, sehingga menghasilkan tampilan kartu yang terstruktur. Penggunaan props memungkinkan komponen yang sama dimanfaatkan berulang kali dengan isi data yang berbeda tanpa perlu membuat ulang struktur antarmukanya.

Selanjutnya, pada komponen App, komponen KartuKaryawan diimpor dan dipanggil beberapa kali untuk menampilkan beberapa data karyawan dalam satu halaman. Setiap pemanggilan diberikan nilai props yang berbeda sehingga menghasilkan kartu dengan informasi yang beragam. Melalui proyek mini ini, dapat dipahami konsep dasar React yang meliputi pembuatan komponen, penggunaan props sebagai media pengiriman data, serta penerapan prinsip penggunaan ulang komponen dalam pengembangan aplikasi React sederhana.

Kode Utama:


```
import React from 'react';
import KartuKaryawan from './Komponen/KartuKaryawan';

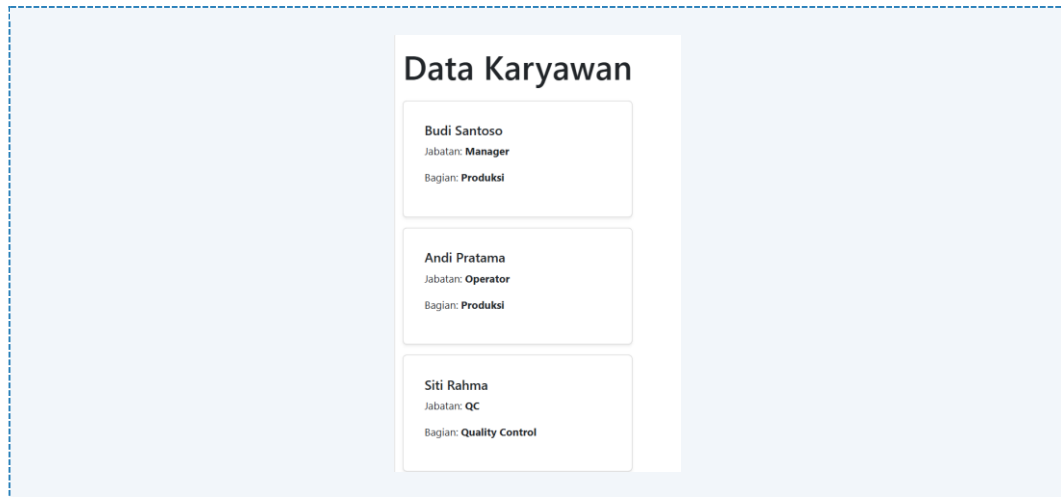
function App() {
  return (
    <div className="container mt-4">
      <h1 className="text-center mb-4">
        Data Karyawan
      </h1>

      <KartuKaryawan
        nama="Budi Santoso"
        jabatan="Manager"
        bagian="Produksi"
      />

      <KartuKaryawan
        nama="Andi Pratama"
        jabatan="Operator"
        bagian="Produksi"
      />

      <KartuKaryawan
        nama="Siti Rahma"
        jabatan="QC"
        bagian="Quality Control"
      />
    </div>
  );
}

export default App;
```



Gambar F.3 – Hasil Pembuatan Mini Proyek Kartu Karyawan

G. PEMBAHASAN

G.1 Analisis Kesesuaian Hasil dengan Teori

Secara teoritis, penerapan pendekatan berbasis komponen dalam pengembangan perangkat lunak bertujuan untuk meningkatkan tingkat penggunaan ulang (reusability) serta kemudahan pemeliharaan (maintainability) sistem. Berdasarkan hasil praktik pembuatan komponen sederhana menggunakan React, diperoleh bahwa satu komponen dapat digunakan kembali pada berbagai bagian aplikasi tanpa perlu menuliskan ulang kode yang sama. Temuan tersebut sejalan dengan konsep dalam rekayasa perangkat lunak yang menyatakan bahwa modularisasi mampu mengurangi kompleksitas sistem sekaligus meningkatkan efisiensi proses pengembangan.

Dari aspek performa, teori menjelaskan bahwa React memanfaatkan mekanisme virtual DOM untuk mengoptimalkan proses pembaruan tampilan. Pada praktiknya, ketika terjadi perubahan data pada komponen, React hanya memperbarui bagian antarmuka yang mengalami perubahan tanpa melakukan render ulang pada seluruh halaman. Hasil tersebut menunjukkan kesesuaian antara teori dan implementasi karena proses rendering menjadi lebih cepat, efisien, dan responsif dibandingkan pendekatan konvensional yang mengandalkan manipulasi DOM secara langsung.

Selain itu, teori mengenai pengiriman data antar komponen melalui props menekankan pentingnya konsistensi dan kejelasan struktur data agar aplikasi dapat berjalan dengan baik. Dalam pelaksanaannya, props yang dikirim dengan format dan tipe data yang sesuai mampu ditampilkan oleh komponen tanpa menimbulkan kesalahan. Hal ini membuktikan bahwa prinsip konsistensi data yang dijelaskan dalam teori pemrograman memiliki pengaruh nyata terhadap keberhasilan implementasi dan stabilitas sistem yang dibangun.

G.2 Kendala yang Ditemukan dan Solusinya

Kendala / Error yang Ditemukan	Solusi yang Diterapkan
Terjadi error saat menjalankan perintah npm create vite@latest karena Node.js atau npm belum terinstal atau versinya tidak sesuai.	Mengatasi masalah ini dapat dilakukan dengan menginstal atau memperbarui Node.js ke versi terbaru, kemudian memastikan instalasi berhasil dengan menjalankan perintah node -v dan npm -v di terminal.
Komponen tidak tampil pada halaman karena kesalahan penulisan nama komponen atau kesalahan import file.	Hal ini dapat diperbaiki dengan mengecek kembali penamaan komponen agar sesuai dengan nama file serta memastikan path

	import sudah benar, termasuk penggunaan huruf besar dan kecil.
Terjadi error pada sintaks JSX karena penulisan tag tidak tertutup atau terdapat lebih dari satu elemen utama dalam satu komponen.	Solusinya adalah memperbaiki struktur JSX dengan memastikan semua tag ditutup dengan benar serta membungkus elemen menggunakan satu parent element atau React fragment. Solusinya adalah memperbaiki struktur JSX dengan memastikan semua tag ditutup dengan benar serta membungkus elemen menggunakan satu parent element atau React fragment.

G.3 Kaitan dengan Penerapan di Industri

Dalam era industri modern, khususnya pada bidang manufaktur dan logistik, banyak perusahaan telah memanfaatkan sistem berbasis web untuk memantau aktivitas operasional secara langsung (real-time). React banyak digunakan untuk membangun dashboard produksi yang menampilkan berbagai informasi seperti jumlah hasil produksi, kondisi mesin, hingga tingkat efisiensi kerja. Melalui pendekatan berbasis komponen, pengembangan fitur baru dapat dilakukan tanpa perlu mengubah keseluruhan sistem, sehingga proses pengembangan menjadi lebih cepat dan biaya operasional dapat ditekan.

Selain itu, React juga diterapkan dalam pengembangan sistem manajemen rantai pasok atau supply chain management. Pada sistem ini, data dari berbagai bagian seperti gudang, produksi, dan distribusi perlu diintegrasikan dan ditampilkan dalam satu tampilan yang saling terhubung. Penggunaan props memungkinkan pertukaran data dari berbagai sumber secara dinamis ke dalam antarmuka pengguna, sehingga informasi dapat disajikan dengan lebih efektif dan mendukung pengambilan keputusan yang lebih cepat dan tepat dalam lingkungan industri yang kompetitif.

Penerapan React juga sejalan dengan perkembangan transformasi digital dan konsep Industry 4.0, di mana sistem informasi dituntut mampu mengelola data dalam jumlah besar secara efisien. Dengan arsitektur yang fleksibel dan modular, React juga mempermudah integrasi dengan teknologi lain seperti Internet of Things (IoT) serta sistem analitik data. Oleh karena itu, pemahaman mengenai React tidak hanya penting dalam bidang teknologi informasi, tetapi juga relevan bagi mahasiswa Teknik Industri yang akan berperan dalam pengembangan serta pengelolaan sistem industri berbasis digital.

H. KESIMPULAN

1	Proyek React menggunakan Vite berhasil disiapkan dan dapat dijalankan dengan baik pada lingkungan pengembangan tanpa adanya kendala konfigurasi.
2	Penggunaan JSX memungkinkan tampilan antarmuka dan data ditampilkan secara dinamis dalam satu file komponen.
3	Mekanisme komunikasi data antar komponen melalui props berjalan dengan baik sehingga alur pertukaran data dalam aplikasi dapat dipahami.
4	Integrasi variabel JavaScript ke dalam JSX berhasil diterapkan sehingga data dapat ditampilkan sesuai dengan logika pemrograman yang dibuat.

I. DAFTAR PUSTAKA

No.	Referensi (Format APA 7th Edition)
[1]	Burhandenny, A. E. (2026). Manual Praktikum Aplikasi Web dan Mobile Semester Genap 2025/2026. Program Studi Teknik Industri, Universitas Negeri Yogyakarta.
[2]	https://react.dev/learn
[3]	https://www.javascripttutorial.net/nodejs-tutorial/
[4]	https://www.javascripttutorial.net/react-tutorial/

J. LEMBAR PENILAIAN DAN PENGESAHAN

RUBRIK PENILAIAN LAPORAN					
Aspek Penilaian		Bobot			
No.	Aspek Penilaian	Bobot (%)	Nilai (0-100)	Skor Akhir	Catatan Penilai
1	Kelengkapan Isi (Semua bagian A–I terisi lengkap)	20%			
2	Kualitas Dasar Teori (Ditulis dengan bahasa sendiri, relevan, minimal 3 paragraf)	15%			
3	Dokumentasi Langkah Kerja (Kode & Screenshot sesuai, jelas, dan terbaca)	25%			
4	Tugas/Latihan Mandiri (Semua latihan & proyek mini dikerjakan dan berfungsi)	20%			
5	Pembahasan & Analisis (Kritis, mendalam, dan relevan dengan industri)	15%			
6	Kesimpulan & Tata Bahasa (Spesifik, rapi, mengikuti format yang ditentukan)	5%			
TOTAL NILAI AKHIR		100%			

Mahasiswa	Rekan Tim	Diperiksa oleh Asisten / Dosen
 _____ Saprina Melita Sari NIM:23051430012	 _____ Ardita Natalia NIM:23051430022	 _____ Dr. Eng. Ir. Aji Ery Burhandenny, S.T., M.AIT. Tanggal: _____

MODUL 10

A. IDENTITAS & INFORMASI MODUL

Mata Kuliah	Aplikasi Web dan Mobile
Kode Modul	Modul 10
Topik Utama	State, Hooks (useState, useEffect), dan Conditional Rendering
Sub-Topik	State Management, Lifecycle Hooks, dan Rendering Bersyarat.
Durasi Praktikum	340 Menit
Tanggal Praktikum	28 April 2026
Nama Mahasiswa	Saprina Melita Sari
NIM	23051430031
Kelas	A1

B. TUJUAN PEMBELAJARAN

1	Memahami apa itu State dan perbedaannya dengan Props.
2	Menguasai penggunaan `useState` untuk menyimpan data yang berubah-ubah.
3	Memahami `useEffect` untuk menangani side effects (seperti timer atau API call).
4	Mampu melakukan Conditional Rendering (menyembunyikan / menampilkan elemen berdasarkan kondisi).

C. ALAT DAN BAHAN

No.	Nama Alat / Bahan / Aplikasi	Versi / Spesifikasi	Keterangan
1	Visual Studio Code (VS Code)	Versi 1.8x ke atas	Code Editor

2	Google Chrome / Browser Modern	Versi terbaru	Testing & Debugging
3	Node.js & npm	Node 18+ / npm 9+	Runtime JS (Modul tertentu)
4	Prettier	Versi 12.3.0	Extensions VS Code untuk merapikan indentansi code

D. DASAR TEORI

D.1 Konsep Utama

Konsep utama dalam materi ini mencakup state, hooks, dan conditional rendering pada aplikasi berbasis JavaScript modern. State berfungsi sebagai tempat penyimpanan data yang dapat berubah selama aplikasi berjalan, seperti jumlah produksi, status mesin, atau durasi proses. Ketika nilai dalam state berubah, tampilan aplikasi akan otomatis diperbarui tanpa perlu melakukan refresh halaman, sehingga aplikasi menjadi lebih interaktif dan responsif.

Hooks seperti `useState` dan `useEffect` digunakan untuk membantu pengembang dalam mengelola perilaku komponen. `useState` digunakan untuk mendeklarasikan dan memperbarui data, sedangkan `useEffect` berfungsi untuk menjalankan proses tambahan seperti pengambilan data dari server atau perhitungan ulang ketika terjadi perubahan. Dengan adanya hooks, struktur logika program menjadi lebih rapi dan mudah dikelola.

Conditional rendering merupakan teknik untuk menampilkan atau menyembunyikan elemen berdasarkan kondisi tertentu. Misalnya, ketika mesin dalam keadaan aktif maka sistem menampilkan status "Aktif", sedangkan ketika mesin berhenti akan ditampilkan status "Maintenance". Teknik ini membuat tampilan aplikasi lebih sesuai dengan kondisi nyata di lapangan.

D.2 Keterkaitan dengan Teknik Industri

Dalam konteks Teknik Industri, state dapat dianalogikan sebagai data operasional yang selalu berubah, seperti output produksi, waktu operasional mesin, atau jumlah produk cacat. Data tersebut perlu terus diperbarui agar pengambilan keputusan oleh manajemen dapat dilakukan secara tepat.

Hooks memiliki hubungan dengan proses pemantauan dan pengendalian sistem. Sebagai contoh, perubahan kondisi mesin dalam sistem produksi harus segera terdeteksi dan ditindaklanjuti. `useEffect` dapat

diibaratkan sebagai mekanisme otomatis yang aktif ketika terjadi perubahan, seperti sistem peringatan saat terjadi gangguan atau penurunan performa.

Sementara itu, conditional rendering berkaitan dengan proses pengambilan keputusan berbasis kondisi dalam Teknik Industri. Contohnya, sistem hanya menampilkan notifikasi ketika nilai efisiensi berada di bawah batas standar. Dengan pendekatan ini, informasi yang disajikan menjadi lebih relevan dan mudah dipahami oleh pengguna.

E. LANGKAH KERJA DAN HASIL PRAKTIKUM

E.1 Langkah 1 – Membuat Komponen Counter Produksi

Deskripsi singkat apa yang dilakukan pada langkah ini:

```
/* ===== KODE / OUTPUT LANGKAH 1 ===== */
import React, { useState } from 'react';
function CounterProduksi() {
  // Deklarasi State: 'jumlah' bernilai awal 0, 'setJumlah'
  fungsi untuk mengubahnya
  const [jumlah, setJumlah] = useState(0);
  const [target, setTarget] = useState(100);
  // Fungsi menambah produksi
  const tambahProduksi = () => {
    setJumlah(jumlah + 1);
  };
  // Fungsi reset
  const reset = () => {
    setJumlah(0);
  };
  return (
    <div className="text-center p-4 border rounded bg-light">
      <h3>Simulasi Hitung Produk</h3>
      <h1 className="display-4">{jumlah}</h1>
      <p>Target: {target} Unit</p>

      {/* Conditional Rendering: Tampilkan pesan jika
target tercapai */}
      {jumlah >= target ? (
        <div className="alert alert-success d-inline-block">Target Tercapai!</div>
      ) : (
        <div className="alert alert-secondary d-inline-block">Produksi Berjalan...</div>
      )}
    </div>
  );
}
```

```

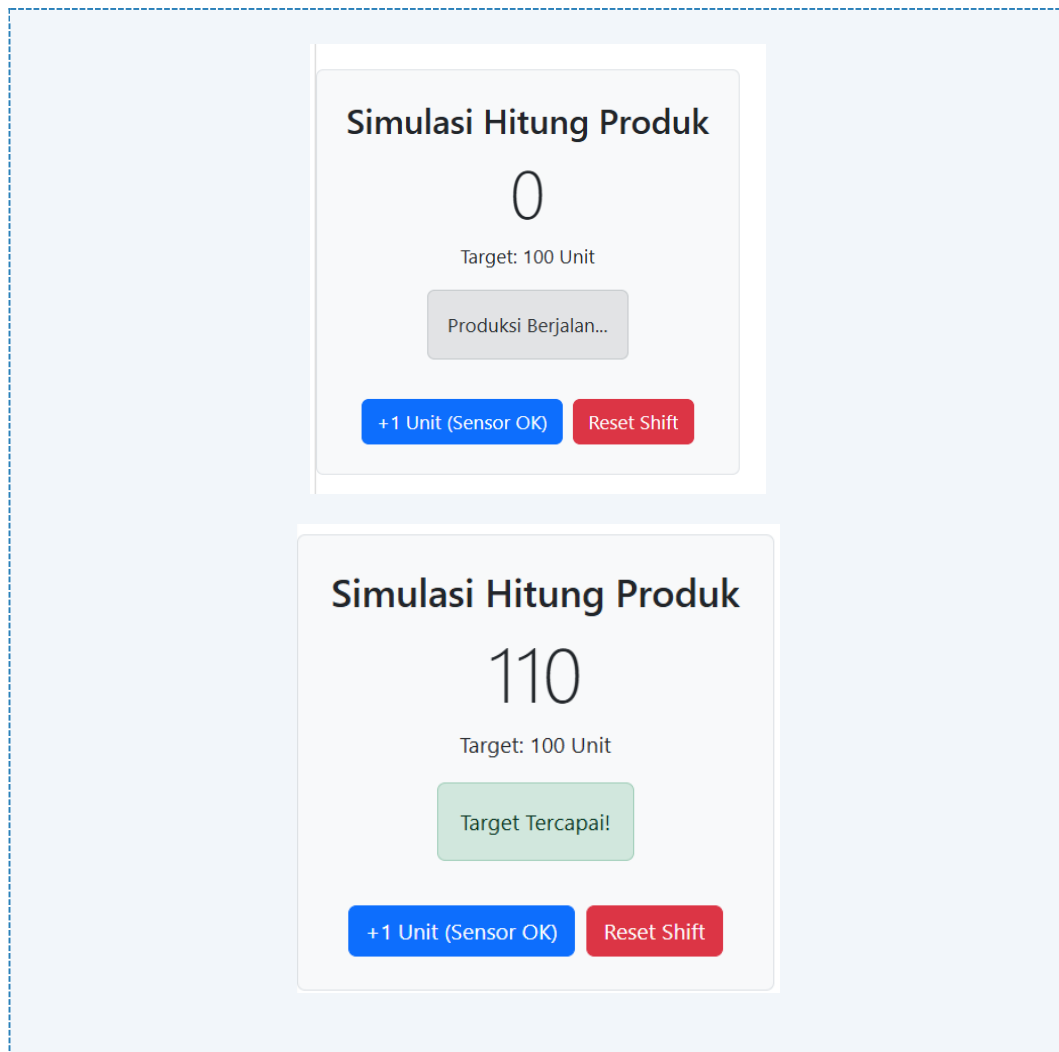
        <div className="mt-3">
          <button className="btn btn-primary me-2"
onClick={tambahProduksi}>
            +1 Unit (Sensor OK)
          </button>
          <button className="btn btn-danger"
onClick={reset}>
            Reset Shift
          </button>
        </div>
      </div>
    );
  }
  export default CounterProduksi;

//Di file app.jsx
  <div
    className="card border-0 mb-4"
    style={{
      borderRadius: '16px',
      boxShadow: '0 8px 20px rgba(0,0,0,0.08)'
    }}
  >
    <div className="card-body p-4 text-center">

      <div
        style={{
          background: '#f8fafc',
          borderRadius: '12px',
          padding: '20px'
        }}
      >
        <CounterProduksi />
      </div>
    </div>
  </div>

```

Screenshot Hasil:



Gambar E.1 – Hasil Pembuatan komponen CounterProduksi.jsx

E.2 Langkah 2 – Menggunakan useEffect untuk Real-time Clock

```

/* ===== KODE / OUTPUT LANGKAH 2 ===== */
import React, { useState, useEffect } from 'react';

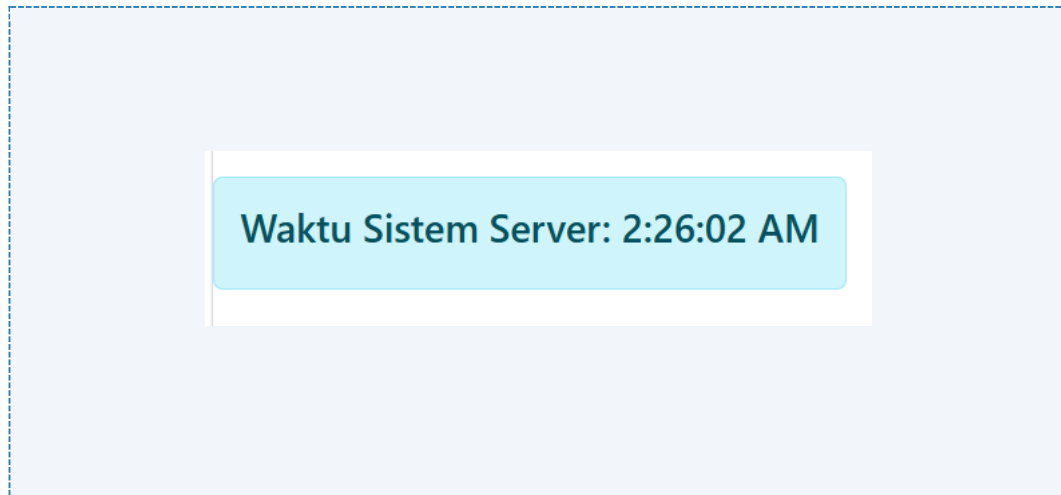
function JamDigital() {
  const [waktu, setWaktu] = useState(new Date());
  // useEffect berjalan sekali setelah komponen dirender
  pertama kali
  useEffect(() => {
    // Membuat interval timer
    const timerID = setInterval(() => {
      setWaktu(new Date()); // Update state waktu setiap
detik
    }, 1000);
    // Cleanup function: Dijalankan saat komponen
dihapus/hancur
  });
}

```

```
// Penting untuk mencegah memory leak
return () => {
  clearInterval(timerID);
};
}, []); // Array kosong [] artinya hanya dijalankan sekali
saat mount
return (
  <div className="alert alert-info text-center">
    <h4>Waktu Sistem Server:
{waktu.toLocaleTimeString()}</h4>
  </div >
);
}
export default JamDigital;

//File app.jsx
<div
  className="card border-0"
  style={{
    borderRadius: '16px',
    boxShadow: '0 8px 20px rgba(0,0,0,0.08)'
  }}
>
  <div className="card-body p-4 text-center">
    <div
      style={{
        background: '#f8fafc',
        borderRadius: '12px',
        padding: '20px'
      }}
    >
      <JamDigital />
    </div>
  </div>
</div>
```

Screenshot Hasil:



Gambar E.2 – Hasil Pembuatan komponen JamDigital.jsx

E.3 Langkah 3 – Contoh Form Input State

```
/* ===== KODE / OUTPUT LANGKAH 3 ===== */
// 1. Import React dan useState
import React, { useState } from 'react';

// 2. Komponen Fungsi
function KartuMesin(props) {

  // Menerima data dari props
  const namaMesin = props.nama;
  const status = props.status;
  const produksi = props.produksi;

  // State lokal untuk status
  const [statusLokal, setStatusLokal] = useState(status);

  // Logika penentuan warna badge berdasarkan statusLokal
  let badgeColor = 'bg-secondary';
  if (statusLokal === 'Running') badgeColor = 'bg-success';
  if (statusLokal === 'Stop') badgeColor = 'bg-danger';
  if (statusLokal === 'Maintenance') badgeColor = 'bg-warning';

  return (
    <div className="card shadow-sm p-3 mb-3">
      <div className="card-body">
        <h5 className="card-title">{namaMesin}</h5>

        <span className={`badge ${badgeColor}`}>
          {statusLokal}
        </span>
      </div>
    </div>
  );
}
```

```

        <hr />

        <p>
            Produksi Saat Ini:
<strong>{produksi}</strong> Unit
        </p>

        <div className="mt-2">
            <select
                className="form-select form-select-sm"
                value={statusLokal}
                onChange={(e) =>
setStatusLokal(e.target.value)}
            >
                <option value="Running">Running</option>
                <option value="Stop">Stop</option>
                <option
value="Maintenance">Maintenance</option>
            </select>
        </div>
    </div>
</div>
    );
}

// 3. Export
export default KartuMesin;

// Di app.jsx
<div
    className="card border-0 mb-5"
    style={{
        borderRadius: '16px',
        boxShadow: '0 8px 20px rgba(0,0,0,0.08)'
    }}
>
    <div className="card-body p-4">
        <h3 className="text-center fw-semibold mb-4">
            Praktik 3 (Pembuatan form input state pada
Monitoring Lini Produksi A)
        </h3>

        <div className="row g-4">
            <div className="col-md-4">
                <div className="h-100 hover-card">

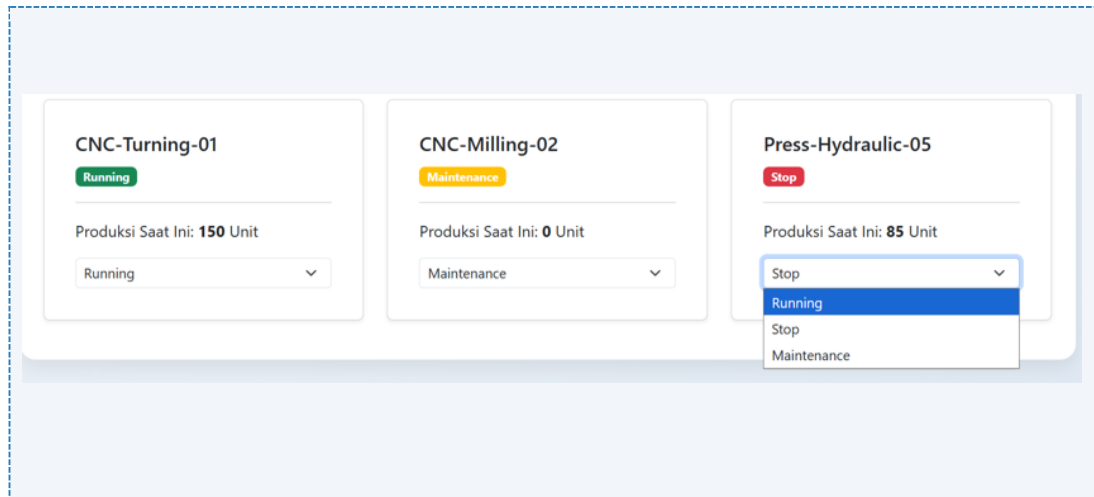
```

```
        <KartuMesin
            nama="CNC-Turning-01"
            status="Running"
            produksi={150}
        />
    </div>
</div>

<div className="col-md-4">
    <div className="h-100 hover-card">
        <KartuMesin
            nama="CNC-Milling-02"
            status="Maintenance"
            produksi={0}
        />
    </div>
</div>

<div className="col-md-4">
    <div className="h-100 hover-card">
        <KartuMesin
            nama="Press-Hydraulic-05"
            status="Stop"
            produksi={85}
        />
    </div>
</div>
</div>
</div>
```

Screenshot Hasil:



Gambar E.3– Hasil Pembuatan Form Input State

F. LATIHAN DAN TUGAS MANDIRI

F.1 Latihan 1 – Dependency Array useEffect

Pertanyaan / Instruksi Latihan:

Modifikasi `JamDigital`. Tambahkan input text untuk memasukkan nama kota. Gunakan `useEffect` untuk mengubah `document.title` menjadi "Jam [Nama Kota]". Gunakan state kota sebagai dependency array `[kota]`

Jawaban / Kode Solusi:

```
import React, { useState, useEffect } from 'react';

function JamDigital() {
  const [waktu, setWaktu] = useState(new Date());
  const [kota, setKota] = useState("Yogyakarta");

  // Effect 1: timer (mount only)
  useEffect(() => {
    const id = setInterval(() => setWaktu(new Date()), 1000);
    return () => clearInterval(id);
  }, []);

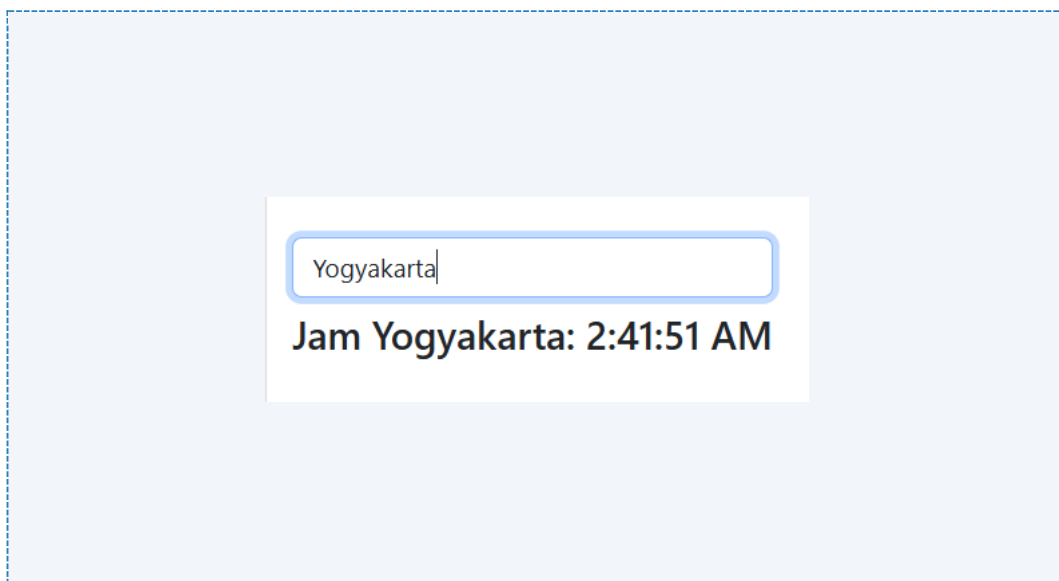
  // Effect 2: update title saat kota berubah
  useEffect(() => {
    document.title = `Jam ${kota}`;
  }, [kota]); // Dependency array berisi kota

  return (
    <div className="text-center p-3">
      <input value={kota}
        onChange={(e) => setKota(e.target.value)}
      />
    </div>
  );
}
```



```
        className="form-control mb-2" />
        <h4>Jam {kota}: {waktu.toLocaleTimeString()}</h4>
      </div>
    );
  }
export default JamDigital;

// Di app.jsx
<div
  className="card border-0"
  style={{
    borderRadius: '16px',
    boxShadow: '0 8px 20px rgba(0,0,0,0.08)'
  }}
>
  <div className="card-body p-4 text-center">
    <div
      style={{
        background: '#f8fafc',
        borderRadius: '12px',
        padding: '20px'
      }}
    >
      <JamDigital />
    </div>
  </div>
</div>
```



Gambar F.1 – Hasil Pembuatan Dependency Array useEffect

F.2 Latihan 2 – Conditional Rendering

Pertanyaan / Instruksi Latihan:

Pada `CounterProduksi`, buat tombol "Emergency Stop". Saat diklik, ubah state status menjadi "EMERGENCY", tombol "+1" menjadi disabled (`disabled`), dan tampilkan pesan merah.

Jawaban / Kode Solusi:

```
import React, { useState } from 'react';
function CounterProduksi() {
  const [jumlah, setJumlah] = useState(0);
  const [emergency, setEmergency] = useState(false);

  const handleEmergency = () => setEmergency(true);
  const reset = () => {
    setJumlah(0);
    setEmergency(false); // reset emergency juga
  };

  return (
    <div className="text-center p-4">
      <h1>{jumlah}</h1>

      {emergency && (
        <div className="alert alert-danger">
          🚨 EMERGENCY STOP AKTIF – Hubungi supervisor
        </div>
      )}

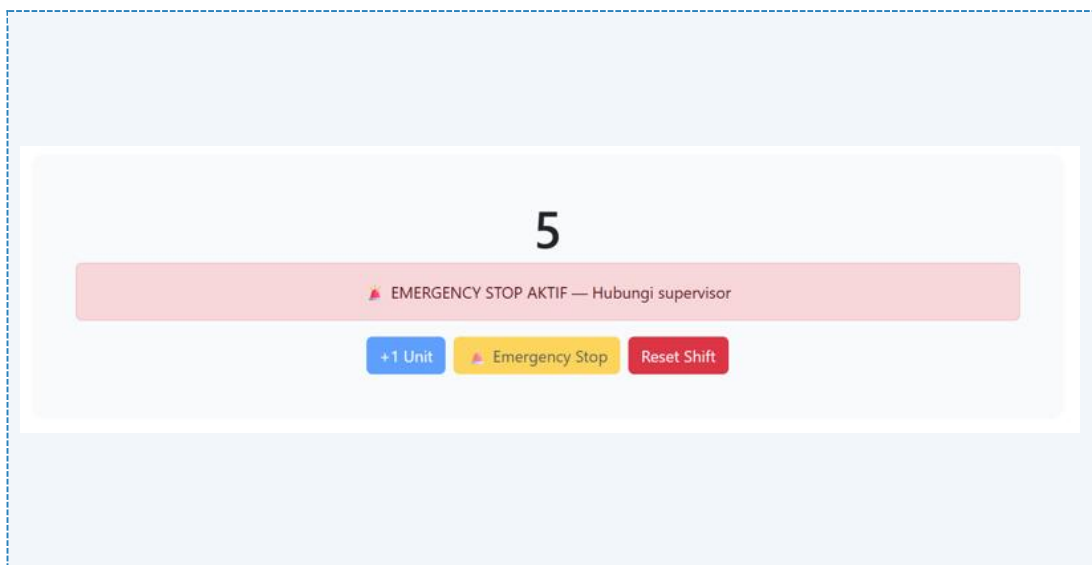
      <button className="btn btn-primary me-2"
        disabled={emergency}
        onClick={() => setJumlah(jumlah + 1)}>+1
      </button>

      <button className="btn btn-warning me-2"
        onClick={handleEmergency}
        disabled={emergency}> 🚨 Emergency Stop</button>

      <button className="btn btn-danger"
        onClick={reset}>Reset Shift</button>
    </div>
  );
}
export default CounterProduksi;
```

```
// Di app.jsx
<div
  className="card border-0 mb-4"
  style={{
    borderRadius: '16px',
    boxShadow: '0 8px 20px rgba(0,0,0,0.08)'
  }}
>
  <div className="card-body p-4 text-center">

    <div
      style={{
        background: '#f8fafc',
        borderRadius: '12px',
        padding: '20px'
      }}
    >
      <CounterProduksi />
    </div>
  </div>
</div>
```



Gambar F.2 – Hasil Pembuatan button emergency stop

F.3 Tugas Proyek Mini – Kalkulator OEE Sederhana

Deskripsi proyek mini:

- a. Buat komponen form input: `Plan Time`, `Run Time`, `Total Parts`, `Good Parts`.
- b. Gunakan State untuk menyimpan nilai-nilai ini.
- c. Hitung OEE secara otomatis (Real-time) setiap kali input berubah.
- d. Rumus OEE = (Availability x Performance x Quality).
- e. Tampilkan hasil OEE dalam persen. Jika OEE < 50%, warnanya merah. Jika > 85%, warnanya hijau. Tampilkan komponen ini 3 kali di `App.jsx` dengan data karyawan berbeda (Manager, Operator, QC).

Penjelasan pendekatan/solusi yang digunakan:

Pendekatan yang digunakan dalam program ini adalah pengukuran kinerja produksi dengan metode Overall Equipment Effectiveness (OEE). Pada sistem ini, efektivitas mesin atau proses produksi dihitung berdasarkan tiga komponen utama, yaitu Availability untuk mengukur tingkat ketersediaan waktu operasi mesin dari waktu yang telah direncanakan, Performance untuk menilai kesesuaian kecepatan produksi terhadap standar, serta Quality untuk mengetahui jumlah produk yang dihasilkan dalam kondisi baik. Ketiga komponen tersebut kemudian dikombinasikan menjadi satu nilai OEE dalam bentuk persentase yang menunjukkan tingkat efektivitas proses produksi secara keseluruhan.

Dari sisi implementasi pemrograman, pendekatan yang digunakan adalah reactive programming dengan React. Setiap data seperti waktu produksi dan jumlah output disimpan dalam bentuk state, sehingga ketika terjadi perubahan nilai, sistem secara otomatis menghitung ulang nilai OEE dan memperbarui tampilan secara langsung. Hal ini membuat aplikasi lebih responsif dan memungkinkan pengguna melihat hasil secara real-time. Selain itu, digunakan pula pendekatan komponen modular, yaitu dengan memecah program menjadi beberapa bagian terpisah seperti komponen input dan komponen tampilan faktor. Pendekatan ini membuat struktur kode lebih terorganisir, mudah dipahami, serta lebih mudah dikembangkan apabila diperlukan penambahan fitur di kemudian hari. Terakhir, diterapkan aturan sederhana berupa threshold untuk mengelompokkan hasil OEE, misalnya kategori "World Class" untuk nilai tinggi dan "Need Improvement" untuk nilai rendah, sehingga hasil perhitungan dapat diinterpretasikan dengan cepat dan mudah.

Kode Utama:

```
import React, { useState } from "react";

function KalkulatorOEE() {
  // STATE
  const [planTime, setPlanTime] = useState(480);
  const [runTime, setRunTime] = useState(420);
  const [totalParts, setTotalParts] = useState(800);
  const [goodParts, setGoodParts] = useState(760);

  const standar = 2;

  // PERHITUNGAN
  const availability = planTime > 0 ? runTime / planTime : 0;
  const performance =
    runTime > 0 ? totalParts / (runTime * standar) : 0;
  const quality =
    totalParts > 0 ? goodParts / totalParts : 0;

  const oee =
    availability * performance * quality * 100;

  // WARNA DINAMIS
  let warna = "#ffc107";
  let bgStatus = "#fff3cd";
  let label = "Acceptable";

  if (oee >= 85) {
    warna = "#198754";
    bgStatus = "#d1e7dd";
    label = "World Class";
  }

  if (oee < 50) {
    warna = "#dc3545";
    bgStatus = "#f8d7da";
    label = "Need Improvement";
  }

  return (
    <div
      className="card shadow-lg border-0 p-4"
      style={{
        borderRadius: "18px",
        background:
          "linear-gradient(135deg,#f8fafc,#e9f2ff)"
      }}
    >
```

```

>
<h3 className="text-center fw-bold mb-4">
  Kalkulator OEE
</h3>

{/* INPUT */}
<div className="row g-3 mb-4">
  <InputBox
    label="Plan Time (min)"
    value={planTime}
    setValue={setPlanTime}
  />

  <InputBox
    label="Run Time (min)"
    value={runTime}
    setValue={setRunTime}
  />

  <InputBox
    label="Total Parts"
    value={totalParts}
    setValue={setTotalParts}
  />

  <InputBox
    label="Good Parts"
    value={goodParts}
    setValue={setGoodParts}
  />
</div>

<hr />

{/* OEE SCORE */}
<div
  className="text-center mb-4 p-4"
  style={{
    background: bgStatus,
    borderRadius: "14px"
  }}
>
  <h6>OEE Score</h6>

  <h1
    className="display-1 fw-bold"

```

```
        style={{
          color: warna,
          textShadow:
            "0 5px 15px rgba(0,0,0,0.15)"
        }}
      >
        {oeo.toFixed(1)}%
      </h1>

      <span
        className="badge fs-6 px-3 py-2"
        style={{
          background: warna
        }}
      >
        {label}
      </span>
    </div>

    { /* FAKTOR */ }
    <div className="row text-center">

      <FactorBox
        title="Availability"
        value={availability}
        color="#198754"
        bg="#e8f5e9"
      />

      <FactorBox
        title="Performance"
        value={performance}
        color="#0d6efd"
        bg="#e3f2fd"
      />

      <FactorBox
        title="Quality"
        value={quality}
        color="#fd7e14"
        bg="#fff3e0"
      />

    </div>
  </div>
);
```

```

}

/* COMPONENT INPUT */

function InputBox({ label, value, setValue }) {
  return (
    <div className="col-md-3">
      <label className="fw-semibold mb-1">
        {label}
      </label>

      <input
        type="number"
        className="form-control shadow-sm"
        style={{
          borderRadius: "10px"
        }}
        value={value}
        onChange={(e) =>
          setValue(
            e.target.value === ""
              ? ""
              : Number(e.target.value)
          )
        }
      />
    </div>
  );
}

/* COMPONENT FAKTOR */

function FactorBox({ title, value, color, bg }) {
  const percent = (value * 100).toFixed(1);

  return (
    <div className="col-md-4">
      <div
        className="p-3 shadow-sm"
        style={{
          borderRadius: "14px",
          background: bg,
          borderLeft:
            "6px solid " + color
        }}
      >

```



```
        <h6 className="text-muted">
          {title}
        </h6>

        <h4
          className="fw-bold"
          style={{
            color: color
          }}
        >
          {percent}%
        </h4>

        { /* PROGRESS BAR */ }

        <div className="progress mt-2">
          <div
            className="progress-bar"
            role="progressbar"
            style={{
              width: percent + "%",
              backgroundColor: color
            }}
          />
        </div>

      </div>
    </div>
  );
}

export default KalkulatorOEE;

//Di app.jsx
import React from 'react';
import KalkulatorOEE from './komponen/KalkulatorOEE';

function App() {
  return (
    <div
      style={{
        minHeight: '100vh',
        background: 'linear-gradient(135deg, #eef2f7, #dfe9f3)',
        display: 'flex',
        justifyContent: 'center'
      }}
    >
```

```
        className="py-5"
    >
    <div
        className="container"
        style={{ maxWidth: '1100px', margin: '0 auto' }}
    >

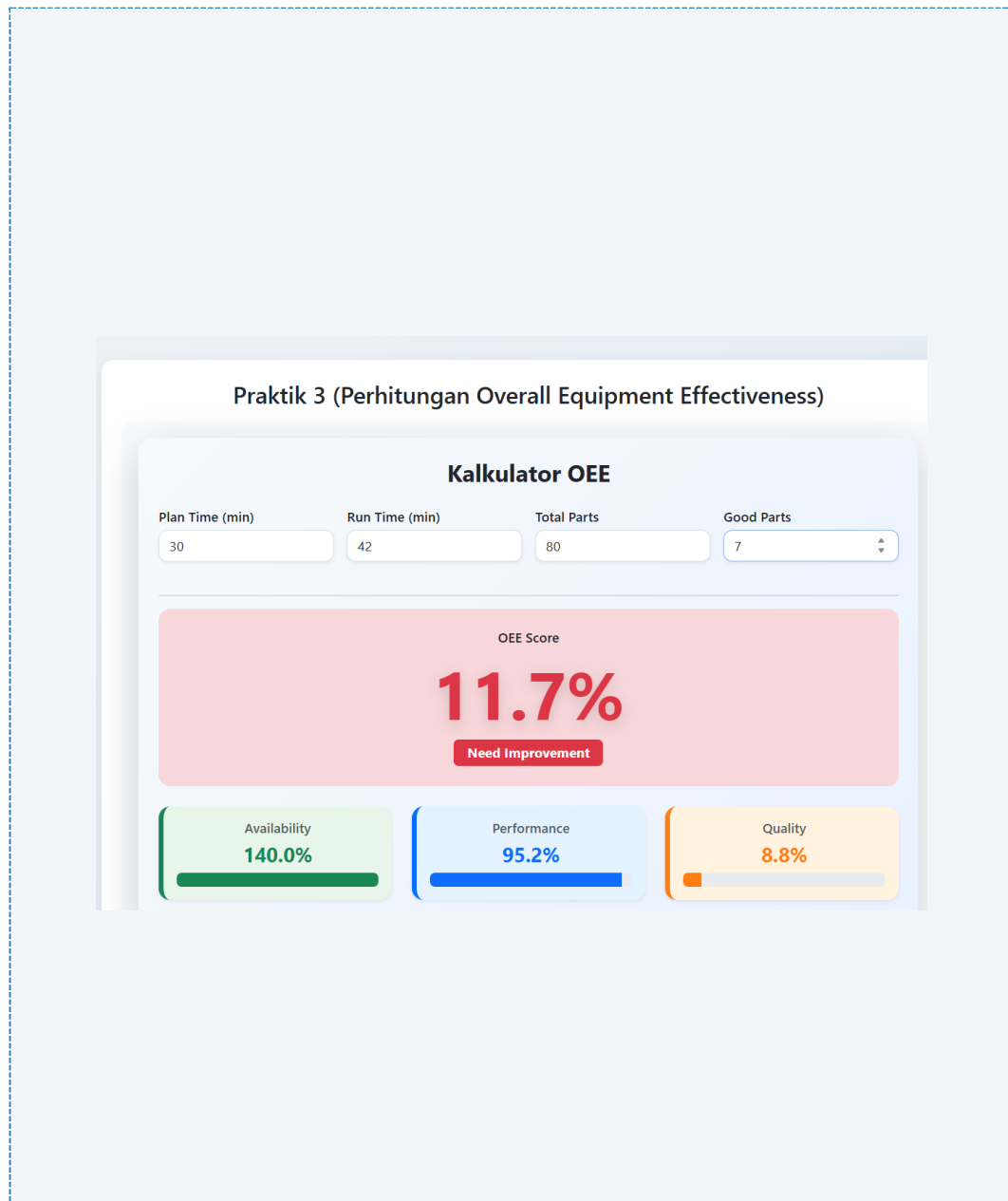
        { /* HEADER */ }
        <div className="text-center mb-5">
            <h1
                className="fw-bold"
                style={{ letterSpacing: '0.5px' }}

            <span
                className="badge px-4 py-2"
                style={{
                    backgroundColor: '#2563eb',
                    fontSize: '14px',
                    borderRadius: '8px'
                }}
            >
                Praktik Kalkulator OEE
            </span>
        </div>

        { /* Section Kalkulator OEE */ }
        <div
            className="card border-0 mb-4"
            style={{
                borderRadius: '16px',
                boxShadow: '0 8px 20px rgba(0,0,0,0.08)'
            }}
        >
            <div className="card-body p-4">
                <div
                    style={{
                        background: '#f8fafc',
                        borderRadius: '12px',
                        padding: '20px'
                    }}
                >
                    <KalkulatorOEE />
                </div>
            </div>
        </div>
    </div>
```

```
    </div>
  </div>
);
}

export default App;
```



Gambar F.3 – Hasil Pembuatan ProyekMini Kalkulator OEE

G. PEMBAHASAN

G.1 Analisis Kesesuaian Hasil dengan Teori

Secara teori, perubahan pada state seharusnya menyebabkan komponen melakukan render ulang. Apabila hasil implementasi menunjukkan bahwa tampilan langsung ikut berubah setelah data diperbarui, maka sistem tersebut telah sesuai dengan konsep dasar React. Hal ini menunjukkan bahwa mekanisme pengelolaan state telah berjalan dengan benar. Penggunaan `useEffect` juga dapat dianalisis berdasarkan waktu eksekusi prosesnya. Jika fungsi dijalankan saat komponen pertama kali ditampilkan atau ketika data mengalami perubahan, maka perilaku tersebut sudah sesuai dengan konsep lifecycle pada React. Sebaliknya, jika tidak berjalan sebagaimana mestinya, biasanya terdapat kesalahan pada dependency array atau penempatan kode. Sementara itu, conditional rendering dapat diuji dengan memberikan berbagai kondisi pada data. Jika elemen dapat muncul atau menghilang sesuai dengan aturan yang ditentukan, maka implementasinya sudah tepat. Namun, jika tampilan tidak berubah meskipun kondisi sudah berbeda, maka kemungkinan terdapat kesalahan pada logika kondisi atau pengelolaan state.

G.2 Kendala yang Ditemukan dan Solusinya

Kendala / Error yang Ditemukan	Solusi yang Diterapkan
tampilan aplikasi belum sepenuhnya menyesuaikan kondisi data yang ada, misalnya status yang ditampilkan tidak berubah meskipun nilai data telah diperbarui.	Dapat diatasi dengan mengevaluasi dan memperbaiki logika kondisi pada proses rendering

G.3 Kaitan dengan Penerapan di Industri

Hooks seperti `useEffect` umumnya dimanfaatkan untuk mengambil data dari sensor atau basis data secara berkala. Sebagai contoh, sistem dapat secara otomatis melakukan pembaruan data setiap beberapa detik guna memantau kinerja mesin. Dengan demikian, proses pengawasan menjadi lebih cepat dan akurat dibandingkan metode konvensional.

H. KESIMPULAN

1	Hook seperti useState dan useEffect membantu mengatur logika program.
2	State digunakan untuk mengelola data yang berubah.

I. DAFTAR PUSTAKA

No.	Referensi (Format APA 7th Edition)
[1]	Burhandenny, A. E. (2026). Manual Praktikum Aplikasi Web dan Mobile Semester Genap 2025/2026. Program Studi Teknik Industri, Universitas Negeri Yogyakarta.
[2]	https://react.dev/learn
[3]	https://react.dev/reference/react-dom/hooks/useFormStatus
[4]	https://www.javascripttutorial.net/react-tutorial/

J. LEMBAR PENILAIAN DAN PENGESAHAN

RUBRIK PENILAIAN LAPORAN					
Aspek Penilaian		Bobot			
No.	Aspek Penilaian	Bobot (%)	Nilai (0-100)	Skor Akhir	Catatan Penilai
1	Kelengkapan Isi (Semua bagian A–I terisi lengkap)	20%			
2	Kualitas Dasar Teori (Ditulis dengan bahasa sendiri, relevan, minimal 3 paragraf)	15%			
3	Dokumentasi Langkah Kerja (Kode & Screenshot sesuai, jelas, dan terbaca)	25%			
4	Tugas/Latihan Mandiri (Semua latihan & proyek mini dikerjakan dan berfungsi)	20%			
5	Pembahasan & Analisis (Kritis, mendalam, dan relevan dengan industri)	15%			
6	Kesimpulan & Tata Bahasa (Spesifik, rapi, mengikuti format yang ditentukan)	5%			
TOTAL NILAI AKHIR		100%			

Mahasiswa	Rekan Tim	Diperiksa oleh Asisten / Dosen
 _____ Saprina Melita Sari NIM:23051430012	 _____ Ardita Natalia NIM:23051430022	 _____ Dr. Eng. Ir. Aji Ery Burhandenny, S.T., M.AIT. Tanggal: _____

MODUL 11

A. IDENTITAS & INFORMASI MODUL

Mata Kuliah	Aplikasi Web dan Mobile
Kode Modul	Modul 11
Topik Utama	React Routing & Multi-Page App + Integrasi API
Sub-Topik	Single Page Application (SPA) Navigation, React Router DOM, dan Fetch API dalam React.
Durasi Praktikum	340 Menit
Tanggal Praktikum	07 Mei 2026
Nama Mahasiswa	Saprina Melita Sari
NIM	23051430031
Kelas	A1

B. TUJUAN PEMBELAJARAN

1	Memahami konsep SPA (Single Page Application).	
2	Mampu mengatur navigasi antar halaman tanpa reload browser menggunakan `react-router-dom`.	
3	Menerapkan `fetch` API di dalam `useEffect` untuk mengambil data eksternal.	
4	Membuat struktur aplikasi multi-halaman (Dashboard, Inventori, Settings)	

C. ALAT DAN BAHAN

No.	Nama Alat / Bahan / Aplikasi	Versi / Spesifikasi	Keterangan
1	Visual Studio Code (VS Code)	Versi 1.8x ke atas	Code Editor
2	Google Chrome / Browser Modern	Versi terbaru	Testing & Debugging
3	Node.js & npm	Node 18+ / npm 9+	Runtime JS

4	Prettier	Versi 12.3.0	Extensioins VS Code untuk merapikan indentansi code
---	----------	--------------	---

D. DASAR TEORI

D.1 Konsep Utama

Konsep utama dalam materi ini berfokus pada arsitektur Single Page Application (SPA) yang memungkinkan aplikasi berjalan dalam satu halaman tanpa perlu melakukan reload penuh pada browser. Proses perpindahan halaman dilakukan secara dinamis melalui pengelolaan state dan perubahan URL di sisi klien, sehingga memberikan pengalaman pengguna yang lebih cepat dan responsif dibandingkan dengan multi-page application tradisional. Selain itu, penggunaan React Router DOM menjadi komponen utama dalam mengatur navigasi antar komponen yang merepresentasikan halaman, seperti dashboard, inventori, dan laporan, sehingga aplikasi menjadi lebih terstruktur, modular, dan mudah dipelihara. Di samping itu, integrasi API menggunakan fungsi seperti fetch di dalam useEffect memungkinkan aplikasi mengambil data dari sumber eksternal secara asynchronous dan memperbarui tampilan secara otomatis, sehingga frontend tidak hanya bersifat statis tetapi juga dapat berinteraksi secara dinamis dengan backend.

D.2 Keterkaitan dengan Teknik Industri

Dalam konteks industri, SPA banyak digunakan pada aplikasi modern seperti sistem manajemen perusahaan, platform e-commerce, dan dashboard analitik karena mampu memberikan pengalaman pengguna yang cepat, responsif, dan menyerupai aplikasi native. Pendekatan ini mengurangi latensi yang biasanya terjadi akibat proses reload halaman. Penggunaan client-side routing juga mendukung kebutuhan industri dalam membangun sistem yang modular, di mana setiap fitur seperti manajemen stok atau laporan keuangan dapat dikembangkan secara terpisah namun tetap terintegrasi dalam satu kesatuan aplikasi, sehingga proses pengembangan menjadi lebih efisien dan kolaboratif. Selain itu, integrasi API mencerminkan arsitektur sistem industri berbasis client-server atau microservices, di mana frontend berperan sebagai pengakses data dari berbagai layanan backend, sehingga pemahaman mengenai pengambilan data dan pengelolaan state menjadi keterampilan penting dalam pengembangan aplikasi industri modern.

E. LANGKAH KERJA DAN HASIL PRAKTIKUM

E.1 Langkah 1 – Setup Router

```
/* ===== KODE / OUTPUT LANGKAH 1 ===== */
import React from 'react'
import ReactDOM from 'react-dom/client'
import { BrowserRouter } from 'react-router-dom'
import App from './App.jsx'
import './index.css'
ReactDOM.createRoot(document.getElementById('root')).render(
  <React.StrictMode>
    <BrowserRouter>
      <App />
    </BrowserRouter>
  </React.StrictMode>,
)
```

Screenshot Hasil:

E.2 Langkah 2 – Membuat Halaman (Pages)

```
src/Halaman/Dashboard.jsx
import React from 'react';
import { Link } from 'react-router-dom';
function Dashboard() {
  return (
    <div className="container mt-4">
      <h1>Dashboard Utama Pabrik</h1>
      <p>Selamat datang di sistem monitoring terpadu.</p>
      <Link to="/inventori" className="btn btn-primary">
        Ke Halaman Inventori
      </Link>
    </div>
  );
}
export default Dashboard;
src/Halaman/Inventori.jsx
import React, { useState, useEffect } from 'react';
import { Link } from 'react-router-dom';
function Inventori() {
  const [products, setProducts] = useState([]);
  // Fetch Data saat komponen mount
  useEffect(() => {
    fetch('https://jsonplaceholder.typicode.com/posts')
```

```

        .then(res => res.json())
        .then(data => {
            // Ambil hanya 5 data pertama untuk contoh
            setProducts(data.slice(0, 5));
        })
        .catch(err => console.log(err));
    }, []);
    return (
        <div className="container mt-4">
            <h1>Data Inventori Bahan Baku</h1>
            <Link to="/" className="btn btn-secondary mb-3">Kembali ke Dashboard</Link>

            <table className="table table-striped">
                <thead>
                    <tr>
                        <th>ID Item</th>
                        <th>Nama Bahan</th>
                        <th>Status Supplier</th>
                    </tr>
                </thead>
                <tbody>
                    {products.map((item) => (
                        <tr key={item.id}>
                            <td>{item.id}</td>
                            <td>{item.title}</td> { /* Menggunakan
title sebagai simulasi nama bahan */}
                            <td><span className="badge bg-
success">Available</span></td>
                        </tr>
                    ))}
                </tbody>
            </table>
        </div>
    );
}
export default Inventori;

src/Halaman/NotFound.jsx
import React from 'react';
import { Link } from 'react-router-dom';
function NotFound() {
    return (
        <div className="text-center mt-5">
            <h1 className="display-1">404</h1>
            <p>Halaman tidak ditemukan dalam sistem pabrik.</p>

```

```

        <Link to="/" className="btn btn-dark">Kembali ke
Home</Link>
      </div>
    );
  }
  export default NotFound

src/Komponen/Navbar.jsx
import React from 'react';
import { NavLink } from 'react-router-dom';

function Navbar() {
  return (
    <nav
      className="navbar navbar-expand-lg shadow-sm mb-4"
      style={{
        background: 'linear-gradient(90deg, #0f2027,
#203a43, #2c5364)'
      }}
    >
      <div className="container">

        {/* Brand */}
        <NavLink className="navbar-brand text-white fw-
bold fs-4" to="/">
          Sistem Pabrik
        </NavLink>

        {/* Toggle */}
        <button
          className="navbar-toggler border-0"
          type="button"
          data-bs-toggle="collapse"
          data-bs-target="#navbarNav"
        >
          <span className="navbar-toggler-icon"></span>
        </button>

        {/* Menu */}
        <div className="collapse navbar-collapse"
id="navbarNav">

          {/* Menu kiri */}
          <div className="navbar-nav mx-auto">

            <NavLink

```

```

        to="/"
        className=(({ isActive }) =>
            isActive
                ? "nav-link text-warning fw-
bold mx-2"
                : "nav-link text-white mx-2"
        )
    >
        Dashboard
    </NavLink>

    <NavLink
        to="/inventori"
        className=(({ isActive }) =>
            isActive
                ? "nav-link text-warning fw-
bold mx-2"
                : "nav-link text-white mx-2"
            )
        >
            Inventori
        </NavLink>
    </div>

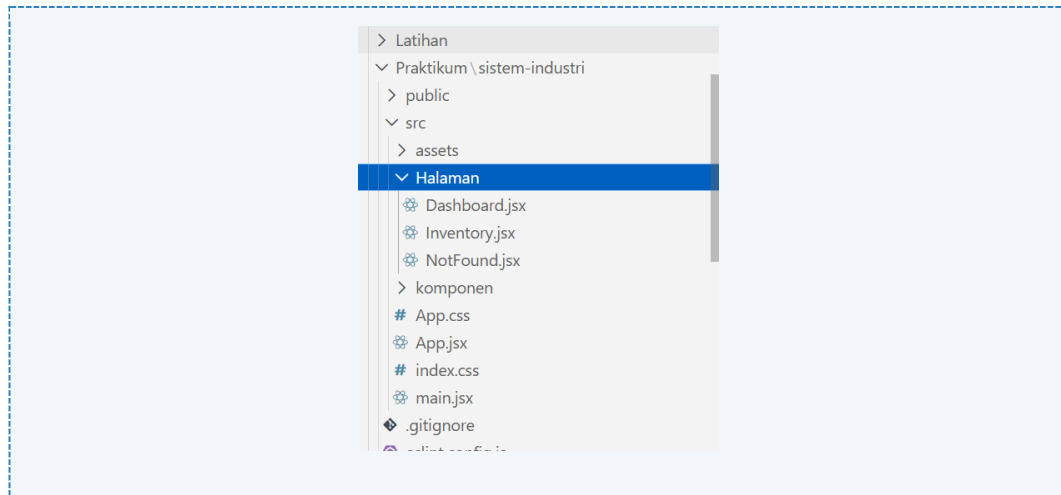
</div>

</div>

</div>
</nav>
);
}

export default Navbar;
```

Screenshot Hasil:



Gambar E.2 – Hasil Langkah 2

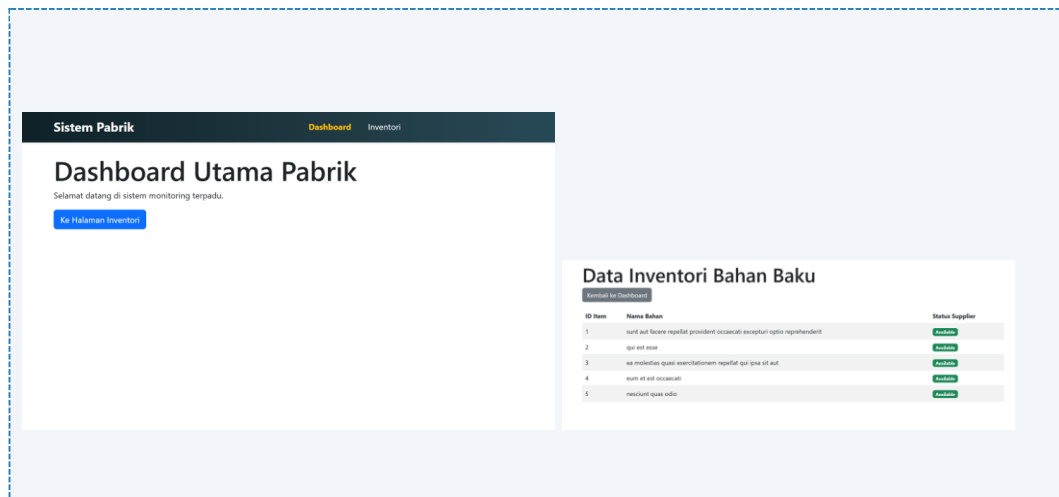
E.3 Langkah 3 – Mengatur Route di App.jsx

```
import React from 'react';
import { Routes, Route } from 'react-router-dom';
import Dashboard from './Halaman/Dashboard';
import Inventori from './Halaman/Inventori';
import NotFound from './Halaman/NotFound';
import Navbar from './Komponen/Navbar';

function App() {
  return (
    <div>
      <Navbar />
      <Routes>
        { /* Route yang tepat akan di-render */ }
        <Route path="/" element={ <Dashboard /> } />
        <Route path="/inventori" element={ <Inventori /> } />

        { /* Route untuk semua path lainnya (404) */ }
        <Route path="*" element={ <NotFound /> } />
      </Routes>
    </div>
  );
}
export default App;
```

Screenshot Hasil:



Gambar E.3– Hasil Langkah 3

F. LATIHAN DAN TUGAS MANDIRI

F.1 Latihan 1 – Dependency Array useEffect

Pertanyaan / Instruksi Latihan:

Tambahkan Route untuk detail item. Misal path `/inventori/:id`. Di halaman Inventori, ubah "Nama Bahan" menjadi Link ke halaman detail.

Jawaban / Kode Solusi:

```
// perubahan di file inventori.jsx
      <td>
        <Link
to={`/inventori/${item.id}`}>
          {item.title}
        </Link>
      </td>

// penambahan route dan import di app.jsx
import InventoriDetail from './Halaman/InventoriDetail';

    <Route path="/inventori/:id" element={<InventoriDetail
/>} />

//halaman InventoriDetail.jsx
import React, { useEffect, useState } from 'react';
import { useParams, Link } from 'react-router-dom';

function InventoriDetail() {
  const { id } = useParams();
  const [item, setItem] = useState(null);
```

```

useEffect(() => {
  fetch(`https://jsonplaceholder.typicode.com/posts/${id}`)
    .then(res => res.json())
    .then(data => setItem(data))
    .catch(err => console.log(err));
}, [id]);

if (!item) return <p>Loading...</p>;

return (
  <div className="container mt-5">
    <div className="row justify-content-center">
      <div className="col-lg-7">

        <div className="card border-0 shadow-lg"
style={{ borderRadius: '16px' }}>

// perubahan di file inventori.jsx
          <td>
            <Link
to={`/inventori/${item.id}`}>
              {item.title}
            </Link>
          </td>
// penambahan route dan import di app.jsx
import InventoriDetail from './Halaman/InventoriDetail';

          <Route path="/inventori/:id" element={<InventoriDetail
/>} />

//halaman InventoriDetail.jsx
import React, { useEffect, useState } from 'react';
import { useParams, Link } from 'react-router-dom';

function InventoriDetail() {
  const { id } = useParams();
  const [item, setItem] = useState(null);

  useEffect(() => {
    fetch(`https://jsonplaceholder.typicode.com/posts/${id}`)
      .then(res => res.json())
      .then(data => setItem(data))
      .catch(err => console.log(err));
  }, [id]);

  if (!item) return <p>Loading...</p>;

```

```

return (
  <div className="container mt-5">
    <div className="row justify-content-center">
      <div className="col-lg-7">

        <div className="card border-0 shadow-lg"
style={{ borderRadius: '16px' }}>
{ /* Header */}

          <div
            className="p-4 text-white"
            style={{
              background: 'linear-
gradient(135deg, #1e3c72, #2a5298)',
              borderTopLeftRadius: '16px',
              borderTopRightRadius: '16px'
            }}
          >
            <h4 className="mb-0 fw-bold">Detail
Bahan</h4>
            <small className="opacity-
75">Informasi lengkap material</small>
          </div>

          { /* Body */}
          <div className="p-4">

            <div className="d-flex justify-
content-between mb-3">
              <div>
                <small className="text-
muted">ID</small>

                <div className="fw-
bold">{item.id}</div>

              </div>

              <span className="badge bg-success
align-self-start">
                Available
              </span>
            </div>

            <hr />

            <div className="mb-3">

```



```

        <small className="text-
muted">Nama Bahan</small>
        <div className="fs-5 fw-
semibold">
            {item.title}
        </div>
    </div>

    <div className="mb-4">
        <small className="text-
muted">Deskripsi</small>
        <p className="mb-0 text-
secondary" style={{ lineHeight: '1.6' }}>
            {item.body}
        </p>
    </div>

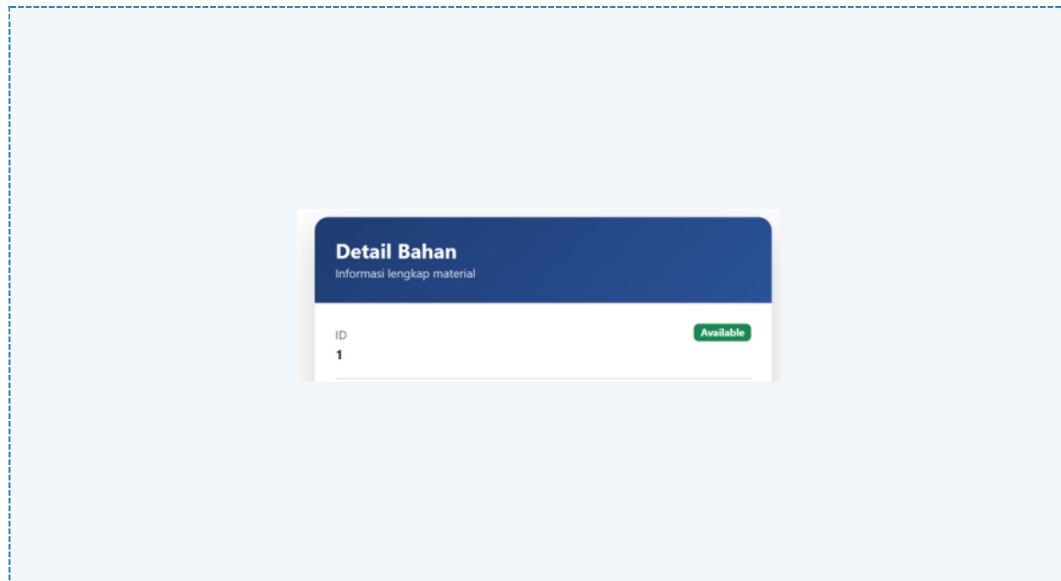
    { /* Action */ }
    <Link
        to="/inventori"
        className="btn btn-dark px-4"
        style={{ borderRadius: '10px' }}
    >
        ← Kembali
    </Link>

    </div>
</div>

</div>
</div>
</div>
);
}

export default InventoriDetail;

```



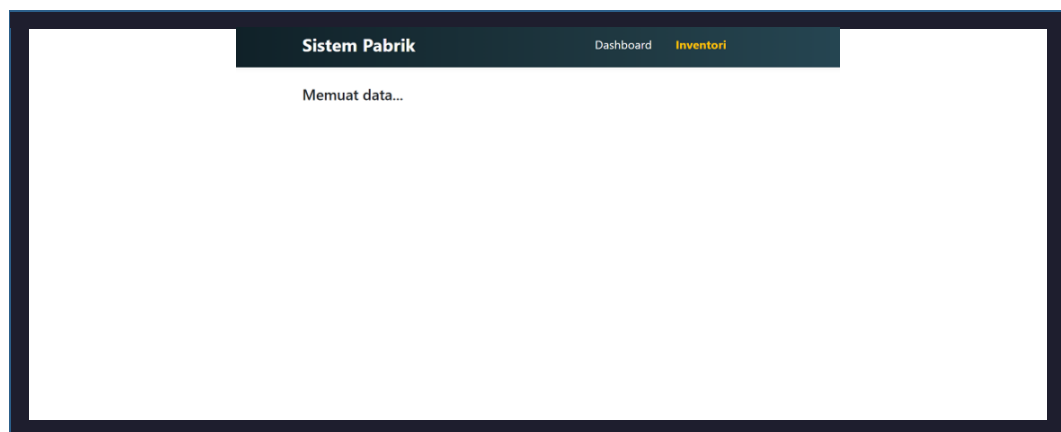
Gambar F.1 – Hasil Latihan 1

F.2 Latihan 2 – Conditional Rendering

Pertanyaan / Instruksi Latihan:

Di halaman `Inventori.jsx`, tambahkan state `loading` (boolean). Set `loading = true` sebelum fetch, dan `false` setelah data masuk. Tampilkan teks "Memuat data..." jika loading true

Jawaban / Kode Solusi:



Gambar F.2 – Hasil Latihan 2

F.3 Tugas Proyek Mini – Kalkulator OEE Sederhana

Deskripsi proyek mini:

- Buat halaman baru bernama `LaporanKualitas.jsx`. Halaman ini menampilkan data "Cacat" (mock data array manual).
- Tambahkan link di Navbar untuk menuju halaman ini.
- Pastikan transisi antar halaman berjalan mulus tanpa reload browser.

Penjelasan pendekatan/solusi yang digunakan:

Pendekatan yang diterapkan dalam implementasi tugas ini adalah Single Page Application (SPA) berbasis React, yang memungkinkan perpindahan halaman berlangsung secara dinamis di sisi klien tanpa perlu memuat ulang seluruh halaman browser, sehingga interaksi pengguna menjadi lebih cepat dan efisien.

Pada pengembangan halaman LaporanKualitas.jsx digunakan pendekatan component-based development, di mana setiap fitur atau halaman dipisahkan ke dalam komponen tersendiri agar struktur aplikasi menjadi lebih modular, terorganisasi, dan mudah dikembangkan.

Data yang digunakan masih berupa mock data dalam bentuk array manual untuk mensimulasikan data cacat produk sebelum diintegrasikan dengan sumber data nyata seperti database atau API. Selain itu, navigasi antar halaman memanfaatkan React Router DOM dengan konsep client-side routing, yang memungkinkan pengaturan rute dilakukan di sisi frontend sehingga perpindahan halaman dapat berlangsung dengan lebih halus tanpa melakukan refresh pada browser.

Kode Utama:

```
import React, { useState } from "react";
//penambahan route di app.jsx
import LaporanKualitas from './Halaman/LaporanKualitas';

    <Route path="/laporan-kualitas" element={<LaporanKualitas
/>} />

//di file /Komponen/Navbar.jsx
    <NavLink
      to="/laporan-kualitas"
      className={({ isActive }) =>
        isActive
          ? "nav-link text-warning fw-
bold mx-2"
          : "nav-link text-white mx-2"
      }
    >
      Laporan Kualitas
    </NavLink>

// Halaman LaporanKualitas.jsx
import React from 'react';

function LaporanKualitas() {
  const dataCacat = [
```

```

        { id: 1, produk: 'Baut A', jenis: 'Retak', jumlah: 5 },
        { id: 2, produk: 'Mur B', jenis: 'Ukuran Tidak Sesuai',
jumlah: 3 },
        { id: 3, produk: 'Plat C', jenis: 'Goresan', jumlah: 8 },
    ];

    return (
        <div className="container mt-5">
            <div className="row justify-content-center">
                <div className="col-lg-9">

                    <div className="card border-0 shadow-lg"
style={{ borderRadius: '16px' }}>

                        {/* Header */}
                        <div
                            className="p-4 text-white"
                            style={{
                                background: 'linear-
gradient(135deg, #1e3c72, #2a5298)',
                                borderTopLeftRadius: '16px',
                                borderTopRightRadius: '16px'
                            }}
                        >
                            <h4 className="mb-1 fw-bold">Laporan
Kualitas</h4>
                            <small className="opacity-75">Data
cacat produk produksi</small>
                        </div>

                        {/* Body */}
                        <div className="p-4">

                            <table className="table align-middle
">

                                <thead className="table-info">
                                    <tr>
                                        <th>ID</th>
                                        <th>Produk</th>
                                        <th>Jenis Cacat</th>
                                        <th className="text-
center">Jumlah</th>
                                        <th className="text-
center">Status</th>
                                    </tr>
                                </thead>

```

```

<tbody>
  {dataCacat.map(item => (
    <tr key={item.id}>
      <td>{item.id}</td>
      <td className="fw-
semibold">{item.produk}</td>
      <td>{item.jenis}</td>
      <td className="text-
center fw-bold">
        {item.jumlah}
      </td>
      <td className="text-
center">
        <span
className={`badge ${item.jumlah > 5 ? 'bg-danger' : 'bg-success'
} `}>
          {item.jumlah
> 5 ? 'Tinggi' : 'Normal'}
        </span>
      </td>
    </tr>
  )
  )
}</tbody>
</table>
</div>
</div>
</div>
</div>
</div>
);
}

```

Sistem Pabrik

Dashboard

Inventori

Laporan Kualitas

Baut A	Retak	5	Normal
Mur B	Ukuran Tidak Sesuai	3	Normal
Plat C	Goresan	8	Tinggi

Gambar F.3 – Hasil Proyek Mini

G. PEMBAHASAN

G.1 Analisis Kesesuaian Hasil dengan Teori

Penggunaan React Router DOM juga sejalan dengan prinsip separation of concerns, di mana setiap komponen bertanggung jawab terhadap tampilan tertentu. Hal ini sesuai dengan teori rekayasa perangkat lunak yang menekankan modularitas dan maintainability. Struktur aplikasi yang terbagi menjadi beberapa halaman menunjukkan penerapan konsep ini secara praktis.

G.2 Kendala yang Ditemukan dan Solusinya

Kendala / Error yang Ditemukan	Solusi yang Diterapkan
Error Module not found ketika menjalankan project	Memeriksa kembali struktur folder dan menyesuaikan path import

G.3 Kaitan dengan Penerapan di Industri

Integrasi API sangat krusial dalam lingkungan industri yang memiliki banyak sumber data. Misalnya, data inventori dapat berasal dari sistem gudang, sementara data penjualan dari sistem transaksi. Frontend bertugas menggabungkan informasi tersebut secara real-time agar dapat digunakan oleh pengguna.

H. KESIMPULAN

1	Navbar berhasil dihubungkan dengan halaman laporan sehingga pengguna dapat berpindah halaman dengan lebih mudah.
2	Bootstrap digunakan untuk membuat tampilan card dan tabel menjadi lebih rapi, responsive, dan mudah dibaca saat aplikasi dijalankan.

I. DAFTAR PUSTAKA

No.	Referensi (Format APA 7th Edition)
[1]	Burhandenny, A. E. (2026). Manual Praktikum Aplikasi Web dan Mobile Semester Genap 2025/2026. Program Studi Teknik Industri, Universitas Negeri Yogyakarta.
[2]	https://react.dev/learn
[3]	https://react.dev/reference/react-dom/hooks/useFormStatus
[4]	https://www.javascripttutorial.net/react-tutorial/

J. LEMBAR PENILAIAN DAN PENGESAHAN

RUBRIK PENILAIAN LAPORAN					
Aspek Penilaian		Bobot			
No.	Aspek Penilaian	Bobot (%)	Nilai (0-100)	Skor Akhir	Catatan Penilai
1	Kelengkapan Isi (Semua bagian A–I terisi lengkap)	20%			
2	Kualitas Dasar Teori (Ditulis dengan bahasa sendiri, relevan, minimal 3 paragraf)	15%			
3	Dokumentasi Langkah Kerja (Kode & Screenshot sesuai, jelas, dan terbaca)	25%			
4	Tugas/Latihan Mandiri (Semua latihan & proyek mini dikerjakan dan berfungsi)	20%			
5	Pembahasan & Analisis (Kritis, mendalam, dan relevan dengan industri)	15%			
6	Kesimpulan & Tata Bahasa (Spesifik, rapi, mengikuti format yang ditentukan)	5%			
TOTAL NILAI AKHIR		100%			

Mahasiswa	Rekan Tim	Diperiksa oleh Asisten / Dosen
		
Saprina Melita Sari NIM:23051430012	Ardita Natalia NIM:23051430022	Dr. Eng. Ir. Aji Ery Burhandenny, S.T., M.AIT. Tanggal:

MODUL 12

A. IDENTITAS & INFORMASI MODUL

Mata Kuliah	Aplikasi Web dan Mobile
Kode Modul	Modul 12
Topik Utama	Proyek Web: Dashboard Monitoring Produksi (React + Chart.js)
Sub-Topik	Integrasi Library Visualisasi Data (Chart.js) dan finalisasi Layout Dashboard.
Durasi Praktikum	340 Menit
Tanggal Praktikum	11 Mei 2026
Nama Mahasiswa	Saprina Melita Sari
NIM	23051430012
Kelas	A1

B. TUJUAN PEMBELAJARAN

1	Mampu mengintegrasikan library pihak ketiga (`react-chartjs-2`) ke dalam React.
2	Mampu mengubah data mentah menjadi visualisasi grafik yang informatif.
3	Menyelesaikan proyek mini dashboard lengkap dari pertemuan 1-12.

C. ALAT DAN BAHAN

No.	Nama Alat / Bahan / Aplikasi	Versi / Spesifikasi	Keterangan
1	Visual Studio Code (VS Code)	Versi 1.8x ke atas	Code Editor
2	Google Chrome / Browser Modern	Versi terbaru	Testing & Debugging
3	Node.js & npm	Node 18+ / npm 9+	Runtime JS

4	Prettier	Versi 12.3.0	Extensions VS Code untuk merapikan indentansi code
---	----------	--------------	--

D. DASAR TEORI

D.1 Konsep Utama

Praktikum ini menggunakan React dan Chart.js untuk membangun dashboard monitoring produksi yang menyajikan data dalam bentuk visual agar lebih mudah dianalisis. Data mentah diolah menjadi grafik interaktif seperti produksi, defect, dan utilisasi mesin, sehingga pengguna dapat memahami informasi dengan lebih cepat dibandingkan tampilan tabel biasa.

Penerapan Single Page Application (SPA) pada React menjadi konsep utama dalam pengembangan ini, di mana perpindahan antar halaman dapat dilakukan tanpa melakukan reload pada browser. Hal ini membuat aplikasi terasa lebih cepat dan responsif. Selain itu, penggunaan komponen-komponen React membuat struktur kode lebih modular, terorganisir, dan mudah untuk dikembangkan lebih lanjut.

Chart.js juga diintegrasikan untuk mendukung visualisasi data yang lebih informatif. Melalui grafik, pengguna dapat melihat tren, pola, serta perbandingan data produksi secara real-time. Dengan adanya visualisasi ini, proses pengambilan keputusan menjadi lebih efektif karena informasi dapat dipahami dengan lebih cepat dan jelas.

D.2 Keterkaitan dengan Teknik Industri

Dalam bidang Teknik Industri, pemantauan proses produksi merupakan aspek yang sangat penting untuk memastikan efisiensi serta kualitas operasional tetap terjaga. Penggunaan dashboard monitoring memungkinkan data produksi dipantau secara terpadu, sehingga memudahkan dalam menganalisis kinerja mesin, jumlah output, hingga tingkat produk cacat. Pemanfaatan visualisasi data juga erat kaitannya dengan konsep pengambilan keputusan berbasis data yang umum diterapkan dalam manajemen industri. Melalui grafik dan indikator kinerja produksi, seorang engineer dapat lebih mudah mendeteksi permasalahan seperti penurunan produktivitas atau meningkatnya jumlah defect pada proses tertentu.

E. LANGKAH KERJA DAN HASIL PRAKTIKUM

E.1 Langkah 1 – Persiapan Data Grafik

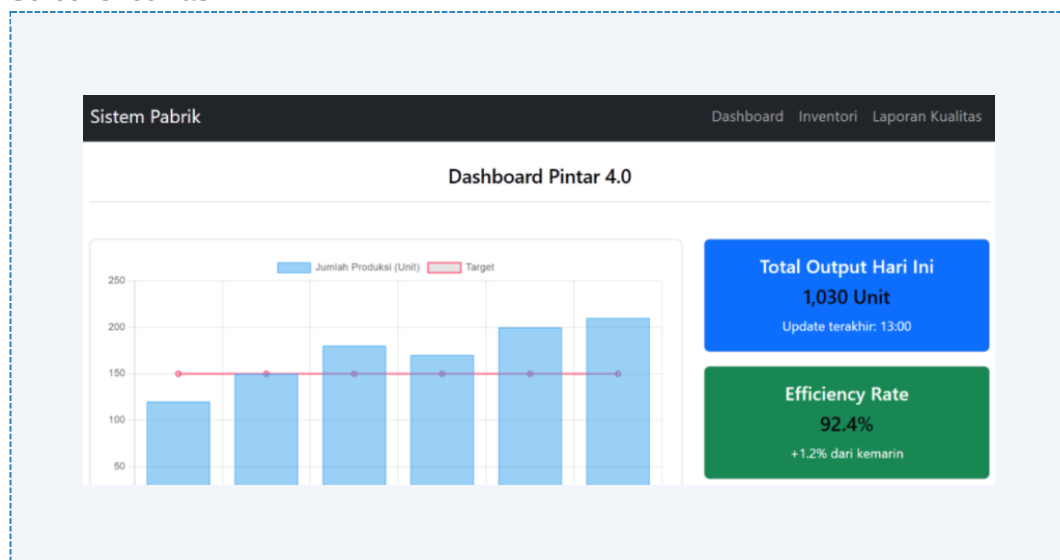
Deskripsi singkat apa yang dilakukan pada langkah ini:

```

/* ===== KODE / OUTPUT LANGKAH 1 ===== */
const data = {
  labels: ['08:00', '09:00', '10:00', '11:00', '12:00',
'13:00'],
  datasets: [
    {
      label: 'Jumlah Produksi (Unit)',
      data: [120, 150, 180, 170, 200, 210], // Data
      statis, bisa diganti state/dinamis
      backgroundColor: 'rgba(54, 162, 235, 0.5)',
      borderColor: 'rgba(54, 162, 235, 1)',
      borderWidth: 1,
    },
    {
      label: 'Target',
      data: [150, 150, 150, 150, 150, 150], // Garis
      target konstan
      type: 'line', // Tipe campuran (Bar + Line)
      borderColor: 'rgb(255, 99, 132)',
      borderWidth: 2,
    },
  ],
};

```

Screenshot Hasil:



Gambar E.1 – Hasil Pembuatan Array dataset

E.2 Langkah 2 – Membuat Komponen Grafik

```
src/Halaman/Dashboard.jsx
import React from 'react';
import { Bar } from 'react-chartjs-2';

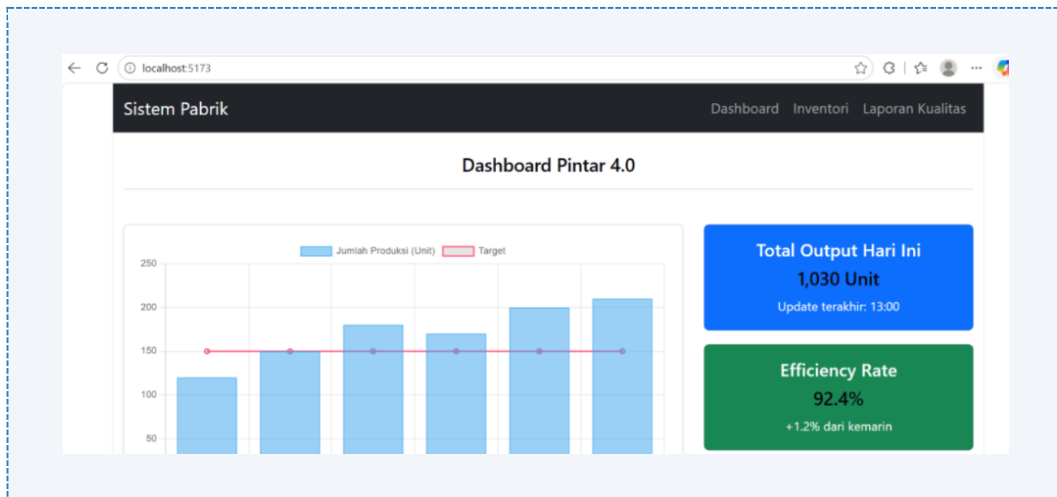
function GrafikProduksi() {
  // Data untuk Grafik
  const data = {
    labels: ['08:00', '09:00', '10:00', '11:00', '12:00',
'13:00'],
    datasets: [
      {
        label: 'Jumlah Produksi (Unit)',
        data: [120, 150, 180, 170, 200, 210], // Data
statis, bisa diganti state/dinamis
        backgroundColor: 'rgba(54, 162, 235, 0.5)',
        borderColor: 'rgba(54, 162, 235, 1)',
        borderWidth: 1,
      },
      {
        label: 'Target',
        data: [150, 150, 150, 150, 150, 150], // Garis
target konstan
        type: 'line', // Tipe campuran (Bar + Line)
        borderColor: 'rgb(255, 99, 132)',
        borderWidth: 2,
      },
    ],
  };
  // Opsi Tampilan Grafik
  const options = {
    responsive: true,
    plugins: {
      legend: {
        position: 'top',
      },
      title: {
        display: true,
        text: 'Grafik Produksi Harian - Lini 1',
      },
    },
    scales: {
      y: {
        beginAtZero: true,
      },
    },
  };
};
```

```

    return <Bar data={data} options={options} />;
  }
  export default GrafikProduksi;

```

Screenshot Hasil:



Gambar E.2 – Hasil Pembuatan Komponen Grafik Produksi

E.3 Langkah 3 – Menggabungkan ke Dashboard Utama

```

/* ===== KODE / OUTPUT LANGKAH 3 ===== */
import React from 'react';
import GrafikProduksi from '../Komponen/GrafikProduksi';
import KartuMesin from '../Komponen/KartuMesin';

function Dashboard() {
  return (
    <div className="container-fluid py-4">

      {/* Header */}
      <div className="d-flex align-items-center justify-content-between mb-4 pb-2 border-bottom">

        <div>
          <h2 className="fw-bold mb-0">
            Dashboard Pintar 4.0
          </h2>
        </div>
      </div>

      {/* Grafik dan KPI */}
      <div className="row g-4 mb-4">

```

```

        { /* Grafik */ }
        <div className="col-lg-8">
            <div className="card shadow-sm border-0 h-
100">
                <div className="card-body">
                    <GrafikProduksi />
                </div>
            </div>
        </div>

        { /* KPI */ }
        <div className="col-lg-4">

            <div className="card bg-primary text-white
shadow-sm border-0 mb-3">
                <div className="card-body">
                    <h6 className="text-uppercase mb-2">
                        Total Output Hari Ini
                    </h6>

                    <h2 className="fw-bold">1,030
Unit</h2>

                    <small>
                        Update terakhir: 13:00
                    </small>
                </div>
            </div>

            <div className="card bg-success text-white
shadow-sm border-0">
                <div className="card-body">
                    <h6 className="text-uppercase mb-2">
                        Efficiency Rate
                    </h6>

                    <h2 className="fw-bold">92.4%</h2>

                    <small>
                        +1.2% dari kemarin
                    </small>
                </div>
            </div>

        </div>

```

```
</div>

{/* Status Mesin */}
<div className="mb-3">
  <h4 className="fw-semibold">
    Status Mesin Aktif
  </h4>
</div>

<div className="row g-4">

  <div className="col-md-6 col-lg-3">
    <KartuMesin
      nama="CNC-01"
      status="Running"
      produksi={320}
    />
  </div>

  <div className="col-md-6 col-lg-3">
    <KartuMesin
      nama="CNC-02"
      status="Running"
      produksi={310}
    />
  </div>

  <div className="col-md-6 col-lg-3">
    <KartuMesin
      nama="Press-01"
      status="Stop"
      produksi={150}
    />
  </div>

  <div className="col-md-6 col-lg-3">
    <KartuMesin
      nama="Weld-04"
      status="Maintenance"
      produksi={0}
    />
  </div>

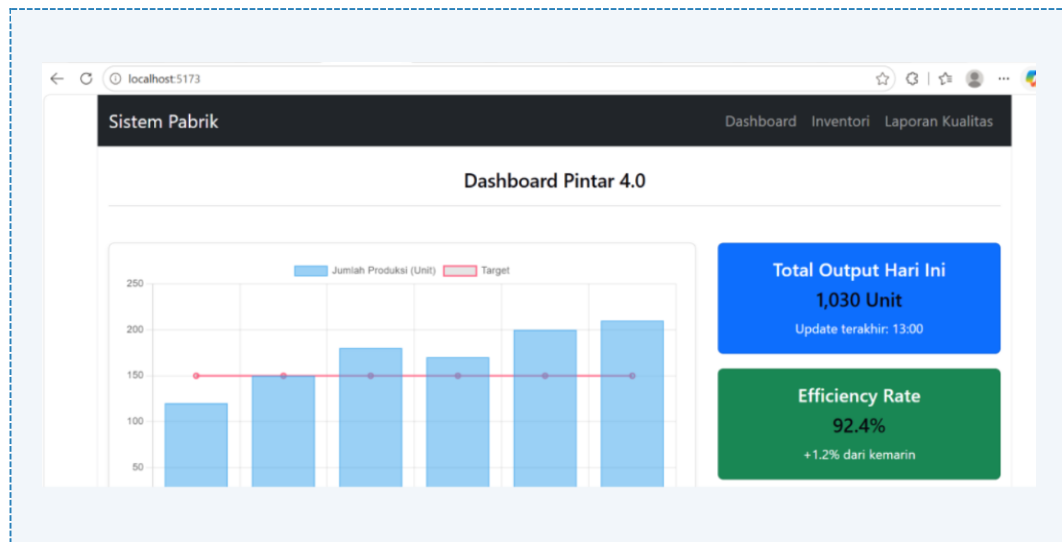
</div>
</div>
);
```



```
}

export default Dashboard;
```

Screenshot Hasil:



Gambar E.3– Hasil Pembuatan Grafik pada Halaman Dashboard

F. LATIHAN DAN TUGAS MANDIRI

F.1 Latihan 1 – Integrasi Data Realistik

Pertanyaan / Instruksi Latihan:

Ubah data statis di `GrafikProduksi` menjadi data yang diambil dari `fetch` API (gunakan mock data dari internet atau buat generator acak).

Jawaban / Kode Solusi:

```
const [produksiData, setProduksiData] = useState([]);

// Simulasi fetch API dengan data random
useEffect(() => {

  const fetchData = async () => {

    // Simulasi delay request API
    await new Promise(resolve => setTimeout(resolve,
1000));
```

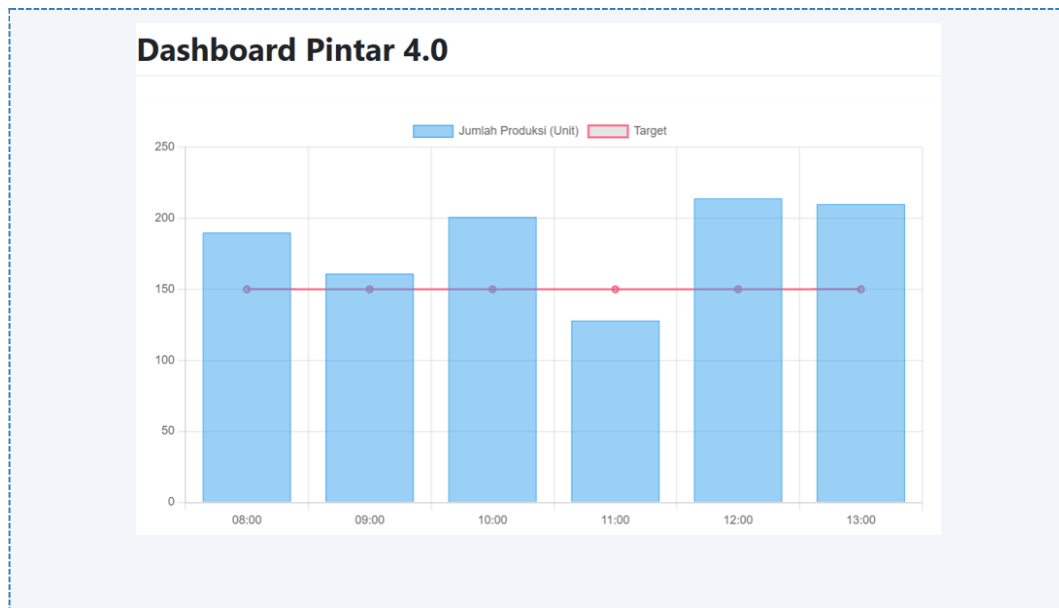
```
// Generate data random
const hasilRandom = Array.from(
  { length: 6 },
  () => Math.floor(Math.random() * 100) + 120
);

setProduksiData(hasilRandom);
};

fetchData();

}, []);

// Data grafik
const data = {
  labels: ['08:00', '09:00', '10:00', '11:00', '12:00',
'13:00'],
  datasets: [
    {
      label: 'Jumlah Produksi (Unit)',
      data: produksiData,
      backgroundColor: 'rgba(54, 162, 235, 0.5)',
      borderColor: 'rgba(54, 162, 235, 1)',
      borderWidth: 1,
    },
    {
      label: 'Target',
      data: [150, 150, 150, 150, 150, 150],
      type: 'line',
      borderColor: 'rgb(255, 99, 132)',
      borderWidth: 2,
    },
  ],
};
```



Gambar F.1 – Hasil Pembuatan Grafik dari Data Random

F.2 Latihan 2 – Grafik Donat

Pertanyaan / Instruksi Latihan:

Buat grafik "Proporsi Cacat" (Misal: Scratch 50%, Dent 30%, Lainnya 20%) menggunakan komponen `Doughnut` dari chart.js.

Jawaban / Kode Solusi:

```
import React from 'react';
import { Doughnut } from 'react-chartjs-2';

function GrafikCacat() {

  const data = {
    labels: ['Scratch', 'Dent', 'Lainnya'],

    datasets: [
      {
        label: 'Proporsi Cacat',
        data: [50, 30, 20],

        backgroundColor: [
          'rgba(255, 99, 132, 0.8)',
          'rgba(255, 206, 86, 0.8)',
          'rgba(54, 162, 235, 0.8)',
        ],

        borderWidth: 0,
```

```
        hoverOffset: 12,
      },
    ],
  };

  const options = {
    responsive: true,
    maintainAspectRatio: false,

    cutout: '68%',

    plugins: {
      legend: {
        position: 'bottom',

        labels: {
          padding: 20,
          boxWidth: 14,
          font: {
            size: 13,
          },
        },
      },

      tooltip: {
        enabled: true,
      },
    },
  };

  return (
    <div
      style={{
        height: '320px',
        width: '100%',
        maxWidth: '340px',
        margin: '0 auto',
      }}
    >
      <Doughnut
        data={data}
        options={options}
      />
    </div>
  );
}
```

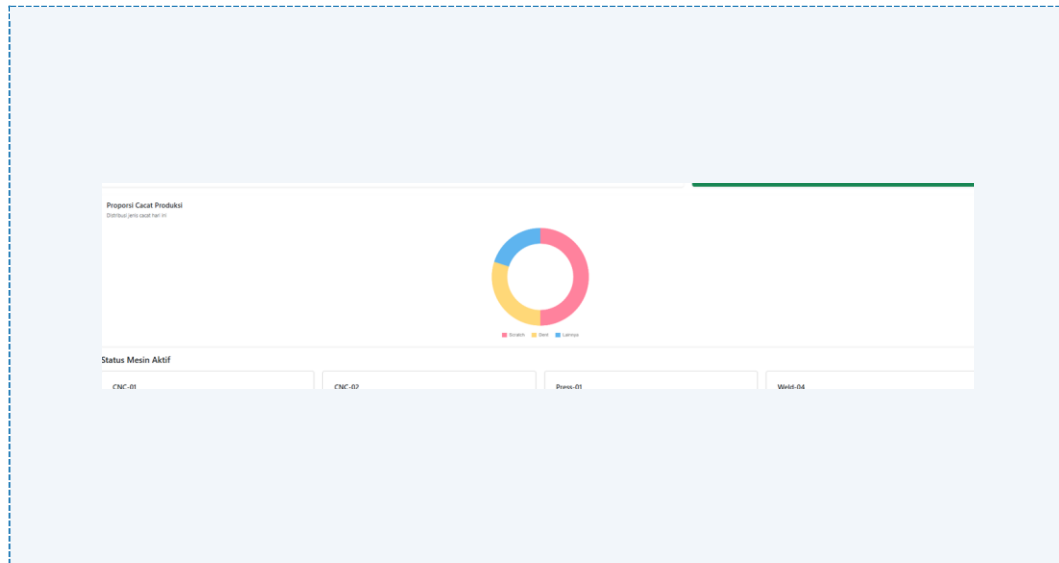
```

    </div>

    );
}

export default GrafikCacat;

```



Gambar F.2 – Hasil Pembuatan Grafik Cacat menggunakan Grafik Doughnut

F.3 Tugas Proyek Mini – Tugas Akhir Web

Deskripsi proyek mini:

Susun ulang halaman `Dashboard` agar memiliki Sidebar (kiri) dan Main Content (kanan).

- Sidebar: Logo, Menu Dashboard, Menu Inventori, Menu Laporan.
- Main Content: Header (User Profile), Grafik, dan Tabel Data.
- Pastikan desain responsif (Sidebar bisa collapse di HP)

Penjelasan pendekatan/solusi yang digunakan:

Pada perancangan dashboard ini, saya membagi tampilan menjadi dua bagian utama yaitu sidebar sebagai menu navigasi dan main content sebagai area utama untuk menampilkan informasi. Sidebar saya buat dalam komponen terpisah (Sidebar.jsx) agar bisa digunakan kembali dan lebih mudah dikelola. Untuk perpindahan halaman saya menggunakan NavLink dari React Router supaya navigasi dapat berjalan tanpa reload dan menu aktif bisa langsung terlihat. Pada tampilan desktop, sidebar dibuat tetap (fixed) agar selalu terlihat, sedangkan pada tampilan mobile digunakan Bootstrap offcanvas supaya sidebar bisa disembunyikan dan ditampilkan melalui tombol toggle agar lebih

hemat ruang. Layout utama saya atur menggunakan Layout.jsx untuk menggabungkan sidebar dan content, serta menggunakan class Bootstrap seperti d-none, d-lg-block, dan d-lg-none agar tampilan dapat menyesuaikan ukuran layar secara responsif. Selain itu, saya juga menggunakan Bootstrap dan inline CSS untuk mengatur tampilan agar lebih rapi, modern, dan mudah digunakan di berbagai perangkat.

Kode Utama:

```
import React from 'react';

function LaporanKualitas() {
  const dataCacat = [
    { id: 1, produk: 'Baut A', jenis: 'Retak', jumlah: 5 },
    { id: 2, produk: 'Mur B', jenis: 'Ukuran Tidak Sesuai',
jumlah: 3 },
    { id: 3, produk: 'Plat C', jenis: 'Goresan', jumlah: 8 },
  ];

  return (
    <div className="container mt-5">

      { /* Identitas */ }
      <div className="text-center mb-3">
        <h6 className="fw-bold">
          Saprina Melita Sari - 23051430012
        </h6>
      </div>

      <div className="row justify-content-center">
        <div className="col-lg-9">

          <div className="card border-0 shadow-lg"
style={{ borderRadius: '16px' }}>

            { /* Header */ }
            <div
              className="p-4 text-white"
              style={{
                background: 'linear-
gradient(135deg, #1e3c72, #2a5298)',
                borderTopLeftRadius: '16px',
                borderTopRightRadius: '16px'
              }}
            >
```

```

    }}
  >
  <h4 className="mb-1 fw-bold">Laporan
Kualitas</h4>
  <small className="opacity-75">
    Data cacat produk produksi
  </small>
</div>

{/* Body */}
<div className="p-4">

  <table className="table align-
middle">
    <thead className="table-info">
      <tr>
        <th>ID</th>
        <th>Produk</th>
        <th>Jenis Cacat</th>
        <th className="text-
center">Jumlah</th>
        <th className="text-
center">Status</th>
      </tr>
    </thead>
    <tbody>
      {dataCacat.map(item => (
        <tr key={item.id}>
          <td>{item.id}</td>
          <td className="fw-
semibold">{item.produk}</td>
          <td>{item.jenis}</td>
          <td className="text-
center fw-bold">
            {item.jumlah}
          </td>
          <td className="text-
center">
            <span
className={`badge ${item.jumlah > 5 ? 'bg-danger' : 'bg-
success'}`}>
              {item.jumlah}
            </span>
            <br>
            <span>
              <span>
                > 5 ? 'Tinggi' : 'Normal'

```

```

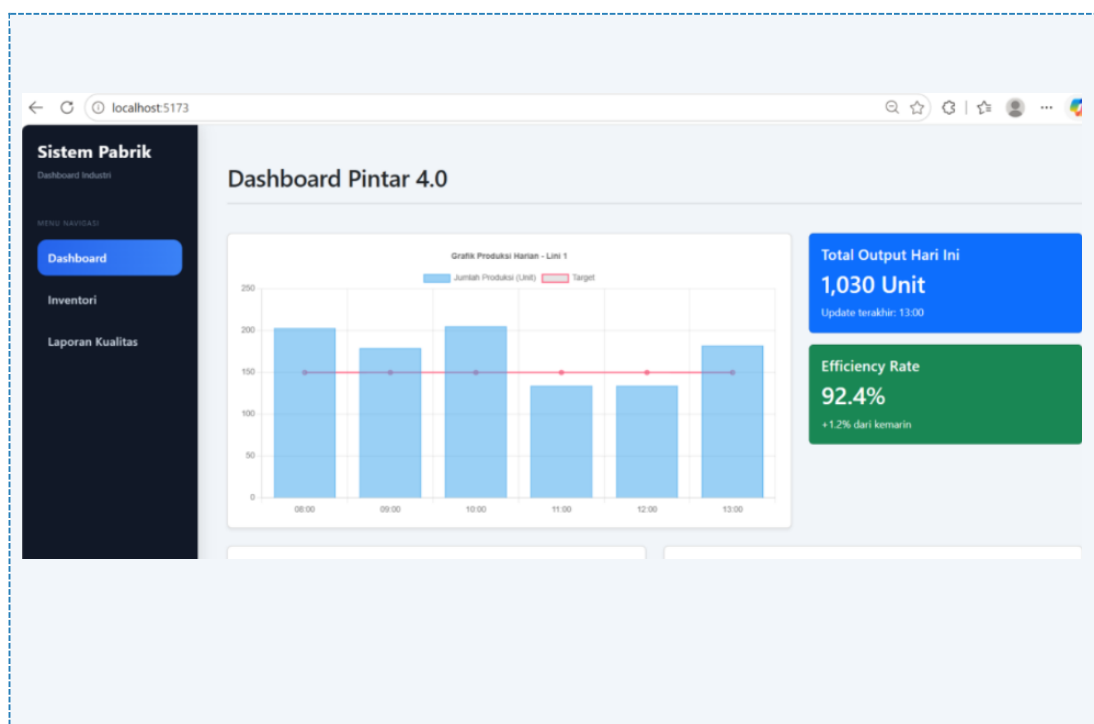
        </span>
      </td>
    </tr>
  </tbody>
</table>

</div>
</div>

</div>
</div>
</div>
);
}

export default LaporanKualitas;

```



Gambar F.3 – Hasil Pembuatan ProyekMini Navigasi Modular

G. PEMBAHASAN

Analisis hasil praktikum secara kritis. Jawab pertanyaan berikut dengan lengkap dan mendalam (bukan hanya "ya/tidak").

G.1 Analisis Kesesuaian Hasil dengan Teori

Penggunaan React juga sesuai dengan teori pengembangan antarmuka modern yang menekankan pada component-based architecture. Setiap bagian dashboard seperti kartu data, sidebar, dan grafik dibuat terpisah sehingga memudahkan pengembangan serta pemeliharaan aplikasi.

G.2 Kendala yang Ditemukan dan Solusinya

Kendala / Error yang Ditemukan	Solusi yang Diterapkan
Layout menjadi berantakkan ketika ukuran browser diubah terutama pada sidebar	Dilakukan penyesuaian layout dengan mengubah tampilan sidebar menjadi navbar/header pada kondisi ukuran layar tertentu (terutama mobile atau layar kecil) agar tampilan tetap responsif dan tidak berantakan.

G.3 Kaitan dengan Penerapan di Industri

Dalam praktikum ini, saya merancang dashboard yang bertujuan untuk membantu penyajian informasi produksi agar lebih mudah dan cepat dipahami oleh pengguna. Dashboard dibuat dengan tampilan yang ringkas dan terstruktur, serta menampilkan data penting dalam satu halaman sehingga supervisor maupun pihak manajemen tidak perlu membaca laporan yang panjang. Pada proses perancangannya, saya juga memperhatikan aspek kemudahan penggunaan (user friendly). Pengaturan layout seperti grafik, kartu informasi, warna, dan navigasi disusun agar pengguna dapat dengan mudah menemukan informasi yang dibutuhkan. Oleh karena itu, penggunaan React dan Chart.js sangat membantu karena dapat menghasilkan tampilan dashboard yang interaktif dan responsif.

H. KESIMPULAN

1	Integrasi Chart.js pada React berhasil digunakan untuk menampilkan grafik produksi dan defect secara dinamis.
2	Penggunaan React mempermudah pengembangan dashboard melalui konsep komponen yang modular dan responsif.
3	
4	

I. DAFTAR PUSTAKA

No.	Referensi (Format APA 7th Edition)
[1]	Burhandenny, A. E. (2026). Manual Praktikum Aplikasi Web dan Mobile Semester Genap 2025/2026. Program Studi Teknik Industri, Universitas Negeri Yogyakarta.
[2]	https://react.dev/learn
[3]	https://react.dev/reference/react-dom/hooks/useFormStatus
[4]	https://www.javascripttutorial.net/react-tutorial/

J. LEMBAR PENILAIAN DAN PENGESAHAN

RUBRIK PENILAIAN LAPORAN					
Aspek Penilaian		Bobot			
No.	Aspek Penilaian	Bobot (%)	Nilai (0-100)	Skor Akhir	Catatan Penilai
1	Kelengkapan Isi (Semua bagian A–I terisi lengkap)	20%			
2	Kualitas Dasar Teori (Ditulis dengan bahasa sendiri, relevan, minimal 3 paragraf)	15%			
3	Dokumentasi Langkah Kerja (Kode & Screenshot sesuai, jelas, dan terbaca)	25%			
4	Tugas/Latihan Mandiri (Semua latihan & proyek mini dikerjakan dan berfungsi)	20%			
5	Pembahasan & Analisis (Kritis, mendalam, dan relevan dengan industri)	15%			
6	Kesimpulan & Tata Bahasa (Spesifik, rapi, mengikuti format yang ditentukan)	5%			
TOTAL NILAI AKHIR		100%			

Mahasiswa	Rekan Tim	Diperiksa oleh Asisten / Dosen
 _____ Saprina Melita Sari NIM:23051430012	 _____ Ardita Natalia NIM:23051430022	 _____ Dr. Eng. Ir. Aji Ery Burhandenny, S.T., M.AIT. Tanggal: _____

MODUL 14

A. IDENTITAS & INFORMASI MODUL

Mata Kuliah	Aplikasi Web dan Mobile
Kode Modul	Modul 14
Topik Utama	Pengenalan React Native & Setup Expo
Sub-Topik	Konsep Cross-Platform, Setup Lingkungan Expo, Komponen Dasar (View, Text, Image), dan styling di Mobile.
Durasi Praktikum	340 Menit
Tanggal Praktikum	21 Mei 2026
Nama Mahasiswa	Saprina Melita Sari
NIM	23051430012
Kelas	A1

B. TUJUAN PEMBELAJARAN

1	Memahami perbedaan antara Web Development (React.js) dan Mobile Development (React Native).
2	Mampu menyiapkan lingkungan kerja React Native menggunakan Expo
3	Menguasai komponen dasar pengganti HTML: `View` (div), `Text` (p/h1), `ScrollView`.
4	Mampu menata tampilan mobile menggunakan `StyleSheet`.

C. ALAT DAN BAHAN

No.	Nama Alat / Bahan / Aplikasi	Versi / Spesifikasi	Keterangan
1	Visual Studio Code (VS Code)	Versi 1.8x ke atas	Code Editor
2	Google Chrome / Browser Modern	Versi terbaru	Testing & Debugging
3	Node.js & npm	Node 18+ / npm 9+	Runtime JS

4	Prettier	Versi 12.3.0	Extensioins VS Code untuk merapikan indentansi code
5	Aplikasi Expo Go	Versi terbaru	Melihat tampilan mobile
6	React Native	SDK Versi 54	Framework pengembangan aplikasi mobile

D. DASAR TEORI

D.1 Konsep Utama

React Native adalah framework untuk mengembangkan aplikasi mobile yang dapat digunakan pada platform Android dan iOS dengan menggunakan JavaScript. Konsep utamanya adalah cross-platform development, yaitu satu kode program dapat dijalankan pada dua sistem operasi sekaligus. Dalam penggunaannya, React Native menyediakan komponen seperti View, Text, Image, dan StyleSheet untuk membangun tampilan aplikasi yang mirip dengan pengembangan web, tetapi sudah disesuaikan dengan lingkungan mobile. Pada pertemuan ini juga dikenalkan Expo sebagai alat bantu dalam pengembangan React Native.

Expo memudahkan proses instalasi, pengujian, dan menjalankan aplikasi tanpa perlu konfigurasi native yang rumit. Dengan Expo Go, aplikasi dapat langsung diuji melalui smartphone sehingga proses pengembangan menjadi lebih cepat dan praktis. Pendekatan ini membantu dalam memahami dasar pembuatan aplikasi mobile secara lebih sederhana dan langsung dipraktikkan.

Selain itu, materi ini juga menjelaskan perbedaan antara pengembangan web dan mobile. Pada web digunakan elemen seperti `<div>` dan `<p>`, sedangkan pada React Native digunakan komponen seperti View dan Text. Hal ini menunjukkan bahwa React Native tidak hanya memindahkan konsep dari web ke mobile, tetapi juga menghadirkan struktur dan pendekatan baru yang memang dirancang khusus untuk pengembangan aplikasi mobile.

D.2 Keterkaitan dengan Teknik Industri

Dalam bidang Teknik Industri, penggunaan React Native berkaitan dengan proses digitalisasi sistem kerja dan pengelolaan informasi di lingkungan industri melalui aplikasi mobile yang dapat mendukung aktivitas seperti produksi, pencatatan data lapangan, monitoring kualitas, hingga pengawasan distribusi secara real-time, sehingga proses pengambilan keputusan menjadi lebih cepat karena data dapat diakses langsung melalui

perangkat seluler. Selain itu, React Native dengan konsep cross-platform juga membantu meningkatkan efisiensi pengembangan sistem informasi karena satu basis kode dapat digunakan untuk Android dan iOS, sehingga waktu dan biaya pengembangan dapat diminimalkan sesuai dengan prinsip Teknik Industri yang berfokus pada efisiensi, optimalisasi sumber daya, dan peningkatan produktivitas. Tidak hanya itu, pengembangan aplikasi mobile ini juga mendukung integrasi antara manusia, sistem, dan teknologi dalam lingkungan industri, sehingga React Native dapat menjadi solusi untuk membangun sistem yang lebih fleksibel, mudah diakses, dan mampu menunjang kebutuhan operasional industri modern.

E. LANGKAH KERJA DAN HASIL PRAKTIKUM

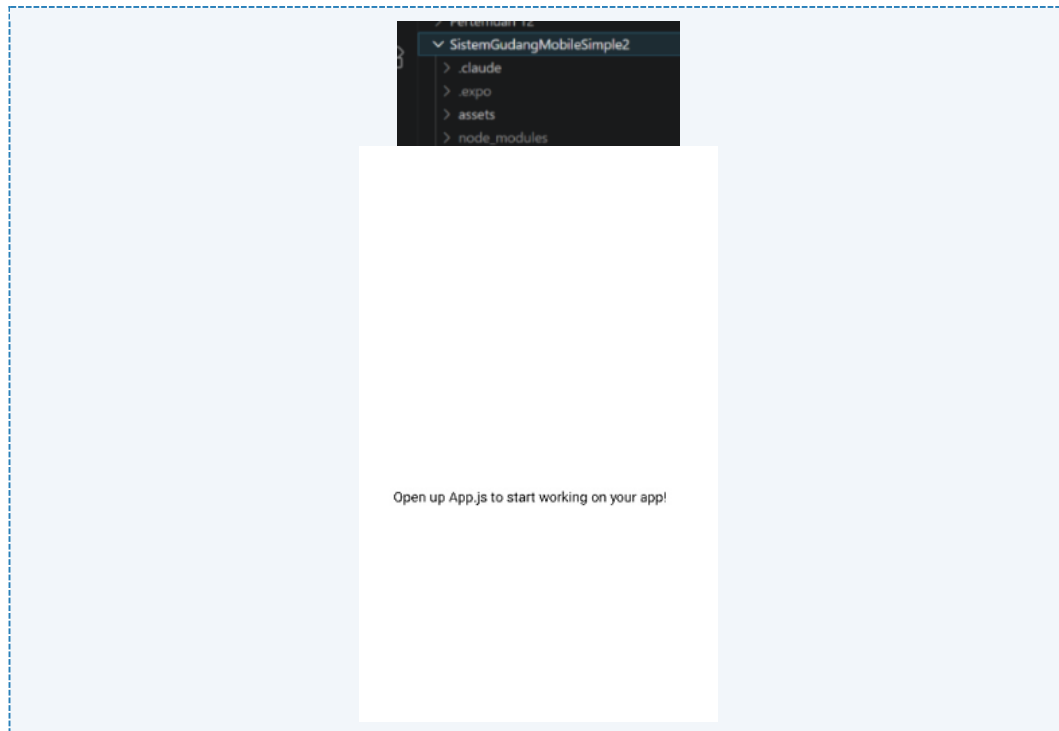
E.1 Langkah 1 – Membuat Project Expo

```
* ===== KODE / OUTPUT LANGKAH 1 ===== */
import React from 'react';
import { StyleSheet, Text, View } from 'react-native';

export default function App() {
  return (
    <View style={styles.container}>
      <Text>
        Open up App.js to start working on your app!
      </Text>
    </View>
  );
}

const styles = StyleSheet.create({
  container: {
    flex: 1,
    backgroundColor: '#fff',
    alignItems: 'center',
    justifyContent: 'center',
  },
});
```

Screenshot Hasil:



Gambar E.1 – Hasil Pembuatan Set Up React

E.2 Langkah 2 – Memodifikasi App.js (UI Dashboard Sederhana)

```

/* ===== KODE / OUTPUT LANGKAH 2 ===== */
import { StatusBar } from 'expo-status-bar';
import { StyleSheet, Text, View, SafeAreaView, Platform,
TouchableOpacity, Alert } from 'react-native';
export default function App() {
  return (
    <SafeAreaView style={styles.container}>
      <StatusBar style="auto" />

      {/* Bagian Header */}
      <View style={styles.header}>
        <Text style={styles.headerTitle}>PT. Manufaktur
Maju</Text>
        <Text style={styles.headerSubtitle}>Aplikasi Monitoring
Gudang</Text>
      </View>
      {/* Bagian Konten Utama */}
      <View style={styles.content}>
        <Text style={styles.welcomeText}>Selamat Datang,
Operator!</Text>
        <TouchableOpacity
          style={styles.card}
          onPress={() => Alert.alert("Info", "Membuka Detail Stok
Gudang A...")}

```

```

        <Text style={styles.cardTitle}>Status Gudang A</Text>
        <Text style={styles.cardValue}>Kapasitas: 85%</Text>
        <Text style={styles.cardStatus}>TEKAN UNTUK
DETAIL</Text>
    </TouchableOpacity>

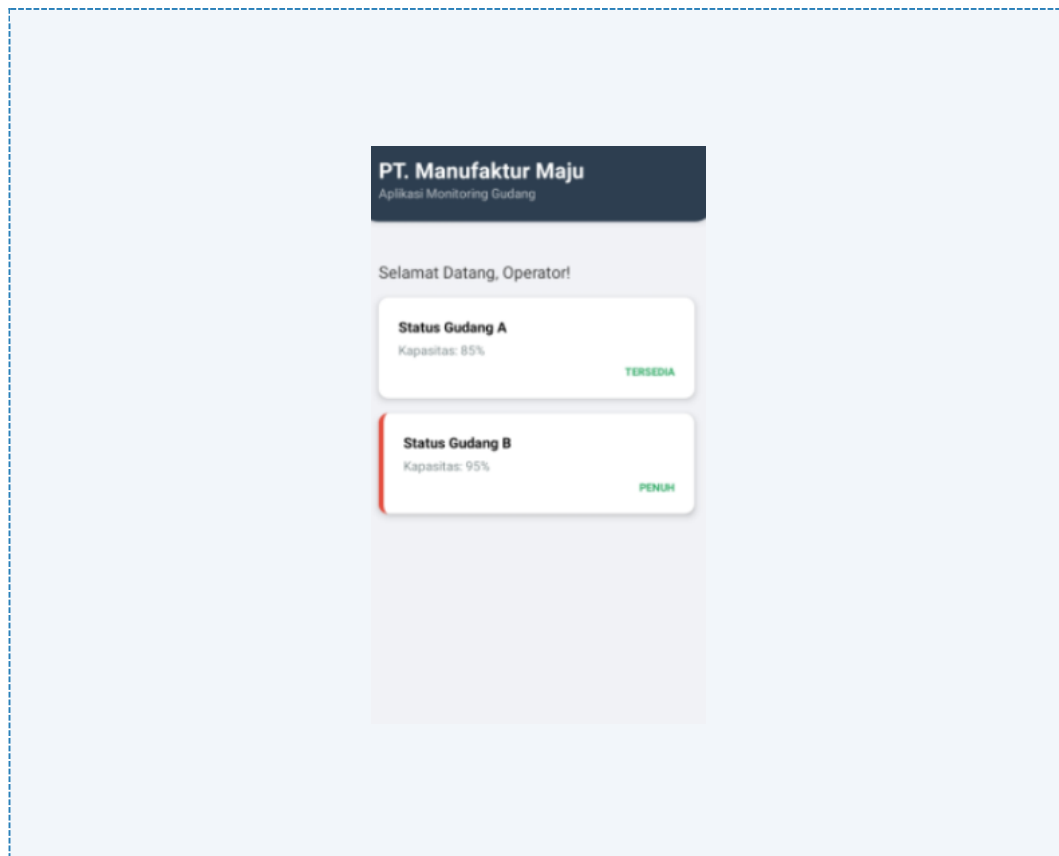
    <View style={[styles.card, styles.cardWarning]}>
        <Text style={styles.cardTitle}>Status Gudang B</Text>
        <Text style={styles.cardValue}>Kapasitas: 95%</Text>
        <Text style={styles.cardStatus}>PENUH</Text>
    </View>
</View>
</SafeAreaView>
);
}
// Styling menggunakan StyleSheet (seperti CSS tapi object JS)
const styles = StyleSheet.create({
  container: {
    flex: 1, // Artinya layar penuh
    backgroundColor: '#f0f2f5',
    paddingTop: Platform.OS === 'android' ? 25 : 0, // Padding
    khusus Android status bar
  },
  header: {
    backgroundColor: '#2c3e50',
    padding: 20,
    borderBottomLeftRadius: 20,
    borderBottomRightRadius: 20,
    marginBottom: 20,
    elevation: 5, // Shadow di Android
  },
  headerTitle: {
    color: 'white',
    fontSize: 24,
    fontWeight: 'bold',
  },
  headerSubtitle: {
    color: '#bdc3c7',
    fontSize: 14,
  },
  content: {
    padding: 20,
  },
  welcomeText: {
    fontSize: 18,

```



```
        marginBottom: 15,
        color: '#333',
    },
    card: {
        backgroundColor: 'white',
        padding: 20,
        borderRadius: 10,
        marginBottom: 15,
        // Shadow khusus iOS dan Android
        shadowColor: "#000",
        shadowOffset: { width: 0, height: 2 },
        shadowOpacity: 0.25,
        shadowRadius: 3.84,
        elevation: 5,
    },
    cardWarning: {
        borderLeftWidth: 5,
        borderLeftColor: '#e74c3c',
    },
    cardTitle: {
        fontSize: 16,
        fontWeight: 'bold',
        marginBottom: 5,
    },
    cardValue: {
        fontSize: 14,
        color: '#7f8c8d',
    },
    cardStatus: {
        fontSize: 12,
        fontWeight: 'bold',
        color: '#27ae60',
        marginTop: 5,
        textAlign: 'right'
    }
  }
});
```

Screenshot Hasil:

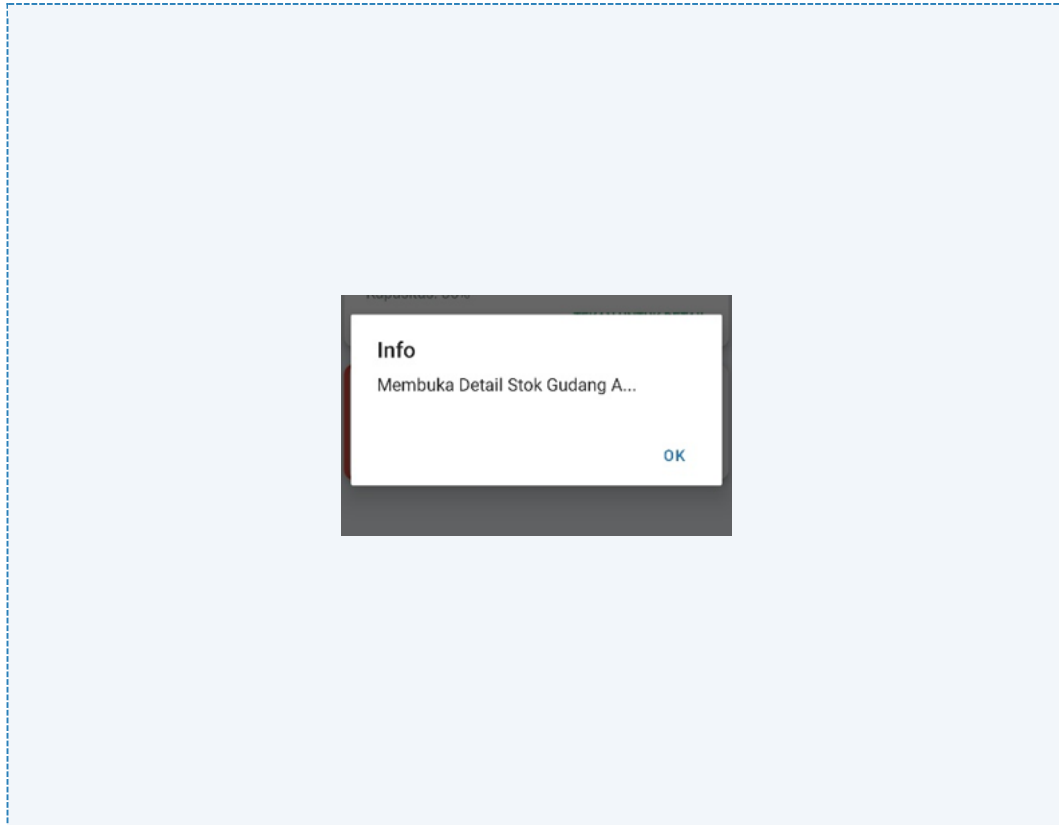


Gambar E.2 – Hasil Pembuatan Dashboard Sederhana

E.3 Langkah 3 – Menambahkan Interaksi (Button & Alert)

```
/* ===== KODE / OUTPUT LANGKAH 3 ===== */
import { StyleSheet, Text, View, SafeAreaView, Platform,
TouchableOpacity, Alert } from 'react-native';
<TouchableOpacity
  style={styles.card}
  onPress={() => Alert.alert("Info", "Membuka Detail Stok
Gudang A...")}
>
  <Text style={styles.cardTitle}>Status Gudang A</Text>
  <Text style={styles.cardValue}>Kapasitas: 85%</Text>
  <Text style={styles.cardStatus}>TEKAN UNTUK
DETAIL</Text>
</TouchableOpacity>
```

Screenshot Hasil:

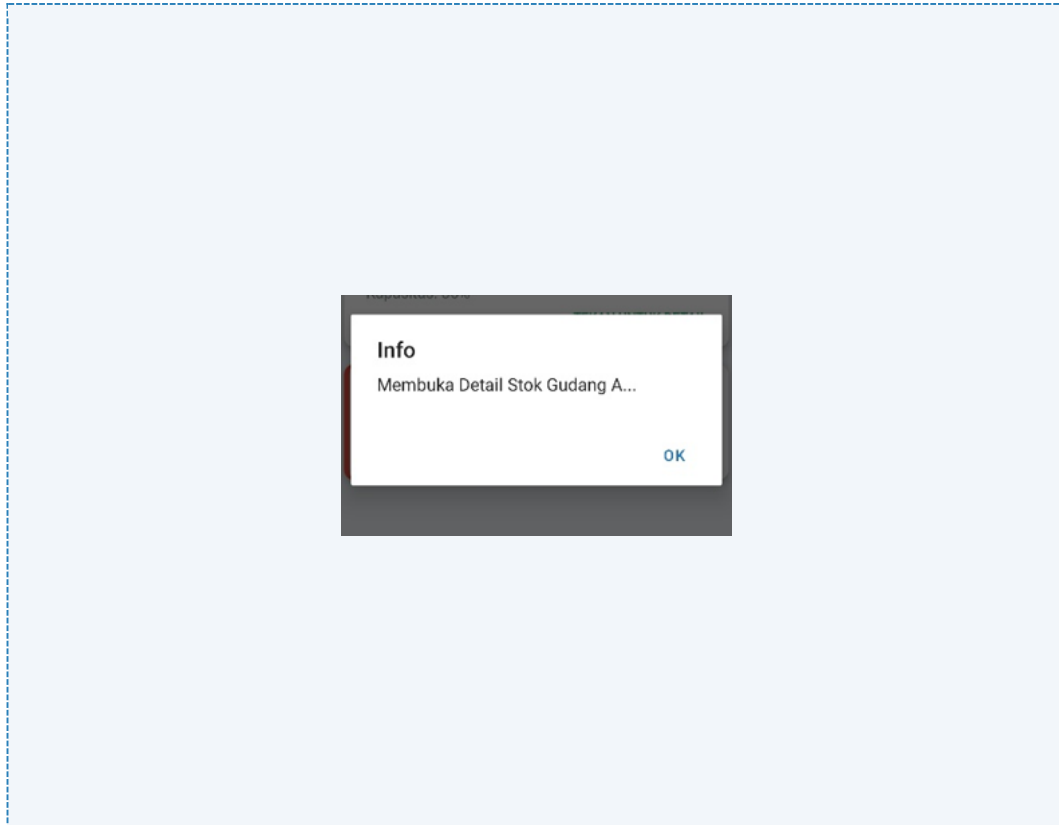


Gambar E.3– Hasil Pembuatan Grafik pada Halaman Dashboard

F. LATIHAN DAN TUGAS MANDIRI

F.1 Latihan 1 – Gambar/Ikon

Pertanyaan / Instruksi Latihan:

Cari gambar logo perusahaan (atau gunakan placeholder), taruh di folder project, lalu tampilkan menggunakan `` style={styles.logo} />`.

Jawaban / Kode Solusi:

```
<Image
  source={require('./logo.png')}
  style={styles.headerLogo}
  resizeMode="contain"
/>
headerLogo: {
  width: 38,
  height: 38,
  borderRadius: 8,
  marginRight: 12,
},
```



Gambar F.1 – Hasil Penambahan Logo

F.2 Latihan 2 – ScrollView

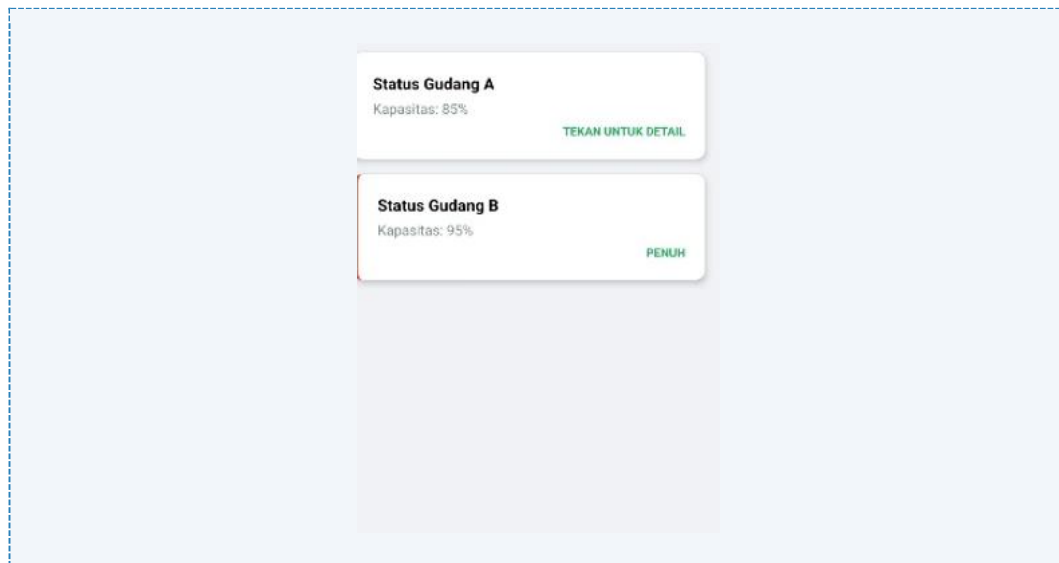
Pertanyaan / Instruksi Latihan:

Jika konten melebihi tinggi layar, gunakan ``<ScrollView>`` untuk membungkus konten agar bisa digulir ke bawah.

Jawaban / Kode Solusi:

```
<ScrollView>
  <View style={styles.content}>
    <Text style={styles.welcomeText}>Selamat Datang,
Operator!</Text>
    <TouchableOpacity
      style={styles.card}
      onPress={() => Alert.alert("Info", "Membuka Detail
Stok Gudang A...")}
    >
      <Text style={styles.cardTitle}>Status Gudang A</Text>
      <Text style={styles.cardValue}>Kapasitas: 85%</Text>
      <Text style={styles.cardStatus}>TEKAN UNTUK
DETAIL</Text>
    </TouchableOpacity>
  </View>
</ScrollView>
```

```
<View style={[styles.card, styles.cardWarning]}>  
  <Text style={styles.cardTitle}>Status Gudang B</Text>  
  <Text style={styles.cardValue}>Kapasitas: 95%</Text>  
  <Text style={styles.cardStatus}>PENUH</Text>  
</View>  
</View>  
</ScrollView>
```



Gambar F.2 – Hasil Pembuatan ScrollView

F.3 Tugas Proyek Mini – Profil Mesin Mobile

Deskripsi proyek mini:

- Buat halaman profil mesin yang menampilkan: Foto Mesin (gunakan placeholder), Nama Mesin, Tahun Pembuatan, dan Status.
- Gunakan Layout Flexbox (`flexDirection: 'row'`) untuk menyusun Foto di kiri dan Teks di kanan.

Penjelasan pendekatan/solusi yang digunakan:

Pendekatan yang digunakan pada proyek ini adalah membangun tampilan aplikasi mobile dengan menggunakan komponen dasar React Native seperti View, Text, Image, dan ScrollView. Layout disusun menggunakan flexDirection: 'row' sehingga gambar mesin ditempatkan di sisi kiri dan informasi mesin berada di sisi kanan agar tampilan lebih terstruktur. Selain itu, penggunaan StyleSheet membantu dalam mengatur warna, ukuran, jarak, serta desain card agar tampilan aplikasi menjadi lebih rapi dan responsif, sedangkan ScrollView digunakan untuk menampilkan daftar mesin secara

vertikal sehingga seluruh data tetap dapat diakses meskipun jumlahnya cukup banyak.

Kode Utama:

```
import { StatusBar } from 'expo-status-bar';
import { StyleSheet, Text, View, SafeAreaView, Platform,
TouchableOpacity, Alert, ScrollView, Image } from 'react-native';

export default function App() {
  return (
    <SafeAreaView style={styles.container}>
      <StatusBar style="auto" />

      {/* Bagian Header */}
      <View style={styles.header}>
        <Image
          source={require('./logo.png')}
          style={styles.headerLogo}
          resizeMode="contain"
        />
        <View style={styles.headerTextContainer}>
          <Text style={styles.headerTitle}>PT. Manufaktur
Maju</Text>
          <Text style={styles.headerSubtitle}>Profil Mesin
Mobile</Text>
        </View>
      </View>

      {/* ScrollView */}
      <ScrollView>
        <View style={styles.content}>
          <Text style={styles.welcomeText}>Selamat Datang,
Operator!</Text>

          {/* ===== BAGIAN PROYEK MINI: PROFIL MESIN ===== */}
          <Text style={styles.sectionTitle}>Daftar Mesin</Text>

          {/* Card Mesin 1 */}
```

```

        <View style={styles.mesinCard}>
            <Image
                source={require('./MesinCNCFrais.png')}
                style={styles.mesinImage}
            />
            <View style={styles.infoContainer}>
                <Text style={styles.mesinTitle}>Mesin CNC
Frais</Text>
                <Text style={styles.mesinDetail}>Tahun: 2021</Text>
                <Text style={[styles.mesinStatus,
styles.statusActive]}>Status: Beroperasi</Text>
            </View>
        </View>

        {/* Card Mesin 2 */}
        <View style={styles.mesinCard}>
            <Image
                source={require('./MesinBubutOtomatis.png')}
                style={styles.mesinImage}
            />
            <View style={styles.infoContainer}>
                <Text style={styles.mesinTitle}>Mesin Bubut
Otomatis</Text>
                <Text style={styles.mesinDetail}>Tahun: 2019</Text>
                <Text style={[styles.mesinStatus,
styles.statusMaintenance]}>Status: Perawatan</Text>
            </View>
        </View>

        {/* Card Mesin 3 */}
        <View style={styles.mesinCard}>
            <Image
                source={require('./MesinPressHidrolik.jpg')}
                style={styles.mesinImage}
            />
            <View style={styles.infoContainer}>
                <Text style={styles.mesinTitle}>Mesin Press
Hidrolik</Text>
                <Text style={styles.mesinDetail}>Tahun: 2018</Text>
                <Text style={[styles.mesinStatus,
styles.statusInactive]}>Status: Rusak</Text>
            </View>
        </View>

        {/* Card Mesin 4 */}
        <View style={styles.mesinCard}>

```

```

        <Image
            source={require('./MesinInjectionMolding.jpg')}
            style={styles.mesinImage}
        />
        <View style={styles.infoContainer}>
            <Text style={styles.mesinTitle}>Mesin Injection
Molding</Text>
            <Text style={styles.mesinDetail}>Tahun: 2022</Text>
            <Text style={[styles.mesinStatus,
styles.statusStandby]}>Status: Standby</Text>
        </View>
    </View>

    {/* Card Mesin 5 */}
    <View style={styles.mesinCard}>
        <Image
            source={require('./MesinPotongPlasma.jpg')}
            style={styles.mesinImage}
        />
        <View style={styles.infoContainer}>
            <Text style={styles.mesinTitle}>Mesin Potong
Plasma</Text>
            <Text style={styles.mesinDetail}>Tahun: 2020</Text>
            <Text style={[styles.mesinStatus,
styles.statusActive]}>Status: Beroperasi</Text>
        </View>
    </View>

    {/* Card Mesin 6 */}
    <View style={styles.mesinCard}>
        <Image
            source={require('./MesinLasRobotik.jpg')}
            style={styles.mesinImage}
        />
        <View style={styles.infoContainer}>
            <Text style={styles.mesinTitle}>Mesin Las
Robotik</Text>
            <Text style={styles.mesinDetail}>Tahun: 2023</Text>
            <Text style={[styles.mesinStatus,
styles.statusStandby]}>Status: Standby</Text>
        </View>
    </View>

    {/* Card Mesin 7 */}
    <View style={styles.mesinCard}>
        <Image

```



```

        source={require('./MesinBendingPlat.jpg')}
        style={styles.mesinImage}
      />
      <View style={styles.infoContainer}>
        <Text style={styles.mesinTitle}>Mesin Bending
Plat</Text>
        <Text style={styles.mesinDetail}>Tahun: 2017</Text>
        <Text style={
[styles.mesinStatus,
styles.statusMaintenance]}>Status: Perawatan</Text>
      </View>
    </View>

    { /* Card Mesin 8 */ }
    <View style={styles.mesinCard}>
      <Image
        source={require('./MesinBorRadial.jpg')}
        style={styles.mesinImage}
      />
      <View style={styles.infoContainer}>
        <Text style={styles.mesinTitle}>Mesin Bor
Radial</Text>
        <Text style={styles.mesinDetail}>Tahun: 2019</Text>
        <Text style={
[styles.mesinStatus,
styles.statusActive]}>Status: Beroperasi</Text>
      </View>
    </View>

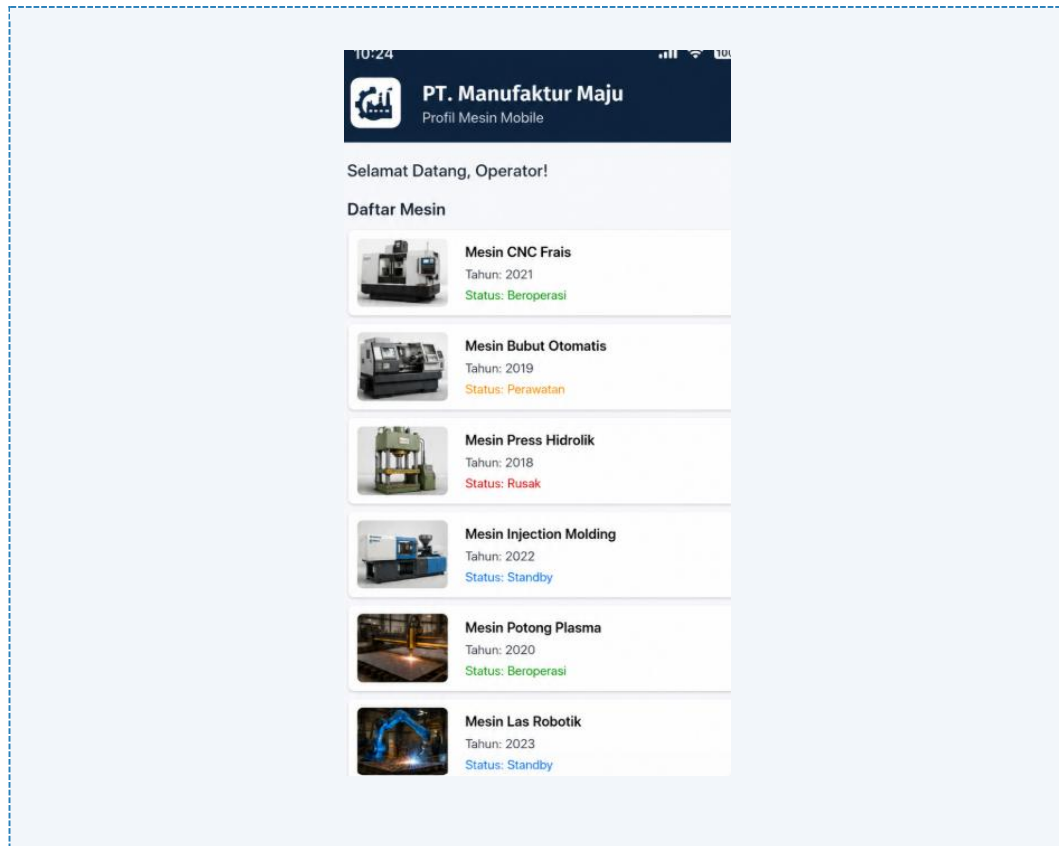
    </View>
  </ScrollView>
</SafeAreaView>
);
}

// Styling
const styles = StyleSheet.create({
  container: {
    flex: 1,
    backgroundColor: '#f0f2f5',
    paddingTop: Platform.OS === 'android' ? 37 : 0,
  },
  header: {
    backgroundColor: '#2c3e50',
    padding: 20,
    borderBottomLeftRadius: 20,
    borderBottomRightRadius: 20,
    marginBottom: 5,
  },
});

```

```
        elevation: 5,
        flexDirection: 'row',
        alignItems: 'center',
    },
    headerLogo: {
        width: 38,
        height: 38,
        borderRadius: 8,
        marginRight: 12,
    },
    headerTextContainer: {
        flex: 1,
    },
    headerTitle: {
        color: 'white',
        fontSize: 22,
        fontWeight: 'bold',
        marginBottom: 2,
    },
    headerSubtitle: {
        color: '#bdc3c7',
        fontSize: 14,
    },
    content: {
        padding: 20,
    },
    welcomeText: {
        fontSize: 18,
        marginBottom: 15,
        color: '#333',
    },
    sectionTitle: {
        fontSize: 18,
        fontWeight: 'bold',
        color: '#2c3e50',
        marginTop: 10,
        marginBottom: 15,
    },
    mesinCard: {
        backgroundColor: 'white',
        padding: 15,
        borderRadius: 10,
        marginBottom: 15,
        flexDirection: 'row',
        alignItems: 'center',
        elevation: 5,
```

```
    },
    mesinImage: {
      width: 70,
      height: 70,
      borderRadius: 8,
      marginRight: 15,
      backgroundColor: '#e0e0e0',
    },
    infoContainer: {
      flex: 1,
    },
    mesinTitle: {
      fontSize: 16,
      fontWeight: 'bold',
      color: '#333',
      marginBottom: 5,
    },
    mesinDetail: {
      fontSize: 14,
      color: '#7f8c8d',
      marginBottom: 5,
    },
    mesinStatus: {
      fontSize: 13,
      fontWeight: 'bold',
    },
    statusActive: { color: '#27ae60' },
    statusMaintenance: { color: '#f39c12' },
    statusInactive: { color: '#e74c3c' },
    statusStandby: { color: '#3498db' },
  });
```



Gambar F.3 – Hasil Pembuatan ProyekMini Profile Mesin Mobile

G. PEMBAHASAN

G.1 Analisis Kesesuaian Hasil dengan Teori

React Native dirancang untuk memberikan pengalaman seperti aplikasi native dengan proses pengembangan yang lebih sederhana. Hasil praktik menunjukkan bahwa konsep ini dapat diterapkan dengan baik melalui penggunaan komponen dasar seperti View, Text, dan Image untuk membangun tampilan aplikasi mobile sederhana. Selain itu, penggunaan Expo terbukti mempermudah proses pengembangan karena aplikasi dapat dijalankan tanpa konfigurasi yang rumit, sehingga pengembang lebih fokus pada desain dan logika aplikasi.

Namun, terdapat beberapa keterbatasan yang sesuai dengan teori, yaitu tetap diperlukannya pemahaman terhadap struktur komponen dan pengaturan tampilan menggunakan StyleSheet. Selain itu, perbedaan antara konsep web dan mobile dapat menjadi tantangan awal bagi pengguna yang terbiasa dengan HTML dan CSS. Hal ini menunjukkan bahwa meskipun React Native mempermudah pengembangan, pemahaman dasar tentang konsep aplikasi mobile tetap diperlukan agar implementasi berjalan optimal.

G.2 Kendala yang Ditemukan dan Solusinya

Kendala / Error yang Ditemukan	Solusi yang Diterapkan
Aplikasi tidak muncul di Expo Go	Memastikan koneksi jaringan dan restart Expo server
Error import komponen React Native	Memperbaiki penulisan path dan syntax import

G.3 Kaitan dengan Penerapan di Industri

Dalam dunia industri, aplikasi mobile berbasis React Native dapat digunakan untuk mendukung berbagai aktivitas operasional seperti monitoring produksi, absensi karyawan, manajemen gudang, hingga pelaporan kondisi mesin. Dengan sistem mobile, data dapat dikirim dan diproses secara real-time sehingga perusahaan dapat merespon masalah di lapangan dengan lebih cepat. Selain itu, penggunaan React Native juga mendukung fleksibilitas kerja karena supervisor atau operator dapat mengakses informasi langsung melalui smartphone serta terintegrasi dengan sistem perusahaan seperti ERP. Dari sisi bisnis, teknologi ini juga lebih efisien karena satu kode program dapat digunakan untuk Android dan iOS, sehingga dapat menghemat biaya pengembangan dan mempercepat proses digitalisasi industri.

H. KESIMPULAN

1	Komponen dasar seperti View, Text, Image, dan StyleSheet memiliki fungsi penting dalam membangun struktur serta tampilan antarmuka aplikasi mobile.
2	Expo berperan sebagai development environment yang menyediakan proses build, live reload, debugging, dan deployment aplikasi mobile tanpa konfigurasi native Android Studio atau Xcode secara langsung.
3	Penggunaan StyleSheet pada React Native menerapkan konsep styling berbasis object JavaScript sehingga pengaturan layout, ukuran, warna, dan positioning dilakukan secara terstruktur dan modular.
4	

I. DAFTAR PUSTAKA

Tuliskan semua sumber referensi yang digunakan dalam format APA 7th Edition. Minimal 3 referensi yang relevan (termasuk Manual Praktikum dan sumber online resmi seperti MDN, Reactjs.org, dll).

No.	Referensi (Format APA 7th Edition)
[1]	Burhandenny, A. E. (2026). Manual Praktikum Aplikasi Web dan Mobile Semester Genap 2025/2026. Program Studi Teknik Industri, Universitas Negeri Yogyakarta.
[2]	
[3]	
[4]	

J. LEMBAR PENILAIAN DAN PENGESAHAN

RUBRIK PENILAIAN LAPORAN					
Aspek Penilaian		Bobot			
No.	Aspek Penilaian	Bobot (%)	Nilai (0-100)	Skor Akhir	Catatan Penilai
1	Kelengkapan Isi (Semua bagian A–I terisi lengkap)	20%			
2	Kualitas Dasar Teori (Ditulis dengan bahasa sendiri, relevan, minimal 3 paragraf)	15%			
3	Dokumentasi Langkah Kerja (Kode & Screenshot sesuai, jelas, dan terbaca)	25%			
4	Tugas/Latihan Mandiri (Semua latihan & proyek mini dikerjakan dan berfungsi)	20%			
5	Pembahasan & Analisis (Kritis, mendalam, dan relevan dengan industri)	15%			
6	Kesimpulan & Tata Bahasa (Spesifik, rapi, mengikuti format yang ditentukan)	5%			
TOTAL NILAI AKHIR		100%			

Mahasiswa	Rekan Tim	Diperiksa oleh Asisten / Dosen
 _____ Saprina Melita Sari NIM:23051430012	 _____ Ardita Natalia NIM:23051430022	 _____ Dr. Eng. Ir. Aji Ery Burhandenny, S.T., M.AIT. Tanggal: _____

MODUL 15

A. IDENTITAS & INFORMASI MODUL

Mata Kuliah	Aplikasi Web dan Mobile
Kode Modul	Modul 15
Topik Utama	Component, Navigation, dan Styling di React Native + Proyek Mobile
Sub-Topik	React Navigation (Stack Navigator), Passing Data Antar Layar, dan Proyek Inventori Sederhana
Durasi Praktikum	340 Menit
Tanggal Praktikum	02 Juni 2026
Nama Mahasiswa	Saprina Melita Sari
NIM	23051430012
Kelas	A1

B. TUJUAN PEMBELAJARAN

1	Memahami konsep navigasi layar di aplikasi mobile (Stack).
2	Mampu menginstall dan mengonfigurasi `react-navigation`.
3	Membuat aplikasi multi-halaman: Beranda (List) → Detail Item.
4	Meneruskan data (params) dari satu layar ke layar lain

C. ALAT DAN BAHAN

No.	Nama Alat / Bahan / Aplikasi	Versi / Spesifikasi	Keterangan
1	Visual Studio Code (VS Code)	Versi 1.8x ke atas	Code Editor
2	Google Chrome / Browser Modern	Versi terbaru	Testing & Debugging
3	Node.js & npm	Node 18+ / npm 9+	Runtime JS

4	Prettier	Versi 12.3.0	Extensioins VS Code untuk merapikan indentansi code
5	Aplikasi Expo GO	Versi yang kompatibel dengan SDK 54	Aplikasi untuk menampilkan hasil
6	React Native	SDK Versi 54	Framework pengembangan aplikasi mobile

D. DASAR TEORI

D.1 Konsep Utama

Pada pertemuan ini, materi berfokus pada pengembangan aplikasi mobile menggunakan React Native. Pembahasan menekankan penggunaan komponen dasar, navigasi antar halaman (stack navigation), serta pengiriman data antar layar menggunakan route params. Selain itu, praktikum diarahkan untuk membangun aplikasi inventori sederhana yang terdiri dari halaman daftar barang dan halaman detail barang. Stack navigation yang digunakan dalam materi ini merupakan konsep perpindahan layar yang bekerja seperti tumpukan. Setiap halaman baru akan berada di atas halaman sebelumnya, sedangkan untuk kembali ke halaman sebelumnya, layar teratas akan dihapus dari tumpukan.

Pola ini umum digunakan dalam pengembangan aplikasi mobile modern karena memberikan alur navigasi yang lebih terstruktur dan mudah dipahami pengguna. Selain navigasi, materi juga membahas proses pengiriman data antar halaman. Data dari halaman utama dapat diteruskan ke halaman detail melalui parameter navigasi. Dengan cara ini, informasi yang ditampilkan pada halaman detail dapat menyesuaikan item yang dipilih oleh pengguna. Hal ini membuat aplikasi menjadi lebih dinamis dan interaktif.

Penerapan konsep ini juga didukung dengan pemahaman dasar mengenai alur data dalam aplikasi mobile. Dengan adanya pengiriman parameter antar halaman, pengembang dapat membangun aplikasi yang lebih terorganisir dan efisien. Selain itu, praktik ini menjadi dasar penting dalam pengembangan aplikasi mobile yang berbasis data dan memiliki banyak halaman.

D.2 Keterkaitan dengan Teknik Industri

Dalam perspektif Teknik Industri, pengembangan aplikasi inventori berkaitan erat dengan pengelolaan informasi dan sistem pendukung operasional dalam organisasi. Inventori merupakan bagian penting dari rantai pasok yang

membutuhkan pencatatan dan akses data secara efektif agar proses pengendalian barang atau aset dapat berjalan lebih optimal. Penggunaan navigasi dan struktur data yang terorganisasi dalam aplikasi mencerminkan aliran informasi dalam sistem, di mana setiap proses saling terhubung untuk mendukung kebutuhan pengguna. Selain itu, pengembangan aplikasi mobile juga menjadi bagian dari transformasi digital di industri, yang bertujuan meningkatkan efisiensi, efektivitas pengambilan keputusan, serta integrasi teknologi informasi dalam pengelolaan sumber daya.

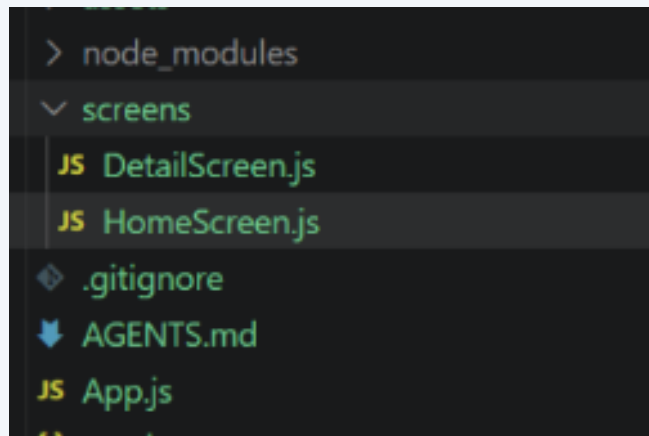
E. LANGKAH KERJA DAN HASIL PRAKTIKUM

E.1 Langkah 1 – Struktur Folder

Deskripsi singkat apa yang dilakukan pada langkah ini:

```
/* ===== KODE / OUTPUT LANGKAH 1 ===== */  
npx create-expo-app praktikum --template blank
```

Screenshot Hasil:



Gambar E.1 – Hasil Set Up Project

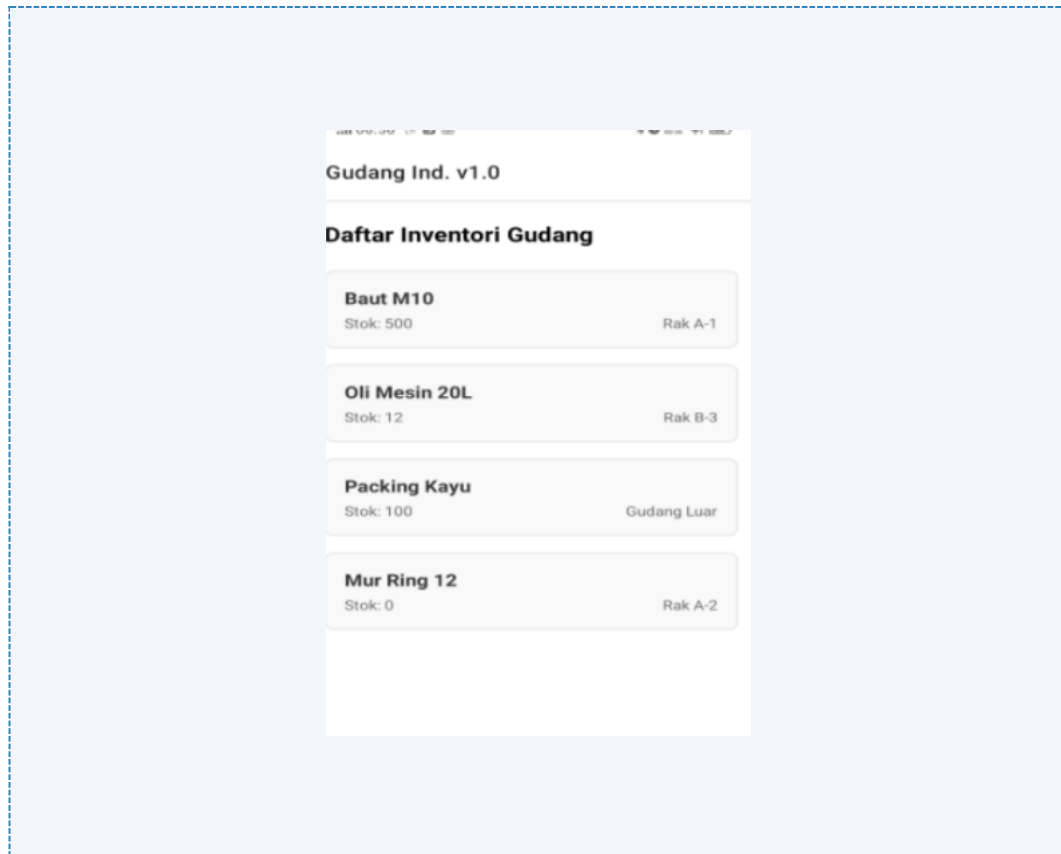
E.2 Langkah 2 – Membuat HomeScreen (Daftar Inventori)

```
/* ===== KODE / OUTPUT LANGKAH 2 ===== */
import React from 'react';
import { View, Text, FlatList, TouchableOpacity, StyleSheet }
from 'react-native';
// Data Mock Inventori
const DATA_INVENTORI = [
  { id: '1', nama: 'Baut M10', stok: 500, lokasi: 'Rak A-1'
},
  { id: '2', nama: 'Oli Mesin 20L', stok: 12, lokasi: 'Rak B-
3' },
  { id: '3', nama: 'Packing Kayu', stok: 100, lokasi: 'Gudang
Luar' },
  { id: '4', nama: 'Mur Ring 12', stok: 0, lokasi: 'Rak A-2'
}, // Stok Habis
];
function HomeScreen({ navigation }) {

  // Fungsi Render Item untuk FlatList
  const renderItem = ({ item }) => (
    <TouchableOpacity
      style={styles.itemContainer}
      onPress={() => navigation.navigate('Detail', {
itemData: item })}
    >
      <Text style={styles.itemTitle}>{item.nama}</Text>
      <View style={styles.itemInfo}>
        <Text style={styles.itemSub}>Stok:
{item.stok}</Text>
        <Text
style={styles.itemSub}>{item.lokasi}</Text>
      </View>
    </TouchableOpacity>
  );
  return (
    <View style={styles.container}>
      <Text style={styles.header}>Daftar Inventori
      <FlatList
        data={DATA_INVENTORI}
        renderItem={renderItem}
        keyExtractor={item => item.id}
      />
    </View>
  );
}
const styles = StyleSheet.create({
  container: {
```

```
        flex: 1,
        backgroundColor: '#fff',
        paddingTop: 20,
      },
      header: {
        fontSize: 22,
        fontWeight: 'bold',
        marginBottom: 15,
        paddingHorizontal: 15,
      },
      itemContainer: {
        backgroundColor: '#f9f9f9',
        padding: 15,
        marginVertical: 8,
        marginHorizontal: 15,
        borderRadius: 8,
        borderWidth: 1,
        borderColor: '#ddd',
      },
      itemTitle: {
        fontSize: 18,
        fontWeight: 'bold',
        color: '#333',
      },
      itemInfo: {
        flexDirection: 'row',
        justifyContent: 'space-between',
        marginTop: 5,
      },
      itemSub: {
        color: '#666',
      },
    });
export default HomeScreen;
```

Screenshot Hasil:



Gambar E.2 – Hasil Pembuatan Home Screen

E.3 Langkah 3 – Membuat DetailScreen

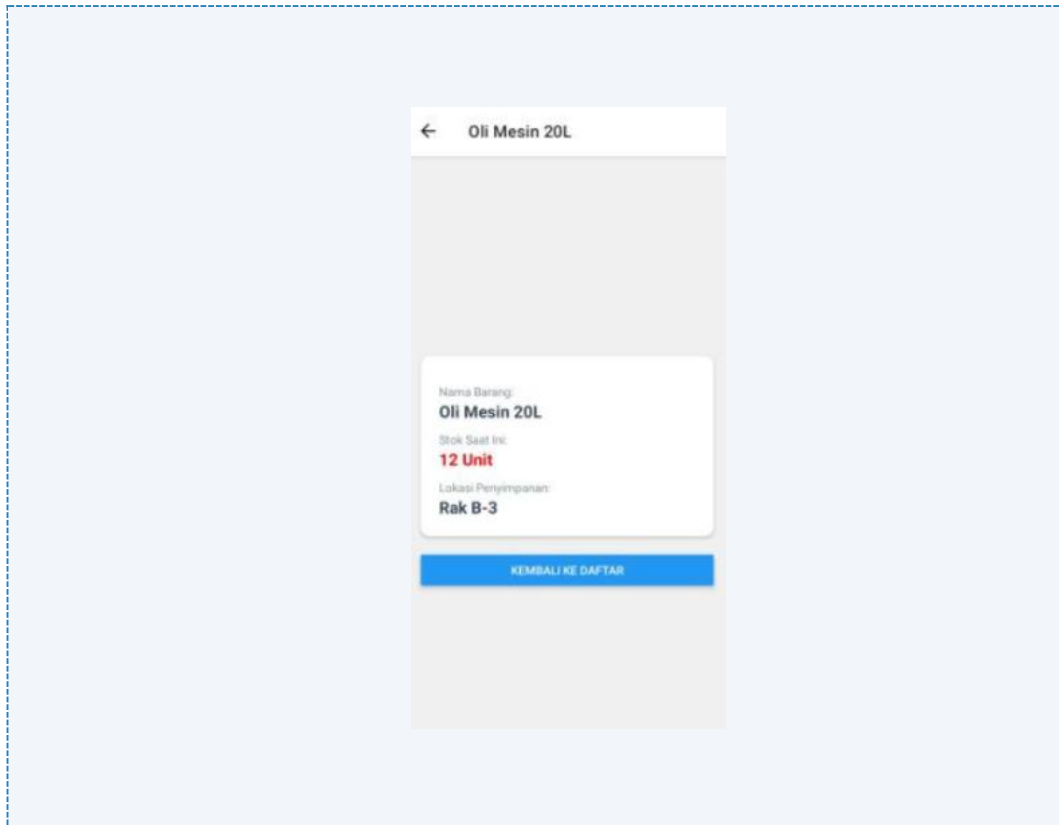
```
/* ===== KODE / OUTPUT LANGKAH 3 ===== */  
import React from 'react';  
import { View, Text, StyleSheet, Button } from 'react-native';  
function DetailScreen({ route, navigation }) {  
  // Menerima data yang dikirim dari HomeScreen  
  const { itemData } = route.params;  
  return (  
    <View style={styles.container}>  
      <View style={styles.card}>  
        <Text style={styles.label}>Nama Barang:</Text>  
        <Text  
style={styles.value}>{itemData.nama}</Text>  
        <Text style={styles.label}>Stok Saat  
Ini:</Text>  
        <Text style={styles.value, itemData.stok < 20  
? styles.dangerText :  
styles.successText}>  
{itemData.stok} Unit  
        </Text>  
      </View>  
    </View>  
  );  
}
```

```
        <Text style={styles.label}>Lokasi
Penyimpanan:</Text>
        <Text
style={styles.value}>{itemData.lokasi}</Text>
    </View>

    <Button title="Kembali ke Daftar" onPress={() =>
navigation.goBack()} />
    </View>
  );
}
const styles = StyleSheet.create({
  container: {
    flex: 1,
    justifyContent: 'center',
    padding: 20,
    backgroundColor: '#f0f0f0',
  },
  header: {
    fontSize: 20,
    fontWeight: 'bold',
    textAlign: 'center',
    color: '#2c3e50',
  },
  card: {
    backgroundColor: 'white',
    padding: 20,
    borderRadius: 10,
    marginBottom: 20,
    elevation: 3,
  },
  label: {
    fontSize: 14,
    color: '#7f8c8d',
    marginTop: 10,
  },
  value: {
    fontSize: 20,
    fontWeight: 'bold',
    color: '#2c3e50',
  },
  dangerText: {
    color: 'red',
  },
  successText: {
    color: 'green',
  }
});
```

```
});  
export default DetailScreen;
```

Screenshot Hasil:



Gambar E.3– Hasil Pembuatan Detail Screen

E.4 Langkah 4 – Menghubungkan Navigation di App.js

```
/* ===== KODE / OUTPUT LANGKAH 3 ===== */  
import { StatusBar } from 'expo-status-bar';  
import { StyleSheet, Text, View, SafeAreaView, Platform,  
TouchableOpacity, Alert, ScrollView, Image } from 'react-  
native';  
import { NavigationContainer } from '@react-navigation/native';  
import { createNativeStackNavigator } from '@react-  
navigation/native-stack';  
import HomeScreen from './screens/HomeScreen';  
import DetailScreen from './screens/DetailScreen';  
const Stack = createNativeStackNavigator();  
export default function App() {  
  return (  

```

```
<NavigationContainer>
  <Stack.Navigator initialRouteName="Home">
    <Stack.Screen
      name="Home"
      component={HomeScreen}
      options={{ title: 'Gudang Ind. v1.0' }}
    />
    <Stack.Screen
      name="Detail"
      component={DetailScreen}
      options={({ route }) => ({ title:
route.params?.itemData.nama || 'Detail' })}
    />
  </Stack.Navigator>
</NavigationContainer>
);
}

// Styling menggunakan StyleSheet
const styles = StyleSheet.create({
  container: {
    flex: 1,
    backgroundColor: '#f0f2f5',
    paddingTop: Platform.OS === 'android' ? 37 : 0,
  },
  header: {
    backgroundColor: '#2c3e50',
    padding: 20,
    borderBottomLeftRadius: 20,
    borderBottomRightRadius: 20,
    marginBottom: 5,
    elevation: 5, // Shadow di Android
    flexDirection: 'row',
    alignItems: 'center',
  },
  headerLogo: {
    width: 38,
    height: 38,
    borderRadius: 8,
    marginRight: 12,
  },
  headerTextContainer: {
    flex: 1,
  },
  headerTitle: {
    color: 'white',
```

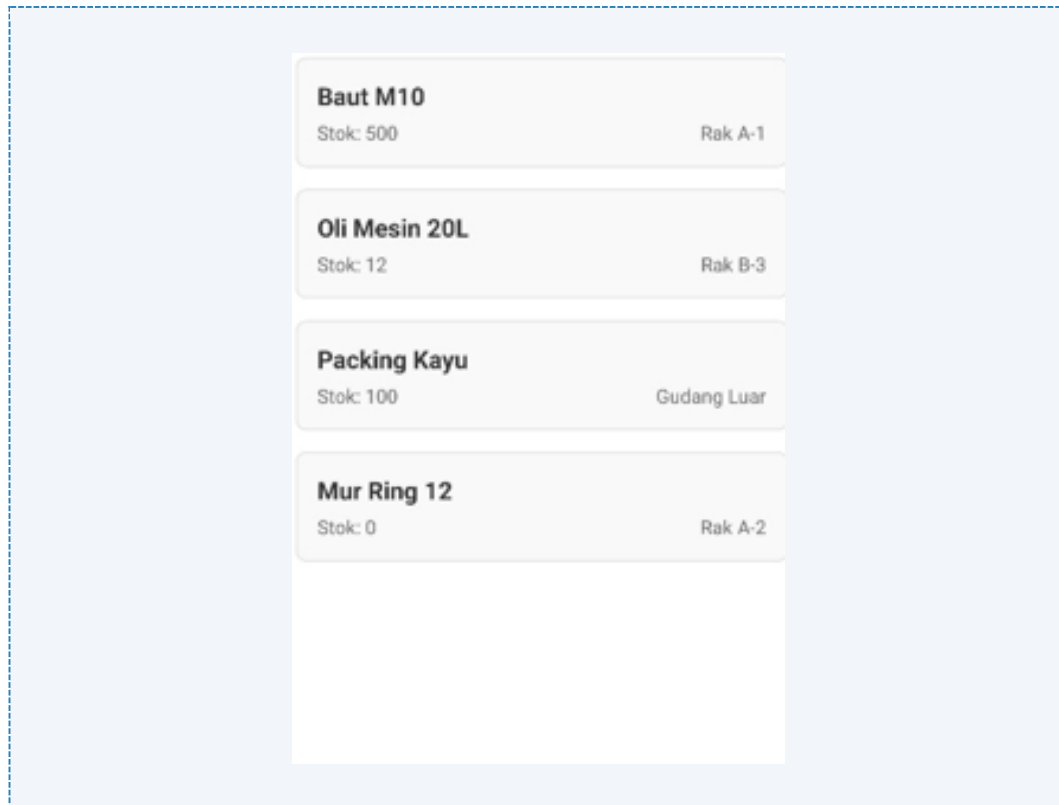


```
        fontSize: 22,
        fontWeight: 'bold',
        marginBottom: 2,
      },
      headerSubtitle: {
        color: '#bdc3c7',
        fontSize: 14,
      },
      content: {
        padding: 20,
      },
      welcomeText: {
        fontSize: 18,
        marginBottom: 15,
        color: '#333',
      },

      // === Styling Proyek Mini (Mesin) ===
      sectionTitle: {
        fontSize: 18,
        fontWeight: 'bold',
        color: '#2c3e50',
        marginTop: 10,
        marginBottom: 15,
      },
      mesinCard: {
        backgroundColor: 'white',
        padding: 15,
        borderRadius: 10,
        marginBottom: 15,
        flexDirection: 'row', // Foto di kiri, teks di kanan
        alignItems: 'center',
        shadowColor: "#000",
        shadowOffset: { width: 0, height: 2 },
        shadowOpacity: 0.25,
        shadowRadius: 3.84,
        elevation: 5,
      },
      mesinImage: {
        width: 70,
        height: 70,
        borderRadius: 8,
        marginRight: 15,
        backgroundColor: '#e0e0e0',
      },
      infoContainer: {
```

```
        flex: 1,
      },
      mesinTitle: {
        fontSize: 16,
        fontWeight: 'bold',
        color: '#333',
        marginBottom: 5,
      },
      mesinDetail: {
        fontSize: 14,
        color: '#7f8c8d',
        marginBottom: 5,
      },
      mesinStatus: {
        fontSize: 13,
        fontWeight: 'bold',
      },
      statusActive: {
        color: '#27ae60',
      },
      statusMaintenance: {
        color: '#f39c12',
      },
      statusInactive: {
        color: '#e74c3c',
      },
      statusStandby: {
        color: '#3498db',
      }
    }
  });
```

Screenshot Hasil:



Gambar E.4– Hasil Menghubungkan Navigasi

F. LATIHAN DAN TUGAS MANDIRI

F.1 Latihan 1 – Conditional Rendering

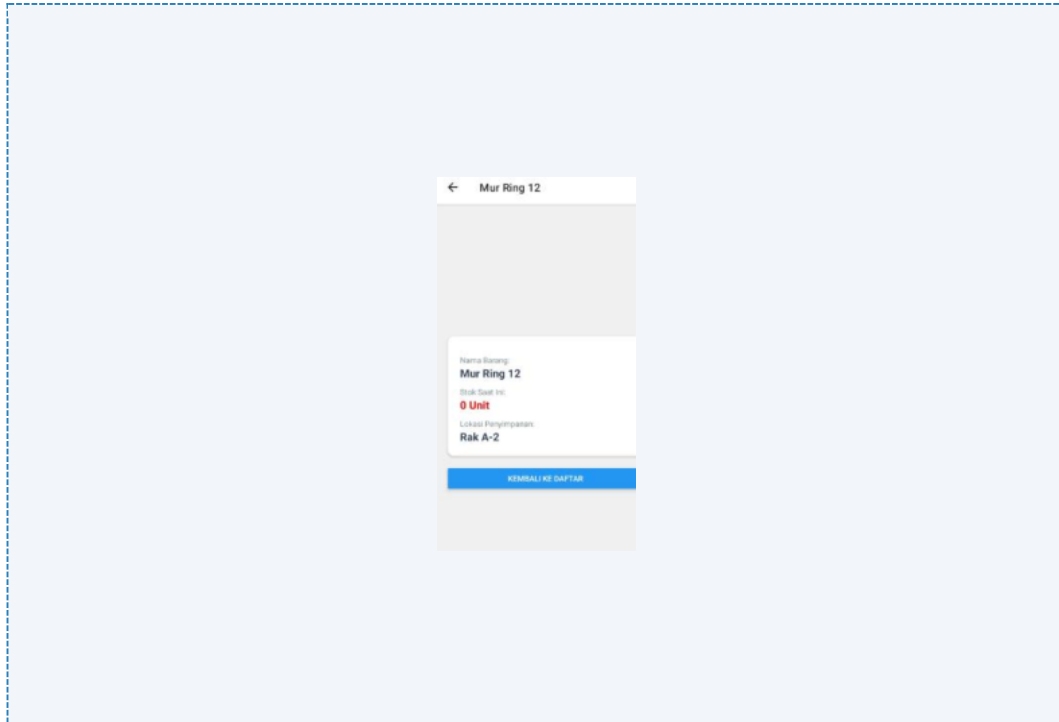
Pertanyaan / Instruksi Latihan:

Pada `DetailScreen`, jika stok = 0, munculkan tombol "Request Stok Darurat" (gunakan Button tambahan).

Jawaban / Kode Solusi:

```
import { View, Text, StyleSheet, Button } from 'react-native';

{itemData.stok === 0 && (
  <Button
    title="Request Stok Darurat"
    onPress={() => alert('Permintaan stok
telah dikirim!')}
  />
)}
```



Gambar F.1 – Hasil Penambahan Button Request Stock Darurat

F.2 Latihan 2 – Passing Data

Pertanyaan / Instruksi Latihan:

Di `HomeScreen`, tambahkan tombol FAB (Floating Action Button) atau Button di bawah untuk "Tambah Barang". Saat ditekan, navigasi ke layar (screen baru bernama `TambahScreen`) yang berisi form kosong.

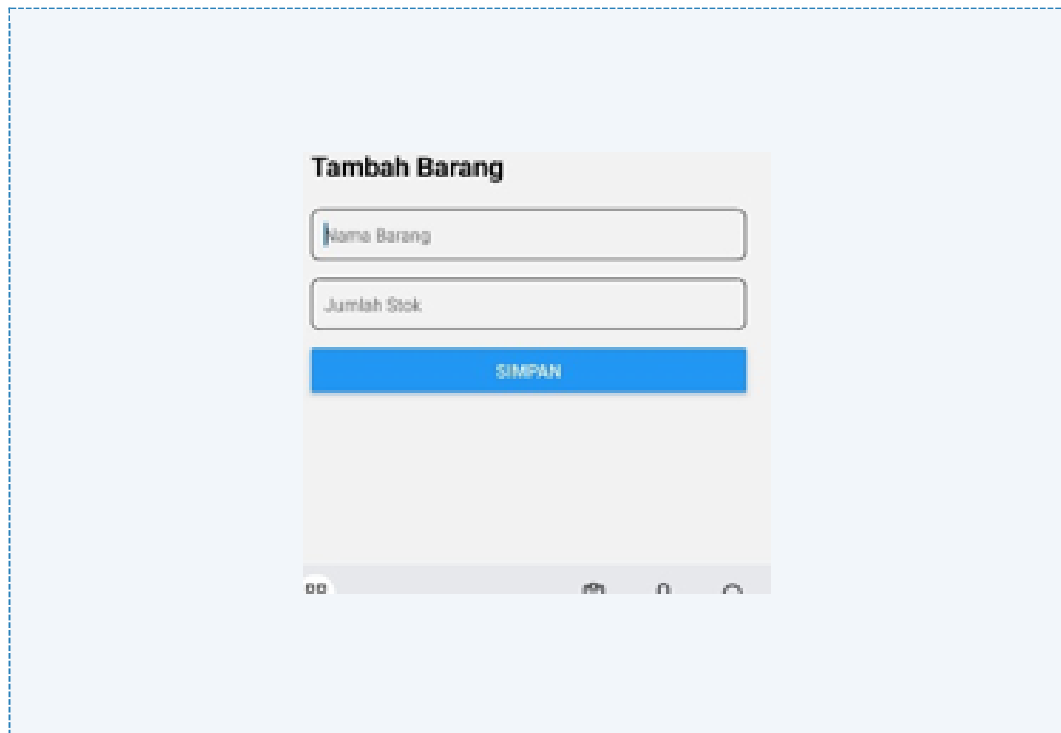
Jawaban / Kode Solusi:

```
// HomeScreen.js
<Button
  title="Tambah Barang"
  onPress={() => navigation.navigate('Tambah')}
/>

// TambahScreen.js
import React, { useState } from 'react';
import {
  View,
  Text,
  TextInput,
  Button,
  StyleSheet
} from 'react-native';
```

```
export default function TambahScreen({ navigation }) {  
  const [nama, setNama] = useState('');  
  const [stok, setStok] = useState('');  
  
  return (  
    <View style={styles.container}>  
      <Text style={styles.title}>Tambah Barang</Text>  
  
      <TextInput  
        placeholder="Nama Barang"  
        value={nama}  
        onChangeText={setNama}  
        style={styles.input}  
      />  
  
      <TextInput  
        placeholder="Jumlah Stok"  
        value={stok}  
        onChangeText={setStok}  
        keyboardType="numeric"  
        style={styles.input}  
      />  
  
      <Button  
        title="Simpan"  
        onPress={() => {  
          alert(`Barang ${nama} berhasil  
ditambahkan`);  
          navigation.goBack();  
        }}  
      />  
    </View>  
  );  
}  
  
const styles = StyleSheet.create({  
  container: {  
    flex: 1,  
    padding: 20,  
  },  
  title: {  
    fontSize: 22,  
    fontWeight: 'bold',  
    marginBottom: 20,  
  },  
  input: {
```

```
borderWidth: 1,  
marginBottom: 15,  
padding: 10,  
borderRadius: 8,  
},  
));
```



Gambar F.2 – Hasil Pembuatan Button dan Screen Tambah Barang

F.3 Tugas Proyek Mini – Aplikasi Inspeksi QC

Deskripsi proyek mini:

Buat aplikasi inspeksi sederhana.

- Home: Daftar item yang perlu diinspeksi.
- Detail: Foto item, standar kualitas, dan dropdown status (Lolos/Gagal).
- Simulasi: Saat user memilih "Gagal" di layar detail dan kembali, ubah warna teks di Home menjadi merah (memerlukan sedikit logika state management sederhana).

Penjelasan pendekatan/solusi yang digunakan:

Proyek ini menggunakan pendekatan pengembangan aplikasi mobile berbasis React Native dengan memanfaatkan komponen fungsional (functional component), state management sederhana menggunakan React Hooks (useState), serta navigasi antarhalaman menggunakan React Navigation. Data produk inspeksi disimpan dalam state terpusat pada komponen utama aplikasi sehingga dapat diakses dan diperbarui oleh beberapa layar. Pendekatan ini dipilih karena sesuai untuk aplikasi berskala kecil hingga menengah yang membutuhkan interaksi data secara real-time tanpa kompleksitas manajemen state yang lebih besar seperti Redux atau Context API.

Kode Utama:

```
// app.js
import React, { useState } from 'react';
import { NavigationContainer } from '@react-navigation/native';
import { createNativeStackNavigator } from '@react-navigation/native-stack';

import HomeScreen from './screens/HomeScreen';
import DetailScreen from './screens/DetailScreen';

const Stack = createNativeStackNavigator();

export default function App() {

  const [items, setItems] = useState([
    {
      id: '1',
      nama: 'Produk A',
      status: 'Belum Dicek'
    },
    {
      id: '2',
      nama: 'Produk B',
      status: 'Belum Dicek'
    },
    {
      id: '3',
      nama: 'Produk C',
      status: 'Belum Dicek'
    }
  ]);
```

```
return (  
  <NavigationContainer>  
    <Stack.Navigator>  
  
      <Stack.Screen name="Home">  
        {(props) => (  
          <HomeScreen  
            {...props}  
            items={items}  
            setItems={setItems}  
          />  
        )}  
      </Stack.Screen>  
  
      <Stack.Screen name="Detail">  
        {(props) => (  
          <DetailScreen  
            {...props}  
            items={items}  
            setItems={setItems}  
          />  
        )}  
      </Stack.Screen>  
  
    </Stack.Navigator>  
  </NavigationContainer>  
);  
}  
  
// HomeScreen.js  
import React from 'react';  
import {  
  View,  
  Text,  
  FlatList,  
  TouchableOpacity,  
  StyleSheet  
} from 'react-native';  
  
export default function HomeScreen({  
  navigation,  
  items  
) {  
  
  return (  
    <View style={styles.container}>
```



```
<Text style={styles.header}>
  Daftar Inspeksi QC
</Text>

<FlatList
  data={items}
  keyExtractor={(item) => item.id}
  renderItem={({ item }) => (
    <TouchableOpacity
      style={styles.card}
      onPress={() =>
        navigation.navigate('Detail', {
          itemData: item
        })
      }
    >
      <Text style={styles.nama}>
        {item.nama}
      </Text>

      <Text
        style={[
          item.status === 'Gagal' && {
color: 'red' },
          item.status === 'Lolos' && {
color: 'green' }
        ]}
      >
        {item.status}
      </Text>
    </TouchableOpacity>
  )}
/>
</View>
);
}

const styles = StyleSheet.create({
  container: {
    flex: 1,
    padding: 20,
  },

  header: {
    fontSize: 22,
    fontWeight: 'bold',
```

```
        marginBottom: 20,
      },

      identitas: {
        fontSize: 16,
        textAlign: 'center',
        marginBottom: 5,
      },

      card: {
        padding: 15,
        borderWidth: 1,
        marginBottom: 10,
        borderRadius: 8,
      },

      nama: {
        fontSize: 18,
        fontWeight: 'bold',
      },
    });

//DetailScreen.js
import React, { useState } from 'react';
import {
  View,
  Text,
  Button,
  StyleSheet,
  Alert
} from 'react-native';
import { Picker } from '@react-native-picker/picker';

export default function DetailScreen({
  route,
  navigation,
  items,
  setItems
}) {

  const { itemData } = route.params;

  const [status, setStatus] = useState(itemData.status);

  const handleStatusChange = (itemValue) => {
```

```
setStatus(itemValue);

setItems(
  items.map((item) =>
    item.id === itemData.id
      ? {
          ...item,
          status: itemValue
        }
      : item
  )
);

if (itemValue === 'Lolos') {
  Alert.alert('Informasi', 'Produk Lolos QC');
}

if (itemValue === 'Gagal') {
  Alert.alert('Informasi', 'Produk Gagal QC');
}
};

return (
  <View style={styles.container}>

    <View style={styles.card}>
      <Text style={styles.nama}>
        Sesa Nugroho Satriva
      </Text>

      <Text style={styles.nim}>
        23051430031
      </Text>

      <Text style={styles.title}>
        {itemData.nama}
      </Text>

      <Text style={styles.label}>
        Standar Kualitas:
      </Text>

      <Text style={styles.deskripsi}>
        Tidak cacat, warna sesuai, ukuran sesuai.
      </Text>
    </View>
  </View>
);
```

```
<Text style={styles.label}>
  Status Inspeksi:
</Text>

<Text
  style={[
    styles.status,
    status === 'Lolos' && styles.lolos,
    status === 'Gagal' && styles.gagal
  ]}
>
  {status}
</Text>
</View>

<View style={styles.dropdownContainer}>
  <Text style={styles.label}>
    Pilih Status:
  </Text>

  <Picker
    selectedValue={status}
    onChange={handleStatusChange}
  >
    <Picker.Item
      label="Belum Dicek"
      value="Belum Dicek"
    />

    <Picker.Item
      label="Lolos"
      value="Lolos"
    />

    <Picker.Item
      label="Gagal"
      value="Gagal"
    />
  </Picker>
</View>

<View style={{ marginTop: 20 }}>
  <Button
    title="Kembali"
    onPress={() => navigation.goBack()}
  />
</View>
```

```
        </View>

        </View>
    );
}

const styles = StyleSheet.create({
  container: {
    flex: 1,
    padding: 20,
    backgroundColor: '#f5f5f5',
  },

  card: {
    backgroundColor: '#fff',
    padding: 20,
    borderRadius: 10,
    elevation: 3,
  },

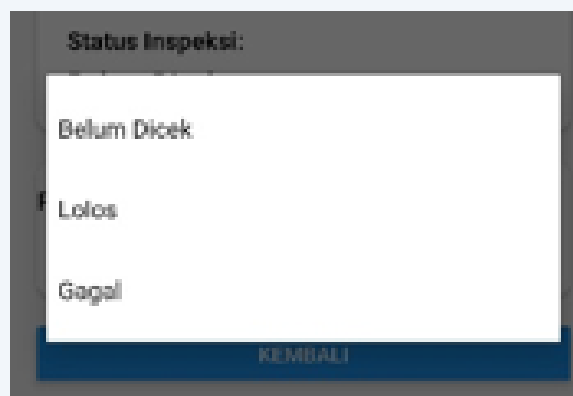
  nama: {
    fontSize: 20,
    fontWeight: 'bold',
    textAlign: 'center',
  },

  nim: {
    fontSize: 16,
    textAlign: 'center',
    color: '#666',
    marginBottom: 20,
  },

  title: {
    fontSize: 24,
    fontWeight: 'bold',
    color: '#2c3e50',
  },

  dropdownContainer: {
    marginTop: 20,
    backgroundColor: '#fff',
    borderRadius: 10,
    elevation: 2,
  },
});
```

```
label: {  
  marginTop: 15,  
  fontWeight: 'bold',  
  fontSize: 16,  
},  
  
deskripsi: {  
  fontSize: 15,  
  marginTop: 5,  
},  
  
status: {  
  fontSize: 18,  
  fontWeight: 'bold',  
  marginTop: 5,  
},  
  
lolos: {  
  color: 'green',  
},  
  
gagal: {  
  color: 'red',  
},  
});
```



Gambar F.3 – Hasil Pembuatan ProyekMini Aplikasi Inspeksi QC

G. PEMBAHASAN

G.1 Analisis Kesesuaian Hasil dengan Teori

Berdasarkan teori pengembangan aplikasi mobile, penggunaan stack navigation merupakan pendekatan yang sesuai untuk aplikasi yang memiliki alur perpindahan berjenjang, seperti halaman daftar menuju halaman detail. Implementasi yang dijelaskan dalam praktikum menunjukkan kesesuaian dengan teori tersebut karena pengguna dapat berpindah dan kembali ke halaman sebelumnya dengan mekanisme yang sistematis.

G.2 Kendala yang Ditemukan dan Solusinya

Kendala / Error yang Ditemukan	Solusi yang Diterapkan
Perbedaan versi library atau SDK menyebabkan error saat menjalankan aplikasi.	Menggunakan versi library yang kompatibel dengan SDK yang digunakan serta melakukan pembaruan dependensi sesuai kebutuhan proyek.

G.3 Kaitan dengan Penerapan di Industri

Dalam lingkungan industri, konsep navigasi dan pertukaran data antar halaman banyak diterapkan pada berbagai sistem seperti inventori, pergudangan, logistik, dan manajemen aset, di mana pengguna dapat melihat daftar data, memilih item tertentu, lalu mengakses informasi detail secara langsung. Prinsip ini juga digunakan dalam sistem Enterprise Resource Planning (ERP), yang memungkinkan data antar modul saling terhubung sehingga informasi yang diperoleh lebih lengkap tanpa perlu pencarian ulang, sekaligus meningkatkan efisiensi kerja dan mengurangi kesalahan. Di era digitalisasi industri saat ini, pengembangan aplikasi mobile menjadi semakin penting karena mendukung akses data secara real-time dari berbagai lokasi, sehingga konsep yang dipelajari dalam praktikum tidak hanya bersifat akademik, tetapi juga relevan dengan kebutuhan nyata sistem informasi industri modern.

H. KESIMPULAN

1	React Native memungkinkan pengembangan aplikasi mobile secara terstruktur menggunakan konsep komponen dan navigasi antar layar.
2	Implementasi Stack Navigation mempermudah perpindahan pengguna dari halaman utama ke halaman detail secara sistematis.
3	
4	

I. DAFTAR PUSTAKA

No.	Referensi (Format APA 7th Edition)
[1]	Burhandenny, A. E. (2026). Manual Praktikum Aplikasi Web dan Mobile Semester Genap 2025/2026. Program Studi Teknik Industri, Universitas Negeri Yogyakarta.
[2]	https://react.dev/learn
[3]	https://react.dev/reference/react-dom/hooks/useFormStatus
[4]	https://www.javascripttutorial.net/react-tutorial/

J. LEMBAR PENILAIAN DAN PENGESAHAN

RUBRIK PENILAIAN LAPORAN					
Aspek Penilaian		Bobot			
No.	Aspek Penilaian	Bobot (%)	Nilai (0-100)	Skor Akhir	Catatan Penilai
1	Kelengkapan Isi (Semua bagian A–I terisi lengkap)	20%			
2	Kualitas Dasar Teori (Ditulis dengan bahasa sendiri, relevan, minimal 3 paragraf)	15%			
3	Dokumentasi Langkah Kerja (Kode & Screenshot sesuai, jelas, dan terbaca)	25%			
4	Tugas/Latihan Mandiri (Semua latihan & proyek mini dikerjakan dan berfungsi)	20%			
5	Pembahasan & Analisis (Kritis, mendalam, dan relevan dengan industri)	15%			
6	Kesimpulan & Tata Bahasa (Spesifik, rapi, mengikuti format yang ditentukan)	5%			
TOTAL NILAI AKHIR		100%			

Mahasiswa	Rekan Tim	Diperiksa oleh Asisten / Dosen
 _____ Saprina Melita Sari NIM:23051430012	 _____ Ardita Natalia NIM:23051430022	 _____ Dr. Eng. Ir. Aji Ery Burhandenny, S.T., M.AIT. Tanggal: _____